

Lean 形式化数学

Mathematics in Lean 中文版

1. 引言

1.1. 入门

本书的目标，是教会读者使用 Lean 4 交互式证明助手来形式化数学。本书假定读者具备一定的数学知识，但所需预备知识并不多。尽管我们将涉及从数论到测度论与分析的诸多例子，重点仍将放在这些领域的初等方面；如果读者尚不熟悉相关内容，也可以在学习过程中逐步掌握。我们也不预设读者具有形式化方法方面的背景。形式化可以看作一种计算机编程：我们将用一种规整的、类似程序语言且 Lean 能够理解的语言，来写出数学定义、定理与证明。作为回报，Lean 会提供反馈和信息，解释表达式并保证它们是良构的，最终还会认证我们的证明确实正确。

关于 Lean 的更多信息，可以参见 [Lean 项目页面](#) 以及 [Lean 社区网页](#)。本教程建立在 Lean 庞大且持续增长的库 *Mathlib* 之上。我们也强烈建议尚未加入的读者加入 [Lean Zulip 在线聊天室](#)。在那里，你会遇到一个活跃而友善的 Lean 爱好者社群，大家乐于回答问题并提供支持。

虽然读者可以在线阅读本书的 PDF 或 HTML 版本，但本书的设计目标是交互式阅读，也就是在 VS Code 编辑器中运行 Lean。开始学习前，请先完成以下步骤：

按照这些[安装说明](#)安装 Lean 4 和 VS Code。

点击 VS Code 右上角的全称量词符号，选择 *Open Project*、*Download Project*，再选择 *Mathematics in Lean*，以获取本仓库。

本书每一节都有一个关联的 Lean 文件，其中包含例子和练习。它们位于按章节组织的 `MIL` 文件夹中。我们强烈建议复制该文件夹，并在副本中实验和完成练习。这样既能保持原文件不变，也能在仓库内容更新时更方便地同步（见下文）。你可以把副本命名为 `my_files`，或任何你喜欢的名字，并用它创建自己的 Lean 文件。

到这一步，你可以按如下方式在 VS Code 的侧边面板中打开本教材：

输入 `ctrl-shift-P`（macOS 中为 `command-shift-P`）。

在出现的输入栏中输入 `Lean 4: Docs: Show Documentation Resources`，然后按回车。（一旦菜单中高亮该项，也可以直接按回车选择。）

在打开的窗口中点击 `Mathematics in Lean`。

另一种方式是在云端通过 [Codespaces](#) 运行 Lean 和 VS Code。关于如何使用 Codespaces，可以在 *Mathematics in Lean* 的 [项目页面](#) 找到说明。即便采用这种方式，我们仍建议按照上文所述，在 *MIL* 文件夹的副本中工作。

本教材及其关联仓库仍在持续开发中。你可以在 `mathematics_in_lean` 文件夹中输入 `git pull`，随后输入 `lake exe cache get` 来更新仓库。（这假定你没有修改 `MIL` 文件夹中的内容；这也正是我们建议创建副本的原因。）

我们希望读者在阅读教材时，在 `MIL` 文件夹中完成练习；教材正文则提供解释、说明与提示。正文中经常会包含如下例子：

```
#eval "Hello, World!"
```

你应当能够在相应的 Lean 文件中找到对应例子。点击该行时，VS Code 会在 `Lean InfoView` 窗口中显示 Lean 的反馈；将光标悬停在 `#eval` 命令上时，VS Code 会在弹出窗口中显示 Lean 对该命令的响应。我们鼓励你编辑文件，并尝试自己构造例子。

本书还提供了大量具有挑战性的练习，供读者尝试。不要匆忙跳过它们！Lean 关乎以交互方式做数学，而不只是阅读关于数学的文字。完成练习是这种体验的核心。当然，你不必完成所有练习；当你确信自己已经掌握相关技能时，完全可以继续向后阅读。你也总是可以将自己的解答同每一节对应的 `solutions` 文件夹中的解答进行比较。

1.2. 概览

简单地说，Lean 是一种用于在称为 *依值类型论* 的形式语言中构造复杂表达式的工具。

每个表达式都有一个类型，你可以使用 `#check` 命令将其打印出来。有些表达式的类型形如 $\mathbb{N}$ 或 $\mathbb{N} \rightarrow \mathbb{N}$ 。这些表达式是数学对象。

```
#check 2 + 2

def f (x :  $\mathbb{N}$ ) :=
  x + 3

#check f
```

有些表达式的类型是 *Prop*。这些表达式是数学命题。

```
#check 2 + 2 = 4

def FermatLastTheorem :=
   $\forall x y z n : \mathbb{N}, n > 2 \wedge x^n + y^n = z^n \rightarrow x = 0 \vee y = 0 \vee z = 0$ 

#check FermatLastTheorem
```

还有一些表达式的类型是 *P*，而 *P* 本身的类型是 *Prop*。这样的表达式就是命题 *P* 的一个证明。

```
theorem easy : 2 + 2 = 4 :=
  rfl

#check easy

theorem hard : FermatLastTheorem :=
  sorry

#check hard
```

如果你能够构造一个类型为 `FermatLastTheorem` 的表达式，并且 Lean 接受它为该类型的项，那么你就完成了一件相当了不起的事。（使用 `sorry` 是作弊，而 Lean 知道这一点。）至此，你已经知道这场游戏的目标；剩下要学习的只是规则。

本书与配套教程 *Theorem Proving in Lean* 相辅相成；后者对 Lean 的底层逻辑框架和核心语法作了更系统的介绍。*Theorem Proving in Lean* 适合那种在使用一台新洗碗机之前，愿意从头到尾读完用户手册的人。如果你更倾向于先按下启动按钮，再慢慢弄清楚如何启用强力洗涤功能，那么从本书开始，并在必要时回头查阅 *Theorem Proving in Lean*，会更合适。

Mathematics in Lean 与 *Theorem Proving in Lean* 的另一项区别在于，本书更强调策略的使用。既然我们的目标是构造复杂表达式，Lean 提供了两种途径：我们可以直接写出这些表达式本身（也就是写出合适的文本描述），也可以向 Lean 给出构造这些表达式的指令。例如，下列表达式表示这样一个事实的证明：若 `n` 为偶数，则 `m * n` 也是偶数。

```
example : ∀ m n : Nat, Even n → Even (m * n) := fun m n (k, (hk : n = k + k)) =>
  have hmn : m * n = m * k + m * k := by rw [hk, mul_add]
  show ∃ l, m * n = l + l from (_, hmn)
```

这个证明项可以压缩成一行：

```
example : ∀ m n : Nat, Even n → Even (m * n) :=
fun m n (k, hk) => (m * k, by rw [hk, mul_add])
```

下面则是同一定理的一个策略式证明，其中以 `--` 开头的行是注释，因此会被 Lean 忽略：

```
example : ∀ m n : Nat, Even n → Even (m * n) := by
  -- Say `m` and `n` are natural numbers, and assume `n = 2 * k`.
  rintro m n (k, hk)
  -- We need to prove `m * n` is twice a natural number. Let's show it's twice `m * k`.
  use m * k
  -- Substitute for `n`,
  rw [hk]
  -- and now it's obvious.
  ring
```

在 VS Code 中逐行输入这样的证明时，Lean 会在单独窗口中显示证明状态，告诉你已经建立了哪些事实，以及为了证明定理还剩下哪些任务。你可以逐行移动光标来回放证明，因为 Lean 会持续显示光标所在位置的证明状态。在这个例子中，你会看到证明的第一行引入了 `m` 和 `n`（如果愿意，此时也可以给它们另取名字），并且把假设 `Even n` 分解为某个 `k` 以及假设 `n = 2 * k`。第二行 `use m * k` 声明：我们将通过证明 `m * n = 2 * (m * k)` 来说明 `m * n` 是偶数。下一行使用 `rw` 策略在目标中把 `n` 替换为 `2 * k`（`rw` 是“rewrite”的缩写），而 `ring` 策略则解出由此得到的目标 `m * (2 * k) = 2 * (m * k)`。

以小步构造证明并获得增量反馈的能力极其强大。正因如此，策略式证明通常比直接写证明项更容易也更快捷。二者之间并没有截然分明的界线：策略式证明可以嵌入证明项中，就像上例中的 `by rw [hk, mul_add]` 那样。我们也会看到，反过来，在策略式证明中插入一个短小的证明项也常常很有用。尽管如此，本书的重点仍将放在策略的使用上。

在我们的例子中，策略式证明也可以缩减为一行：

```
example : ∀ m n : Nat, Even n → Even (m * n) := by
  rintro m n (k, hk); use m * k; rw [hk]; ring
```

这里我们使用策略完成了一些小的证明步骤。但策略也可以提供相当程度的自动化，并为更长的计算和更大的推理步骤给出论证。例如，我们可以调用 Lean 的简化器，并指定若干关于奇偶性的化简规则，从而自动证明我们的定理。

```
example : ∀ m n : Nat, Even n → Even (m * n) := by
  intros; simp [*, parity_simps]
```

这两本入门教程之间还有一个重要区别：*Theorem Proving in Lean* 只依赖核心 Lean 及其内建策略，而 *Mathematics in Lean* 则建立在 Lean 强大且持续增长的库 *Mathlib* 之上。因此，我们可以展示如何使用库中的若干数学对象和定理，以及一些非常有用的策略。本书并不打算作为该库的完整概览；社区网页中已有大量文档。我们的目标毋宁是介绍这种形式化背后的思维方式，并指出一些基本入口，使读者能够从容地浏览库并自行寻找所需内容。

交互式定理证明有时令人挫败，而且学习曲线陡峭。不过，Lean 社区对新手非常友好，[Lean Zulip 聊天室](#) 中几乎随时都有人可以回答问题。我们希望在那里见到你；也毫不怀疑，过不了多久，你也能够回答这类问题，并为 *Mathlib* 的发展作出贡献。

因此，如果你愿意接受，这就是你的任务：投入其中，尝试练习，带着问题来到 Zulip，并享受这个过程。不过也请预先知道：交互式定理证明会迫使你以根本上新的方式思考数学和数学推理。你的生活也许从此不再相同。

致谢。感谢 Gabriel Ebner 搭建了在 VS Code 中运行本教程的基础设施；感谢 Kim Morrison 和 Mario Carneiro 帮助将本教程从 Lean 4 早期版本迁移而来。我们也感谢 Takeshi Abe、Julian Berman、Alex Best、Axel Boldt、Thomas Browning、Bulwi Cha、Hanson Char、Bryan Gin-ge Chen、Steven Clontz、Mauricio Collaris、Johan Commelin、Mark Czubin、Alexandru Duca、Pierpaolo Frasa、Denis Gorbachev、Winston de Greef、Darij Grinberg、Mathieu Guay-Paquet、Rik Heurter、Marc Huisinga、Benjamin Jones、Julian Külshammer、Victor Liu、Jimmy Lu、Martin C. Martin、Giovanni Mascellani、John McDowell、Joseph McKinsey、Bhavik Mehta、Sebastian Miele、Isaiah Mindich、Kabelo Moiloa、Hunter Monroe、Pietro Monticone、Oliver Nash、Emanuelle Natale、Filippo A. E. Nuccio、Pim Otte、Nicolas Rolland、Keith Rush、Yannick Seurin、Guilherme Silva、Bernardo Subercaseaux、Pedro Sánchez Terraf、Matthew Toohey、Alistair Tucker、Floris van Doorn、Veniamin Viflyantsev、Eric Wieser 以及其他提供帮助的人与修正。我们的工作得到了 Hoskinson Center for Formal Mathematics 的部分支持。

2. 基础

本章旨在介绍 Lean 中数学推理的基本环节：计算、应用引理和定理，以及对一般结构进行推理。

2.1. 计算

我们通常学会进行数学计算时，并不会把这些计算看作证明。但是，当我们像 Lean 要求的那样，为计算中的每一步给出理由时，最终得到的正是一个证明：它证明该计算左端等于右端。

在 Lean 中，陈述一个定理等同于陈述一个目标，也就是证明该定理这一目标。Lean 提供重写策略 `rw`，用于在目标中把某个恒等式的左端替换为右端。若 `a`、`b` 和 `c` 是实数，则 `mul_assoc a b c` 是恒等式 `a * b * c = a * (b * c)`，而 `mul_comm a b` 是恒等式 `a * b = b * a`。Lean 提供的自动化通常使我们不必显式引用这类事实，但它们作为说明材料很有用。在 Lean 中，乘法向左结合，因此 `mul_assoc` 的左端也可以写作 `(a * b) * c`。不过，通常更好的风格是注意 Lean 的记号约定：Lean 省略括号时，我们也相应省略。

让我们试用一下 `rw`。

```
example (a b c : ℝ) : a * b * c = b * (a * c) := by
  rw [mul_comm a b]
  rw [mul_assoc b a c]
```

关联例子文件开头的 `import` 行，从 Mathlib 导入了实数理论以及有用的自动化工具。为了简洁，本教材通常省略这类信息。

欢迎你修改代码，观察会发生什么。在 VS Code 中，可以用 `\R` 或 `\real` 输入字符 `ℝ`。只有在按下空格键或 Tab 键之后，该符号才会出现。阅读 Lean 文件时，如果把鼠标悬停在某个符号上，VS Code 会显示可用于输入该符号的语法。如果想查看所有可用缩写，可以按 Ctrl-Shift-P，然后输入 abbreviations，以打开 `Lean 4: Show Unicode Input Abbreviations` 命令。如果键盘上不便输入反斜杠，也可以修改 `lean4.input.leader` 设置来改变前导字符。

当光标位于一个策略证明中间时，Lean 会在 *Lean Infoview* 窗口中报告当前的证明状态。随着你把光标逐步移过证明中的每一步，可以看到状态发生变化。Lean 中一个典型的证明状态可能如下所示：

```
1 goal
x y : ℕ,
h₁ : Prime x,
h₂ : ¬Even x,
h₃ : y > x
├ y ≥ 4
```

以 `├` 开头的那一行之前的各行表示语境：它们是当前正在讨论的对象和假设。在这个例子中，语境包括两个对象 `x` 和 `y`，它们都是自然数；还包括三个假设，分别标记为 `h₁`、`h₂` 和 `h₃`。在 Lean 中，语境中的一切都有一个标识符。你可以将这些带下标的标号输入为 `h\1`、`h\2` 和 `h\3`，但任何合法标识符都可以；例如也可以使用 `h1`、`h2`、`h3`，或者 `foo`、`bar` 和 `baz`。最后一行表示目标，也就是待证明的事实。有时人们用靶标指待证明的事实，而用目标指语境与靶标的组合。在实践中，具体含义通常由上下文决定。

请尝试证明这些恒等式；每一次都把 `sorry` 替换为一个策略证明。使用 `rw` 策略时，可以用左箭头 (`<`) 反向使用一个恒等式。例如，`rw [← mul_assoc a b c]` 会在当前目标中把 `a * (b * c)` 替换为 `a * b * c`。注意，左箭头表示按 `mul_assoc` 给出的恒等式从右向左使用；它与目标本身的左端或右端无关。

```
example (a b c : ℝ) : c * b * a = b * (a * c) := by
  sorry

example (a b c : ℝ) : a * (b * c) = b * (a * c) := by
  sorry
```

你也可以不带参数地使用 `mul_assoc`、`mul_comm` 这样的恒等式。在这种情况下，重写策略会试图用恒等式的左端匹配目标中的某个表达式，并采用它找到的第一个匹配模式。

```
example (a b c : ℝ) : a * b * c = b * c * a := by
  rw [mul_assoc]
  rw [mul_comm]
```

你也可以只提供部分信息。例如，`mul_comm a` 会匹配任何形如 `a * ?` 的模式，并将其重写为 `? * a`。请尝试在第一个例子中完全不提供参数，在第二个例子中只提供一个参数。

```
example (a b c : ℝ) : a * (b * c) = b * (c * a) := by
  sorry

example (a b c : ℝ) : a * (b * c) = b * (a * c) := by
  sorry
```

也可以把 `rw` 用于局部语境中的事实。

```
example (a b c d e f : ℝ) (h : a * b = c * d) (h' : e = f) : a * (b * e) = c * (d * f) := by
  rw [h']
  rw [← mul_assoc]
  rw [h]
  rw [mul_assoc]
```

请尝试下面的例子；第二个例子可使用定理 `sub_self`：

```
example (a b c d e f : ℝ) (h : b * c = e * f) : a * b * c * d = a * e * f * d := by
  sorry

example (a b c d : ℝ) (hyp : c = b * a - d) (hyp' : d = a * b) : c = 0 := by
  sorry
```

多个重写命令可以合并为一个命令：只需在方括号中用逗号列出相关恒等式即可。

```
example (a b c d e f : ℝ) (h : a * b = c * d) (h' : e = f) : a * (b * e) = c * (d * f) := by
  rw [h', ← mul_assoc, h, mul_assoc]
```

把光标放在任意重写列表中的逗号之后，仍然可以看到逐步推进的中间状态。

另一个技巧是，可以在例子或定理之外一次性声明变量。Lean 随后会自动把这些变量纳入相关语境。

```
variable (a b c d e f : ℝ)

example (h : a * b = c * d) (h' : e = f) : a * (b * e) = c * (d * f) := by
  rw [h', ← mul_assoc, h, mul_assoc]
```

检查上述证明开头的策略状态，可以看到 Lean 的确纳入了所有变量。我们可以把声明放进 `section ... end` 块中，以限定其作用域。最后，回忆引言中提到的内容：Lean 为我们提供了一个命令，用于确定表达式的类型。

```

section
variable (a b c : ℝ)

#check a
#check a + b
#check (a : ℝ)
#check mul_comm a b
#check (mul_comm a b : a * b = b * a)
#check mul_assoc c a b
#check mul_comm a
#check mul_comm

end

```

`#check` 命令既适用于对象，也适用于事实。对于命令 `#check a`，Lean 会报告 `a` 的类型是 `ℝ`。对于命令 `#check mul_comm a b`，Lean 会报告 `mul_comm a b` 是事实 `a * b = b * a` 的一个证明。命令 `#check (a : ℝ)` 表明我们预期 `a` 的类型是 `ℝ`；若事实并非如此，Lean 会报错。最后三个 `#check` 命令的输出我们稍后会解释；在此之前，你可以先查看它们，并自行尝试一些 `#check` 命令。

再看几个例子。定理 `two_mul a` 表示 $2 * a = a + a$ 。定理 `add_mul` 和 `mul_add` 表达乘法对加法的分配律，定理 `add_assoc` 表达加法的结合律。请使用 `#check` 命令查看它们的精确陈述。

```

example : (a + b) * (a + b) = a * a + 2 * (a * b) + b * b := by
  rw [mul_add, add_mul, add_mul]
  rw [← add_assoc, add_assoc (a * a)]
  rw [mul_comm b a, ← two_mul]

```

虽然可以在编辑器中逐步查看这个证明，从而弄清它究竟在做什么，但单独阅读它并不容易。Lean 提供了一种更结构化的方式，用关键字 `calc` 来书写这类证明。

```

example : (a + b) * (a + b) = a * a + 2 * (a * b) + b * b :=
  calc
    (a + b) * (a + b) = a * a + b * a + (a * b + b * b) := by
      rw [mul_add, add_mul, add_mul]
    _ = a * a + (b * a + a * b) + b * b := by
      rw [← add_assoc, add_assoc (a * a)]
    _ = a * a + 2 * (a * b) + b * b := by
      rw [mul_comm b a, ← two_mul]

```

注意，这个证明并不以 `by` 开头：以 `calc` 开头的表达式是一个*证明项*。`calc` 表达式也可以用于策略证明内部，但 Lean 会把它解释为一条指令：使用由此得到的证明项来解决当前目标。`calc` 语法比较严格：下划线和论证必须采用上面展示的格式。Lean 还会使用缩进来判断策略块或 `calc` 块从何处开始、到何处结束；可以尝试改变上面证明的缩进，看看会发生什么。

书写 `calc` 证明的一种方法，是先用 `sorry` 策略作为论证占位，勾勒出证明框架，确认 Lean 在这些占位下接受该表达式，然后再用策略逐步证明各个步骤。

```

example : (a + b) * (a + b) = a * a + 2 * (a * b) + b * b :=
  calc
    (a + b) * (a + b) = a * a + b * a + (a * b + b * b) := by
      sorry
    _ = a * a + (b * a + a * b) + b * b := by
      sorry
    _ = a * a + 2 * (a * b) + b * b := by
      sorry

```

请尝试用一个纯粹的 `rw` 证明，以及一个更结构化的 `calc` 证明，来证明下面的恒等式：

```

example : (a + b) * (c + d) = a * c + a * d + b * c + b * d := by
  sorry

```

下面的练习稍有挑战。你可以使用列在其后的定理。


```
example (a b : ℝ) : (a + b) * (a - b) = a ^ 2 - b ^ 2 := by
  sorry

#check pow_two a
#check mul_sub a b c
#check add_mul a b c
#check add_sub a b c
#check sub_sub a b c
#check add_zero a
```

我们也可以在语境中的某个假设里进行重写。例如，`rw [mul_comm a b] at hyp` 会在假设 `hyp` 中把 `a * b` 替换为 `b * a`。

```
example (a b c d : ℝ) (hyp : c = d * a + b) (hyp' : b = a * d) : c = 2 * a * d := by
  rw [hyp'] at hyp
  rw [mul_comm d a] at hyp
  rw [← two_mul (a * d)] at hyp
  rw [← mul_assoc 2 a d] at hyp
  exact hyp
```

在最后一步，`exact` 策略可以使用 `hyp` 解决目标，因为此时 `hyp` 与目标完全匹配。

本节最后指出，Mathlib 通过 `ring` 策略提供了一项有用的自动化。只要某个恒等式纯粹由环公理推出，而不使用任何局部假设，该策略就被设计用来在任意交换环中证明它。

```
example : c * b * a = b * (a * c) := by
  ring

example : (a + b) * (a + b) = a * a + 2 * (a * b) + b * b := by
  ring

example : (a + b) * (a - b) = a ^ 2 - b ^ 2 := by
  ring

example (hyp : c = d * a + b) (hyp' : b = a * d) : c = 2 * a * d := by
  rw [hyp, hyp']
  ring
```

当我们导入 `Mathlib.Data.Real.Basic` 时，`ring` 策略会被间接导入；不过下一节会看到，它也可用于实数以外的结构上的计算。它也可以通过命令 `import Mathlib.Tactic` 显式导入。我们还会看到，对于其他常见的代数结构，也存在类似的策略。

`rw` 有一个变体叫 `nth_rw`，允许你只替换目标中某个表达式的特定出现。可能的匹配从 1 开始编号；因此在下面的例子中，`nth_rw 2 [h]` 会把第二次出现的 `a + b` 替换为 `c`。

```
example (a b c : ℕ) (h : a + b = c) : (a + b) * (a + b) = a * c + b * c := by
  nth_rw 2 [h]
  rw [add_mul]
```

2.2. 证明代数结构中的恒等式

从数学上说，一个环由一族对象 R 、运算 $+$ 与 $\times$ 、常元 0 和 1 ，以及运算 $x \mapsto -x$ 组成，并满足如下条件：

R 连同 $+$ 构成一个阿贝尔群，其中 0 是加法单位元，取负给出逆元。

乘法满足结合律，以 1 为单位元，并且乘法对加法满足分配律。

在 Lean 中，这族对象表示为一个类型 `ℝ`。环公理如下：

```

variable (R : Type*) [Ring R]

#check (add_assoc : ∀ a b c : R, a + b + c = a + (b + c))
#check (add_comm : ∀ a b : R, a + b = b + a)
#check (zero_add : ∀ a : R, 0 + a = a)
#check (neg_add_cancel : ∀ a : R, -a + a = 0)
#check (mul_assoc : ∀ a b c : R, a * b * c = a * (b * c))
#check (mul_one : ∀ a : R, a * 1 = a)
#check (one_mul : ∀ a : R, 1 * a = a)
#check (mul_add : ∀ a b c : R, a * (b + c) = a * b + a * c)
#check (add_mul : ∀ a b c : R, (a + b) * c = a * c + b * c)

```

第一行中的方括号稍后还会详细解释；目前只需知道，该声明给出了一个类型 R ，以及 R 上的一个环结构。随后，Lean 允许我们对 R 的元素使用一般的环记号，并使用关于环的定理库。

有些定理的名称应该看起来很熟悉：它们正是上一节中我们用来对实数进行计算的定理。Lean 不仅适合证明关于自然数、整数等具体数学结构的事实，也适合证明关于由公理刻画的抽象结构（例如环）的事实。此外，Lean 支持对抽象结构和具体结构进行一般性推理，并且可以被训练为识别适当的实例。因此，任何关于环的定理都可以应用于整数 $\mathbb{Z}$ 、有理数 $\mathbb{Q}$ 、复数 $\mathbb{C}$ 等具体环。它也可以应用于任何扩展环的抽象结构实例，例如任意有序环或任意域。

不过，并非实数的重要性质都在任意环中成立。例如，实数乘法是交换的，但一般并非如此。如果你学过线性代数，就会知道，对每个 n ，实数上的 n 阶方阵构成一个环，而其中乘法通常不交换。若我们声明 R 是一个交换环，那么上一节中的所有定理，在把 $\mathbb{R}$ 替换为 R 后事实上仍然成立。

```

variable (R : Type*) [CommRing R]
variable (a b c d : R)

example : c * b * a = b * (a * c) := by ring

example : (a + b) * (a + b) = a * a + 2 * (a * b) + b * b := by ring

example : (a + b) * (a - b) = a ^ 2 - b ^ 2 := by ring

example (hyp : c = d * a + b) (hyp' : b = a * d) : c = 2 * a * d := by
  rw [hyp, hyp']
  ring

```

我们把检查其他所有证明是否原样通过留给读者。注意，当证明很短，例如 `by ring`、`by linarith` 或 `by sorry` 时，通常也允许把它和 `by` 写在同一行。良好的证明书写风格应当在简洁性与可读性之间取得平衡。

本节的目标是强化你在上一节形成的技能，并把这些技能应用于关于环的公理化推理。我们将从上面列出的公理出发，用它们推出其他事实。我们证明的大多数事实已经存在于 Mathlib 中。为了帮助你熟悉库的内容以及命名约定，我们会给自己证明的版本使用相同名称。

Lean 提供了一种类似程序语言中的组织机制：当一个定义或定理 `foo` 在命名空间 `bar` 中引入时，它的全名是 `bar.foo`。之后命令 `open bar` 会打开该命名空间，使我们可以使用较短的名称 `foo`。为了避免名称冲突导致错误，在下一个例子中，我们把自己版本的库定理放进一个名为 `MyRing.` 的新命名空间。

下一个例子表明，我们不必把 `add_zero` 或 `add_neg_cancel` 作为环公理，因为它们可以由其他公理推出。

```

namespace MyRing
variable {R : Type*} [Ring R]

theorem add_zero (a : R) : a + 0 = a := by rw [add_comm, zero_add]

theorem add_neg_cancel (a : R) : a + -a = 0 := by rw [add_comm, neg_add_cancel]

#check MyRing.add_zero
#check add_zero

end MyRing

```

其净效果是：我们可以暂时重新证明库中的一个定理，随后继续使用库中的版本。不过不要作弊！在接下来的练习中，请注意只使用本节前面已经证明的一般环论事实。

（如果你非常仔细，可能已经注意到我们把 `(R : Type*)` 中的圆括号改成了 `{R : Type*}` 中的花括号。这把 `R` 声明为一个隐式参数。我们马上会解释这意味着什么；暂时不必担心。）

下面是一个有用的定理：

```

theorem neg_add_cancel_left (a b : R) : -a + (a + b) = b := by
  rw [← add_assoc, neg_add_cancel, zero_add]

```

请证明对应的伴随版本：

```

theorem add_neg_cancel_right (a b : R) : a + b + -b = a := by
  sorry

```

用这些定理证明如下命题：

```

theorem add_left_cancel {a b c : R} (h : a + b = a + c) : b = c := by
  sorry

theorem add_right_cancel {a b c : R} (h : a + b = c + b) : a = c := by
  sorry

```

只要事先规划得当，每一个命题都可以用三次重写完成。

现在我们解释花括号的用法。设想你处在如下情形：语境中有 `a`、`b` 和 `c`，还有假设 `h : a + b = a + c`，而你想推出结论 `b = c`。在 Lean 中，定理可以像应用到对象那样应用到假设和事实上，所以你可能会以为 `add_left_cancel a b c h` 是事实 `b = c` 的一个证明。但请注意，显式写出 `a`、`b` 和 `c` 是冗余的，因为假设 `h` 已经清楚表明了我们所考虑的对象。在这个例子中，多敲几个字符并不麻烦；但如果要把 `add_left_cancel` 应用于更复杂的表达式，写出这些参数就会很繁琐。在这类情况下，Lean 允许我们把参数标记为隐式，意思是这些参数应当省略，并由其他方式推断出来，例如由后续参数和假设推断。`{a b c : R}` 中的花括号正是这个作用。因此，在给定上述定理陈述后，正确的表达式只是

```
add_left_cancel h。
```

作为说明，我们来证明 `a * 0 = 0` 可由环公理推出。

```

theorem mul_zero (a : R) : a * 0 = 0 := by
  have h : a * 0 + a * 0 = a * 0 + 0 := by
    rw [← mul_add, add_zero, add_zero]
  rw [add_left_cancel h]

```

我们使用了一个新技巧！如果逐步查看这个证明，就能看出发生了什么。`have` 策略引入一个新目标

`a * 0 + a * 0 = a * 0 + 0`，其语境与原目标相同。下一行缩进这一事实表明，Lean 正期待一个用于证明该新目标的策略块。因此，缩进促成了一种模块化的证明风格：被缩进的子证明建立由 `have` 引入的目标。之后，我们回到原目标，只是语境中多了一个新假设 `h`：既然已经证明了它，我们便可以使用它。此时，目标恰好就是 `add_left_cancel h` 的结果。

我们同样可以用 `apply add_left_cancel h` 或 `exact add_left_cancel h` 结束证明。`exact` 策略的参数是一个能完全证明当前目标的证明项，并且不会产生任何新目标。`apply` 策略则是一个变体，它的参数不一定是完整证明；缺失的部分要么由 Lean 自动推断，要么成为新的待证明目标。虽然从技术上说 `exact` 是冗余的，因为它严格弱于 `apply`，但它能使证明脚本对人类读者稍微更清晰，也更便于在库演化时维护。

请记住，这里并未假定乘法交换，因此下面的定理同样需要一些工作。

```
theorem zero_mul (a : R) : 0 * a = 0 := by
  sorry
```

到现在为止，你也应当能够在下一个练习中把每个 `sorry` 替换为一个证明；仍然只使用本节已经建立的关于环的事实，以及公理 `eq_symm`。

```
theorem neg_eq_of_add_eq_zero {a b : R} (h : a + b = 0) : -a = b := by
  sorry

theorem eq_neg_of_add_eq_zero {a b : R} (h : a + b = 0) : a = -b := by
  sorry

theorem neg_zero : (-0 : R) = 0 := by
  apply neg_eq_of_add_eq_zero
  rw [add_zero]

theorem neg_neg (a : R) : - -a = a := by
  sorry
```

在第三个定理中，我们不得不使用注解 `(-0 : R)` 而不是 `0`，因为如果不指定 `R`，Lean 就无法推断我们指的是哪一个 `0`，并且默认会把它解释为自然数。

在 Lean 中，环中的减法可证明地等于加上加法逆元。

```
example (a b : R) : a - b = a + -b :=
  sub_eq_add_neg a b
```

在实数上，减法正是以这种方式定义的：

```
example (a b : ℝ) : a - b = a + -b :=
  rfl

example (a b : ℝ) : a - b = a + -b := by
  rfl
```

证明项 `rfl` 是“reflexivity”（自反性）的缩写。把它作为 `a - b = a + -b` 的证明，会迫使 Lean 展开定义，并识别等式两端其实相同。`rfl` 策略做的是同一件事。这是 Lean 底层逻辑中所谓定义相等的一个例子。这意味着，不仅可以用 `sub_eq_add_neg` 进行重写，把 `a - b` 替换为 `a + -b`；在某些处理实数的语境中，还可以互换使用等式两端。例如，你现在已经有足够信息证明上一节中的定理 `self_sub`：

```
theorem self_sub (a : R) : a - a = 0 := by
  sorry
```

请证明你可以使用 `rw` 完成这一点；但如果把任意环 `R` 替换为实数，也可以使用 `apply` 或 `exact` 来证明。

Lean 知道 `1 + 1 = 2` 在任意环中都成立。稍加努力，你可以用它证明上一节中的定理 `two_mul`：

```
theorem one_add_one_eq_two : 1 + 1 = (2 : R) := by
  norm_num

theorem two_mul (a : R) : 2 * a = a + a := by
  sorry
```

本节最后指出，我们上面建立的某些关于加法和负元的事实，并不需要完整的环公理，甚至不需要加法交换。较弱的概念群可以公理化如下：

```
variable (A : Type*) [AddGroup A]

#check (add_assoc : ∀ a b c : A, a + b + c = a + (b + c))
#check (zero_add : ∀ a : A, 0 + a = a)
#check (neg_add_cancel : ∀ a : A, -a + a = 0)
```

约定上，当群运算交换时使用加法记号，否则使用乘法记号。因此，Lean 在加法版本之外也定义了乘法版本（并且还定义了它们的阿贝尔变体 `AddCommGroup` 和 `CommGroup`）。

```
variable {G : Type*} [Group G]

#check (mul_assoc : ∀ a b c : G, a * b * c = a * (b * c))
#check (one_mul : ∀ a : G, 1 * a = a)
#check (inv_mul_cancel : ∀ a : G, a⁻¹ * a = 1)
```

如果你信心十足，可以尝试只使用这些公理证明下面关于群的事实。在此过程中，你需要证明若干辅助引理。本节已经完成的证明可以提供一些提示。

```
theorem mul_inv_cancel (a : G) : a * a⁻¹ = 1 := by
  sorry

theorem mul_one (a : G) : a * 1 = a := by
  sorry

theorem mul_inv_rev (a b : G) : (a * b)⁻¹ = b⁻¹ * a⁻¹ := by
  sorry
```

显式调用这些引理很繁琐，因此 Mathlib 提供了若干类似 *ring* 的策略来覆盖大多数用法：*group* 用于非交换乘法群，*abel* 用于阿贝尔加法群，*noncomm_ring* 用于非交换环。代数结构称为 *Ring* 和 *CommRing*，而策略却命名为 *noncomm_ring* 和 *ring*，这看起来或许有些奇怪。这部分出于历史原因，也部分是为了便利：处理交换环的策略使用更频繁，因此给它一个更短的名称。

2.3. 使用定理和引理

重写非常适合用来证明等式；但其他类型的定理又该如何处理？例如，若 $b \leq c$ ，如何证明诸如 $a + e^b \leq a + e^c$ 这样的不等式？我们已经看到，定理可以应用于参数和假设，而 `apply` 与 `exact` 策略可以用来解决目标。本节将充分利用这些工具。

考虑库中的定理 `le_refl` 和 `le_trans`：

```
#check (le_refl : ∀ a : ℝ, a ≤ a)
#check (le_trans : a ≤ b → b ≤ c → a ≤ c)
```

正如我们将在第 3.1 节中更详细说明的，`le_trans` 陈述中的隐含括号向右结合，因此它应解释为

`a ≤ b → (b ≤ c → a ≤ c)`。库的设计者把 `le_trans` 的参数 `a`、`b` 和 `c` 设为隐式参数，所以 Lean 不会允许你显式提供它们（除非你坚持如此；我们稍后会讨论这一点）。相反，Lean 期望从使用它们的语境中推断这些参数。例如，当语境中有假设 `h : a ≤ b` 和 `h' : b ≤ c` 时，下列写法都可行：

```
variable (h : a ≤ b) (h' : b ≤ c)

#check (le_refl : ∀ a : Real, a ≤ a)
#check (le_refl a : a ≤ a)
#check (le_trans : a ≤ b → b ≤ c → a ≤ c)
#check (le_trans h : b ≤ c → a ≤ c)
#check (le_trans h h' : a ≤ c)
```

`apply` 策略接受一个一般命题或蕴含命题的证明，尝试把它的结论与当前目标匹配，并把其中的假设（若有）留下来作为新的目标。如果给出的证明与目标完全匹配（在定义相等意义下），则可以用 `exact` 策略代替 `apply`。因此，下面这些写法都成立：

```
example (x y z : ℝ) (h₀ : x ≤ y) (h₁ : y ≤ z) : x ≤ z := by
  apply le_trans
  · apply h₀
  · apply h₁

example (x y z : ℝ) (h₀ : x ≤ y) (h₁ : y ≤ z) : x ≤ z := by
  apply le_trans h₀
  apply h₁

example (x y z : ℝ) (h₀ : x ≤ y) (h₁ : y ≤ z) : x ≤ z :=
  le_trans h₀ h₁

example (x : ℝ) : x ≤ x := by
  apply le_refl

example (x : ℝ) : x ≤ x :=
  le_refl x
```

在第一个例子中，应用 `le_trans` 会产生两个目标，我们用圆点标明每个目标的证明从何处开始。圆点不是必需的，但它们可以聚焦目标：在由圆点引入的代码块中，只显示一个目标，并且必须在该代码块结束前完成它。这里我们通过另起一个圆点块来结束第一个代码块；同样也可以通过减少缩进来达到这一效果。在第三个例子和最后一个例子中，我们完全避免进入策略模式：`le_trans h₀ h₁` 与 `le_refl x` 正是所需的证明项。

下面还有几个库定理：

```
#check (le_refl : ∀ a, a ≤ a)
#check (le_trans : a ≤ b → b ≤ c → a ≤ c)
#check (lt_of_le_of_lt : a ≤ b → b < c → a < c)
#check (lt_of_lt_of_le : a < b → b ≤ c → a < c)
#check (lt_trans : a < b → b < c → a < c)
```

请把它们与 `apply` 和 `exact` 结合起来，证明如下命题：

```
example (h₀ : a ≤ b) (h₁ : b < c) (h₂ : c ≤ d) (h₃ : d < e) : a < e := by
  sorry
```

事实上，Lean 有一个策略可以自动处理这类事情：

```
example (h₀ : a ≤ b) (h₁ : b < c) (h₂ : c ≤ d) (h₃ : d < e) : a < e := by
  linarith
```

`linarith` 策略专门用于处理线性算术。

```
example (h : 2 * a ≤ 3 * b) (h' : 1 ≤ a) (h'' : d = 2) : d + a ≤ 5 * b := by
  linarith
```

除了语境中的等式和不等式之外，`linarith` 还会使用你作为参数传入的额外不等式。在下一个例子中，`exp_le_exp.mpr h'` 是 `exp b ≤ exp c` 的一个证明，我们稍后会解释这一点。注意，在 Lean 中，我们写 `f x` 来表示函数 `f` 作用于参数 `x`，这与写 `h x` 表示事实或定理 `h` 作用于参数 `x` 所得结果完全相同。只有复合参数才需要括号，例如 `f (x + y)`。若没有括号，`f x + y` 会被解析为 `(f x) + y`。

```
example (h : 1 ≤ a) (h' : b ≤ c) : 2 + a + exp b ≤ 3 * a + exp c := by
  linarith [exp_le_exp.mpr h']
```

下面是库中另外一些可用于建立实数不等式的定理。

```
#check (exp_le_exp : exp a ≤ exp b ↔ a ≤ b)
#check (exp_lt_exp : exp a < exp b ↔ a < b)
#check (log_le_log : 0 < a → a ≤ b → log a ≤ log b)
#check (log_lt_log : 0 < a → a < b → log a < log b)
#check (add_le_add : a ≤ b → c ≤ d → a + c ≤ b + d)
#check (add_le_add_right : a ≤ b → ∀ c, c + a ≤ c + b)
#check (add_le_add_left : a ≤ b → ∀ c, a + c ≤ b + c)
#check (add_lt_add_of_le_of_lt : a ≤ b → c < d → a + c < b + d)
#check (add_lt_add_of_lt_of_le : a < b → c ≤ d → a + c < b + d)
#check (add_lt_add_right : a < b → ∀ c, c + a < c + b)
#check (add_lt_add_left : a < b → ∀ c, a + c < b + c)
#check (add_nonneg : 0 ≤ a → 0 ≤ b → 0 ≤ a + b)
#check (add_pos : 0 < a → 0 < b → 0 < a + b)
#check (add_pos_of_pos_of_nonneg : 0 < a → 0 ≤ b → 0 < a + b)
#check (exp_pos : ∀ a, 0 < exp a)
#check add_le_add_right
```

其中一些定理，如 `exp_le_exp` 和 `exp_lt_exp`，使用双向蕴含，它表示“当且仅当”。（在 VS Code 中可以用 `\lr` 或 `\iff` 输入。）我们将在下一章更详细讨论这个连接词。这类定理可以配合 `rw` 使用，把目标重写成一个等价目标：

```
example (h : a ≤ b) : exp a ≤ exp b := by
  rw [exp_le_exp]
  exact h
```

不过在本节中，我们将使用如下事实：若 `h : A ↔ B` 是这样的等价命题，则 `h.mp` 给出正向方向 `A → B`，而 `h.mpr` 给出反向方向 `B → A`。这里 `mp` 表示“modus ponens”，`mpr` 表示“modus ponens reverse”。如果愿意，也可以分别用 `h.1` 和 `h.2` 代替 `h.mp` 与 `h.mpr`。因此，下面的证明成立：

```
example (h₀ : a ≤ b) (h₁ : c < d) : a + exp c + e < b + exp d + e := by
  apply add_lt_add_of_lt_of_le
  · apply add_lt_add_of_le_of_lt h₀
    apply exp_lt_exp.mpr h₁
  apply le_refl
```

第一行 `apply add_lt_add_of_lt_of_le` 产生两个目标，我们再次用圆点把第一个目标的证明与第二个目标的证明分开。

请自行尝试下面的例子。中间的例子表明，`norm_num` 策略可以用来解决具体的数值目标。

```
example (h₀ : d ≤ e) : c + exp (a + d) ≤ c + exp (a + e) := by sorry

example : (0 : ℝ) < 1 := by norm_num

example (h : a ≤ b) : log (1 + exp a) ≤ log (1 + exp b) := by
  have h₀ : 0 < 1 + exp a := by sorry
  apply log_le_log h₀
  sorry
```

从这些例子应当可以看出，能够找到所需的库定理，是形式化工作的重要组成部分。你可以使用若干策略：

可以在 Mathlib 的 GitHub 仓库中浏览 Mathlib。

可以使用 Mathlib 网页上的 API 文档。

可以使用 Loogle 通过模式搜索 Lean 与 Mathlib 中的定义和定理。

可以依靠 Mathlib 的命名约定以及编辑器中的 Ctrl-space 补全（Mac 键盘上为 Cmd-space）来猜测定理名称。在 Lean 中，名为 `A_of_B_of_C` 的定理，通常从形如 `B` 和 `C` 的假设推出形如 `A` 的结论，其中 `A`、`B` 和 `C` 近似反映我们朗读目标的方式。因此，一个证明类似 `x + y ≤ ...` 的定理很可能以 `add_le` 开头。输入 `add_le` 并按 Ctrl-space，会出现一些有用的候选。注意，连续按两次 Ctrl-space 会显示关于可用补全的更多信息。

如果在 VS Code 中右击一个已有定理名，编辑器会显示一个菜单，其中有跳转到该定理定义所在文件的选项；你也可以在附近找到类似定理。

可以使用 `apply?` 策略，它会尝试在库中寻找相关定理。

```
example : 0 ≤ a ^ 2 := by
  -- apply?
  exact sq_nonneg a
```

若要在例子中尝试 `apply?`，请删除 `exact` 命令，并取消前一行注释。借助这些技巧，看看能否找到完成下一个例子所需的内容：

```
example (h : a ≤ b) : c - exp b ≤ c - exp a := by
  sorry
```

使用同样的技巧，确认用 `linarith` 代替 `apply?` 也可以完成证明。

下面是另一个不等式的例子：

```
example : 2*a*b ≤ a^2 + b^2 := by
  have h : 0 ≤ a^2 - 2*a*b + b^2
  calc
    a^2 - 2*a*b + b^2 = (a - b)^2 := by ring
    _ ≥ 0 := by apply pow_two_nonneg

  calc
    2*a*b = 2*a*b + 0 := by ring
    _ ≤ 2*a*b + (a^2 - 2*a*b + b^2) := add_le_add (le_refl _) h
    _ = a^2 + b^2 := by ring
```

Mathlib 通常会在 `*` 和 `^` 这样的二元运算符两侧加入空格；但在这个例子中，更紧凑的格式反而提高了可读性。这里有几件事情值得注意。第一，表达式 `s ≥ t` 与 `t ≤ s` 定义相等。原则上，这意味着二者应当可以互换使用。但 Lean 的某些自动化不能识别这种等价，因此 Mathlib 往往偏好使用 `≤` 而不是 `≥`。第二，我们大量使用了 `ring` 策略。它确实节省时间！最后，注意在第二个 `calc` 证明的第二行中，不必写 `by exact add_le_add (le_refl _) h`，可以直接写证明项 `add_le_add (le_refl _) h`。

事实上，上述证明中唯一需要一点技巧的地方，是找出假设 `h`。一旦有了它，第二个计算只涉及线性算术，`linarith` 可以处理：

```
example : 2*a*b ≤ a^2 + b^2 := by
  have h : 0 ≤ a^2 - 2*a*b + b^2
  calc
    a^2 - 2*a*b + b^2 = (a - b)^2 := by ring
    _ ≥ 0 := by apply pow_two_nonneg
  linarith
```

很好！我们邀请你用这些想法证明下面的定理。可以使用定理 `abs_le'.mpr`。还需要使用 `constructor` 策略把合取拆分成两个目标；参见第 3.4 节。

```
example : |a*b| ≤ (a^2 + b^2)/2 := by
  sorry

#check abs_le'.mpr
```

如果你成功解决了它，恭喜！你已经很好地走在成为形式化大师的路上。

2.4. 使用 `apply` 和 `rw` 的更多例子

实数上的 `min` 函数由以下三个事实唯一刻画：

```
#check (min_le_left a b : min a b ≤ a)
#check (min_le_right a b : min a b ≤ b)
#check (le_min : c ≤ a → c ≤ b → c ≤ min a b)
```

你能猜出以类似方式刻画 `max` 的那些定理叫什么名字吗？

注意，我们必须通过写 `min a b` 而不是 `min (a, b)`，把 `min` 应用于一对参数 `a` 和 `b`。形式上，`min` 的类型是 $\mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R}$ 。当我们写出带有多个箭头的类型时，约定是隐含括号向右结合，因此该类型解释为 $\mathbb{R} \rightarrow (\mathbb{R} \rightarrow \mathbb{R})$ 。其效果是：若 `a` 和 `b` 的类型都是 $\mathbb{R}$ ，则 `min a` 的类型是 $\mathbb{R} \rightarrow \mathbb{R}$ ，而 `min a b` 的类型是 $\mathbb{R}$ ，所以 `min` 如预期那样表现为一个二元函数。以这种方式处理多个参数称为*柯里化*，得名于逻辑学家 Haskell Curry。

Lean 中运算的优先级也需要一些时间适应。函数应用比中缀运算结合得更紧，因此表达式 `min a b + c` 会被解释为 `(min a b) + c`。随着练习增加，这些约定会逐渐变成自然习惯。

使用定理 `le_antisymm`，如果两个实数彼此小于等于对方，就可以证明它们相等。借助这一点和上述事实，我们可以证明 `min` 是交换的：

```
example : min a b = min b a := by
  apply le_antisymm
  · show min a b ≤ min b a
    apply le_min
    · apply min_le_right
    apply min_le_left
  · show min b a ≤ min a b
    apply le_min
    · apply min_le_right
    apply min_le_left
```

这里我们使用圆点来分隔不同目标的证明。我们的用法并不完全一致：在外层，我们对两个目标都使用圆点和缩进；而在嵌套证明中，只在还剩多个目标时使用圆点。两种约定都是合理且有用的。我们还使用 `show` 策略来组织证明，并指明每个代码块正在证明什么。即使没有这些 `show` 命令，证明仍然成立；但使用它们会使证明更易阅读和维护。

这个证明可能显得重复。为了预示稍后将学习的技巧，我们指出避免重复的一种方法是先陈述一个局部引理，然后使用它：

```
example : min a b = min b a := by
  have h : ∀ x y : ℝ, min x y ≤ min y x := by
    intro x y
    apply le_min
    apply min_le_right
    apply min_le_left
  apply le_antisymm
  apply h
  apply h
```

我们将在第 3.1 节中更多讨论全称量词；这里暂且只需知道，假设 `h` 表示对任意 `x` 和 `y`，所需不等式都成立，而 `intro` 策略引入任意的 `x` 和 `y` 来建立结论。`le_antisymm` 之后的第一个 `apply` 隐式使用 `h a b`，第二个则使用 `h b a`。

另一个解法是使用 `repeat` 策略；它会尽可能多次地应用某个策略（或策略块）。

```
example : min a b = min b a := by
  apply le_antisymm
  repeat
    apply le_min
    apply min_le_right
    apply min_le_left
```

我们鼓励你把下面这些命题作为练习来证明。第一个可以用刚才描述的任一技巧来缩短证明。

```
example : max a b = max b a := by
  sorry
example : min (min a b) c = min a (min b c) := by
  sorry
```

当然，你也可以证明 `max` 的结合律。

一个有趣的事实是，`min` 对 `max` 的分配方式类似于乘法对加法的分配，反之亦然。换言之，在实数上，我们有恒等式 `min a (max b c) = max (min a b) (min a c)`，也有把 `max` 与 `min` 对调后的对应版本。但下一节会看到，这些结论并不只由 `≤` 的传递性、自反性以及上面列出的 `min` 和 `max` 的刻画性质推出。我们还需要使用实数上的 `≤` 是全序这一事实，也就是说，它满足 `∀ x y, x ≤ y ∨ y ≤ x`。这里的析取符号 `∨` 表示“或”。在第一种情形中有 `min x y = x`，在第二种情形中有 `min x y = y`。我们将在第 3.5 节中学习如何按情形推理；但目前我们只处理不需要拆分情形的例子。

下面就是这样的例子：

```
theorem aux : min a b + c ≤ min (a + c) (b + c) := by
  sorry
example : min a b + c = min (a + c) (b + c) := by
  sorry
```

显然，`aux` 提供了证明等式所需的两个不等式之一；但把它应用于合适的值，也能得到另一个方向。作为提示，可以使用定理 `add_neg_cancel_right` 和策略 `linarith`。

库中三角不等式的名称清楚体现了 Lean 的命名约定：

```
#check (abs_add_le : ∀ a b : ℝ, |a + b| ≤ |a| + |b|)
```

请使用它证明下面的变体，同时也使用 `add_sub_cancel_right`：

```
example : |a| - |b| ≤ |a - b| :=
  sorry
end
```

看看你能否在三行以内完成。可以使用定理 `sub_add_cancel`。

我们在后续各节还会使用的另一个重要关系，是自然数上的整除关系 `x | y`。请注意，整除符号不是键盘上的普通竖线；它是一个 Unicode 字符，在 VS Code 中可通过输入 `\|` 获得。按约定，Mathlib 在定理名中用 `dvd` 表示它。

```
example (h₀ : x | y) (h₁ : y | z) : x | z :=
  dvd_trans h₀ h₁

example : x | y * x * z := by
  apply dvd_mul_of_dvd_left
  apply dvd_mul_left

example : x | x ^ 2 := by
  apply dvd_mul_left
```

在最后一个例子中，指数是自然数，应用 `dvd_mul_left` 会迫使 Lean 把 `x^2` 的定义展开为 `x^1 * x`。看看你能否猜出证明下面命题所需的定理名称：

```
example (h : x | w) : x | y * (x * z) + x ^ 2 + w ^ 2 := by
  sorry
end
```

就整除关系而言，最大公因数 `gcd` 和最小公倍数 `lcm` 类似于 `min` 和 `max`。由于每个数都整除 `0`，因此 `0` 在整除意义下实际上是最大元素：

```
variable (m n : ℕ)

#check (Nat.gcd_zero_right n : Nat.gcd n 0 = n)
#check (Nat.gcd_zero_left n : Nat.gcd 0 n = n)
#check (Nat.lcm_zero_right n : Nat.lcm n 0 = 0)
#check (Nat.lcm_zero_left n : Nat.lcm 0 n = 0)
```

看看你能否猜出证明下面命题所需的定理名称：

```
example : Nat.gcd m n = Nat.gcd n m := by
  sorry
```

提示：可以使用 `dvd_antisymm`；但若这样做，Lean 会抱怨该表达式在一般定理和专门用于自然数的版本 `Nat.dvd_antisymm` 之间有歧义。可以用 `_root_.dvd_antisymm` 指明一般版本；两者都能工作。

2.5. 证明关于代数结构的事实

在第 2.2 节中，我们看到支配实数的许多常见恒等式，也适用于更一般的代数结构类，例如交换环。描述代数结构时，我们可以使用任意想要的公理，而不只限于等式。例如，一个偏序由一个集合及其上的一个二元关系组成，该关系满足自反性、传递性和反对称性，就像实数上的 $\leq$ 一样。Lean 知道偏序：

```
variable {α : Type*} [PartialOrder α]
variable (x y z : α)

#check x ≤ y
#check (le_refl x : x ≤ x)
#check (le_trans : x ≤ y → y ≤ z → x ≤ z)
#check (le_antisymm : x ≤ y → y ≤ x → x = y)
```

这里我们采用 Mathlib 的约定，用 α 、 β 和 γ （分别输入为 `\a`、`\b` 和 `\g`）表示任意类型。库中常用 $\mathbb{R}$ 和 $\mathbb{G}$ 这样的字母分别表示环、群等代数结构的载体；但一般来说，类型通常用希腊字母表示，尤其是在它们几乎没有附加结构时。

任一偏序 $\leq$ 还伴随一个严格偏序 $<$ ，其作用有些类似于实数上的 $<$ 。说 x 在该序下小于 y ，等价于说它小于等于 y 且不等于 y 。

```
#check x < y
#check (lt_irrefl x : ¬ (x < x))
#check (lt_trans : x < y → y < z → x < z)
#check (lt_of_le_of_lt : x ≤ y → y < z → x < z)
#check (lt_of_lt_of_le : x < y → y ≤ z → x < z)

example : x < y ↔ x ≤ y ∧ x ≠ y :=
  lt_iff_le_and_ne
```

在这个例子中，符号 $\wedge$ 表示“且”，符号 $\neg$ 表示“非”，而 $x \neq y$ 是 $\neg (x = y)$ 的缩写。在第 3 章中，你将学习如何使用这些逻辑连接词证明 $<$ 具有上述性质。

格是一种结构，它在偏序上增加两个运算 $\sqcap$ 和 $\sqcup$ ，分别类似于实数上的 `min` 和 `max`：

```
variable {α : Type*} [Lattice α]
variable (x y z : α)

#check x ⊓ y
#check (inf_le_left : x ⊓ y ≤ x)
#check (inf_le_right : x ⊓ y ≤ y)
#check (le_inf : z ≤ x → z ≤ y → z ≤ x ⊓ y)
#check x ⊔ y
#check (le_sup_left : x ≤ x ⊔ y)
#check (le_sup_right : y ≤ x ⊔ y)
#check (sup_le : x ≤ z → y ≤ z → x ⊔ y ≤ z)
```

$\sqcap$ 和 $\sqcup$ 的刻画分别说明它们可称为最大下界和最小上界。在 VS Code 中，可以用 `\glb` 和 `\lub` 输入这些符号。它们也常称为下确界和上确界，Mathlib 在定理名中用 `inf` 和 `sup` 指称它们。更复杂的是，它们还常称为交和并。因此，如果处理格，就需要记住如下术语对应表：

$\sqcap$ 是最大下界、下确界，或交。

$\sqcup$ 是最小上界、上确界，或并。

格的一些实例包括：

任意全序上的 `min` 和 `max`，例如整数或实数上的 `≤`

某个论域的子集族上的 $\sqcap$ 和 $\sqcup$ ，其序关系为 $\subseteq$

布尔真值上的 $\wedge$ 和 $\vee$ ，其序关系 $x \leq y$ 表示 x 为假或 y 为真

自然数（或正自然数）上的 `gcd` 和 `lcm`，其序关系是整除关系 $\mid$

向量空间的线性子空间族，其中最大下界由交给出，最小上界由两个空间的和给出，序关系为包含关系

一个集合（或在 Lean 中，一个类型）上的拓扑族，其中两个拓扑的最大下界是由它们的并生成的拓扑，最小上界是它们的交，序关系为反向包含

你可以验证，正如 `min` / `max` 以及 `gcd` / `lcm` 那样，仅凭下确界和上确界的刻画公理，再加上 `le_refl` 与 `le_trans`，即可证明它们的交换律和结合律。

当目标为 $x \leq z$ 时使用 `apply le_trans` 并不是好主意。实际上，Lean 无法猜出我们想使用哪个中间元素 y 。因此，`apply le_trans` 会产生三个形如 $x \leq ?a$ 、 $?a \leq z$ 和 α 的目标，其中 $?a$ （可能有一个更复杂的自动生成名称）代表神秘的 y 。最后一个类型为 α 的目标，是要求给出 y 的值。它排在最后，是因为 Lean 希望能从第一个目标 $x \leq ?a$ 的证明中自动推断它。为了避免这种不便，可以使用 `calc` 策略显式给出 y 。或者，也可以使用 `trans` 策略；它以 y 为参数，并产生预期的目标 $x \leq y$ 和 $y \leq z$ 。当然，也可以直接给出完整证明来避免这个问题，例如 `exact le_trans inf_le_left inf_le_right`，但这需要更多预先规划。

```
example : x ⊓ y = y ⊓ x := by
  sorry

example : x ⊓ y ⊓ z = x ⊓ (y ⊓ z) := by
  sorry

example : x ⊔ y = y ⊔ x := by
  sorry

example : x ⊔ y ⊔ z = x ⊔ (y ⊔ z) := by
  sorry
```

这些定理在 Mathlib 中分别名为 `inf_comm`、`inf_assoc`、`sup_comm` 和 `sup_assoc`。

另一个很好的练习是仅用这些公理证明吸收律：

```
theorem absorb1 : x ⊓ (x ⊔ y) = x := by
  sorry

theorem absorb2 : x ⊔ x ⊓ y = x := by
  sorry
```

这些定理在 Mathlib 中可分别以名称 `inf_sup_self` 和 `sup_inf_self` 找到。

若一个格还满足额外恒等式 $x \sqcap (y \sqcup z) = (x \sqcap y) \sqcup (x \sqcap z)$ 和 $x \sqcup (y \sqcap z) = (x \sqcup y) \sqcap (x \sqcup z)$ ，则称为分配格。Lean 也知道这些结构：

```
variable {α : Type*} [DistribLattice α]
variable (x y z : α)

#check (inf_sup_left x y z : x ⊓ (y ⊔ z) = x ⊓ y ⊔ x ⊓ z)
#check (inf_sup_right x y z : (x ⊔ y) ⊓ z = x ⊓ z ⊔ y ⊓ z)
#check (sup_inf_left x y z : x ⊔ y ⊓ z = (x ⊔ y) ⊓ (x ⊔ z))
#check (sup_inf_right x y z : x ⊓ y ⊔ z = (x ⊔ z) ⊓ (y ⊔ z))
```

在 $\sqcap$ 和 $\sqcup$ 交换律成立的前提下，左右两个版本很容易证明等价。给出一个有限元素的非分配格的显式描述，从而说明并非每个格都是分配格，是一个很好的练习。另一个好练习是证明：在任意格中，任一条分配律都蕴含另一条分配律：

```
variable {α : Type*} [Lattice α]
variable (a b c : α)

example (h : ∀ x y z : α, x ⊓ (y ⊔ z) = x ⊓ y ⊔ x ⊓ z) : a ⊔ b ⊓ c = (a ⊔ b) ⊓ (a ⊔ c) := by
  sorry

example (h : ∀ x y z : α, x ⊔ y ⊓ z = (x ⊔ y) ⊓ (x ⊔ z)) : a ⊓ (b ⊔ c) = a ⊓ b ⊔ a ⊓ c := by
  sorry
```

可以把公理化结构组合成更大的结构。例如，严格有序环由一个环以及其载体上的一个偏序组成，并满足额外公理，表明环运算与该序相容：

```
variable {R : Type*} [Ring R] [PartialOrder R] [IsStrictOrderedRing R]
variable (a b c : R)

#check (add_le_add_right : a ≤ b → ∀ c, c + a ≤ c + b)
#check (mul_pos : 0 < a → 0 < b → 0 < a * b)
```

第 3 章 将提供从 `mul_pos` 和 `<` 的定义推出下列结论的方法：

```
#check (mul_nonneg : 0 ≤ a → 0 ≤ b → 0 ≤ a * b)
```

随后可以把它作为一个较长练习来证明：许多用于实数算术和实数序关系推理的常见事实，对任意有序环也一般成立。下面有几个例子可供尝试，只使用环、偏序的性质，以及最后两个例子中列出的事实（注意，这些环并未假定为交换环，因此不能使用 *ring* 策略）：

```
example (h : a ≤ b) : 0 ≤ b - a := by
  sorry

example (h : 0 ≤ b - a) : a ≤ b := by
  sorry

example (h : a ≤ b) (h' : 0 ≤ c) : a * c ≤ b * c := by
  sorry
```

最后，再看一个例子。度量空间由一个集合以及一个距离概念 `dist x y` 组成，后者把任意一对元素映射到一个实数。距离函数假定满足如下公理：

```
variable {X : Type*} [MetricSpace X]
variable (x y z : X)

#check (dist_self x : dist x x = 0)
#check (dist_comm x y : dist x y = dist y x)
#check (dist_triangle x y z : dist x z ≤ dist x y + dist y z)
```

掌握本节内容之后，你可以证明这些公理推出距离总是非负的：

```
example (x y : X) : 0 ≤ dist x y := by
  sorry
```

如你可能已经猜到的，这个定理在 Mathlib 中名为 `dist_nonneg`。

3. 逻辑

上一章中，我们处理了等式、不等式，以及诸如“ x 整除 y ”这样的基本数学命题。复杂的数学命题由这类简单命题借助逻辑术语构成，例如“且”“或”“非”，以及“若.....则.....”“每个”和“存在某个”。本章将说明如何处理以这种方式构成的命题。

3.1. 蕴含与全称量词

考虑 `#check` 之后的命题：

```
#check ∀ x : ℝ, 0 ≤ x → |x| = x
```

用文字来说，我们会说：“对每个实数 x ，若 $0 \leq x$ ，则 x 的绝对值等于 x 。”我们也可以有更复杂的命题，例如：

```
#check ∀ x y ε : ℝ, 0 < ε → ε ≤ 1 → |x| < ε → |y| < ε → |x * y| < ε
```

用文字来说，我们会说：“对每个 x 、 y 和 ϵ ，若 $0 < \epsilon \leq 1$ ， x 的绝对值小于 ϵ ，且 y 的绝对值小于 ϵ ，则 $x * y$ 的绝对值小于 ϵ 。”在 Lean 中，一串蕴含中的隐含括号向右结合。因此，上面的表达式意为：“若 $0 < \epsilon$ ，则若 $\epsilon \leq 1$ ，则若 $|x| < \epsilon$ ，.....”于是，该表达式表示所有假设合在一起推出结论。

你已经看到，尽管这个命题中的全称量词遍历对象，而蕴含箭头引入假设，Lean 对二者的处理方式非常相似。特别地，如果你证明了这种形式的定理，就可以用同样方式把它应用于对象和假设。我们将以下命题作为例子；稍后会帮助你证明它：

```
theorem my_lemma : ∀ x y ε : ℝ, 0 < ε → ε ≤ 1 → |x| < ε → |y| < ε → |x * y| < ε :=
  sorry

section
variable (a b δ : ℝ)
variable (h₀ : 0 < δ) (h₁ : δ ≤ 1)
variable (ha : |a| < δ) (hb : |b| < δ)

#check my_lemma a b δ
#check my_lemma a b δ h₀ h₁
#check my_lemma a b δ h₀ h₁ ha hb

end
```

你也已经看到，在 Lean 中，当量化变量可以由后续假设推断出来时，常常使用花括号把它们设为隐式变量。这样做之后，应用引理时只需把它应用于假设，而不必提及那些对象。

```
theorem my_lemma2 : ∀ {x y ε : ℝ}, 0 < ε → ε ≤ 1 → |x| < ε → |y| < ε → |x * y| < ε :=
  sorry

section
variable (a b δ : ℝ)
variable (h₀ : 0 < δ) (h₁ : δ ≤ 1)
variable (ha : |a| < δ) (hb : |b| < δ)

#check my_lemma2 h₀ h₁ ha hb

end
```

到此为止，你也知道：若使用 `apply` 策略把 `my_lemma` 应用于形如 $|a * b| < \delta$ 的目标，就会留下若干新目标，要求你证明每一个假设。

要证明这样的命题，可以使用 `intro` 策略。请看它在这个例子中的作用：

```
theorem my_lemma3 :
  ∀ {x y ε : ℝ}, 0 < ε → ε ≤ 1 → |x| < ε → |y| < ε → |x * y| < ε := by
  intro x y ε epos ele1 xlt ylt
  sorry
```

对于全称量化的变量，我们可以使用任意名称；它们不必叫作 `x`、`y` 和 `ε`。注意，即使这些变量被标记为隐式，我们仍然必须引入它们：把它们设为隐式，意味着在写使用 `my_lemma` 的表达式时省略它们；但它们仍是我们正在证明的命题的本质组成部分。在 `intro` 命令之后，目标就变成了如果我们像上一节那样在冒号之前列出所有变量和假设时一开始会看到的目标。稍后我们会看到，为什么有时必须在证明开始之后再引入变量和假设。

为了帮助你证明这个引理，我们先给出开头部分：

```
theorem my_lemma4 :
  ∀ {x y ε : ℝ}, 0 < ε → ε ≤ 1 → |x| < ε → |y| < ε → |x * y| < ε := by
  intro x y ε epos ele1 xlt ylt
  calc
    |x * y| = |x| * |y| := sorry
  _ ≤ |x| * ε := sorry
  _ < 1 * ε := sorry
  _ = ε := sorry
```

请使用定理 `abs_mul`、`mul_le_mul`、`abs_nonneg`、`mul_lt_mul_of_pos_right` 和 `one_mul` 完成证明。记住，可以用 Ctrl-space 补全（Mac 上为 Cmd-space 补全）寻找这类定理。还要记住，可以使用 `.mp` 与 `.mpr`，或 `.1` 与 `.2`，从一个当且仅当命题中取出两个方向。

全称量词常常隐藏在定义之中；必要时，Lean 会展开定义以显露它们。例如，让我们定义两个谓词 `FnUb f a` 和 `FnLb f a`，其中 `f` 是从实数到实数的函数，`a` 是实数。前者表示 `a` 是 `f` 的取值的上界，后者表示 `a` 是 `f` 的取值的下界。

```
def FnUb (f : ℝ → ℝ) (a : ℝ) : Prop :=
  ∀ x, f x ≤ a

def FnLb (f : ℝ → ℝ) (a : ℝ) : Prop :=
  ∀ x, a ≤ f x
```

在下一个例子中，`fun x ↦ f x + g x` 是把 `x` 映到 `f x + g x` 的函数。在类型论中，从表达式 `f x + g x` 到这个函数的过程称为 lambda 抽象。

```
example (hfa : FnUb f a) (hgb : FnUb g b) : FnUb (fun x ↦ f x + g x) (a + b) := by
  intro x
  dsimp
  apply add_le_add
  apply hfa
  apply hgb
```

对目标 `FnUb (fun x ↦ f x + g x) (a + b)` 应用 `intro`，会迫使 Lean 展开 `FnUb` 的定义，并为全称量词引入 `x`。此时目标变为 `(fun (x : ℝ) ↦ f x + g x) x ≤ a + b`。但是，把 `(fun x ↦ f x + g x)` 应用于 `x` 应得到 `f x + g x`，而 `dsimp` 命令执行这一化简。（其中“d”表示“definitional”。）删除该命令后证明仍然成立；Lean 无论如何都必须执行这个约简，才能理解下一个 `apply`。`dsimp` 只是让目标更可读，并帮助我们判断下一步该做什么。另一种选择是使用 `change` 策略，写成 `change f x + g x ≤ a + b`。这有助于提高证明的可读性，并使你能更好地控制目标如何变形。

证明的其余部分是例行的。最后两个 `apply` 命令迫使 Lean 展开假设中 `FnUb` 的定义。请尝试以类似方式完成下面这些证明：

```

example (hfa : FnLb f a) (hgb : FnLb g b) : FnLb (fun x ↦ f x + g x) (a + b) :=
  sorry

example (nnf : FnLb f 0) (nng : FnLb g 0) : FnLb (fun x ↦ f x * g x) 0 :=
  sorry

example (hfa : FnUb f a) (hgb : FnUb g b) (nng : FnLb g 0) (nna : 0 ≤ a) :
  FnUb (fun x ↦ f x * g x) (a * b) :=
  sorry

```

尽管我们为从实数到实数的函数定义了 `FnUb` 和 `FnLb`，但你应当认识到，这些定义和证明要一般得多。只要函数的陪域带有某种序结构，这些定义就适用于任意两个类型之间的函数。检查定理 `add_le_add` 的类型会发现，它适用于任何与相关序相容的幺半群结构；这究竟是什么意思目前并不重要，但值得知道自然数、整数、有理数和实数都是其实例。因此，如果我们在这个一般性层次证明定理 `fnUb_add`，它就可用于所有这些实例。

```

variable {α : Type*} {R : Type*} [AddCommMonoid R] [PartialOrder R] [IsOrderedCancelAddMonoid R]

#check add_le_add

def FnUb' (f : α → R) (a : R) : Prop :=
  ∀ x, f x ≤ a

theorem fnUb_add {f g : α → R} {a b : R} (hfa : FnUb' f a) (hgb : FnUb' g b) :
  FnUb' (fun x ↦ f x + g x) (a + b) := fun x ↦ add_le_add (hfa x) (hgb x)

```

你已经在第 2.2 节中见过这样的方括号，尽管我们尚未解释它们的含义。为了具体起见，本书多数例子会坚持使用实数；但值得知道的是，Mathlib 包含许多在高度一般性下工作的定义和定理。

作为隐藏全称量词的另一个例子，Mathlib 定义了谓词 `Monotone`，表示函数关于其参数单调不减：

```

example (f : ℝ → ℝ) (h : Monotone f) : ∀ {a b}, a ≤ b → f a ≤ f b :=
  @h

```

性质 `Monotone f` 按定义恰好就是冒号之后的表达式。我们需要在 `h` 前写符号 `@`；否则 Lean 会展开 `h` 的隐式参数并插入占位符。

证明关于单调性的命题时，通常使用 `intro` 引入两个变量，例如 `a` 和 `b`，以及假设 `a ≤ b`。要使用单调性假设，可以把它应用于合适的参数和假设，然后把得到的表达式应用于目标。也可以把它应用于目标，让 Lean 通过把剩余假设显示为新的子目标来帮助你反向推理。

```

example (mf : Monotone f) (mg : Monotone g) : Monotone fun x ↦ f x + g x := by
  intro a b aleb
  apply add_le_add
  apply mf aleb
  apply mg aleb

```

当证明如此简短时，直接给出证明项往往很方便。为了描述一个临时引入对象 `a`、`b` 和假设 `aleb` 的证明，Lean 使用记号 `fun a b aleb ↦ ...`。这类似于表达式 `fun x ↦ x^2` 通过临时命名一个对象 `x`，然后用它描述一个值来定义函数。因此，前一个证明中的 `intro` 命令对应下一个证明项中的 lambda 抽象，而 `apply` 命令则对应构造定理对其参数的应用。

```

example (mf : Monotone f) (mg : Monotone g) : Monotone fun x ↦ f x + g x :=
  fun _a _b aleb ↦ add_le_add (mf aleb) (mg aleb)

```

这里有一个有用技巧：如果你开始写证明项 `fun a b aleb ↦ _`，用下划线表示剩余表达式应出现的位置，Lean 会标出错误，说明它无法猜出该表达式的值。若在 VS Code 中查看 Lean InfoView 窗口，或把鼠标悬停在波浪线错误标记上，Lean 会显示剩余表达式必须解决的目标。

你会注意到，Lean 会对上述证明打印警告，说 a 和 b 未在函数体中使用。这是一种通用机制，例如可用于避免定理中含有不必要的假设。这里它并没有告诉我们什么真正有用的信息。我们可以写 `set_option linter.unusedVariables false`，将这个警告禁用到当前小节结束；也可以只在该例子中禁用，在例子上方写 `set_option linter.unusedVariables false in`。

我们也可以把 a 和 b 替换为下划线，告诉 Lean 不想给它们命名。Lean 随后会使用自动生成的不可访问名称，例如 $x!$ 。另一种选择是在名称前加下划线，告诉 Lean 我们并不打算使用它们。

请尝试用策略或证明项证明下面的例子：

```
example {c : ℝ} (mf : Monotone f) (nnc : 0 ≤ c) : Monotone fun x ↦ c * f x :=
  sorry

example (mf : Monotone f) (mg : Monotone g) : Monotone fun x ↦ f (g x) :=
  sorry
```

下面还有一些例子。从 $\mathbb{R}$ 到 $\mathbb{R}$ 的函数 f ，若对每个 x 都有 $f(-x) = f(x)$ ，则称为偶函数；若对每个 x 都有 $f(-x) = -f(x)$ ，则称为奇函数。下面的例子形式化定义这两个概念，并证明其中一个事实。你可以完成其余证明。

```
def FnEven (f : ℝ → ℝ) : Prop :=
  ∀ x, f x = f (-x)

def FnOdd (f : ℝ → ℝ) : Prop :=
  ∀ x, f x = -f (-x)

example (ef : FnEven f) (eg : FnEven g) : FnEven fun x ↦ f x + g x := by
  intro x
  calc
    (fun x ↦ f x + g x) x = f x + g x := rfl
    _ = f (-x) + g (-x) := by rw [ef, eg]

example (of : FnOdd f) (og : FnOdd g) : FnEven fun x ↦ f x * g x := by
  sorry

example (ef : FnEven f) (og : FnOdd g) : FnOdd fun x ↦ f x * g x := by
  sorry

example (ef : FnEven f) (og : FnOdd g) : FnEven fun x ↦ f (g x) := by
  sorry
```

第一个证明可以用 `dsimp` 或 `change` 缩短，以消除 lambda 抽象。但你可以检查，如果不显式消除 lambda 抽象，后续 `rw` 将无法工作，因为它无法在表达式中找到模式 `f x` 和 `g x`。与某些其他策略不同，`rw` 在句法层面操作；它不会替你展开定义或进行约简（它有一个变体 `erw`，会在这个方向上稍微多尝试一点，但也不会多很多）。

一旦知道如何识别隐式全称量词，就会发现它们无处不在。

Mathlib 包含一个很好的集合操作库。回忆一下，Lean 并不使用基于集合论的基础，因此这里“集合”一词具有日常数学意义：给定某个类型 α 的数学对象所组成的一个汇集。若 x 的类型是 α ，而 s 的类型是 `Set  $\alpha$` ，则 $x \in s$ 是一个命题，断言 x 是 s 的元素。若 y 具有某个不同类型 β ，则表达式 $y \in s$ 没有意义。这里“没有意义”指的是“没有类型，因而 Lean 不接受它作为良构命题”。这与 Zermelo-Fraenkel 集合论形成对比；在后者中，对任意数学对象 a 和 b ， $a \in b$ 都是良构命题。例如， $\sin \in \cos$ 在 ZF 中是良构命题。集合论基础的这一缺陷，是证明助手不采用它的重要动机之一：证明助手应通过发现无意义表达式来帮助我们。在 Lean 中，`sin` 的类型是 $\mathbb{R} \rightarrow \mathbb{R}$ ，`cos` 的类型也是 $\mathbb{R} \rightarrow \mathbb{R}$ ，即使展开定义后也不等于 `Set ( $\mathbb{R} \rightarrow \mathbb{R}$ )`，所以命题 $\sin \in \cos$ 没有意义。人们也可以用 Lean 研究集合论本身。例如，连续统假设相对于 Zermelo-Fraenkel 公理的独立性已经在 Lean 中形式化。但这种集合论元理论完全超出了本书范围。

若 s 和 t 的类型是 $\text{Set } \alpha$ ，则子集关系 $s \subseteq t$ 定义为 $\forall \{x : \alpha\}, x \in s \rightarrow x \in t$ 。量词中的变量被标记为隐式变量，因此给定 $h : s \subseteq t$ 和 $h' : x \in s$ 时，可以写 $h h'$ 作为 $x \in t$ 的证明。下面的例子给出了子集关系自反性的策略证明和证明项，并要求你对传递性做同样的事情。

```
variable {α : Type*} (r s t : Set α)

example : s ⊆ s := by
  intro x xs
  exact xs

theorem Subset.refl : s ⊆ s := fun _x xs => xs

theorem Subset.trans : r ⊆ s → s ⊆ t → r ⊆ t := by
  sorry
```

正如我们为函数定义 FnUb 一样，也可以定义 $\text{SetUb } s \ a$ ，表示 a 是集合 s 的上界；这里假定 s 是某个带序类型中元素组成的集合。在下一个例子中，我们要求你证明：如果 a 是 s 的上界且 $a \leq b$ ，那么 b 也是 s 的上界。

```
variable {α : Type*} [PartialOrder α]
variable (s : Set α) (a b : α)

def SetUb (s : Set α) (a : α) :=
  ∀ x, x ∈ s → x ≤ a

example (h : SetUb s a) (h' : a ≤ b) : SetUb s b :=
  sorry
```

本节最后看一个重要例子。若对每个 x_1 和 x_2 ，由 $f(x_1) = f(x_2)$ 可推出 $x_1 = x_2$ ，则称函数 f 是单射。Mathlib 定义了 $\text{Function.Injective } f$ ，其中 x_1 和 x_2 为隐式参数。下一个例子说明，在实数上，任何加上常数的函数都是单射。随后我们要求你证明，乘以非零常数的函数也是单射；可以从例子中的引理名获得启发。记住，在猜出引理名开头之后，应使用 Ctrl-space 补全。

```
open Function

example (c : ℝ) : Injective fun x => x + c := by
  intro x₁ x₂ h'
  exact (add_left_inj c).mp h'

example {c : ℝ} (h : c ≠ 0) : Injective fun x => c * x := by
  sorry
```

最后，请证明两个单射函数的复合仍为单射：

```
variable {α : Type*} {β : Type*} {γ : Type*}
variable {g : β → γ} {f : α → β}

example (injg : Injective g) (injf : Injective f) : Injective fun x => g (f x) := by
  sorry
```

3.2. 存在量词

存在量词可以在 VS Code 中输入为 $\backslash\text{ex}$ ，用于表示“存在”。Lean 中的形式表达式 $\exists x : \mathbb{R}, 2 < x \wedge x < 3$ 表示在 2 与 3 之间存在一个实数。（我们将在第 3.4 节中讨论合取符号 $\wedge$ 。）证明这类命题的标准方式，是给出一个实数，并证明它具有所陈述的性质。数 2.5 具有所需性质；它可以输入为 $5 / 2$ ，或在 Lean 无法从语境推断我们指的是实数时输入为 $(5 : \mathbb{R}) / 2$ ；`norm_num` 策略可以证明它满足该描述。

有几种方式可以把这些信息组合起来。若目标以存在量词开头，`use` 策略用于给出对象，并留下证明该对象满足相应性质的目标。

```
example :  $\exists x : \mathbb{R}, 2 < x \wedge x < 3 := \text{by}$ 
  use 5 / 2
  norm_num
```

除了数据之外，也可以向 `use` 策略给出证明：

```
example :  $\exists x : \mathbb{R}, 2 < x \wedge x < 3 := \text{by}$ 
  have h1 :  $2 < (5 : \mathbb{R}) / 2 := \text{by}$  norm_num
  have h2 :  $(5 : \mathbb{R}) / 2 < 3 := \text{by}$  norm_num
  use 5 / 2, h1, h2
```

事实上，`use` 策略还会自动尝试使用可用假设。

```
example :  $\exists x : \mathbb{R}, 2 < x \wedge x < 3 := \text{by}$ 
  have h :  $2 < (5 : \mathbb{R}) / 2 \wedge (5 : \mathbb{R}) / 2 < 3 := \text{by}$  norm_num
  use 5 / 2
```

另一种方式是使用 Lean 的匿名构造子记号来构造存在量词的证明。

```
example :  $\exists x : \mathbb{R}, 2 < x \wedge x < 3 :=$ 
  have h :  $2 < (5 : \mathbb{R}) / 2 \wedge (5 : \mathbb{R}) / 2 < 3 := \text{by}$  norm_num
  (5 / 2, h)
```

注意，这里没有 `by`；我们给出的是显式证明项。左右尖括号可分别输入为 `\<` 和 `\>`，它们告诉 Lean 使用适合当前目标的构造方式，把给定数据组合起来。我们也可以不先进入策略模式而直接使用这个记号：

```
example :  $\exists x : \mathbb{R}, 2 < x \wedge x < 3 :=$ 
  (5 / 2, by norm_num)
```

现在我们知道如何证明存在命题。那么如何使用存在命题呢？如果已知存在一个具有某性质的对象，就应当能够给其中任意一个命名并对它推理。例如，回忆上一节中的谓词 `FnUb f a` 和 `FnLb f a`，它们分别表示 `a` 是 `f` 的上界或下界。我们可以用存在量词说“`f` 有界”，而不指明具体界：

```
def FnUb (f :  $\mathbb{R} \rightarrow \mathbb{R}$ ) (a :  $\mathbb{R}$ ) : Prop :=
   $\forall x, f\ x \leq a$ 

def FnLb (f :  $\mathbb{R} \rightarrow \mathbb{R}$ ) (a :  $\mathbb{R}$ ) : Prop :=
   $\forall x, a \leq f\ x$ 

def FnHasUb (f :  $\mathbb{R} \rightarrow \mathbb{R}$ ) :=
   $\exists a, \text{FnUb } f\ a$ 

def FnHasLb (f :  $\mathbb{R} \rightarrow \mathbb{R}$ ) :=
   $\exists a, \text{FnLb } f\ a$ 
```

我们可以使用上一节中的定理 `FnUb_add` 证明：若 `f` 和 `g` 都有上界，则 `fun x ↦ f x + g x` 也有上界。

```
variable {f g :  $\mathbb{R} \rightarrow \mathbb{R}$ }

example (ubf : FnHasUb f) (ubg : FnHasUb g) : FnHasUb fun x ↦ f x + g x := by
  rcases ubf with (a, ubfa)
  rcases ubg with (b, ubgb)
  use a + b
  apply fnUb_add ubfa ubgb
```

`rcases` 策略会拆开存在量词中的信息。像 `(a, ubfa)` 这样的标注使用与匿名构造子相同的尖括号，称为模式；它描述了我们在拆开主参数时预期得到的信息。给定假设 `ubf`，表示 `f` 有上界，命令 `rcases ubf with (a, ubfa)` 会向语境加入一个新变量 `a`，表示某个上界，同时加入假设 `ubfa`，表示它具有给定性质。目标保持不变；改变的是我们现在可以使用这个新对象和新假设来证明目标。这是数学推理中的常见方法：我们拆开由某个假设断言或蕴含其存在性的对象，然后用它建立其他事物的存在性。

请尝试使用这种方法建立下面的命题。你可能会发现，把上一节中的某些例子像 `fn_ub_add` 那样改成命名定理会很有用；也可以直接把参数插入证明中。

```
example (lbf : FnHasLb f) (lbg : FnHasLb g) : FnHasLb fun x ↦ f x + g x := by
  sorry

example {c : ℝ} (ubf : FnHasUb f) (h : c ≥ 0) : FnHasUb fun x ↦ c * f x := by
  sorry
```

`rcases` 中的 “r” 表示 “recursive”，因为它允许我们使用任意复杂的模式来拆开嵌套数据。`rintro` 策略是 `intro` 与 `rcases` 的组合：

```
example : FnHasUb f → FnHasUb g → FnHasUb fun x ↦ f x + g x := by
  rintro ⟨a, ubfa⟩ ⟨b, ubgb⟩
  exact ⟨a + b, fnUb_add ubfa ubgb⟩
```

事实上，Lean 也支持在表达式和证明项中使用带模式匹配的 `fun`：

```
example : FnHasUb f → FnHasUb g → FnHasUb fun x ↦ f x + g x :=
  fun ⟨a, ubfa⟩ ⟨b, ubgb⟩ => ⟨a + b, fnUb_add ubfa ubgb⟩
```

拆开假设中的信息是如此重要，以至于 Lean 和 Mathlib 提供了许多方法。例如，`obtain` 策略提供了富有提示性的语法：

```
example (ubf : FnHasUb f) (ubg : FnHasUb g) : FnHasUb fun x ↦ f x + g x := by
  obtain ⟨a, ubfa⟩ := ubf
  obtain ⟨b, ubgb⟩ := ubg
  exact ⟨a + b, fnUb_add ubfa ubgb⟩
```

可以把第一条 `obtain` 指令理解为：把 `ubf` 的“内容”与给定模式匹配，并把各组成部分赋给命名变量。我们说 `rcases` 和 `obtain` 会析构它们的参数。

Lean 还支持类似其他函数式编程语言中的语法：

```
example (ubf : FnHasUb f) (ubg : FnHasUb g) : FnHasUb fun x ↦ f x + g x := by
  cases ubf
  case intro a ubfa =>
    cases ubg
    case intro b ubgb =>
      exact ⟨a + b, fnUb_add ubfa ubgb⟩

example (ubf : FnHasUb f) (ubg : FnHasUb g) : FnHasUb fun x ↦ f x + g x := by
  cases ubf
  next a ubfa =>
    cases ubg
    next b ubgb =>
      exact ⟨a + b, fnUb_add ubfa ubgb⟩

example (ubf : FnHasUb f) (ubg : FnHasUb g) : FnHasUb fun x ↦ f x + g x := by
  match ubf, ubg with
  | ⟨a, ubfa⟩, ⟨b, ubgb⟩ =>
    exact ⟨a + b, fnUb_add ubfa ubgb⟩

example (ubf : FnHasUb f) (ubg : FnHasUb g) : FnHasUb fun x ↦ f x + g x :=
  match ubf, ubg with
  | ⟨a, ubfa⟩, ⟨b, ubgb⟩ =>
    ⟨a + b, fnUb_add ubfa ubgb⟩
```

在第一个例子中，如果把光标放在 `cases ubf` 之后，会看到该策略产生一个目标，Lean 给它标记为 `intro`。（这个具体名称来自用于构造存在命题证明的公理化原语的内部名称。）随后 `case` 策略为各组成部分命名。第二个例子类似，只是使用 `next` 而非 `case`，从而可以避免提及 `intro`。最后两个例子中的 `match` 一词强调了这里所做的正是计算机科学家所谓的“模式匹配”。注意，第三个证明以 `by` 开始；在它之后，策略版本的 `match` 要求箭头右侧是一个策略证明。最后一个例子是证明项：完全没有策略出现。

在本书余下部分，我们会坚持使用 `rcases`、`rintro` 和 `obtain`，作为使用存在量词的首选方式。不过，了解替代语法并无坏处，尤其是在你可能与计算机科学家共处时。

为了说明 `rcases` 的一种用法，我们证明一个古老的数学常识：若两个整数 `x` 和 `y` 都可以写成两个平方数之和，则它们的乘积 `x * y` 也可以写成两个平方数之和。事实上，这个命题对任意交换环都成立，而不仅限于整数。在下一个例子中，`rcases` 一次拆开两个存在量词。然后我们把把 `x * y` 表示为平方和所需的神奇取值作为列表提供给 `use` 语句，并用 `ring` 验证它们有效。

```
variable {α : Type*} [CommRing α]

def SumOfSquares (x : α) :=
  ∃ a b, x = a ^ 2 + b ^ 2

theorem sumOfSquares_mul {x y : α} (sosx : SumOfSquares x) (sosy : SumOfSquares y) :
  SumOfSquares (x * y) := by
  rcases sosx with ⟨a, b, xeq⟩
  rcases sosy with ⟨c, d, yeq⟩
  rw [xeq, yeq]
  use a * c - b * d, a * d + b * c
  ring
```

这个证明没有提供太多洞见，但可以用以下方式来说明它的动机。高斯整数是形如 $a + bi$ 的数，其中 a 和 b 是整数，且 $i = \sqrt{-1}$ 。按定义，高斯整数 $a + bi$ 的范数是 $a^2 + b^2$ 。因此，高斯整数的范数是平方和，且任意平方和都可以这样表示。上述定理反映了这样一个事实：高斯整数乘积的范数等于范数的乘积；如果 x 是 $a + bi$ 的范数，而 y 是 $c + di$ 的范数，则 xy 是 $(a + bi)(c + di)$ 的范数。这个晦涩的证明说明，最容易形式化的证明并不总是最清楚的证明。在 [第 7.3 节](#) 中，我们会提供定义高斯整数的手段，并用它们给出一个替代证明。

拆开存在量词内部的等式，再用它重写目标中的表达式，这种模式经常出现；频繁到 `rcases` 策略为它提供了一个缩写：若使用关键字 `rfl` 代替新的标识符，`rcases` 会自动执行重写（这个技巧不适用于模式匹配 `lambda`）。

```
theorem sumOfSquares_mul' {x y : α} (sosx : SumOfSquares x) (sosy : SumOfSquares y) :
  SumOfSquares (x * y) := by
  rcases sosx with ⟨a, b, rfl⟩
  rcases sosy with ⟨c, d, rfl⟩
  use a * c - b * d, a * d + b * c
  ring
```

与全称量词一样，只要知道如何识别，存在量词也会隐藏在许多地方。例如，整除关系隐含地就是一个“存在”命题。

```
example (divab : a | b) (divbc : b | c) : a | c := by
  rcases divab with ⟨d, beq⟩
  rcases divbc with ⟨e, ceq⟩
  rw [ceq, beq]
  use d * e; ring
```

同样，这为配合 `rfl` 使用 `rcases` 提供了一个很好的场景。请在上面的证明中试一试；效果相当顺手！

然后请尝试证明下面的命题：

```
example (divab : a | b) (divac : a | c) : a | b + c := by
  sorry
```

另一个重要例子是满射。若对陪域 β 中每个 y ，都存在定义域 α 中的 x ，使得 $f(x) = y$ ，则称函数 $f : \alpha \rightarrow \beta$ 是满射。注意，这个陈述同时包含全称量词和存在量词，这解释了为什么下一个例子同时使用 `intro` 和 `use`。

```
example {c : ℝ} : Surjective fun x ↦ x + c := by
  intro y
  use y - c
  dsimp; ring
```

请自行使用定理 `mul_div_cancel`。尝试这个例子：

```
example {c : ℝ} (h : c ≠ 0) : Surjective fun x ↦ c * x := by
  sorry
```

此时值得一提的是，有一个策略 `field_simp`，常常能以有用的方式消去分母。它可以与 `ring` 策略配合使用。

```
example (x y : ℝ) (h : x - y ≠ 0) : (x ^ 2 - y ^ 2) / (x - y) = x + y := by
  field_simp [h]
  ring
```

下一个例子通过把满射性假设应用于合适的值来使用它。注意，`rcases` 可以用于任意表达式，而不仅仅用于假设。

```
example {f : ℝ → ℝ} (h : Surjective f) : ∃ x, f x ^ 2 = 4 := by
  rcases h 2 with (x, hx)
  use x
  rw [hx]
  norm_num
```

看看能否用这些方法证明满射函数的复合仍然是满射。

```
variable {α : Type*} {β : Type*} {γ : Type*}
variable {g : β → γ} {f : α → β}

example (surjg : Surjective g) (surjf : Surjective f) : Surjective fun x ↦ g (f x) := by
  sorry
```

3.3. 否定

符号 `¬` 用来表示否定，因此 `¬ x < y` 表示 `x` 不小于 `y`，`¬ x = y`（或等价地，`x ≠ y`）表示 `x` 不等于 `y`，而 `¬ ∃ z, x < z ∧ z < y` 表示不存在严格位于 `x` 与 `y` 之间的 `z`。在 Lean 中，记号 `¬ A` 是 `A → False` 的缩写，可以理解为 `A` 蕴含矛盾。实际而言，这意味着你已经知道一些处理否定的方法：可以通过引入假设 `h : A` 并证明 `False` 来证明 `¬ A`；若有 `h : ¬ A` 和 `h' : A`，则把 `h` 应用于 `h'` 就得到 `False`。

为说明这一点，考虑严格序的非自反性原理 `lt_irrefl`，它说对每个 `a` 都有 `¬ a < a`。不对称性原理 `lt_asymm` 说 `a < b → ¬ b < a`。我们来证明 `lt_asymm` 可由 `lt_irrefl` 推出。

```
example (h : a < b) : ¬ b < a := by
  intro h'
  have : a < a := lt_trans h h'
  apply lt_irrefl a this
```

这个例子引入了几个新技巧。首先，当你使用 `have` 而不给出标签时，Lean 会使用名称 `this`，从而提供一个方便的方式回指它。由于证明很短，我们给出显式证明项。但这个证明中真正应当关注的是 `intro` 策略的结果：它留下一个 `False` 目标；以及我们最终通过把 `lt_irrefl` 应用于 `a < a` 的证明来证明 `False`。

下面是另一个例子，它使用上一节定义的谓词 `FnHasUb`，该谓词表示函数有上界。

```
example (h : ∀ a, ∃ x, f x > a) : ¬FnHasUb f := by
  intro fnub
  rcases fnub with (a, fnuba)
  rcases h a with (x, hx)
  have : f x ≤ a := fnuba x
  linarith
```

请记住，当目标可由语境中的线性等式和不等式推出时，使用 `linarith` 往往很方便。

看看能否用类似方式证明这些命题：

```
example (h : ∀ a, ∃ x, f x < a) : ¬FnHasLb f :=
  sorry

example : ¬FnHasUb fun x ↦ x :=
  sorry
```

Mathlib 提供了若干有用定理，用于联系序关系与否定：

```
#check (not_le_of_gt : a > b → ¬a ≤ b)
#check (not_lt_of_ge : a ≥ b → ¬a < b)
#check (lt_of_not_ge : ¬a ≥ b → a < b)
#check (le_of_not_gt : ¬a > b → a ≤ b)
```

回忆谓词 `Monotone f`，它表示 `f` 单调不减。请使用刚刚列出的某些定理证明如下命题：

```
example (h : Monotone f) (h' : f a < f b) : a < b := by
  sorry

example (h : a ≤ b) (h' : f b < f a) : ¬Monotone f := by
  sorry
```

我们可以证明，如果把上一段代码中的第一个例子里的 `<` 替换为 `≤`，则该命题无法证明。注意，可以通过给出反例来证明全称量化命题的否定。请完成证明。

```
example : ¬∀ {f : ℝ → ℝ}, Monotone f → ∀ {a b}, f a ≤ f b → a ≤ b := by
  intro h
  let f := fun x : ℝ ↦ (0 : ℝ)
  have monof : Monotone f := by sorry
  have h' : f 1 ≤ f 0 := le_refl _
  sorry
```

这个例子引入了 `let` 策略，它向语境添加一个局部定义。如果把光标放在 `let` 命令之后，在目标窗口中会看到定义 `f : ℝ → ℝ := fun x ↦ 0` 已被加入语境。必要时，Lean 会展开 `f` 的定义。特别地，当我们用 `le_refl` 证明 `f 1 ≤ f 0` 时，Lean 会把 `f 1` 和 `f 0` 约简为 `0`。

请使用 `le_of_not_gt` 证明下面的命题：

```
example (x : ℝ) (h : ∀ ε > 0, x < ε) : x ≤ 0 := by
  sorry
```

在我们刚完成的许多证明中，隐含着这样一个事实：若 `P` 是任意性质，则说没有任何对象具有性质 `P`，等价于说所有对象都不具有性质 `P`；而说并非所有对象都具有性质 `P`，等价于说存在某个对象不具有性质 `P`。换言之，以下四个蕴含都是有效的（但其中一个不能用目前已经解释的方法证明）：

```
variable {α : Type*} (P : α → Prop) (Q : Prop)

example (h : ¬∃ x, P x) : ∀ x, ¬P x := by
  sorry

example (h : ∀ x, ¬P x) : ¬∃ x, P x := by
  sorry

example (h : ¬∀ x, P x) : ∃ x, ¬P x := by
  sorry

example (h : ∃ x, ¬P x) : ¬∀ x, P x := by
  sorry
```

第一、第二和第四个蕴含可以直接用你已经见过的方法证明。我们鼓励你尝试。第三个更困难，因为它要从某个对象不存在会导致矛盾这一事实推出该对象存在。这是经典数学推理的一个实例。我们可以如下用反证法证明第三个蕴含。

```
example (h :  $\neg \forall x, P x$ ) :  $\exists x, \neg P x$  := by
  by_contra h'
  apply h
  intro x
  show P x
  by_contra h''
  exact h' (x, h'')
```

请确保理解其工作方式。`by_contra` 策略允许我们通过假设 $\neg Q$ 并导出矛盾来证明目标 Q 。事实上，它等价于使用等价命题 `not_not :  $\neg \neg Q \leftrightarrow Q$` 。请确认你可以用 `by_contra` 证明这个等价的正向方向，而反向方向则由否定的一般规则推出。

```
example (h :  $\neg \neg Q$ ) :  $Q$  := by
  sorry

example (h :  $Q$ ) :  $\neg \neg Q$  := by
  sorry
```

请使用反证法建立下面的命题，它是上面已证明某个蕴含的逆命题。（提示：先使用 `intro`。）

```
example (h :  $\neg \text{FnHasUb } f$ ) :  $\forall a, \exists x, f x > a$  := by
  sorry
```

处理前面带有否定的复合命题常常很繁琐；数学中常见的做法，是把这类命题替换为等价形式，使否定向内推进。为此，Mathlib 提供了 `push_neg` 策略，它会以这种方式重述目标（这包括把 $\neg \neg A$ 化简为 A ）。命令 `push_neg at h` 会重述假设 `h`。

```
example (h :  $\neg \forall a, \exists x, f x > a$ ) :  $\text{FnHasUb } f$  := by
  push_neg at h
  exact h

example (h :  $\neg \text{FnHasUb } f$ ) :  $\forall a, \exists x, f x > a$  := by
  dsimp only [FnHasUb, FnUb] at h
  push_neg at h
  exact h
```

在第二个例子中，我们用 `dsimp` 展开 `FnHasUb` 和 `FnUb` 的定义。（展开 `FnUb` 时需要用 `dsimp` 而不是 `rw`，因为它出现在量词作用域内。）你可以验证，在上面含有 $\neg \exists x, P x$ 和 $\neg \forall x, P x$ 的例子中，`push_neg` 策略会得到预期结果。即使还不知道如何使用合取符号，你也应该能够用 `push_neg` 证明下面的命题：

```
example (h :  $\neg \text{Monotone } f$ ) :  $\exists x y, x \leq y \wedge f y < f x$  := by
  sorry
```

Mathlib 还有一个策略 `contrapose`，它把目标 $A \rightarrow B$ 转化为 $\neg B \rightarrow \neg A$ 。类似地，若要从假设 $h : A$ 证明 B ，则 `contrapose h` 会留下一个目标：从假设 $\neg B$ 证明 $\neg A$ 。使用 `contrapose!` 而不是 `contrapose`，还会同时对目标和相关假设应用 `push_neg`。

```
example (h :  $\neg \text{FnHasUb } f$ ) :  $\forall a, \exists x, f x > a$  := by
  contrapose! h
  exact h

example (x :  $\mathbb{R}$ ) (h :  $\forall \epsilon > 0, x \leq \epsilon$ ) :  $x \leq 0$  := by
  contrapose! h
  use x / 2
  constructor <.> linarith
```

我们还没有解释 `constructor` 命令，也没有解释其后的分号用法；这些将在下一节说明。

本节最后介绍 *ex falso* 原理，即从矛盾可以推出任何命题。在 Lean 中，这由 `False.elim` 表示，它对任意命题 P 建立 $\text{False} \rightarrow P$ 。这个原理看起来也许奇怪，但经常出现。我们常常通过分情况来证明定理，有时可以证明其中一种情形是矛盾的。在这种情况下，需要断言该矛盾建立了当前目标，以便继续处理下一个情形。（我们将在第 3.5 节中看到分情况推理的实例。）

一旦已经得到矛盾，Lean 提供了若干关闭目标的方法。

```
example (h : 0 < 0) : a > 37 := by
  ex falso
  apply lt_irrefl 0 h

example (h : 0 < 0) : a > 37 :=
  absurd h (lt_irrefl 0)

example (h : 0 < 0) : a > 37 := by
  have h' : ¬0 < 0 := lt_irrefl 0
  contradiction
```

`ex falso` 策略把当前目标替换为证明 `False` 的目标。给定 `h : P` 和 `h' : ¬ P`，项 `absurd h h'` 可以建立任意命题。最后，`contradiction` 策略会尝试通过在假设中寻找矛盾来关闭目标，例如寻找形如 `h : P` 与 `h' : ¬ P` 的一对假设。当然，在这个例子中，`linarith` 也能工作。

3.4. 合取与当且仅当

你已经看到，合取符号 `^` 用于表示“且”。`constructor` 策略允许通过先证明 `A` 再证明 `B`，来证明形如 `A ^ B` 的命题。

```
example {x y : ℝ} (h₀ : x ≤ y) (h₁ : ¬y ≤ x) : x ≤ y ^ x ≠ y := by
  constructor
  · assumption
  intro h
  apply h₁
  rw [h]
```

在这个例子中，`assumption` 策略告诉 Lean 寻找一个可以解决目标的假设。注意最后的 `rw` 通过应用 `≤` 的自反性完成目标。下面是使用匿名构造子尖括号完成前面例子的其他方式。第一个是前一个证明的简洁证明项版本，并在关键字 `by` 处进入策略模式。

```
example {x y : ℝ} (h₀ : x ≤ y) (h₁ : ¬y ≤ x) : x ≤ y ^ x ≠ y :=
  (h₀, fun h ↦ h₁ (by rw [h]))

example {x y : ℝ} (h₀ : x ≤ y) (h₁ : ¬y ≤ x) : x ≤ y ^ x ≠ y :=
  have h : x ≠ y := by
    contrapose! h₁
    rw [h₁]
  (h₀, h)
```

使用合取而不是证明合取时，需要拆开其两个部分的证明。可以使用 `rcases` 策略，也可以使用 `rintro` 或模式匹配 `fun`；其方式都类似于它们在存在量词中的用法。

```
example {x y : ℝ} (h : x ≤ y ^ x ≠ y) : ¬y ≤ x := by
  rcases h with (h₀, h₁)
  contrapose! h₁
  exact le_antisymm h₀ h₁

example {x y : ℝ} : x ≤ y ^ x ≠ y → ¬y ≤ x := by
  rintro (h₀, h₁) h'
  exact h₁ (le_antisymm h₀ h')
```

```
example {x y : ℝ} : x ≤ y ^ x ≠ y → ¬y ≤ x :=
  fun (h₀, h₁) h' ↦ h₁ (le_antisymm h₀ h')
```

与 `obtain` 策略类似，也存在一种带模式匹配的 `have`：

```
example {x y : ℝ} (h : x ≤ y ^ x ≠ y) : ¬y ≤ x := by
  have (h₀, h₁) := h
  contrapose! h₁
  exact le_antisymm h₀ h₁
```

与 `rcases` 不同，这里的 `have` 策略会把 `h` 保留在语境中。尽管我们不会使用它们，还是再次给出计算机科学家常用的模式匹配语法：

```
example {x y : ℝ} (h : x ≤ y ∧ x ≠ y) : ¬y ≤ x := by
  cases h
  case intro h₀ h₁ =>
    contrapose! h₁
    exact le_antisymm h₀ h₁

example {x y : ℝ} (h : x ≤ y ∧ x ≠ y) : ¬y ≤ x := by
  cases h
  next h₀ h₁ =>
    contrapose! h₁
    exact le_antisymm h₀ h₁

example {x y : ℝ} (h : x ≤ y ∧ x ≠ y) : ¬y ≤ x := by
  match h with
  | ⟨h₀, h₁⟩ =>
    contrapose! h₁
    exact le_antisymm h₀ h₁
```

与存在量词不同，对于假设 `h : A ∧ B`，也可以写 `h.left` 和 `h.right` 来取出两个组成部分的证明；等价地，也可以写 `h.1` 和 `h.2`。

```
example {x y : ℝ} (h : x ≤ y ∧ x ≠ y) : ¬y ≤ x := by
  intro h'
  apply h.right
  exact le_antisymm h.left h'

example {x y : ℝ} (h : x ≤ y ∧ x ≠ y) : ¬y ≤ x :=
  fun h' => h.right (le_antisymm h.left h')
```

请尝试使用这些技巧，以多种方式证明下面的命题：

```
example {m n : ℕ} (h : m < n ∧ m ≠ n) : m < n ∧ ¬n < m :=
  sorry
```

可以把 `∃` 与 `∧` 嵌套使用，并配合匿名构造子、`rintro` 和 `rcases`。

```
example : ∃ x : ℝ, 2 < x ∧ x < 4 :=
  (5 / 2, by norm_num, by norm_num)

example (x y : ℝ) : (∃ z : ℝ, x < z ∧ z < y) → x < y := by
  rintro ⟨z, xltz, zltz⟩
  exact lt_trans xltz zltz

example (x y : ℝ) : (∃ z : ℝ, x < z ∧ z < y) → x < y :=
  fun ⟨z, xltz, zltz⟩ => lt_trans xltz zltz
```

回忆一下，在 `z` 前加下划线，是告诉 Lean 我们打算使用 `z`。否则 Lean 会输出警告，说 `z` 未被使用。

也可以使用 `use` 策略：

```
example : ∃ x : ℝ, 2 < x ∧ x < 4 := by
  use 5 / 2
  constructor <.> norm_num

example : ∃ m n : ℕ, 4 < m ∧ m < n ∧ n < 10 ∧ Nat.Prime m ∧ Nat.Prime n := by
  use 5
  use 7
  norm_num

example {x y : ℝ} : x ≤ y ∧ x ≠ y → x < y ∧ ¬y ≤ x := by
  rintro ⟨h₀, h₁⟩
  use h₀
  exact fun h' => h₁ (le_antisymm h₀ h')
```

在第一个例子中，`constructor` 命令之后的 `<.>` 告诉 Lean 对由此产生的两个目标都使用 `norm_num` 策略。

在 Lean 中, $A \leftrightarrow B$ 并未定义为 $(A \rightarrow B) \wedge (B \rightarrow A)$, 但本来可以这样定义, 而且它的行为大致相同。你已经看到, 对于 $h : A \leftrightarrow B$, 可以写 `h.mp` 和 `h.mpr`, 或 `h.1` 和 `h.2`, 来获得两个方向。也可以使用 `cases` 及其相关方法。要证明一个当且仅当命题, 可以像证明合取一样使用 `constructor` 或尖括号。

```
example {x y : ℝ} (h : x ≤ y) : ¬y ≤ x ↔ x ≠ y := by
  constructor
  · contrapose!
    rintro rfl
    rfl
  contrapose!
  exact le_antisymm h

example {x y : ℝ} (h : x ≤ y) : ¬y ≤ x ↔ x ≠ y :=
  (fun h₀ h₁ ↦ h₀ (by rw [h₁]), fun h₀ h₁ ↦ h₀ (le_antisymm h h₁))
```

最后一个证明项难以读懂。请记住, 在写这类表达式时, 可以使用下划线查看 Lean 期望什么。

请尝试使用刚刚看到的各种技巧和工具证明下面的命题:

```
example {x y : ℝ} : x ≤ y ∧ ¬y ≤ x ↔ x ≤ y ∧ x ≠ y :=
  sorry
```

作为一个更有趣的练习, 请证明: 对任意两个实数 x 和 y , $x^2 + y^2 = 0$ 当且仅当 $x = 0$ 且 $y = 0$ 。我们建议使用 `linarith`、`pow_two_nonneg` 和 `eq_zero_of_pow_eq_zero` 证明一个辅助引理。

```
theorem aux {x y : ℝ} (h : x ^ 2 + y ^ 2 = 0) : x = 0 :=
  have h' : x ^ 2 = 0 := by sorry
  eq_zero_of_pow_eq_zero h'

example (x y : ℝ) : x ^ 2 + y ^ 2 = 0 ↔ x = 0 ∧ y = 0 :=
  sorry
```

在 Lean 中, 双向蕴含具有双重身份。可以像合取一样对待它, 并分别使用其两个部分。但 Lean 也知道它是命题之间的自反、对称且传递的关系, 因此也可以把它与 `calc` 和 `rw` 一起使用。把一个命题重写为等价命题常常很方便。在下一个例子中, 我们用 `abs_lt` 把形如 $|x| < y$ 的表达式替换为等价表达式 $-y < x \wedge x < y$; 在再下一个例子中, 我们用 `Nat.dvd_gcd_iff` 把形如 $m \mid \text{Nat.gcd } n \ k$ 的表达式替换为等价表达式 $m \mid n \wedge m \mid k$ 。

```
example (x : ℝ) : |x + 3| < 5 → -8 < x ∧ x < 2 := by
  rw [abs_lt]
  intro h
  constructor <.> linarith

example : 3 ∣ Nat.gcd 6 15 := by
  rw [Nat.dvd_gcd_iff]
  constructor <.> norm_num
```

看看能否用下面的定理配合 `rw` 给出一个简短证明, 说明取负不是单调不减函数。(注意, `push_neg` 不会替你展开定义, 因此该定理证明中的 `rw [Monotone]` 是必要的。)

```
theorem not_monotone_iff {f : ℝ → ℝ} : ¬Monotone f ↔ ∃ x y, x ≤ y ∧ f x > f y := by
  rw [Monotone]
  push_neg
  rfl

example : ¬Monotone fun x : ℝ ↦ -x := by
  sorry
```

本节剩余练习旨在让你进一步练习合取与双向蕴含。记住, 偏序是一个满足传递性、自反性和反对称性的二元关系。有时还会出现更弱的概念: 预序只是自反且传递的关系。对任意预序 $\leq$, Lean 通过 $a < b \leftrightarrow a \leq b \wedge \neg b \leq a$ 公理化相关的严格预序。请证明, 如果 $\leq$ 是偏序, 则 $a < b$ 等价于 $a \leq b \wedge a \neq b$:

```
variable {α : Type*} [PartialOrder α]
variable (a b : α)

example : a < b ↔ a ≤ b ∧ a ≠ b := by
  rw [lt_iff_le_not_ge]
  sorry
```

除了逻辑操作之外，你不需要比 `le_refl`、`le_trans` 和 `le_antisymm` 更多的工具。请证明：即使 `≤` 只假定为预序，也能证明严格序是非自反且传递的。在第二个例子中，为方便起见，我们使用化简器而不是 `rw`，把 `<` 用 `≤` 和 `≠` 表示出来。我们稍后会回到化简器；这里仅依赖这样一个事实：即使需要把指定引理实例化为不同的值，化简器也会反复使用该引理。

```
variable {α : Type*} [Preorder α]
variable (a b c : α)

example : ¬a < a := by
  rw [lt_iff_le_not_ge]
  sorry

example : a < b → b < c → a < c := by
  simp only [lt_iff_le_not_ge]
  sorry
```

3.5. 析取

证明析取 `A ∨ B` 的标准方式，是证明 `A` 或证明 `B`。`left` 策略选择 `A`，而 `right` 策略选择 `B`。

```
variable {x y : ℝ}

example (h : y > x ^ 2) : y > 0 ∨ y < -1 := by
  left
  linarith [pow_two_nonneg x]

example (h : -y > x ^ 2 + 1) : y > 0 ∨ y < -1 := by
  right
  linarith [pow_two_nonneg x]
```

我们不能用匿名构造子来构造“或”的证明，因为 Lean 必须猜测我们要证明哪一个析取支。写证明项时，可以改用 `Or.inl` 和 `Or.inr` 来显式作出选择。这里 `inl` 是“introduction left”的缩写，`inr` 是“introduction right”的缩写。

```
example (h : y > 0) : y > 0 ∨ y < -1 :=
  Or.inl h

example (h : y < -1) : y > 0 ∨ y < -1 :=
  Or.inr h
```

通过证明某一边来证明析取，可能看起来有些奇怪。实践中，哪种情形成立通常取决于假设和数据中隐含或显式的情形区分。`rcases` 策略允许我们使用形如 `A ∨ B` 的假设。不同于在合取或存在量词中使用 `rcases`，这里 `rcases` 会产生两个目标。二者有相同结论，但第一种情形中假定 `A` 为真，第二种情形中假定 `B` 为真。换言之，正如其名所示，`rcases` 策略执行分情况证明。照常，我们可以告诉 Lean 使用什么名称来命名这些假设。在下一个例子中，我们告诉 Lean 在每个分支都使用名称 `h`。

```
example : x < |y| → x < y ∨ x < -y := by
  rcases le_or_gt 0 y with h | h
  · rw [abs_of_nonneg h]
    intro h; left; exact h
  · rw [abs_of_neg h]
    intro h; right; exact h
```

注意，在合取情形中模式从 $\langle h_0, h_1 \rangle$ 变成析取情形中的 $h_0 \mid h_1$ 。可以把第一个模式理解为匹配同时包含 h_0 和 h_1 的数据；而第二个带竖线的模式匹配包含 h_0 或 h_1 的数据。在这个例子中，由于两个目标彼此分离，我们选择在每种情形中使用同一名称 h 。

绝对值函数的定义方式使我们能够立即证明： $x \geq 0$ 蕴含 $|x| = x$ （这是定理 `abs_of_nonneg`），而 $x < 0$ 蕴含 $|x| = -x$ （这是 `abs_of_neg`）。表达式 `le_or_gt 0 x` 建立 $0 \leq x \vee x < 0$ ，使我们可以按这两种情形拆分。

Lean 也支持用于析取的计算机科学式模式匹配语法。现在 `cases` 策略更有吸引力，因为它允许我们命名每个 `case`，并在更接近使用位置的地方命名引入的假设。

```
example : x < |y| → x < y ∨ x < -y := by
  cases le_or_gt 0 y
  case inl h =>
    rw [abs_of_nonneg h]
    intro h; left; exact h
  case inr h =>
    rw [abs_of_neg h]
    intro h; right; exact h
```

名称 `inl` 和 `inr` 分别是“intro left”和“intro right”的缩写。使用 `case` 的优点是可以按任意顺序证明各情形；Lean 会使用标签找到相关目标。若不关心这一点，可以使用 `next`、`match`，甚至使用模式匹配 `have`。

```
example : x < |y| → x < y ∨ x < -y := by
  cases le_or_gt 0 y
  next h =>
    rw [abs_of_nonneg h]
    intro h; left; exact h
  next h =>
    rw [abs_of_neg h]
    intro h; right; exact h

example : x < |y| → x < y ∨ x < -y := by
  match le_or_gt 0 y with
  | Or.inl h =>
    rw [abs_of_nonneg h]
    intro h; left; exact h
  | Or.inr h =>
    rw [abs_of_neg h]
    intro h; right; exact h
```

在 `match` 的情形中，需要使用证明析取的标准方式的全名 `Or.inl` 和 `Or.inr`。本书通常会使用 `rcases` 来对析取进行分情况讨论。

请尝试使用下面代码片段中的前两个定理证明三角不等式。这些定理使用与 Mathlib 中相同的名称。

```
namespace MyAbs

theorem le_abs_self (x : ℝ) : x ≤ |x| := by
  sorry

theorem neg_le_abs (x : ℝ) : -x ≤ |x| := by
  sorry

theorem abs_add_le (x y : ℝ) : |x + y| ≤ |x| + |y| := by
  sorry
```

如果你喜欢这些例子（这里有个双关），并且想进一步练习析取，请尝试下面这些题。

```
theorem lt_abs : x < |y| ↔ x < y ∨ x < -y := by
  sorry

theorem abs_lt : |x| < y ↔ -y < x ∧ x < y := by
  sorry
```

也可以把 `rcases` 和 `rintro` 用于嵌套析取。当它们产生真正的多目标分情况讨论时，各个新目标的模式用竖线分隔。

```
example {x : ℝ} (h : x ≠ 0) : x < 0 ∨ x > 0 := by
  rcases lt_trichotomy x 0 with xlt | xeq | xgt
  · left
    exact xlt
  · contradiction
  · right; exact xgt
```

仍然可以嵌套模式，并使用关键字 `rfl` 来替换等式：

```
example {m n k : ℕ} (h : m | n ∨ m | k) : m | n * k := by
  rcases h with ⟨a, rfl⟩ | ⟨b, rfl⟩
  · rw [mul_assoc]
    apply dvd_mul_right
  · rw [mul_comm, mul_assoc]
    apply dvd_mul_right
```

看看你能否用一行（较长的）代码证明下面的命题。使用 `rcases` 拆开假设并分情况讨论，然后用 `<;> linarith` 解决每个分支。

```
example {z : ℝ} (h : ∃ x y, z = x ^ 2 + y ^ 2 ∨ z = x ^ 2 + y ^ 2 + 1) : z ≥ 0 := by
  sorry
```

在实数上，等式 $x * y = 0$ 告诉我们 $x = 0$ 或 $y = 0$ 。在 Mathlib 中，这一事实名为

`eq_zero_or_eq_zero_of_mul_eq_zero`，它也是析取如何出现的一个漂亮例子。看看你能否用它证明下面的命题：

```
example {x : ℝ} (h : x ^ 2 = 1) : x = 1 ∨ x = -1 := by
  sorry

example {x y : ℝ} (h : x ^ 2 = y ^ 2) : x = y ∨ x = -y := by
  sorry
```

记住，可以使用 `ring` 策略帮助完成计算。

在任意环 R 中，若元素 x 满足对某个非零 y 有 $xy = 0$ ，则称 x 为左零因子；若满足对某个非零 y 有 $yx = 0$ ，则称 x 为右零因子；一个左零因子或右零因子统称为零因子。定理 `eq_zero_or_eq_zero_of_mul_eq_zero` 表明实数没有非平凡零因子。具有这一性质的交换环称为整环。你对上面两个定理的证明，在任意整环中也同样有效：

```
variable {R : Type*} [CommRing R] [IsDomain R]
variable (x y : R)

example (h : x ^ 2 = 1) : x = 1 ∨ x = -1 := by
  sorry

example (h : x ^ 2 = y ^ 2) : x = y ∨ x = -y := by
  sorry
```

事实上，如果足够小心，第一个定理不必使用乘法交换性即可证明。在这种情况下，只需假定 R 是 `Ring`，而不必是 `CommRing`。

有时在证明中，我们想根据某个命题是否为真来分情况讨论。对任意命题 P ，可以使用 `em P : P ∨ ¬ P`。名称 `em` 是“excluded middle”（排中律）的缩写。

```
example (P : Prop) : ¬¬P → P := by
  intro h
  cases em P
  · assumption
  · contradiction
```

另一种选择是使用 `by_cases` 策略。

```
example (P : Prop) : ¬¬P → P := by
  intro h
  by_cases h' : P
  · assumption
  · contradiction
```

注意，`by_cases` 策略允许你指定在每个分支中引入的假设标签；在这个例子中，一个分支里是 `h' : P`，另一个分支里是 `h' : ¬ P`。如果省略标签，Lean 默认使用 `h`。请尝试证明下面的等价命题，并用 `by_cases` 建立其中一个方向。

```
example (P Q : Prop) : P → Q ↔ ¬P ∨ Q := by
  sorry
```

3.6. 序列与收敛

现在我们掌握了足够多的技巧，可以做一些真正的数学。在 Lean 中，可以把实数序列 $s_0, s_1, s_2, \dots$ 表示为函数

`s : ℕ → ℝ`。若对每个 $\varepsilon > 0$ ，都存在某个位置，从那里往后序列始终位于 a 的 ε 邻域内，也就是说，存在自然数 N ，使得对每个 $n \geq N$ ，都有 $|s_n - a| < \varepsilon$ ，则称该序列收敛到数 a 。在 Lean 中，可以把它写成如下形式：

```
def ConvergesTo (s : ℕ → ℝ) (a : ℝ) :=
  ∀ ε > 0, ∃ N, ∀ n ≥ N, |s n - a| < ε
```

记号 `∀ ε > 0, ...` 是 `∀ ε, ε > 0 → ...` 的便利缩写；类似地，`∀ n ≥ N, ...` 是 `∀ n, n ≥ N → ...` 的缩写。还要记住，`ε > 0` 又定义为 `0 < ε`，而 `n ≥ N` 定义为 `N ≤ n`。

本节将建立一些关于收敛的性质。但首先，我们讨论三个处理等式时有用的策略。第一个是 `ext` 策略，它提供了一种证明两个函数相等的方法。设 $f(x) = x + 1$ 和 $g(x) = 1 + x$ 是从实数到实数的函数。那么当然有 $f = g$ ，因为它们在每个 x 处返回相同的值。`ext` 策略使我们能够通过证明两个函数在所有参数值处取值相同，来证明函数之间的等式。

```
example : (fun x y : ℝ ↦ (x + y) ^ 2) = fun x y : ℝ ↦ x ^ 2 + 2 * x * y + y ^ 2 := by
  ext
  ring
```

稍后我们会看到，`ext` 实际上更加一般，并且可以指定出现的变量名。例如，你可以尝试把上面证明中的 `ext` 替换为 `ext u v`。第二个策略 `congr` 允许我们通过调和两个表达式中不同的部分来证明等式：

```
example (a b : ℝ) : |a| = |a - b + b| := by
  congr
  ring
```

这里，`congr` 策略剥去两边的 `abs`，留下要证明的目标 `a = a - b + b`。

最后，`convert` 策略用于在某个定理的结论与当前目标不完全匹配时，把该定理应用于目标。例如，假设我们想从 `1 < a` 证明 `a < a * a`。库中的一个定理 `mul_lt_mul_iff_left` 可让我们证明 `1 * a < a * a`。一种做法是反向工作，把目标重写为这种形式。相反，`convert` 策略允许我们按原样应用该定理，并留下证明若干等式的任务，这些等式正是使目标匹配所需的。

```
example {a : ℝ} (h : 1 < a) : a < a * a := by
  convert (mul_lt_mul_iff_left _).2 h
  · rw [one_mul]
  exact lt_trans zero_lt_one h
```

这个例子展示了另一个有用技巧：当我们应用一个带下划线的表达式而 Lean 无法自动填入该下划线时，它会把它简单地留下来，作为另一个目标交给我们。

下面证明任意常值序列 $a, a, a, \dots$ 都收敛。

```
theorem convergesTo_const (a : ℝ) : ConvergesTo (fun _x : ℕ → a) a := by
  intro ε εpos
  use 0
  intro n nge
  rw [sub_self, abs_zero]
  apply εpos
```

Lean 有一个策略 `simp`，它常常能省去手工执行类似 `rw [sub_self, abs_zero]` 步骤的麻烦。我们很快会进一步介绍它。

作为一个更有趣的定理，我们来证明：若 s 收敛到 a ，且 t 收敛到 b ，则 $\text{fun } n \mapsto s\ n + t\ n$ 收敛到 $a + b$ 。在开始写形式化证明之前，心中有一个清楚的纸笔证明会很有帮助。给定大于 0 的 ϵ ，思路是使用假设得到一个 N_s ，使得从该点以后， s 与 a 的距离小于 $\epsilon / 2$ ；再得到一个 N_t ，使得从该点以后， t 与 b 的距离小于 $\epsilon / 2$ 。于是，只要 n 大于等于 N_s 和 N_t 的最大值，序列 $\text{fun } n \mapsto s\ n + t\ n$ 就应当与 $a + b$ 的距离小于 ϵ 。下面的例子开始实现这个策略。看看你能否把它完成。

```
theorem convergesTo_add {s t : ℕ → ℝ} {a b : ℝ}
  (cs : ConvergesTo s a) (ct : ConvergesTo t b) :
  ConvergesTo (fun n ↦ s n + t n) (a + b) := by
  intro ε εpos
  dsimp -- this line is not needed but cleans up the goal a bit.
  have ε2pos : 0 < ε / 2 := by linarith
  rcases cs (ε / 2) ε2pos with (Ns, hs)
  rcases ct (ε / 2) ε2pos with (Nt, ht)
  use max Ns Nt
  sorry
```

作为提示，可以使用 `le_of_max_le_left` 和 `le_of_max_le_right`，而 `norm_num` 可以证明 $\epsilon / 2 + \epsilon / 2 = \epsilon$ 。此外，使用 `congr` 策略证明 $|s\ n + t\ n - (a + b)|$ 等于 $|(s\ n - a) + (t\ n - b)|$ 也很有帮助，因为这样就可以使用三角不等式。注意，我们把所有变量 s 、 t 、 a 和 b 都标记为隐式变量，因为它们可以由假设推断出来。

把同一个定理中的加法替换为乘法后，证明会更棘手。我们将先证明一些辅助命题。请看看能否也完成下一个证明，它表明若 s 收敛到 a ，则 $\text{fun } n \mapsto c * s\ n$ 收敛到 $c * a$ 。按 c 是否等于零分情况讨论会很有帮助。我们已经处理了零的情形，并留下了带有额外假设 c 非零时的结果让你证明。

```
theorem convergesTo_mul_const {s : ℕ → ℝ} {a : ℝ} (c : ℝ) (cs : ConvergesTo s a) :
  ConvergesTo (fun n ↦ c * s n) (c * a) := by
  by_cases h : c = 0
  · convert convergesTo_const 0
  · rw [h]
    ring
  rw [h]
  ring
  have acpos : 0 < |c| := abs_pos.mpr h
  sorry
```

下一个定理本身也很有意思：它表明收敛序列最终在绝对值意义下有界。我们已经给出开头；看看你能否完成它。

```
theorem exists_abs_le_of_convergesTo {s : ℕ → ℝ} {a : ℝ} (cs : ConvergesTo s a) :
  ∃ N b, ∀ n, N ≤ n → |s n| < b := by
  rcases cs 1 zero_lt_one with (N, h)
  use N, |a| + 1
  sorry
```

事实上，这个定理还可以加强为：存在一个界 b ，对所有 n 都成立。但当前版本已足以满足我们的需要，并且我们将在本节末尾看到它在更一般情形下也成立。

下一个引理是辅助性的：我们证明若 s 收敛到 a ，且 t 收敛到 0 ，则 $\text{fun } n \mapsto s\ n * t\ n$ 收敛到 0 。为此，我们使用前一个定理找到一个 B ，它在某个点 N_0 之后界住 s 。看看你能否理解我们勾勒出的策略并完成证明。


```

theorem aux {s t :  $\mathbb{N} \rightarrow \mathbb{R}$ } {a :  $\mathbb{R}$ } (cs : ConvergesTo s a) (ct : ConvergesTo t 0) :
  ConvergesTo (fun n  $\mapsto$  s n * t n) 0 := by
  intro  $\epsilon$   $\epsilon$ pos
  dsimp
  rcases exists_abs_le_of_convergesTo cs with  $\langle N_0, B, h_0 \rangle$ 
  have Bpos :  $0 < B$  := lt_of_le_of_lt (abs_nonneg _) ( $h_0$   $N_0$  (le_refl _))
  have pos_0 :  $\epsilon / B > 0$  := div_pos  $\epsilon$ pos Bpos
  rcases ct _ pos_0 with  $\langle N_1, h_1 \rangle$ 
  sorry

```

如果你已经做到这里，恭喜！我们距离目标定理已经很近了。下面的证明将完成它。

```

theorem convergesTo_mul {s t :  $\mathbb{N} \rightarrow \mathbb{R}$ } {a b :  $\mathbb{R}$ }
  (cs : ConvergesTo s a) (ct : ConvergesTo t b) :
  ConvergesTo (fun n  $\mapsto$  s n * t n) (a * b) := by
  have h1 : ConvergesTo (fun n  $\mapsto$  s n * (t n + -b)) 0 := by
    apply aux cs
    convert convergesTo_add ct (convergesTo_const (-b))
    ring
  have := convergesTo_add h1 (convergesTo_mul_const b cs)
  convert convergesTo_add h1 (convergesTo_mul_const b cs) using 1
  · ext; ring
  ring

```

作为另一个有挑战性的练习，请尝试补全下面的证明草稿，证明极限唯一。（如果你很有信心，可以删除证明草稿并尝试从零开始证明。）

```

theorem convergesTo_unique {s :  $\mathbb{N} \rightarrow \mathbb{R}$ } {a b :  $\mathbb{R}$ }
  (sa : ConvergesTo s a) (sb : ConvergesTo s b) :
  a = b := by
  by_contra abne
  have :  $|a - b| > 0$  := by sorry
  let  $\epsilon$  :=  $|a - b| / 2$ 
  have  $\epsilon$ pos :  $\epsilon > 0$  := by
    change  $|a - b| / 2 > 0$ 
    linarith
  rcases sa  $\epsilon$ pos with  $\langle N_a, hNa \rangle$ 
  rcases sb  $\epsilon$ pos with  $\langle N_b, hNb \rangle$ 
  let N := max Na Nb
  have absa :  $|s N - a| < \epsilon$  := by sorry
  have absb :  $|s N - b| < \epsilon$  := by sorry
  have :  $|a - b| < |a - b|$  := by sorry
  exact lt_irrefl _ this

```

本节最后指出，我们的证明可以推广。例如，我们对自然数所使用的唯一性质，是其结构带有偏序以及 `min` 和 `max`。可以检查，如果把各处的 `$\mathbb{N}$` 替换为任意线性序 `$\alpha$` ，一切仍然成立：

```

variable { $\alpha$  : Type*} [LinearOrder  $\alpha$ ]

def ConvergesTo' (s :  $\alpha \rightarrow \mathbb{R}$ ) (a :  $\mathbb{R}$ ) :=
   $\forall \epsilon > 0, \exists N, \forall n \geq N, |s n - a| < \epsilon$ 

```

在第 11.1 节中，我们会看到 Mathlib 有机制以远为一般的方式处理收敛；它不仅抽象掉定义域和陪域的具体特征，也抽象不同类型的收敛。

4. 集合与函数

集合、关系和函数的词汇，为数学各分支中的构造提供了一种统一语言。由于函数和关系可以用集合来定义，公理化集合论可以作为数学的基础。

Lean 的基础则建立在类型这一原始概念之上，并包含定义类型之间函数的方法。Lean 中每个表达式都有类型：有自然数、实数、从实数到实数的函数、群、向量空间等等。有些表达式本身是类型，也就是说，它们的类型是 `Type`。Lean 和 Mathlib 提供了定义新类型的方法，也提供了定义这些类型中对象的方法。

从概念上说，可以把类型看作对象的集合。要求每个对象都有类型具有若干优点。例如，它使得像 `+` 这样的记号可以重载；并且有时可以减少输入的冗长程度，因为 Lean 能从对象的类型推断出大量信息。类型系统还使 Lean 能够在你把函数应用于错误数量的参数，或把函数应用于错误类型的参数时指出错误。

Lean 的库确实定义了初等集合论概念。与集合论不同，在 Lean 中，集合总是某个类型对象的集合，例如自然数集合，或从实数到实数的函数集合。类型与集合之间的区别需要一些时间适应，但本章会带你掌握其要点。

4.1. 集合

若 α 是任意类型，则类型 `Set  $\alpha$` 由 α 的元素组成的集合构成。这个类型支持通常的集合论运算和关系。例如， `$s \subseteq t$` 表示 s 是 t 的子集， `$s \cap t$` 表示 s 与 t 的交， `$s \cup t$` 表示它们的并。子集关系可输入为 `\ss` 或 `\sub`，交可输入为 `\i` 或 `\cap`，并可输入为 `\un` 或 `\cup`。库还定义了集合 `univ`，它包含类型 α 的所有元素；以及空集 $\emptyset$ ，可输入为 `\empty`。给定 $x : \alpha$ 和 $s : \text{Set } \alpha$ ，表达式 `$x \in s$` 表示 x 是 s 的元素。提到集合成员关系的定理名中常包含 `mem`。表达式 `$x \notin s$` 是 `$\neg x \in s$` 的缩写。可以用 `\in` 或 `\mem` 输入 $\in$ ，用 `\notin` 输入 $\notin$ 。

证明关于集合的命题的一种方法，是使用 `rw` 或化简器展开定义。在下面第二个例子中，我们使用 `simp only`，告诉化简器只使用我们给出的恒等式列表，而不使用其完整恒等式数据库。不同于 `rw`，`simp` 可以在全称或存在量词内部进行化简。逐步查看证明时，可以看到这些命令的效果。

```
variable { $\alpha$  : Type*}
variable (s t u : Set  $\alpha$ )
open Set

example (h :  $s \subseteq t$ ) :  $s \cap u \subseteq t \cap u$  := by
  rw [subset_def, inter_def, inter_def]
  rw [subset_def] at h
  simp only [mem_setOf]
  rintro x ⟨xs, xu⟩
  exact ⟨h _ xs, xu⟩

example (h :  $s \subseteq t$ ) :  $s \cap u \subseteq t \cap u$  := by
  simp only [subset_def, mem_inter_iff] at *
  rintro x ⟨xs, xu⟩
  exact ⟨h _ xs, xu⟩
```

在这个例子中，我们打开 `Set` 命名空间，以便使用这些定理的较短名称。但事实上，可以完全删除对 `rw` 和 `simp` 的调用：

```
example (h :  $s \subseteq t$ ) :  $s \cap u \subseteq t \cap u$  := by
  intro x xsu
  exact ⟨h xsu.1, xsu.2⟩
```

这里发生的事情称为定义约简：为了理解 `intro` 命令和匿名构造子，Lean 被迫展开定义。下面的例子也说明了这一现象：

```
example (h : s ⊆ t) : s ∩ u ⊆ t ∩ u :=
  fun _x (xs, xu) ↦ ⟨h xs, xu⟩
```

处理并集时，可以使用 `Set.union_def` 和 `Set.mem_union`。由于 $x \in s \cup t$ 展开为 $x \in s \vee x \in t$ ，也可以使用 `cases` 策略迫使定义约简发生。

```
example : s ∩ (t ∪ u) ⊆ s ∩ t ∪ s ∩ u := by
  intro x hx
  have xs : x ∈ s := hx.1
  have xtu : x ∈ t ∪ u := hx.2
  rcases xtu with xt | xu
  · left
    show x ∈ s ∩ t
    exact ⟨xs, xt⟩
  · right
    show x ∈ s ∩ u
    exact ⟨xs, xu⟩
```

由于交的结合优先级高于并，表达式 $(s \cap t) \cup (s \cap u)$ 中的括号并非必要，但它们使表达式的含义更清楚。下面是同一事实的一个较短证明：

```
example : s ∩ (t ∪ u) ⊆ s ∩ t ∪ s ∩ u := by
  rintro x (xs, xt | xu)
  · left; exact ⟨xs, xt⟩
  · right; exact ⟨xs, xu⟩
```

作为练习，请尝试证明另一个包含关系：

```
example : s ∩ t ∪ s ∩ u ⊆ s ∩ (t ∪ u) := by
  sorry
```

也许有帮助的是：使用 `rintro` 时，有时需要给析取模式 `h1 | h2` 加括号，才能让 Lean 正确解析它。

库还定义了集合差 $s \setminus t$ ，其中反斜杠是输入为 `\\` 的特殊 Unicode 字符。表达式 $x \in s \setminus t$ 展开为 $x \in s \wedge x \notin t$ 。（`∉` 可输入为 `\notin`。）可以用 `Set.diff_eq` 和 `dsimp`，或用 `Set.mem_diff` 手动重写它；但下面两个证明同一包含关系的证明展示了如何避免使用它们。

```
example : (s \ t) \ u ⊆ s \ (t ∪ u) := by
  intro x xstu
  have xs : x ∈ s := xstu.1.1
  have xnt : x ∉ t := xstu.1.2
  have xnu : x ∉ u := xstu.2
  constructor
  · exact xs
  intro xtu
  -- x ∈ t ∨ x ∈ u
  rcases xtu with xt | xu
  · show False; exact xnt xt
  · show False; exact xnu xu

example : (s \ t) \ u ⊆ s \ (t ∪ u) := by
  rintro x (⟨xs, xnt⟩, xnu)
  use xs
  rintro (xt | xu) <|> contradiction
```

作为练习，请证明反向包含：

```
example : s \ (t ∪ u) ⊆ (s \ t) \ u := by
  sorry
```

要证明两个集合相等，只需证明一个集合的每个元素都是另一个集合的元素。这个原则称为“外延性”；不出意料，`ext` 策略正是为处理它而配备的。

```
example : s n t = t n s := by
  ext x
  simp only [mem_inter_iff]
  constructor
  · rintro ⟨xs, xt⟩; exact ⟨xt, xs⟩
  · rintro ⟨xt, xs⟩; exact ⟨xs, xt⟩
```

同样，删除 `simp only [mem_inter_iff]` 这一行并不会破坏证明。事实上，如果你喜欢难以读懂的证明项，下面这个单行证明就是为你准备的：

```
example : s n t = t n s :=
  Set.ext fun _x ↦ ⟨fun ⟨xs, xt⟩ ↦ ⟨xt, xs⟩, fun ⟨xt, xs⟩ ↦ ⟨xs, xt⟩⟩
```

下面是一个使用化简器的更短证明：

```
example : s n t = t n s := by ext x; simp [and_comm]
```

使用 `ext` 的另一种选择，是使用定理 `Subset.antisymm`；它允许我们通过证明 `s ⊆ t` 和 `t ⊆ s`，来证明集合等式 `s = t`。

```
example : s n t = t n s := by
  apply Subset.antisymm
  · rintro x ⟨xs, xt⟩; exact ⟨xt, xs⟩
  · rintro x ⟨xt, xs⟩; exact ⟨xs, xt⟩
```

请尝试完成这个证明项：

```
example : s n t = t n s :=
  Subset.antisymm sorry sorry
```

记住，可以把 `sorry` 替换为下划线；当你把鼠标悬停在它上面时，Lean 会显示此处期望什么。

下面是一些你或许会乐于证明的集合论恒等式：

```
example : s n (s ∪ t) = s := by
  sorry

example : s ∪ s n t = s := by
  sorry

example : s \ t ∪ t = s ∪ t := by
  sorry

example : s \ t ∪ t \ s = (s ∪ t) \ (s n t) := by
  sorry
```

关于集合的表示，其底层机制如下。在类型论中，类型 `α` 上的性质或谓词只是函数 `P : α → Prop`。这很自然：给定 `a : α`，`P a` 正是断言 `P` 对 `a` 成立的命题。在库中，`Set α` 定义为 `α → Prop`，而 `x ∈ s` 定义为 `s x`。换句话说，集合其实就是性质，只是被当作对象来处理。

库还定义了集合构造式记号。表达式 `{ y | P y }` 展开为 `(fun y ↦ P y)`，因此 `x ∈ { y | P y }` 约简为 `P x`。因此，我们可以把“为偶数”这一性质转换为偶数集合：

```
def evens : Set ℕ :=
  { n | Even n }

def odds : Set ℕ :=
  { n | ¬Even n }

example : evens ∪ odds = univ := by
  rw [evens, odds]
  ext n
  simp [-Nat.not_even_iff_odd]
  apply Classical.em
```

你应当逐步查看这个证明，确保理解发生了什么。注意，我们告诉化简器不要使用引理 `Nat.not_even_iff`，因为希望在目标中保留 `¬ Even n`。请尝试删除 `rw [evens, odds]` 这一行，并确认证明仍然成立。

事实上，集合构造式记号用于定义

`s n t` 定义为 $\{x \mid x \in s \wedge x \in t\}$ ，
`s U t` 定义为 $\{x \mid x \in s \vee x \in t\}$ ，
`∅` 定义为 $\{x \mid \text{False}\}$ ，并且
`univ` 定义为 $\{x \mid \text{True}\}$ 。

我们常常需要显式指出 `∅` 和 `univ` 的类型，因为 Lean 很难猜出我们指的是哪一个。下面的例子展示了必要时 Lean 如何展开最后两个定义。在第二个例子中，`trivial` 是库中 `True` 的标准证明。

```
example (x : ℕ) (h : x ∈ (∅ : Set ℕ)) : False :=
  h

example (x : ℕ) : x ∈ (univ : Set ℕ) :=
  trivial
```

作为练习，请证明下面的包含关系。使用 `intro n` 展开子集定义，并使用化简器把集合论构造化归为逻辑。我们还建议使用定理 `Nat.Prime.eq_two_or_odd` 和 `Nat.odd_iff`。

```
example : { n | Nat.Prime n } ∩ { n | n > 2 } ⊆ { n | ¬Even n } := by
  sorry
```

请小心：库中有多个版本的谓词 `Prime`，这有些容易混淆。最一般的版本在任意带零元的交换幺半群中都有意义。谓词 `Nat.Prime` 则专门用于自然数。幸运的是，有一个定理说明在这个特定情形下二者一致，因此总可以把一个重写为另一个。

```
#print Prime

#print Nat.Prime

example (n : ℕ) : Prime n ↔ Nat.Prime n :=
  Nat.prime_iff.symm

example (n : ℕ) (h : Prime n) : Nat.Prime n := by
  rw [Nat.prime_iff]
  exact h
```

`rwa` 策略先执行重写，然后执行 `assumption` 策略。

```
example (n : ℕ) (h : Prime n) : Nat.Prime n := by
  rwa [Nat.prime_iff]
```

Lean 引入记号 $\forall x \in s, \dots$ ，读作“对 `s` 中每个 `x`”，作为 $\forall x, x \in s \rightarrow \dots$ 的缩写。它也引入记号 $\exists x \in s, \dots$ ，读作“存在 `s` 中的 `x` 使得.....”。这些有时称为**有界量词**，因为这种构造把量词的意义限制到集合 `s` 上。因此，库中使用它们的定理名常包含 `ball` 或 `bex`。定理 `bex_def` 断言 $\exists x \in s, \dots$ 等价于 $\exists x, x \in s \wedge \dots$ ；但当它们与 `rintro`、`use` 和匿名构造子一起使用时，这两个表达式行为大致相同。因此，我们通常不需要显式用 `bex_def` 变换它们。下面是一些用法示例：

```

variable (s t : Set ℕ)

example (h₀ : ∀ x ∈ s, ¬Even x) (h₁ : ∀ x ∈ s, Prime x) : ∀ x ∈ s, ¬Even x ∧ Prime x := by
  intro x xs
  constructor
  · apply h₀ x xs
  apply h₁ x xs

example (h : ∃ x ∈ s, ¬Even x ∧ Prime x) : ∃ x ∈ s, Prime x := by
  rcases h with ⟨x, xs, _, prime_x⟩
  use x, xs

```

看看能否证明下面这些稍作变化的命题：

```

section
variable (ssubst : s ⊆ t)

example (h₀ : ∀ x ∈ t, ¬Even x) (h₁ : ∀ x ∈ t, Prime x) : ∀ x ∈ s, ¬Even x ∧ Prime x := by
  sorry

example (h : ∃ x ∈ s, ¬Even x ∧ Prime x) : ∃ x ∈ t, Prime x := by
  sorry

end

```

指标化并集和交集是另一类重要的集合论构造。可以把 α 的元素集合所成的序列 $A_0, A_1, A_2, \dots$ 表示为函数

$A : \mathbb{N} \rightarrow \text{Set } \alpha$ ；此时 $\bigcup i, A i$ 表示它们的并， $\bigcap i, A i$ 表示它们的交。这里自然数没有特殊之处， $\mathbb{N}$ 可以替换为任意用于给集合编号的类型 I 。下面展示它们的用法。

```

variable {α I : Type*}
variable (A B : I → Set α)
variable (s : Set α)

open Set

example : (s ∩ ⋂ i, A i) = ⋂ i, A i ∩ s := by
  ext x
  simp only [mem_inter_iff, mem_iUnion]
  constructor
  · rintro ⟨xs, ⟨i, xAi⟩⟩
    exact ⟨i, xAi, xs⟩
  rintro ⟨i, xAi, xs⟩
  exact ⟨xs, ⟨i, xAi⟩⟩

example : (⋂ i, A i ∩ B i) = (⋂ i, A i) ∩ ⋂ i, B i := by
  ext x
  simp only [mem_inter_iff, mem_iInter]
  constructor
  · intro h
    constructor
    · intro i
      exact (h i).1
    intro i
      exact (h i).2
  rintro ⟨h1, h2⟩ i
  constructor
  · exact h1 i
  exact h2 i

```

指标化并集或交集经常需要括号，因为和量词一样，绑定变量的作用域会尽可能向右延伸。

请尝试证明下面的恒等式。其中一个方向需要经典逻辑！我们建议在证明的适当位置使用 `by_cases xs : x ∈ s`。

```

example : (s ∪ ⋂ i, A i) = ⋂ i, A i ∪ s := by
  sorry

```

Mathlib 也有有界并集和有界交集，它们类似于有界量词。可以用 `mem_iUnion₂` 和 `mem_iInter₂` 展开其含义。如下例所示，Lean 的化简器也会执行这些替换。

```

def primes : Set ℕ :=
  { x | Nat.Prime x }

example : (⋃ p ∈ primes, { x | p ^ 2 ∣ x }) = { x | ∃ p ∈ primes, p ^ 2 ∣ x } := by
  ext
  rw [mem_iUnion₂]
  simp

example : (⋃ p ∈ primes, { x | p ^ 2 ∣ x }) = { x | ∃ p ∈ primes, p ^ 2 ∣ x } := by
  ext
  simp

example : (⋂ p ∈ primes, { x | ¬p ∣ x }) ⊆ { x | x = 1 } := by
  intro x
  contrapose!
  simp
  apply Nat.exists_prime_and_dvd

```

请尝试解决下面这个类似的例子。如果开始输入 `eq_univ`，Tab 补全会提示 `apply eq_univ_of_forall` 是一个好的证明开头。我们还建议使用定理 `Nat.exists_infinite_primes`。

```

example : (⋃ p ∈ primes, { x | x ≤ p }) = univ := by
  sorry

```

给定集合族 $s : \text{Set} (\text{Set } \alpha)$ ，它们的并 $\bigcup_0 s$ 的类型是 $\text{Set } \alpha$ ，并定义为 $\{x \mid \exists t \in s, x \in t\}$ 。类似地，它们的交 $\bigcap_0 s$ 定义为 $\{x \mid \forall t \in s, x \in t\}$ 。这些运算分别称为 `sUnion` 和 `sInter`。下面的例子展示它们与有界并集和有界交集的关系。

```

variable {α : Type*} (s : Set (Set α))

example : ⋃_0 s = ⋃ t ∈ s, t := by
  ext x
  rw [mem_iUnion₂]
  simp

example : ⋂_0 s = ⋂ t ∈ s, t := by
  ext x
  rw [mem_iInter₂]
  rfl

```

在库中，这些恒等式分别称为 `sUnion_eq_biUnion` 和 `sInter_eq_biInter`。

4.2. 函数

若 $f : \alpha \rightarrow \beta$ 是函数，且 p 是类型 β 的元素组成的集合，则库定义 `preimage f p`，记作 $f^{-1} p$ ，为 $\{x \mid f x \in p\}$ 。表达式 $x \in f^{-1} p$ 约简为 $f x \in p$ 。这常常很方便，如下面的例子所示：

```

variable {α β : Type*}
variable (f : α → β)
variable (s t : Set α)
variable (u v : Set β)

open Function
open Set

example : f ⁻¹' (u ∩ v) = f ⁻¹' u ∩ f ⁻¹' v := by
  ext
  rfl

```

若 s 是类型 α 的元素组成的集合，库还定义 `image f s`，记作 $f '' s$ ，为 $\{y \mid \exists x, x \in s \wedge f x = y\}$ 。因此，假设 $y \in f '' s$ 会分解为三元组 $\langle x, xs, xeq \rangle$ ，其中 $x : \alpha$ 满足假设 $xs : x \in s$ 和 $xeq : f x = y$ 。策略中的 `rfl` 标记（见第 3.2 节）正是为这类情形准备的。

```
example : f '' (s ∪ t) = f '' s ∪ f '' t := by
  ext y; constructor
  · rintro ⟨x, xs | xt, rfl⟩
    · left
      use x, xs
    right
      use x, xt
  rintro ⟨(x, xs, rfl) | (x, xt, rfl)⟩
  · use x, Or.inl xs
  use x, Or.inr xt
```

还要注意，`use` 策略会在可能时应用 `rfl` 来关闭目标。

下面还有一个例子：

```
example : s ⊆ f ⁻¹ (f '' s) := by
  intro x xs
  show f x ∈ f '' s
  use x, xs
```

如果想使用专门为此目的设计的定理，可以把 `use x, xs` 这一行替换为 `apply mem_image_of_mem f xs`。不过，知道像集是以存在量词定义的，往往很方便。

下面这个等价命题是一个很好的练习：

```
example : f '' s ⊆ v ↔ s ⊆ f ⁻¹ v := by
  sorry
```

它表明 `image f` 和 `preimage f` 构成了 `Set α` 与 `Set β` 之间所谓 Galois 连接的一个实例，其中二者都按子集关系部分有序。在库中，这个等价命题名为 `image_subset_iff`。实践中，右侧通常是更有用的表示，因为 `x ∈ f ⁻¹ t` 会展开为 `f x ∈ t`，而处理 `y ∈ f '' s` 则需要拆开一个存在量词。

下面是一长串集合论恒等式，供你享用。不必一次全部完成；先做其中几个，其余可以留到有空时再做。


```

example (h : Injective f) : f ⁻¹' (f '' s) ⊆ s := by
  sorry

example : f '' (f ⁻¹' u) ⊆ u := by
  sorry

example (h : Surjective f) : u ⊆ f '' (f ⁻¹' u) := by
  sorry

example (h : s ⊆ t) : f '' s ⊆ f '' t := by
  sorry

example (h : u ⊆ v) : f ⁻¹' u ⊆ f ⁻¹' v := by
  sorry

example : f ⁻¹' (u ∪ v) = f ⁻¹' u ∪ f ⁻¹' v := by
  sorry

example : f '' (s ∩ t) ⊆ f '' s ∩ f '' t := by
  sorry

example (h : Injective f) : f '' s ∩ f '' t ⊆ f '' (s ∩ t) := by
  sorry

example : f '' s \ f '' t ⊆ f '' (s \ t) := by
  sorry

example : f ⁻¹' u \ f ⁻¹' v ⊆ f ⁻¹' (u \ v) := by
  sorry

example : f '' s ∩ v = f '' (s ∩ f ⁻¹' v) := by
  sorry

example : f '' (s ∩ f ⁻¹' u) ⊆ f '' s ∩ u := by
  sorry

example : s ∩ f ⁻¹' u ⊆ f ⁻¹' (f '' s ∩ u) := by
  sorry

example : s ∪ f ⁻¹' u ⊆ f ⁻¹' (f '' s ∪ u) := by
  sorry

```

也可以尝试下一组练习，它们刻画像和原像关于指标化并集与交集的行为。在第三个练习中，需要参数 `i : I` 来保证指标集非空。证明这些命题时，我们建议使用 `ext` 或 `intro` 展开集合之间等式或包含关系的含义，然后调用 `simp` 来拆解成员关系的条件。

```

variable {I : Type*} (A : I → Set α) (B : I → Set β)

example : (f '' ∪ i, A i) = ∪ i, f '' A i := by
  sorry

example : (f '' ∩ i, A i) ⊆ ∩ i, f '' A i := by
  sorry

example (i : I) (inj f : Injective f) : (∩ i, f '' A i) ⊆ f '' ∩ i, A i := by
  sorry

example : (f ⁻¹' ∪ i, B i) = ∪ i, f ⁻¹' B i := by
  sorry

example : (f ⁻¹' ∩ i, B i) = ∩ i, f ⁻¹' B i := by
  sorry

```

库定义谓词 `InjOn f s`，表示 `f` 在 `s` 上是单射。它定义如下：

```

example : InjOn f s ↔ ∀ x₁ ∈ s, ∀ x₂ ∈ s, f x₁ = f x₂ → x₁ = x₂ :=
  Iff.refl _

```

命题 `Injective f` 可证明地等价于 `InjOn f univ`。类似地，库把 `range f` 定义为 `{x | ∃ y, f y = x}`，因此 `range f` 可证明地等于 `f '' univ`。这是 Mathlib 中的一个常见主题：虽然函数的许多性质是相对于其完整定义域定义的，但常常也有把这些陈述限制到定义域类型的某个子集上的相对化版本。

下面是使用 `InjOn` 和 `range` 的一些例子：

```
open Set Real

example : InjOn log { x | x > 0 } := by
  intro x xpos y ypos e
  calc
    x = exp (log x) := by rw [exp_log xpos]
    _ = exp (log y) := by rw [e]
    _ = y := by rw [exp_log ypos]

example : range exp = { y | y > 0 } := by
  ext y; constructor
  · rintro ⟨x, rfl⟩
    apply exp_pos
  intro ypos
  use log y
  rw [exp_log ypos]
```

请尝试证明这些命题：

```
example : InjOn sqrt { x | x ≥ 0 } := by
  sorry

example : InjOn (fun x ↦ x ^ 2) { x : ℝ | x ≥ 0 } := by
  sorry

example : sqrt '' { x | x ≥ 0 } = { y | y ≥ 0 } := by
  sorry

example : (range fun x ↦ x ^ 2) = { y : ℝ | y ≥ 0 } := by
  sorry
```

要定义函数 $f : \alpha \rightarrow \beta$ 的逆，我们将使用两个新成分。首先，需要处理 Lean 中任意类型可能为空这一事实。当不存在满足 $f x = y$ 的 x 时，要定义 f 在 y 处的逆值，我们希望给 α 中指定一个默认值。把标注 `[Inhabited  $\alpha$ ]` 作为变量加入，就等同于假定 α 有一个首选元素，记作 `default`。其次，在有多多个 x 满足 $f x = y$ 的情况下，逆函数需要从中选择一个。这需要诉诸选择公理。Lean 允许以多种方式使用它；一种方便方法是使用经典的 `choose` 运算符，如下所示。

```
variable {α β : Type*} [Inhabited α]

#check (default : α)

variable (P : α → Prop) (h : ∃ x, P x)

#check Classical.choose h

example : P (Classical.choose h) :=
  Classical.choose_spec h
```

给定 $h : \exists x, P x$ ，值 `Classical.choose h` 是某个满足 $P x$ 的 x 。定理 `Classical.choose_spec h` 说明 `Classical.choose h` 满足这个规范。

有了这些准备，就可以如下定义逆函数：

```
noncomputable section

open Classical

def inverse (f : α → β) : β → α := fun y : β ↦
  if h : ∃ x, f x = y then Classical.choose h else default

theorem inverse_spec {f : α → β} (y : β) (h : ∃ x, f x = y) : f (inverse f y) = y := by
  rw [inverse, dif_pos h]
  exact Classical.choose_spec h
```

这里需要 `noncomputable section` 和 `open Classical` 这两行，因为我们在本质上使用了经典逻辑。输入 y 时，函数 `inverse f` 在存在满足 $f x = y$ 的 x 时返回某个这样的 x ，否则返回 α 的默认元素。这是依赖式 `if` 构造的一个实例，因为在肯定情形下，返回值 `Classical.choose h` 依赖于假设 h 。恒等式 `dif_pos h` 在给定 $h : e$ 时，把

`if h : e then a else b` 重写为 `a`；类似地，`dif_neg h` 在给定 `h : ¬ e` 时把它重写为 `b`。还有适用于非依赖式 `if` 构造的版本 `if_pos` 和 `if_neg`，下一节会用到。定理 `inverse_spec` 说明 `inverse f` 满足该规范的第一部分。

如果没有完全理解这些机制，也不必担心。仅定理 `inverse_spec` 就应足以说明：`inverse f` 是左逆当且仅当 `f` 是单射，是右逆当且仅当 `f` 是满射。请在 VS Code 中双击或右击 `LeftInverse` 和 `RightInverse` 来查看它们的定义，或使用命令 `#print LeftInverse` 和 `#print RightInverse`。然后尝试证明两个定理。它们很有挑战！开始处理细节前，先在纸上完成证明会有帮助。每个证明大约可用半打短行完成。如果想额外挑战自己，可以尝试把每个证明压缩成一个单行证明项。

```
variable (f :  $\alpha \rightarrow \beta$ )

open Function

example : Injective f  $\leftrightarrow$  LeftInverse (inverse f) f :=
  sorry

example : Surjective f  $\leftrightarrow$  RightInverse (inverse f) f :=
  sorry
```

本节最后给出 Cantor 著名定理的类型论陈述：不存在从一个集合并到其幂集的满射函数。看看你能否理解这个证明，然后补全缺失的两行。

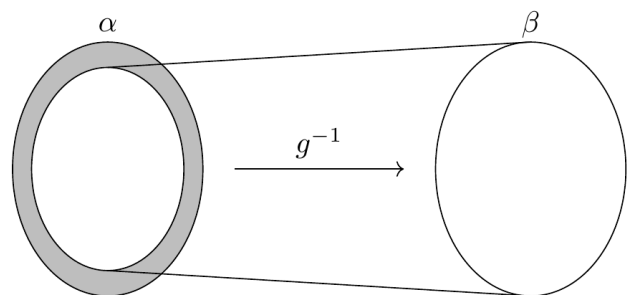
```
theorem Cantor :  $\forall f : \alpha \rightarrow \text{Set } \alpha, \neg \text{Surjective } f := \text{by}$ 
  intro f surjf
  let S := { i | i  $\notin$  f i }
  rcases surjf S with ⟨j, h⟩
  have h1 : j  $\notin$  f j := by
    intro h'
    have : j  $\notin$  f j := by rwa [h] at h'
    contradiction
  have h2 : j  $\in$  S
  sorry
  have h3 : j  $\notin$  S
  sorry
  contradiction
```

4.3. 施罗德-伯恩斯坦定理

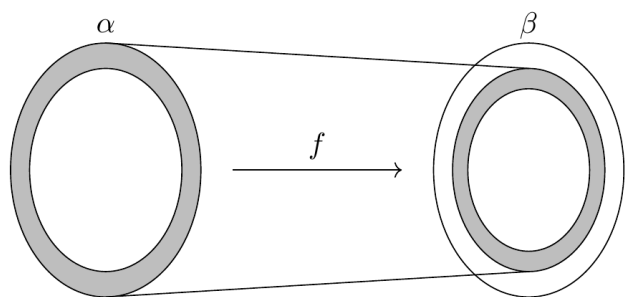
本章最后以集合论中的一个初等但非平凡的定理收尾。设 α 和 β 为集合。（在我们的形式化中，它们实际上是类型。）假设 $f : \alpha \rightarrow \beta$ 和 $g : \beta \rightarrow \alpha$ 都是单射。直观地说，这意味着 α 不大于 β ，反之亦然。如果 α 和 β 有限，这就蕴含它们有相同基数，等价地说，二者之间存在一个双射。十九世纪时，Cantor 陈述了即使在 α 和 β 无限时同样成立的结果。后来 Dedekind、Schröder 和 Bernstein 分别独立证明了它。

我们的形式化将引入一些新方法，后续章节会更详细解释。如果这里讲得太快，也不必担心。我们的目标是让你看到，你已经具备为真实数学结果的形式化证明作出贡献的技能。

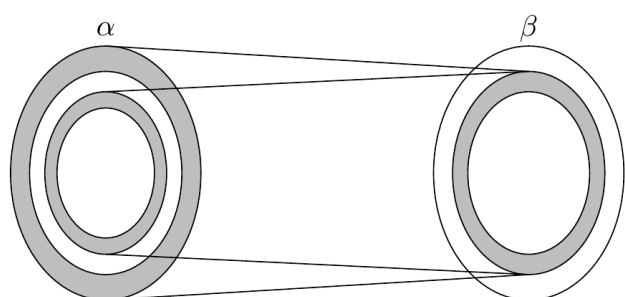
为了理解证明背后的思想，考虑映射 g 在 α 中的像。在这个像上， g 的逆是有定义的，并且给出它与 β 的一个双射。



问题在于，如果 g 不是满射，这个双射没有覆盖图中阴影区域；该区域非空。另一种做法是用 f 把整个 α 映到 β ，但那样的问题是，如果 f 不是满射，就会遗漏 β 中的一些元素。



现在考虑从 α 到自身的复合 $g \circ f$ 。因为该复合是单射，它在 α 与其像之间形成一个双射，从而在 α 内部得到一个缩小的 α 副本。



这个复合把内侧阴影环映到另一个同类集合，我们可以把它看作更小的同心阴影环，如此继续。于是得到一个同心的阴影环序列，每个环都与下一个环双射对应。如果把每个环映到下一个环，并保持 α 的非阴影部分不变，就得到 α 与 g 的像之间的一个双射。再与 g^{-1} 复合，就得到所需的 α 与 β 之间的双射。

可以更简单地描述这个双射。设 A 为阴影区域序列的并，并如下定义 $h : \alpha \rightarrow \beta$ ：

$$h(x) = \begin{cases} f(x) & \text{if } x \in A \\ g^{-1}(x) & \text{otherwise.} \end{cases}$$

换言之，在阴影部分使用 f ，在其他地方使用 g 的逆。所得映射 h 是单射，因为每个组成部分都是单射，且两个组成部分的像互不相交。要看它是满射，设给定 β 中的 y ，并考虑 $g(y)$ 。若 $g(y)$ 位于某个阴影区域，则它不能在第一个环中，所以有 $g(y) = g(f(x))$ ，其中 x 位于前一个环中。由 g 的单射性，得到 $h(x) = f(x) = y$ 。若 $g(y)$ 不在阴影区域，则由 h 的定义有 $h(g(y)) = y$ 。无论哪种情况， y 都在 h 的像中。

这个论证听起来应当是可信的，但细节相当微妙。形式化证明不仅会增强我们对结果的信心，也会帮助我们更好地理解它。由于证明使用经典逻辑，我们告诉 Lean：我们的定义通常不是可计算的。

```
noncomputable section
open Classical
variable {α β : Type*} [Nonempty β]
```

标注 `[Nonempty β]` 指明 β 非空。我们使用它，是因为将来用来构造 g^{-1} 的 Mathlib 原语需要这一点。当 β 为空时，定理的情形是平凡的；虽然把形式化推广到涵盖该情形并不困难，但我们不费这个功夫。具体地说，我们需要假设

`[Nonempty β]` 来使用 Mathlib 中定义的运算 `invFun`。给定 `x : α`，`invFun g x` 在存在 `x` 在 β 中的原像时选择一个

这样的原像，否则返回 β 的任意元素。若 g 是单射，则函数 `invFun g` 总是一个左逆；若 g 是满射，则它总是一个右逆。

```
#check (invFun g :  $\alpha \rightarrow \beta$ )
#check (leftInverse_invFun : Injective g  $\rightarrow$  LeftInverse (invFun g) g)
#check (leftInverse_invFun : Injective g  $\rightarrow \forall y, \text{invFun } g (g y) = y$ )
#check (invFun_eq : ( $\exists y, g y = x$ )  $\rightarrow g (\text{invFun } g x) = x$ )
```

我们如下定义与阴影区域之并对应的集合。

```
variable (f :  $\alpha \rightarrow \beta$ ) (g :  $\beta \rightarrow \alpha$ )

def sbAux :  $\mathbb{N} \rightarrow \text{Set } \alpha$ 
| 0 => univ \ g '' univ
| n + 1 => g '' (f '' sbAux n)

def sbSet :=
   $\bigcup n, \text{sbAux } f g n$ 
```

定义 `sbAux` 是递归定义的一个例子，我们将在下一章解释。它定义了一个集合序列

$$S_0 = \alpha \setminus g(\beta)$$

$$S_{n+1} = g(f(S_n)).$$

定义 `sbSet` 对应证明草图中的集合 $A = \bigcup_{n \in \mathbb{N}} S_n$ 。上面描述的函数 h 现在定义如下：

```
def sbFun (x :  $\alpha$ ) :  $\beta$  :=
  if x  $\in$  sbSet f g then f x else invFun g x
```

我们将需要这样一个事实：我们对 g^{-1} 的定义，在 A 的补集上是右逆，也就是说，在 α 的非阴影区域上是右逆。这是因为最外侧环 S_0 等于 $\alpha \setminus g(\beta)$ ，所以 A 的补集包含于 $g(\beta)$ 。因此，对补集中的每个 x ，都存在 y 使得 $g(y) = x$ 。（由 g 的单射性，这个 y 是唯一的；但下一个定理只说明 `invFun g x` 返回某个满足 $g y = x$ 的 y 。）

请逐步查看下面的证明，确保理解其中发生了什么，并补全剩余部分。最后需要使用 `invFun_eq`。注意，这里用 `sbAux` 重写，会把 `sbAux f g 0` 替换为相应定义方程的右端。

```
theorem sb_right_inv {x :  $\alpha$ } (hx : x  $\notin$  sbSet f g) : g (invFun g x) = x := by
  have : x  $\in$  g '' univ := by
    contrapose! hx
    rw [sbSet, mem_iUnion]
    use 0
    rw [sbAux, mem_diff]
  sorry
  have :  $\exists y, g y = x$  := by
    sorry
  sorry
```

现在转向证明 h 是单射。非形式地说，证明如下。首先，假设 $h(x_1) = h(x_2)$ 。若 x_1 在 A 中，则 $h(x_1) = f(x_1)$ ，并且可以如下证明 x_2 也在 A 中。若它不在，则 $h(x_2) = g^{-1}(x_2)$ 。由 $f(x_1) = h(x_1) = h(x_2)$ 得到 $g(f(x_1)) = x_2$ 。根据 A 的定义，由于 x_1 在 A 中， x_2 也在 A 中，矛盾。因此，若 x_1 在 A 中，则 x_2 也在；此时有 $f(x_1) = h(x_1) = h(x_2) = f(x_2)$ 。由 f 的单射性推出 $x_1 = x_2$ 。对称论证表明，若 x_2 在 A 中，则 x_1 也在，从而同样推出 $x_1 = x_2$ 。

唯一剩下的可能性是 x_1 与 x_2 都不在 A 中。在这种情况下，有 $g^{-1}(x_1) = h(x_1) = h(x_2) = g^{-1}(x_2)$ 。对等式两边应用 g ，得到 $x_1 = x_2$ 。

我们再次鼓励你逐步查看下面的证明，看看这个论证如何在 Lean 中展开。看看能否使用 `sb_right_inv` 完成证明。

```
theorem sb_injective (hf : Injective f) : Injective (sbFun f g) := by
  set A := sbSet f g with A_def
  set h := sbFun f g with h_def
  intro x₁ x₂ (hxeq : h x₁ = h x₂)
  show x₁ = x₂
  simp only [h_def, sbFun, ← A_def] at hxeq
  by_cases xA : x₁ ∈ A ∨ x₂ ∈ A
  · wlog x₁A : x₁ ∈ A generalizing x₁ x₂ hxeq xA
    · symm
      apply this hxeq.symm xA.symm (xA.resolve_left x₁A)
  have x₂A : x₂ ∈ A := by
    apply _root_.not_imp_self.mp
    intro (x₂nA : x₂ ∉ A)
    rw [if_pos x₁A, if_neg x₂nA] at hxeq
    rw [A_def, sbSet, mem_iUnion] at x₁A
  have x₂eq : x₂ = g (f x₁) := by
    sorry
  rcases x₁A with ⟨n, hn⟩
  rw [A_def, sbSet, mem_iUnion]
  use n + 1
  simp [sbAux]
  exact ⟨x₁, hn, x₂eq.symm⟩
sorry
push_neg at xA
sorry
```

这个证明引入了一些新策略。首先，注意 `set` 策略，它为 `sbSet f g` 和 `sb_fun f g` 分别引入缩写 `A` 和 `h`。我们把相应的定义方程命名为 `A_def` 和 `h_def`。这些缩写是定义性的，也就是说，必要时 Lean 有时会自动展开它们。但并非总是如此；例如使用 `rw` 时，通常需要显式使用 `A_def` 和 `h_def`。因此，这些定义带来一种权衡：它们可以使表达式更短、更可读，但有时也要求我们做更多工作。

更有趣的策略是 `wlog`，它封装了上面非形式证明中的对称性论证。我们现在不详细讨论它，但请注意它恰好完成了我们想要的事情。若把鼠标悬停在该策略上，可以查看它的文档。

满射性的证明更容易。给定 β 中的 y ，我们根据 $g(y)$ 是否在 A 中分两种情况。如果在，它不可能位于最外侧环 S_0 ，因为按定义该环与 g 的像不相交。因此，它对某个 n 是 S_{n+1} 的元素。这意味着它形如 $g(f(x))$ ，其中 x 在 S_n 中。由 g 的单射性，得到 $f(x) = y$ 。若 $g(y)$ 在 A 的补集中，则立即有 $h(g(y)) = y$ ，证明完成。

我们再次鼓励你逐步查看证明并补全缺失部分。策略 `rcases n with _ | n` 会把情况分为 `g y ∈ sbAux f g 0` 和 `g y ∈ sbAux f g (n + 1)`。在两种情形中，调用 `simp [sbAux]` 都会应用 `sbAux` 的相应定义方程。

```
theorem sb_surjective (hg : Injective g) : Surjective (sbFun f g) := by
  set A := sbSet f g with A_def
  set h := sbFun f g with h_def
  intro y
  by_cases gyA : g y ∈ A
  · rw [A_def, sbSet, mem_iUnion] at gyA
    rcases gyA with ⟨n, hn⟩
    rcases n with _ | n
    · simp [sbAux] at hn
      simp [sbAux] at hn
      rcases hn with ⟨x, xmem, hx⟩
      use x
      have : x ∈ A := by
        rw [A_def, sbSet, mem_iUnion]
        exact ⟨n, xmem⟩
      rw [h_def, sbFun, if_pos this]
      apply hg hx
    sorry
```

现在可以把一切组合起来。最终陈述简短漂亮，而证明使用了 `Bijjective h` 展开为 `Injective h ∧ Surjective h` 这一事实。

```
theorem schroeder_bernstein {f :  $\alpha \rightarrow \beta$ } {g :  $\beta \rightarrow \alpha$ } (hf : Injective f) (hg : Injective g) :  
   $\exists$  h :  $\alpha \rightarrow \beta$ , Bijective h :=  
  ⟨sbFun f g, sb_injective f g hf, sb_surjective f g hg⟩
```

5. 初等数论

本章说明如何形式化数论中的一些初等结果。随着我们处理更有实质性的数学内容，证明会变得更长、更复杂，并建立在已经掌握的技能之上。

5.1. 无理根

先从古希腊人已知的一个事实开始：2 的平方根是无理数。若反设不然，则可以把 $\sqrt{2} = a/b$ 写成既约分数。两边平方得到 $a^2 = 2b^2$ ，这推出 a 是偶数。若写 $a = 2c$ ，则得到 $4c^2 = 2b^2$ ，从而 $b^2 = 2c^2$ 。这又推出 b 也是偶数，与我们假定 a/b 已经约成既约分数相矛盾。

说 a/b 是既约分数，意味着 a 与 b 没有公共因子，也就是说它们互素。Mathlib 把谓词 `Nat.Coprime m n` 定义为

`Nat.gcd m n = 1`。使用 Lean 的匿名投影记法，若 `s` 和 `t` 是类型 `Nat` 的表达式，就可以写 `s.Coprime t` 代替 `Nat.Coprime s t`；`Nat.gcd` 也类似。照常，在必要时 Lean 常会自动展开 `Nat.Coprime` 的定义，但我们也可以用标识符 `Nat.Coprime` 手动重写或化简。`norm_num` 策略足够聪明，可以计算具体数值。

```
#print Nat.Coprime

example (m n : Nat) (h : m.Coprime n) : m.gcd n = 1 :=
  h

example (m n : Nat) (h : m.Coprime n) : m.gcd n = 1 := by
  rw [Nat.Coprime] at h
  exact h

example : Nat.Coprime 12 7 := by norm_num

example : Nat.gcd 12 8 = 4 := by norm_num
```

我们已经在第 2.4 节中遇到过 `gcd` 函数。整数上也有一个 `gcd` 版本；稍后我们会回到不同数系之间关系的讨论。甚至还有一般的 `gcd` 函数，以及在一般代数结构类中有意义的一般 `Prime` 和 `Coprime` 概念。下一章我们将理解 Lean 如何管理这种一般性。在此之前，本节只关注自然数。

我们还需要素数概念 `Nat.Prime`。定理 `Nat.prime_def_lt` 给出一个熟悉的刻画，而 `Nat.Prime.eq_one_or_self_of_dvd` 给出另一个刻画。

```
#check Nat.prime_def_lt

example (p : ℕ) (prime_p : Nat.Prime p) : 2 ≤ p ∧ ∀ m : ℕ, m < p → m ∣ p → m = 1 := by
  rwa [Nat.prime_def_lt] at prime_p

#check Nat.Prime.eq_one_or_self_of_dvd

example (p : ℕ) (prime_p : Nat.Prime p) : ∀ m : ℕ, m ∣ p → m = 1 ∨ m = p :=
  prime_p.eq_one_or_self_of_dvd

example : Nat.Prime 17 := by norm_num

-- commonly used
example : Nat.Prime 2 :=
  Nat.prime_two

example : Nat.Prime 3 :=
  Nat.prime_three
```


在自然数中，素数具有不能写成非平凡因子乘积的性质。在更广泛的数学语境中，环中具有这一性质的元素称为不可约元素。若环中的一个元素满足：只要它整除一个乘积，就整除其中某个因子，则称它为素元素。自然数的一个重要性质是，在该语境中这两个概念重合，从而得到定理 `Nat.Prime.dvd_mul`。

可以用这个事实建立上述论证中的关键性质：若一个数的平方是偶数，则该数本身也是偶数。Mathlib 在 `Algebra.Group.Even` 中定义了谓词 `Even`，但出于稍后会清楚的原因，我们将简单地使用 `2 ∣ m` 来表示 `m` 是偶数。

```
#check Nat.Prime.dvd_mul
#check Nat.Prime.dvd_mul Nat.prime_two
#check Nat.prime_two.dvd_mul

theorem even_of_even_sqr {m : ℕ} (h : 2 ∣ m ^ 2) : 2 ∣ m := by
  rw [pow_two, Nat.prime_two.dvd_mul] at h
  cases h <;> assumption

example {m : ℕ} (h : 2 ∣ m ^ 2) : 2 ∣ m :=
  Nat.Prime.dvd_of_dvd_pow Nat.prime_two h
```

随着学习推进，你需要熟练找到所需事实。记住，如果能猜出名称前缀并已导入相关库，就可以使用 Tab 补全（有时是 Ctrl-Tab）寻找目标。可以在任意标识符上 Ctrl-点击，跳转到定义它的文件，从而浏览附近的定义和定理。也可以使用 Lean 社区网页上的搜索引擎；如果都失败了，不要犹豫，到 Zulip 上提问。

```
example (a b c : Nat) (h : a * b = a * c) (h' : a ≠ 0) : b = c :=
  -- apply? suggests the following:
  (mul_right_inj' h').mp h
```

我们证明 2 的平方根无理性的核心包含在下面的定理中。请尝试使用 `even_of_even_sqr` 和定理 `Nat.dvd_gcd` 补全证明草图。

```
example {m n : ℕ} (coprime_mn : m.Coprime n) : m ^ 2 ≠ 2 * n ^ 2 := by
  intro sqr_eq
  have : 2 ∣ m := by
    sorry
  obtain (k, meq) := dvd_iff_exists_eq_mul_left.mp this
  have : 2 * (2 * k ^ 2) = 2 * n ^ 2 := by
    rw [← sqr_eq, meq]
    ring
  have : 2 * k ^ 2 = n ^ 2 :=
    sorry
  have : 2 ∣ n := by
    sorry
  have : 2 ∣ m.gcd n := by
    sorry
  have : 2 ∣ 1 := by
    sorry
  norm_num at this
```

事实上，只需很少修改，就可以把 2 替换为任意素数。请在下一个例子中试一试。在证明末尾，需要从 `p ∣ 1` 导出矛盾。可以使用 `Nat.Prime.two_le`，它说明任意素数都大于等于 2；以及 `Nat.le_of_dvd`。

```
example {m n p : ℕ} (coprime_mn : m.Coprime n) (prime_p : p.Prime) : m ^ 2 ≠ p * n ^ 2 := by
  sorry
```

让我们考虑另一种方法。下面快速证明：若 p 是素数，则 $m^2 \neq pn^2$ 。如果假设 $m^2 = pn^2$ ，并考虑 m 和 n 的素因子分解，则 p 在等式左侧出现偶数次，而在右侧出现奇数次，矛盾。注意，这个论证要求 n ，从而 m ，不等于零。下面的形式化确认这个假设足够。

唯一分解定理说，每个非零自然数都能以唯一方式写成素数的乘积。Mathlib 包含该事实的形式版本，用函数 `Nat.primeFactorsList` 表示；它返回一个数按非递减顺序排列的素因子列表。库证明了 `Nat.primeFactorsList n` 的所有元素都是素数，任意大于零的 `n` 等于其因子的乘积，并且如果 `n` 等于另一个素数列表的乘积，那么该列表是 `Nat.primeFactorsList n` 的一个排列。

```
#check Nat.primeFactorsList
#check Nat.prime_of_mem_primeFactorsList
#check Nat.prod_primeFactorsList
#check Nat.primeFactorsList_unique
```

即使我们还没有讨论列表成员关系、乘积或排列，你也可以浏览这些定理以及附近的其他定理。当前任务不需要其中任何内容。相反，我们将使用 Mathlib 有一个函数 `Nat.factorization` 这一事实，它把同样的数据表示为函数。具体地说，`Nat.factorization n p`，也可写作 `n.factorization p`，返回 `p` 在 `n` 的素因子分解中的重数。我们将使用以下三个事实。

```
theorem factorization_mul' {m n : ℕ} (mnez : m ≠ 0) (nnez : n ≠ 0) (p : ℕ) :
  (m * n).factorization p = m.factorization p + n.factorization p := by
  rw [Nat.factorization_mul mnez nnez]
  rfl

theorem factorization_pow' (n k p : ℕ) :
  (n ^ k).factorization p = k * n.factorization p := by
  rw [Nat.factorization_pow]
  rfl

theorem Nat.Prime.factorization' {p : ℕ} (prime_p : p.Prime) :
  p.factorization p = 1 := by
  rw [prime_p.factorization]
  simp
```

事实上，`n.factorization` 在 Lean 中定义为一个有限支撑函数；这解释了你逐步查看上面证明时会看到的奇怪记号。现在不必担心这些。就这里的目的而言，可以把上面三个定理当作黑箱使用。

下一个例子说明，化简器足够聪明，可以把 `n^2 ≠ 0` 替换为 `n ≠ 0`。策略 `simp` 只是调用 `simp` 后接 `assumption`。

看看能否使用上述恒等式补全证明中缺失的部分。

```
example {m n p : ℕ} (nnz : n ≠ 0) (prime_p : p.Prime) : m ^ 2 ≠ p * n ^ 2 := by
  intro sqr_eq
  have nsqr_nez : n ^ 2 ≠ 0 := by simp
  have eq1 : Nat.factorization (m ^ 2) p = 2 * m.factorization p := by
    sorry
  have eq2 : (p * n ^ 2).factorization p = 2 * n.factorization p + 1 := by
    sorry
  have : 2 * m.factorization p % 2 = (2 * n.factorization p + 1) % 2 := by
    rw [← eq1, sqr_eq, eq2]
  rw [add_comm, Nat.add_mul_mod_self_left, Nat.mul_mod_right] at this
  norm_num at this
```

这个证明的一个优点是它也能推广。2 并没有特殊之处；稍作修改后，该证明表明：只要写出 `m^k = r * n^k`，任意素数 `p` 在 `r` 中的重数必须是 `k` 的倍数。

为了把 `Nat.count_factors_mul_of_pos` 用于 `r * n^k`，需要知道 `r` 为正。但当 `r` 为零时，下面的定理是平凡的，并且容易由化简器证明。因此证明按情形进行。行 `rcases r with _ | r` 把目标替换为两个版本：一个把 `r` 替换为 `0`，另一个把 `r` 替换为 `r + 1`。在第二种情形中，可以使用定理 `r.succ_ne_zero`，它建立 `r + 1 ≠ 0`（`succ` 表示后继）。

还要注意，以 `have : npow_nz` 开头的那一行给出了 `n^k ≠ 0` 的一个短证明项证明。为了理解它如何工作，可以尝试把它替换为策略证明，然后思考这些策略如何描述该证明项。

看看能否补全下面证明中的缺失部分。在最后，可以使用 `Nat.dvd_sub` 和 `Nat.dvd_mul_right` 完成证明。

注意，这个例子并不假定 `p` 是素数；但当 `p` 不是素数时，结论是平凡的，因为此时 `r.factorization p` 按定义为零，而证明无论如何都适用于所有情形。

```

example {m n k r : ℕ} (nnz : n ≠ 0) (pow_eq : m ^ k = r * n ^ k) {p : ℕ} :
  k ∣ r.factorization p := by
  rcases r with _ | r
  · simp
  have npow_nz : n ^ k ≠ 0 := fun npowz ↦ nnz (eq_zero_of_pow_eq_zero npowz)
  have eq1 : (m ^ k).factorization p = k * m.factorization p := by
    sorry
  have eq2 : ((r + 1) * n ^ k).factorization p =
    k * n.factorization p + (r + 1).factorization p := by
    sorry
  have : r.succ.factorization p = k * m.factorization p - k * n.factorization p := by
    rw [← eq1, pow_eq, eq2, add_comm, Nat.add_sub_cancel]
  rw [this]
  sorry

```

我们可能希望以多种方式改进这些结果。首先，一个证明 2 的平方根无理性的证明，应当真正谈论 2 的平方根，也就是实数或复数中的一个元素。而说它无理，应当谈论有理数，即没有任何有理数等于它。此外，还应把本节定理扩展到整数。虽然数学上显然，如果能把 2 的平方根写成两个整数的商，也就能把它写成两个自然数的商，但形式化证明这一点需要一些工作。

在 Mathlib 中，自然数、整数、有理数、实数和复数分别由不同的数据类型表示。把注意力限制在单独的论域上往往很有帮助：我们会看到，对自然数做归纳很容易；而当实数不在图景中时，关于整数整除性的推理最容易。但必须在不同论域之间协调是一件麻烦事，我们将不得不处理它。本章稍后会回到这个问题。

我们还应期待能够加强最后一个定理的结论，说明数 $\sqrt[k]{r}$ 是一个 k 次幂，因为它的 k 次根就是每个整除 r 的素数，按其在 r 中的重数除以 k 后所得指数相乘。为了做到这一点，需要更好的手段来推理有限集上的乘积和求和，这也是我们稍后会回到主题。

事实上，本节中的结果都已在 Mathlib 的 `Data.Real.Irrational` 中以远为一般的形式建立。重数概念对任意交换幺半群都有定义，并且取值于扩展自然数 `enat`，它是在自然数中加入无穷大所得的类型。下一章我们将开始发展理解 Lean 如何支持这种一般性的手段。

5.2. 归纳与递归

自然数集 $\mathbb{N} = \{0, 1, 2, \dots\}$ 不仅本身极为重要，也在构造新的数学对象时起核心作用。Lean 的基础允许我们声明归纳类型，即由给定构造子列表归纳生成的类型。在 Lean 中，自然数声明如下。

```

inductive Nat where
  | zero : Nat
  | succ (n : Nat) : Nat

```

可以通过写 `#check Nat`，然后在标识符 `Nat` 上 Ctrl-点击，在库中找到这个定义。该命令说明 `Nat` 是由两个构造子 `zero : Nat` 和 `succ : Nat → Nat` 自由且归纳生成的数据类型。当然，库分别为 `Nat` 和 `zero` 引入记号 $\mathbb{N}$ 和 0 。（数字会被翻译为二进制表示，但现在不必关心这些细节。）

对工作数学家来说，“自由”意味着类型 `Nat` 有一个元素 `zero`，以及一个单射的后继函数 `succ`，且 `succ` 的像不包含 `zero`。

```

example (n : Nat) : n.succ ≠ Nat.zero :=
  Nat.succ_ne_zero n

example (m n : Nat) (h : m.succ = n.succ) : m = n :=
  Nat.succ.inj h

```

对工作数学家来说，“归纳”意味着自然数带有归纳证明原理和递归定义原理。本节将说明如何使用它们。

下面是阶乘函数递归定义的一个例子。

```
def fac : ℕ → ℕ
| 0 => 1
| n + 1 => (n + 1) * fac n
```

这种语法需要一些时间适应。注意第一行没有 `:=`。接下来的两行给出递归定义的基本情形和归纳步骤。这些方程在定义意义下成立，但也可以通过把名称 `fac` 提供给 `simp` 或 `rw` 来手动使用。

```
example : fac 0 = 1 :=
  rfl

example : fac 0 = 1 := by
  rw [fac]

example : fac 0 = 1 := by
  simp [fac]

example (n : ℕ) : fac (n + 1) = (n + 1) * fac n :=
  rfl

example (n : ℕ) : fac (n + 1) = (n + 1) * fac n := by
  rw [fac]

example (n : ℕ) : fac (n + 1) = (n + 1) * fac n := by
  simp [fac]
```

阶乘函数实际上已经在 Mathlib 中定义为 `Nat.factorial`。同样，可以输入 `#check Nat.factorial` 并使用 Ctrl-点击跳转到它。出于说明目的，我们将在例子中继续使用 `fac`。`Nat.factorial` 定义前的标注 `@[simp]` 指定把定义方程加入化简器默认使用的恒等式数据库。

归纳原理说，可以通过证明命题对 `0` 成立，并证明一旦它对自然数 n 成立也对 $n + 1$ 成立，来证明关于自然数的一般命题。因此，下面证明中的行 `induction' n with n ih` 会产生两个目标：第一个需要证明 `0 < fac 0`，第二个带有附加假设 `ih : 0 < fac n`，并要求证明 `0 < fac (n + 1)`。短语 `with n ih` 用于给归纳变量和归纳假设命名，你可以为它们选择任意名称。

```
theorem fac_pos (n : ℕ) : 0 < fac n := by
  induction' n with n ih
  · rw [fac]
    exact zero_lt_one
  rw [fac]
  exact mul_pos n.succ_pos ih
```

`induction'` 策略足够聪明，会把依赖于归纳变量的假设纳入归纳假设。请逐步查看下一个例子，看看发生了什么。

```
theorem dvd_fac {i n : ℕ} (ipos : 0 < i) (ile : i ≤ n) : i ∣ fac n := by
  induction' n with n ih
  · exact absurd ipos (not_lt_of_ge ile)
  rw [fac]
  rcases Nat.of_le_succ ile with h | h
  · apply dvd_mul_of_dvd_right (ih h)
  rw [h]
  apply dvd_mul_right
```

下面的例子给出阶乘函数的一个粗略下界。事实证明，先按情形证明更容易，这样证明余下部分从 $n = 1$ 的情形开始。看看能否使用 `pow_succ` 或 `pow_succ'` 通过归纳证明完成论证。

```
theorem pow_two_le_fac (n : ℕ) : 2 ^ (n - 1) ≤ fac n := by
  rcases n with _ | n
  · simp [fac]
  sorry
```

归纳常用于证明涉及有限和与有限乘积的恒等式。Mathlib 定义表达式 `Finset.sum s f`，其中 `s : Finset  $\alpha$` 是类型 `$\alpha$` 的元素组成的有限集，而 `f` 是定义在 `$\alpha$` 上的函数。`f` 的陪域可以是任意支持带零元的交换结合加法运算的类型。若导入 `Algebra.BigOperators.Ring` 并执行命令 `open BigOperators`，就可以使用更直观的记号 `$\sum x \in s, f x$` 。当然，有限乘积也有类似运算和记号。

我们将在下一节以及后续章节中讨论 `Finset` 类型及其支持的运算。现在只使用 `Finset.range n`，它是小于 `n` 的自然数组成的有限集。

```
variable { $\alpha$  : Type*} (s : Finset  $\mathbb{N}$ ) (f :  $\mathbb{N} \rightarrow \mathbb{N}$ ) (n :  $\mathbb{N}$ )

#check Finset.sum s f
#check Finset.prod s f

open BigOperators
open Finset

example : s.sum f =  $\sum x \in s, f x$  :=
  rfl

example : s.prod f =  $\prod x \in s, f x$  :=
  rfl

example : (range n).sum f =  $\sum x \in \text{range } n, f x$  :=
  rfl

example : (range n).prod f =  $\prod x \in \text{range } n, f x$  :=
  rfl
```

事实 `Finset.sum_range_zero` 和 `Finset.sum_range_succ` 给出了到 n 为止的求和的递归描述；乘积也类似。

```
example (f :  $\mathbb{N} \rightarrow \mathbb{N}$ ) :  $\sum x \in \text{range } 0, f x = 0$  :=
  Finset.sum_range_zero f

example (f :  $\mathbb{N} \rightarrow \mathbb{N}$ ) (n :  $\mathbb{N}$ ) :  $\sum x \in \text{range } n.\text{succ}, f x = \sum x \in \text{range } n, f x + f n$  :=
  Finset.sum_range_succ f n

example (f :  $\mathbb{N} \rightarrow \mathbb{N}$ ) :  $\prod x \in \text{range } 0, f x = 1$  :=
  Finset.prod_range_zero f

example (f :  $\mathbb{N} \rightarrow \mathbb{N}$ ) (n :  $\mathbb{N}$ ) :  $\prod x \in \text{range } n.\text{succ}, f x = (\prod x \in \text{range } n, f x) * f n$  :=
  Finset.prod_range_succ f n
```

每一对中的第一个恒等式按定义成立，也就是说，可以把证明替换为 `rfl`。

下面把我们定义的阶乘函数表示为一个乘积。

```
example (n :  $\mathbb{N}$ ) : fac n =  $\prod i \in \text{range } n, (i + 1)$  := by
  induction' n with n ih
  · simp [fac]
  · simp [fac, ih, prod_range_succ, mul_comm]
```

把 `mul_comm` 作为化简规则纳入值得说明。用恒等式 `$x * y = y * x$` 化简看起来应当很危险，因为它通常会无限循环。Lean 的化简器足够聪明，能够识别这一点；它只在所得项相对于某个固定但任意的项排序更小时才应用该规则。下面的例子表明，使用 `mul_assoc`、`mul_comm` 和 `mul_left_comm` 三条规则化简，能够识别那些只在括号位置和变量顺序上不同的相同乘积。

```
example (a b c d e f :  $\mathbb{N}$ ) : a * (b * c * f * (d * e)) = d * (a * f * e) * (c * b) := by
  simp [mul_comm, mul_left_comm]
```

粗略地说，这些规则通过把括号向右推，然后按同一个标准顺序重新排列两边表达式来工作。使用这些规则以及对应的加法规则化简，是一个方便技巧。

回到求和恒等式，我们建议逐步查看下面的证明：从 0 到 n 的自然数之和为 $n(n+1)/2$ 。证明的第一步清除分母。形式化恒等式时，这通常很有用，因为带除法的计算一般有旁条件。（类似地，在自然数上尽可能避免使用减法也很有用。）

```
theorem sum_id (n : ℕ) : ∑ i ∈ range (n + 1), i = n * (n + 1) / 2 := by
  symm; apply Nat.div_eq_of_eq_mul_right (by norm_num : 0 < 2)
  induction' n with n ih
  · simp
  rw [Finset.sum_range_succ, mul_add 2, ← ih]
  ring
```

我们鼓励你证明平方和的类似恒等式，以及你能在网上找到的其他恒等式。

```
theorem sum_sqr (n : ℕ) : ∑ i ∈ range (n + 1), i ^ 2 = n * (n + 1) * (2 * n + 1) / 6 := by
  sorry
```

在 Lean 的核心库中，加法和乘法本身也是用递归定义的，并且它们的基本性质由归纳证明建立。如果你喜欢思考这类基础性主题，也许会乐于完成乘法和加法的交换律、结合律以及乘法对加法分配律的证明。可以按照下面的提纲，在自然数的一个副本上完成这些工作。注意，可以对 `MyNat` 使用 `induction` 策略；Lean 足够聪明，知道应使用相关归纳原理（当然，它与 `Nat` 的归纳原理相同）。

我们先从加法交换律开始。一个好的经验法则是：由于加法和乘法是按第二个参数递归定义的，通常对出现在该位置的变量做归纳更有利。在结合律证明中决定使用哪个变量稍微有些微妙。

如果没有通常的零、一、加法和乘法记号，书写内容可能会让人困惑。稍后我们会学习如何定义这样的记号。在命名空间 `MyNat` 中工作，意味着可以写 `zero` 和 `succ`，而不必写 `MyNat.zero` 和 `MyNat.succ`，并且这些名称的解释优先于其他解释。在命名空间之外，例如下面定义的 `add` 的完整名称是 `MyNat.add`。

如果你发现自己确实喜欢这类事情，可以尝试定义截断减法和幂运算，并证明它们的一些性质。记住，截断减法会在零处截断。为了定义它，先定义先驱函数 `pred` 很有用；它从任意非零数中减去一，并保持零不变。函数 `pred` 可以由一个简单的递归实例定义。

```

inductive MyNat where
| zero : MyNat
| succ : MyNat → MyNat

namespace MyNat

def add : MyNat → MyNat → MyNat
| x, zero => x
| x, succ y => succ (add x y)

def mul : MyNat → MyNat → MyNat
| _, zero => zero
| x, succ y => add (mul x y) x

theorem zero_add (n : MyNat) : add zero n = n := by
  induction' n with n ih
  · rfl
  rw [add, ih]

theorem succ_add (m n : MyNat) : add (succ m) n = succ (add m n) := by
  induction' n with n ih
  · rfl
  rw [add, ih]
  rfl

theorem add_comm (m n : MyNat) : add m n = add n m := by
  induction' n with n ih
  · rw [zero_add]
    rfl
  rw [add, succ_add, ih]

theorem add_assoc (m n k : MyNat) : add (add m n) k = add m (add n k) := by
  sorry

theorem mul_add (m n k : MyNat) : mul m (add n k) = add (mul m n) (mul m k) := by
  sorry

theorem zero_mul (n : MyNat) : mul zero n = zero := by
  sorry

theorem succ_mul (m n : MyNat) : mul (succ m) n = add (mul m n) n := by
  sorry

theorem mul_comm (m n : MyNat) : mul m n = mul n m := by
  sorry
end MyNat

```

5.3. 无穷多个素数

让我们用另一个数学标准例子继续探索归纳与递归：证明有无穷多个素数。其一种表述是：对每个自然数 n ，都存在一个大于 n 的素数。为了证明这一点，令 p 为 $n! + 1$ 的任意素因子。若 p 小于等于 n ，则它整除 $n!$ 。由于它也整除 $n! + 1$ ，于是它整除 1，矛盾。因此 p 大于 n 。

要形式化该证明，需要说明任意大于等于 2 的数都有素因子。为此，需要说明任意不等于 0 或 1 的自然数都大于等于 2。这引出了形式化的一个怪异特征：常常正是这种平凡陈述最令人烦恼。这里我们考察几种处理方法。

首先，可以使用 `cases` 策略，以及后继函数尊重自然数序关系这一事实。

```

theorem two_le {m : ℕ} (h0 : m ≠ 0) (h1 : m ≠ 1) : 2 ≤ m := by
  cases m; contradiction
  case succ m =>
    cases m; contradiction
    repeat apply Nat.succ_le_succ
    apply zero_le

```

另一种策略是使用 `interval_cases`，当相关变量位于自然数或整数的某个区间内时，它会自动把目标拆成各个情形。记住，可以把鼠标悬停在它上面查看文档。

```
example {m : ℕ} (h0 : m ≠ 0) (h1 : m ≠ 1) : 2 ≤ m := by
  by_contra h
  push_neg at h
  interval_cases m <| contradiction
```

回忆一下，`interval_cases m` 后面的分号表示下一个策略应用于它生成的每一个情形。还有一种选择是使用 `decide` 策略，它会尝试寻找判定过程来解决问题。Lean 知道，若一个命题以有界量词 $\forall x, x < n \rightarrow \dots$ 或 $\exists x, x < n \wedge \dots$ 开始，就可以通过判定有限多个实例来判定其真值。

```
example {m : ℕ} (h0 : m ≠ 0) (h1 : m ≠ 1) : 2 ≤ m := by
  by_contra h
  push_neg at h
  revert h0 h1
  revert h m
  decide
```

有了定理 `two_le`，我们先证明每个大于 2 的自然数都有素因子。Mathlib 包含函数 `Nat.minFac`，它返回最小素因子；但为了学习库中的新内容，我们会避免使用它，直接证明该定理。

这里，普通归纳还不够。我们想使用**强归纳**，它允许我们通过证明对每个数 n ，若性质 P 对所有小于 n 的值成立，则它在 n 处也成立，来证明每个自然数 n 都具有性质 P 。在 Lean 中，这个原理称为 `Nat.strong_induction_on`，我们可以用关键字 `using` 告诉 `induction` 策略使用它。注意，这样做时没有基本情形；它被包含在一般归纳步骤中。

论证很简单。假设 $n \geq 2$ ，若 n 是素数，则完成。若不是，则由素数含义的某个刻画，它有一个非平凡因子 m ，于是可以对该因子应用归纳假设。请逐步查看下一个证明，看看它如何展开。

```
theorem exists_prime_factor {n : Nat} (h : 2 ≤ n) : ∃ p : Nat, p.Prime ∧ p ∣ n := by
  by_cases np : n.Prime
  · use n, np
  induction' n using Nat.strong_induction_on with n ih
  rw [Nat.prime_def_lt] at np
  push_neg at np
  rcases np h with ⟨m, mltn, mdvdn, mne1⟩
  have : m ≠ 0 := by
    intro mz
    rw [mz, zero_dvd_iff] at mdvdn
    linarith
  have mgt2 : 2 ≤ m := two_le this mne1
  by_cases mp : m.Prime
  · use m, mp
  · rcases ih m mltn mgt2 mp with ⟨p, pp, pdvd⟩
    use p, pp
    apply pdvd.trans mdvdn
```

现在可以证明我们定理的如下表述。看看能否补全草图。可以使用 `Nat.factorial_pos`、`Nat.dvd_factorial` 和 `Nat.dvd_sub!`。

```
theorem primes_infinite : ∀ n, ∃ p > n, Nat.Prime p := by
  intro n
  have : 2 ≤ Nat.factorial n + 1 := by
    sorry
  rcases exists_prime_factor this with ⟨p, pp, pdvd⟩
  refine ⟨p, ?_, pp⟩
  show p > n
  by_contra ple
  push_neg at ple
  have : p ∣ Nat.factorial n := by
    sorry
  have : p ∣ 1 := by
    sorry
  show False
  sorry
```


我们来考虑上述证明的一个变体：不使用阶乘函数，而是假设给定有限集 $\{p_1, \dots, p_n\}$ ，并考虑 $\prod_{i=1}^n p_i + 1$ 的一个素因子。这个素因子必然不同于每个 p_i ，从而说明不存在包含所有素数的有限集。

形式化这个论证要求我们推理有限集。在 Lean 中，对任意类型 α ，类型 `Finset  $\alpha$` 表示类型 α 的元素组成的有限集。计算性地推理有限集需要有一个判定 α 上相等性的过程，这就是为什么下面代码片段包含假设 `[DecidableEq  $\alpha$ ]`。对 $\mathbb{N}$ 、 $\mathbb{Z}$ 和 $\mathbb{Q}$ 这样的具体数据类型，该假设会自动满足。对实数推理时，可以通过使用经典逻辑并放弃计算解释来满足它。

我们使用命令 `open Finset`，以便使用相关定理的较短名称。不同于集合的情形，大多数涉及有限集的等价命题并不按定义成立，因此需要使用 `Finset.subset_iff`、`Finset.mem_union`、`Finset.mem_inter` 和 `Finset.mem_sdiff` 这类等价手动展开。

`ext` 策略仍可用于通过证明一个有限集的每个元素是另一个有限集的元素，来说明两个有限集相等。

```
open Finset

section
variable { $\alpha$  : Type*} [DecidableEq  $\alpha$ ] (r s t : Finset  $\alpha$ )

example : r  $\cap$  (s  $\cup$  t)  $\subseteq$  r  $\cap$  s  $\cup$  r  $\cap$  t := by
  rw [subset_iff]
  intro x
  rw [mem_inter, mem_union, mem_union, mem_inter, mem_inter]
  tauto

example : r  $\cap$  (s  $\cup$  t)  $\subseteq$  r  $\cap$  s  $\cup$  r  $\cap$  t := by
  simp [subset_iff]
  intro x
  tauto

example : r  $\cap$  s  $\cup$  r  $\cap$  t  $\subseteq$  r  $\cap$  (s  $\cup$  t) := by
  simp [subset_iff]
  intro x
  tauto

example : r  $\cap$  s  $\cup$  r  $\cap$  t = r  $\cap$  (s  $\cup$  t) := by
  ext x
  simp
  tauto

end
```

我们使用了一个新技巧：`tauto` 策略（以及使用经典逻辑的加强版本 `tauto!`）可以用来处理命题逻辑重言式。看看能否使用这些方法证明下面两个例子。

```
example : (r  $\cup$  s)  $\cap$  (r  $\cup$  t) = r  $\cup$  s  $\cap$  t := by
  sorry
example : (r  $\setminus$  s)  $\setminus$  t = r  $\setminus$  (s  $\cup$  t) := by
  sorry
```

定理 `Finset.dvd_prod_of_mem` 告诉我们，如果 n 是有限集 s 的元素，则 n 整除 $\prod_{i \in s} i$ 。

```
example (s : Finset  $\mathbb{N}$ ) (n :  $\mathbb{N}$ ) (h : n  $\in$  s) : n  $\mid$   $\prod_{i \in s} i$  :=
  Finset.dvd_prod_of_mem _ h
```

还需要知道，当 n 是素数且 s 是素数集合时，反向命题成立。为了说明这一点，需要下面的引理；你应当能用定理 `Nat.Prime.eq_one_or_self_of_dvd` 证明它。

```
theorem _root_.Nat.Prime.eq_of_dvd_of_prime {p q :  $\mathbb{N}$ }
  (prime_p : Nat.Prime p) (prime_q : Nat.Prime q) (h : p  $\mid$  q) :
  p = q := by
  sorry
```

可以用这个引理证明：若素数 p 整除一组有限素数的乘积，则它等于其中某个素数。Mathlib 提供了关于有限集的有效归纳原理：要证明一个性质对任意有限集 s 成立，只需证明它对空集成立，并且证明当加入单个新元素 $a \notin s$ 时它保持成立。这个原理称为 `Finset.induction_on`。当我们告诉 `induction` 策略使用它时，也可以指定名称 a 和 s 、归纳步骤中假设 $a \notin s$ 的名称，以及归纳假设的名称。表达式 `Finset.insert a s` 表示 s 与单元集 $\{a\}$ 的并。恒等式 `Finset.prod_empty` 和 `Finset.prod_insert` 则提供乘积的相关重写规则。在下面的证明中，第一个 `simp` 应用了 `Finset.prod_empty`。请逐步查看证明开头，观察归纳如何展开，然后完成它。

```
theorem mem_of_dvd_prod_primes {s : Finset ℕ} {p : ℕ} (prime_p : p.Prime) :
  (∀ n ∈ s, Nat.Prime n) → (p ∣ ∏ n ∈ s, n) → p ∈ s := by
  intro h₀ h₁
  induction' s using Finset.induction_on with a s ans ih
  · simp at h₁
    linarith [prime_p.two_le]
  simp [Finset.prod_insert ans, prime_p.dvd_mul] at h₀ h₁
  rw [mem_insert]
  sorry
```

我们还需要有限集的最后一个性质。给定元素 $s : \text{Set } \alpha$ 和 α 上的谓词 P ，在第 4 章中我们写 $\{x \in s \mid P x\}$ 表示 s 中满足 P 的元素集合。给定 $s : \text{Finset } \alpha$ ，类似概念写作 `s.filter P`。

```
example (s : Finset ℕ) (x : ℕ) : x ∈ s.filter Nat.Prime ↔ x ∈ s ∧ x.Prime :=
  mem_filter
```

现在证明“有无穷多个素数”的另一个表述：给定任意 $s : \text{Finset } \mathbb{N}$ ，存在一个不属于 s 的素数 p 。为了导出矛盾，我们假设所有素数都在 s 中，然后缩减到一个只包含且包含所有素数的集合 s' 。取这个集合的乘积，加一，再找结果的一个素因子，就会得到所需矛盾。看看能否完成下面的草图。在证明第一个 `have` 时，可以使用 `Finset.prod_pos`。

```
theorem primes_infinite' : ∀ s : Finset ℕ, ∃ p, Nat.Prime p ∧ p ∉ s := by
  intro s
  by_contra h
  push_neg at h
  set s' := s.filter Nat.Prime with s'_def
  have mem_s' : ∀ {n : ℕ}, n ∈ s' ↔ n.Prime := by
    intro n
    simp [s'_def]
    apply h
  have : 2 ≤ (∏ i ∈ s', i) + 1 := by
    sorry
  rcases exists_prime_factor this with (p, pp, pdvd)
  have : p ∣ ∏ i ∈ s', i := by
    sorry
  have : p ∣ 1 := by
    convert Nat.dvd_sub pdvd this
    simp
  show False
  sorry
```

我们已经看到两种说“有无穷多个素数”的方式：说它们不被任何 n 所界定，以及说它们不包含于任何有限集 s 。下面两个证明表明这些表述等价。在第二个证明中，为了构造 `s.filter Q`，必须假设有一个过程能判定 Q 是否成立。Lean 知道 `Nat.Prime` 有这样的判定过程。一般而言，如果通过写 `open Classical` 使用经典逻辑，就可以去掉这个假设。

在 Mathlib 中，`Finset.sup s f` 表示当 x 遍历 s 时 $f x$ 的值的上确界；当 s 为空且 f 的陪域为 $\mathbb{N}$ 时返回 0 。在第一个证明中，我们使用 `s.sup id`，其中 `id` 是恒等函数，来指代 s 中的最大值。

```

theorem bounded_of_ex_finset (Q : ℕ → Prop) :
  (∃ s : Finset ℕ, ∀ k, Q k → k ∈ s) → ∃ n, ∀ k, Q k → k < n := by
  rintro ⟨s, hs⟩
  use s.sup id + 1
  intro k Qk
  apply Nat.lt_succ_of_le
  show id k ≤ s.sup id
  apply le_sup (hs k Qk)

theorem ex_finset_of_bounded (Q : ℕ → Prop) [DecidablePred Q] :
  (∃ n, ∀ k, Q k → k ≤ n) → ∃ s : Finset ℕ, ∀ k, Q k → k ∈ s := by
  rintro ⟨n, hn⟩
  use (range (n + 1)).filter Q
  intro k
  simpa using hn k

```

对我们第二个“素数无穷多”证明稍作变化，可以证明有无穷多个模 4 余 3 的素数。论证如下。首先注意，若两个数 m 与 n 的乘积模 4 等于 3，则这两个数之一模 4 余 3。毕竟二者都必须为奇数；若它们都模 4 余 1，则乘积也模 4 余 1。可以用这个观察证明：若某个大于 2 的数模 4 余 3，则它有一个同样模 4 余 3 的素因子。

现在假设只有有限多个模 4 余 3 的素数，设为 $p_1, \dots, p_k$ 。不失一般性，可以假设 $p_1 = 3$ 。考虑乘积 $4 \prod_{i=2}^k p_i + 3$ 。容易看出它模 4 余 3，因此有一个模 4 余 3 的素因子 p 。不可能有 $p = 3$ ；因为 p 整除 $4 \prod_{i=2}^k p_i + 3$ ，若 p 等于 3，则它也整除 $\prod_{i=2}^k p_i$ ，从而推出 p 等于某个 p_i ($i = 2, \dots, k$)；而我们已把 3 从这个列表中排除。因此 p 必须是其他某个元素 p_i 。但在这种情况下， p 整除 $4 \prod_{i=2}^k p_i$ ，于是也整除 3，这与它不等于 3 相矛盾。

在 Lean 中，记号 `n % m` 读作“ n 模 m ”，表示 n 除以 m 的余数。

```
example : 27 % 4 = 3 := by norm_num
```

于是可以把“ n 模 4 余 3”写成 `n % 4 = 3`。下面的例子和定理总结了稍后需要使用的关于该函数的事实。第一个命名定理再次展示了按少数情形推理。在第二个命名定理中，记住分号表示随后的策略块应用于前一个策略生成的所有目标。

```

example (n : ℕ) : (4 * n + 3) % 4 = 3 := by
  rw [add_comm, Nat.add_mul_mod_self_left]

theorem mod_4_eq_3_or_mod_4_eq_3 {m n : ℕ} (h : m * n % 4 = 3) : m % 4 = 3 ∨ n % 4 = 3 := by
  revert h
  rw [Nat.mul_mod]
  have : m % 4 < 4 := Nat.mod_lt m (by norm_num)
  interval_cases m % 4 <> simp [-Nat.mul_mod_mod]
  have : n % 4 < 4 := Nat.mod_lt n (by norm_num)
  interval_cases n % 4 <> simp

theorem two_le_of_mod_4_eq_3 {n : ℕ} (h : n % 4 = 3) : 2 ≤ n := by
  apply two_le <>
  · intro neq
    rw [neq] at h
    norm_num at h

```

我们还需要下面这个事实：若 m 是 n 的非平凡因子，则 n / m 也是如此。看看能否使用 `Nat.div_dvd_of_dvd` 和 `Nat.div_lt_self` 完成证明。

```

theorem aux {m n : ℕ} (h₀ : m ∣ n) (h₁ : 2 ≤ m) (h₂ : m < n) : n / m ∣ n ∧ n / m < n := by
  sorry

```

现在把所有部分组合起来，证明任意模 4 余 3 的数都有一个具有同样性质的素因子。

```

theorem exists_prime_factor_mod_4_eq_3 {n : Nat} (h : n % 4 = 3) :
  ∃ p : Nat, p.Prime ∧ p ∣ n ∧ p % 4 = 3 := by
  by_cases np : n.Prime
  · use n
  induction' n using Nat.strong_induction_on with n ih
  rw [Nat.prime_def_lt] at np
  push_neg at np
  rcases np (two_le_of_mod_4_eq_3 h) with ⟨m, mltn, mdvdn, mne1⟩
  have mge2 : 2 ≤ m := by
    apply two_le _ mne1
  intro mz
  rw [mz, zero_dvd_iff] at mdvdn
  linarith
  have neq : m * (n / m) = n := Nat.mul_div_cancel' mdvdn
  have : m % 4 = 3 ∨ n / m % 4 = 3 := by
    apply mod_4_eq_3_or_mod_4_eq_3
  rw [neq, h]
  rcases this with h1 | h1
  . sorry
  . sorry

```

我们已接近终点。给定一个素数集合 `s`，需要谈论从该集合中移除 3（若它存在）后的结果。函数 `Finset.erase` 处理这一点。

```

example (m n : ℕ) (s : Finset ℕ) (h : m ∈ erase s n) : m ≠ n ∧ m ∈ s := by
  rwa [mem_erase] at h

example (m n : ℕ) (s : Finset ℕ) (h : m ∈ erase s n) : m ≠ n ∧ m ∈ s := by
  simp at h
  assumption

```

现在可以证明有无穷多个模 4 余 3 的素数。请补全下面缺失部分。我们的解法在过程中使用了 `Nat.dvd_add_iff_left` 和 `Nat.dvd_sub'`。

```

theorem primes_mod_4_eq_3_infinite : ∀ n, ∃ p > n, Nat.Prime p ∧ p % 4 = 3 := by
  by_contra h
  push_neg at h
  rcases h with ⟨n, hn⟩
  have : ∃ s : Finset Nat, ∀ p : ℕ, p.Prime ∧ p % 4 = 3 → p ∈ s := by
    apply ex_finset_of_bounded
    use n
    contrapose! hn
    rcases hn with ⟨p, ⟨pp, p4⟩, pltn⟩
    exact ⟨p, pltn, pp, p4⟩
  rcases this with ⟨s, hs⟩
  have h₁ : ((4 * ∏ i ∈ erase s 3, i) + 3) % 4 = 3 := by
    sorry
  rcases exists_prime_factor_mod_4_eq_3 h₁ with ⟨p, pp, pdvd, p4eq⟩
  have ps : p ∈ s := by
    sorry
  have pne3 : p ≠ 3 := by
    sorry
  have : p ∣ 4 * ∏ i ∈ erase s 3, i := by
    sorry
  have : p ∣ 3 := by
    sorry
  have : p = 3 := by
    sorry
  contradiction

```

如果你成功完成了这个证明，恭喜！这是一项严肃的形式化成就。

5.4. 更多归纳法

在 第 5.2 节 中，我们看到如何通过自然数上的递归来定义阶乘函数。

```

def fac : ℕ → ℕ
| 0 => 1
| n + 1 => (n + 1) * fac n

```

我们也看到如何使用 `induction'` 策略证明定理。

```

theorem fac_pos (n : ℕ) : 0 < fac n := by
  induction' n with n ih
  · rw [fac]
    exact zero_lt_one
  rw [fac]
  exact mul_pos n.succ_pos ih

```

`induction` 策略（不带撇号）允许使用更结构化的语法。

```

example (n : ℕ) : 0 < fac n := by
  induction n
  case zero =>
    rw [fac]
    exact zero_lt_one
  case succ n ih =>
    rw [fac]
    exact mul_pos n.succ_pos ih

example (n : ℕ) : 0 < fac n := by
  induction n with
  | zero =>
    rw [fac]
    exact zero_lt_one
  | succ n ih =>
    rw [fac]
    exact mul_pos n.succ_pos ih

```

照常，可以把鼠标悬停在 `induction` 关键字上阅读文档。情形名称 `zero` 和 `succ` 来自类型 `ℕ` 的定义。注意，`succ` 情形允许你为归纳变量和归纳假设选择任意名称，这里是 `n` 和 `ih`。甚至可以用与定义递归函数相同的记号来证明定理。

```

theorem fac_pos' : ∀ n, 0 < fac n
| 0 => by
  rw [fac]
  exact zero_lt_one
| n + 1 => by
  rw [fac]
  exact mul_pos n.succ_pos (fac_pos' n)

```

还要注意这里没有 `:=`，冒号之后没有 `∀ n`，每个情形中有关键字 `by`，并且在归纳步骤中调用 `fac_pos' n` 作为归纳调用。它仿佛把该定理看作 `n` 的递归函数，并在归纳步骤中进行递归调用。

这种定义风格非常灵活。Lean 的设计者内置了复杂的递归函数定义机制，而这些机制也延伸到归纳证明。例如，可以用多个基本情形定义 Fibonacci 函数。

```

@[simp] def fib : ℕ → ℕ
| 0 => 0
| 1 => 1
| n + 2 => fib n + fib (n + 1)

```

标注 `@[simp]` 表示化简器将使用这些定义方程。也可以写 `rw [fib]` 来应用它们。下面给 `n + 2` 情形命名会很有帮助。

```

theorem fib_add_two (n : ℕ) : fib (n + 2) = fib n + fib (n + 1) := rfl

example (n : ℕ) : fib (n + 2) = fib n + fib (n + 1) := by rw [fib]

```

使用 Lean 的递归函数记法，可以对自然数进行归纳证明，并使证明形式与 `fib` 的递归定义相呼应。下面的例子用黄金比例 φ 及其共轭 φ' 给出第 `n` 个 Fibonacci 数的显式公式。我们必须告诉 Lean 不期望这些定义生成代码，因为实数上的算术运算不可计算。

我们将使用 `grind` 策略完成计算，并告诉它使用 `phi` 和 `phi'` 的定义，以及归纳假设 `fib_eq n` 和 `fib_eq (n+1)`。

```

noncomputable section

def phi : ℝ := (1 + √5) / 2
def phi' : ℝ := (1 - √5) / 2

theorem fib_eq : ∀ n, fib n = (phi^n - phi'^n) / √5
| 0 => by simp
| 1 => by unfold fib; grind [phi, phi']
| n+2 => by unfold fib; simp [fib_eq n, fib_eq (n+1), phi, phi']; grind

end

```

涉及 Fibonacci 函数的归纳证明并不一定都采用这种形式。下面复现 Mathlib 中证明相邻 Fibonacci 数互素的证明。

```

theorem fib_coprime_fib_succ (n : ℕ) : Nat.Coprime (fib n) (fib (n + 1)) := by
  induction n with
  | zero => simp
  | succ n ih =>
    simp only [fib, Nat.coprime_add_self_right]
    exact ih.symm

```

利用 Lean 的计算解释，可以求出 Fibonacci 数。

```

#eval fib 6
#eval List.range 20 |>.map fib

```

`fib` 的直接实现计算效率很低。事实上，它的运行时间关于参数呈指数增长。（你应当思考为什么。）在 Lean 中，可以实现如下尾递归版本，其运行时间关于 `n` 线性，并证明它计算同一个函数。

```

def fib' (n : Nat) : Nat :=
  aux n 0 1
where aux
| 0, x, _ => x
| n+1, x, y => aux n y (x + y)

theorem fib'.aux_eq (m n : ℕ) : fib'.aux n (fib m) (fib (m + 1)) = fib (n + m) := by
  induction n generalizing m with
  | zero => simp [fib'.aux]
  | succ n ih => rw [fib'.aux, ←fib_add_two, ih, add_assoc, add_comm 1]

theorem fib'_eq_fib : fib' = fib := by
  ext n
  erw [fib', fib'.aux_eq 0 n]; rfl

#eval fib' 10000

```

注意 `fib'.aux_eq` 证明中的关键字 `generalizing`。它的作用是在归纳假设前插入 $\forall m$ ，使得在归纳步骤中，`m` 可以取不同值。你可以逐步查看证明，并确认在这个例子中，归纳步骤需要把该量词实例化为 `m + 1`。

还要注意使用的是 `erw`（“extended rewrite”）而不是 `rw`。这是因为要重写目标 `fib'.aux_eq`，必须把 `fib 0` 和 `fib 1` 分别约简为 `0` 和 `1`。策略 `erw` 在展开定义以匹配参数方面比 `rw` 更积极。这并不总是好主意；在某些情况下它会浪费很多时间，因此应谨慎使用 `erw`。

下面是 `generalizing` 关键字在另一个恒等式证明中的例子，该恒等式也出现在 Mathlib 中。该恒等式的非形式证明可以在[这里](#)找到。我们提供形式证明的两个变体。

```

theorem fib_add (m n : ℕ) : fib (m + n + 1) = fib m * fib n + fib (m + 1) * fib (n + 1) := by
  induction n generalizing m with
  | zero => simp
  | succ n ih =>
    specialize ih (m + 1)
    rw [add_assoc m 1 n, add_comm 1 n] at ih
    simp only [fib_add_two, ih]
    ring

theorem fib_add' : ∀ m n, fib (m + n + 1) = fib m * fib n + fib (m + 1) * fib (n + 1)
| _, 0 => by simp
| m, n + 1 => by
  have := fib_add' (m + 1) n
  rw [add_assoc m 1 n, add_comm 1 n] at this
  simp only [fib_add_two, this]
  ring

```

作为练习，请使用 `fib_add` 证明如下命题。

```

example (n : ℕ) : (fib n) ^ 2 + (fib (n + 1)) ^ 2 = fib (2 * n + 1) := by
  sorry

```

Lean 定义递归函数的机制足够灵活，可以允许任意递归调用，只要参数的复杂度按照某个良基度量下降。在下一个例子中，我们利用这样一个事实证明每个自然数 `n ≠ 1` 都有素因子：若 `n` 非零且不是素数，则它有一个更小的因子。（可以检查 Mathlib 在 `Nat` 命名空间中有同名定理，虽然证明不同于这里给出的证明。）

```

#check (@Nat.not_prime_iff_exists_dvd_lt :
  ∀ {n : ℕ}, 2 ≤ n → (¬Nat.Prime n ↔ ∃ m, m | n ∧ 2 ≤ m ∧ m < n))

theorem ne_one_iff_exists_prime_dvd : ∀ {n}, n ≠ 1 ↔ ∃ p : ℕ, p.Prime ∧ p | n
| 0 => by simpa using Exists.intro 2 Nat.prime_two
| 1 => by simp [Nat.not_prime_one]
| n + 2 => by
  have hn : n + 2 ≠ 1 := by omega
  simp only [Ne, not_false_iff, true_iff, hn]
  by_cases h : Nat.Prime (n + 2)
  · use n + 2, h
  · have : 2 ≤ n + 2 := by omega
    rw [Nat.not_prime_iff_exists_dvd_lt this] at h
    rcases h with ⟨m, mdv, mge2, -⟩
    have : m ≠ 1 := by omega
    rw [ne_one_iff_exists_prime_dvd] at this
    rcases this with ⟨p, primep, pdvd⟩
    use p, primep
    exact pdvd.trans mdv

```

行 `rw [ne_one_iff_exists_prime_dvd] at this` 像魔术一样：我们在一个定理自己的证明中使用了正在证明的这个定理。它之所以有效，是因为递归调用实例化在 `m` 处，而当前情形是 `n + 2`，并且语境中有 `m < n + 2`。Lean 能找到这个假设，并用它说明归纳是良基的。Lean 很擅长判断什么在下降；在这个例子中，定理陈述中 `n` 的选择以及小于关系都很明显。在更复杂的情形中，Lean 提供机制让我们显式提供这些信息。参见 Lean Reference Manual 中关于良基递归的章节。

有时在证明中，需要根据自然数 `n` 是零还是后继数分情况讨论，但不需要在后继情形中获得归纳假设。为此，可以使用 `cases` 和 `rcases` 策略。

```

theorem zero_lt_of_mul_eq_one (m n : ℕ) : n * m = 1 → 0 < n ∧ 0 < m := by
  cases n <;> cases m <;> simp

example (m n : ℕ) : n * m = 1 → 0 < n ∧ 0 < m := by
  rcases m with (| m); simp
  rcases n with (| n) <;> simp

```

这是一个有用技巧。你常常会遇到关于自然数 `n` 的定理，其中零情形很容易。如果对 `n` 分情况并迅速处理零情形，剩下的就是把原目标中的 `n` 替换为 `n + 1` 后的目标。

6. 离散数学

Discrete Mathematics 研究有限集合、对象与结构。我们可以计数有限集合的元素，可以计算其元素上的有限和或有限积，可以计算最大值与最小值，等等。我们还可以研究由某些生成函数经有限次应用而生成的对象，可以用结构递归定义函数，并用结构归纳证明定理。本章介绍 Mathlib 中支持这些工作的若干部分。

6.1. 有限集与有限类型

在 Mathlib 中处理有限集合与有限类型可能令人困惑，因为库提供了多种处理方式。本节将讨论其中最常用的几种。

我们已经在 [Section 5.2](#) 和 [Section 5.3](#) 中见过类型 `Finset`。顾名思义，类型 `Finset α` 的一个元素就是由类型 `α` 的元素构成的有限集。我们称这类对象为“有限集”（finset）。数据类型 `Finset` 被设计为具有计算解释；`Finset α` 上许多基本操作都假定 `α` 具有可判定相等性，这保证存在一种算法可检验 `a : α` 是否为有限集 `s` 的元素。

```
section
variable {α : Type*} [DecidableEq α] (a : α) (s t : Finset α)

#check a ∈ s
#check s n t

end
```

如果移除声明 `[DecidableEq α]`，Lean 会在 `#check s n t` 这一行报错，因为它无法计算交。尽管如此，所有你期望能够进行计算的数据类型都有可判定相等性；而若通过打开 `Classical` 命名空间并声明 `noncomputable section` 来以经典方式工作，就可以对任意类型的元素所成有限集进行推理。

有限集支持集合所支持的大多数集合论操作：

```
open Finset

variable (a b c : Finset ℕ)
variable (n : ℕ)

#check a n b
#check a ∪ b
#check a \ b
#check (∅ : Finset ℕ)

example : a n (b ∪ c) = (a n b) ∪ (a n c) := by
  ext x; simp only [mem_inter, mem_union]; tauto

example : a n (b ∪ c) = (a n b) ∪ (a n c) := by rw [inter_union_distrib_left]
```

注意，我们已经打开了 `Finset` 命名空间，其中包含有限集专用的定理。如果逐步查看下面最后一个例子，会看到先应用 `ext` 再应用 `simp` 会把该恒等式化归为命题逻辑问题。作为练习，你可以尝试把 [Chapter 4](#) 中的一些集合恒等式迁移到有限集上来证明。

你已经见过记号 `Finset.range n`，它表示自然数的有限集 $\{0, 1, \dots, n - 1\}$ 。`Finset` 还允许通过列举元素来定义有限集：

```
#check ({0, 2, 5} : Finset Nat)

def example1 : Finset ℕ := {0, 1, 2}
```

有多种方法可以让 Lean 识别：以这种方式给出的集合中，元素的顺序和重复出现并不重要。


```

example : ({0, 1, 2} : Finset ℕ) = {1, 2, 0} := by decide

example : ({0, 1, 2} : Finset ℕ) = {0, 1, 1, 2} := by decide

example : ({0, 1} : Finset ℕ) = {1, 0} := by rw [Finset.pair_comm]

example (x : Nat) : ({x, x} : Finset ℕ) = {x} := by simp

example (x y z : Nat) : ({x, y, z, y, z, x} : Finset ℕ) = {x, y, z} := by
  ext i; simp [or_comm, or_left_comm]

example (x y z : Nat) : ({x, y, z, y, z, x} : Finset ℕ) = {x, y, z} := by
  ext i; simp; tauto

```

可以使用 `insert` 向一个有限集中加入单个元素，也可以使用 `Finset.erase` 删除单个元素。注意，`erase` 位于 `Finset` 命名空间中，而 `insert` 位于根命名空间中。

```

example (s : Finset ℕ) (a : ℕ) (h : a ∉ s) : (insert a s |>.erase a) = s :=
  Finset.erase_insert h

example (s : Finset ℕ) (a : ℕ) (h : a ∈ s) : insert a (s.erase a) = s :=
  Finset.insert_erase h

```

事实上，`{0, 1, 2}` 只是 `insert 0 (insert 1 (singleton 2))` 的记号。

```

set_option pp.notation false in
#check ({0, 1, 2} : Finset ℕ)

```

给定有限集 `s` 和谓词 `P`，可以用集合构造记号 `{x ∈ s | P x}` 定义 `s` 中满足 `P` 的元素所成集合。这是 `Finset.filter P s` 的记号，也可以写作 `s.filter P`。

```

example : {m ∈ range n | Even m} = (range n).filter Even := rfl
example : {m ∈ range n | Even m ∧ m ≠ 3} = (range n).filter (fun m ↦ Even m ∧ m ≠ 3) := rfl

example : {m ∈ range 10 | Even m} = {0, 2, 4, 6, 8} := by decide

```

Mathlib 知道，有限集在函数下的像仍然是有限集。

```

#check (range 5).image (fun x ↦ x * 2)

example : (range 5).image (fun x ↦ x * 2) = {x ∈ range 10 | Even x} := by decide

```

Lean 也知道两个有限集的笛卡尔积 `s ×s t` 是有限集，并且有限集的幂集也是有限集。（注意，记号 `s ×s t` 对集合同样适用。）

```

#check s ×s t
#check s.powerset

```

按有限集的元素来定义操作是微妙的，因为任何这样的定义都必须与元素给出的顺序无关。当然，总可以通过复合已有操作来定义函数。另一种做法是使用 `Finset.fold` 将一个二元运算折叠到各个元素上，前提是该运算满足结合律和交换律；这些性质保证结果与应用运算的顺序无关。有限和、有限积以及有限并正是以这种方式定义的。在下面最后一个例子中，`biUnion` 表示“有界指标并”。用通常的数学记号，该表达式可写为 $\bigcup_{i \in s} g(i)$ 。

```

#check Finset.fold

def f (n : ℕ) : Int := (↑n)^2

#check (range 5).fold (fun x y : Int ↦ x + y) 0 f
#eval (range 5).fold (fun x y : Int ↦ x + y) 0 f

#check ∑ i ∈ range 5, i^2
#check ∏ i ∈ range 5, i + 1

variable (g : Nat → Finset Int)

#check (range 5).biUnion g

```

有限集上有一个自然的归纳原则：要证明每个有限集都具有某性质，只需证明空集具有该性质，并证明当向一个有限集加入一个新元素时该性质得以保持。（下一个例子的归纳步骤中需要在 `@insert` 里使用符号 `@`，以便给参数 `a` 和 `s` 命名，因为它们已被标记为隐式参数。）

```
#check Finset.induction

example {α : Type*} [DecidableEq α] (f : α → ℕ) (s : Finset α) (h : ∀ x ∈ s, f x ≠ 0) :
  ∀ x ∈ s, f x ≠ 0 := by
  induction s using Finset.induction_on with
  | empty => simp
  | @insert a s anins ih =>
    rw [prod_insert anins]
    apply mul_ne_zero
    · apply h; apply mem_insert_self
    apply ih
  intros x xs
  exact h x (mem_insert_of_mem xs)
```

若 `s` 是有限集，则 `Finset.Nonempty s` 被定义为 `∃ x, x ∈ s`。可以使用经典选择从非空有限集中选取一个元素。类似地，库定义了 `Finset.toList s`，它使用选择以某种顺序选取 `s` 的元素。

```
noncomputable example (s : Finset ℕ) (h : s.Nonempty) : ℕ := Classical.choose h

example (s : Finset ℕ) (h : s.Nonempty) : Classical.choose h ∈ s := Classical.choose_spec h

noncomputable example (s : Finset ℕ) : List ℕ := s.toList

example (s : Finset ℕ) (a : ℕ) : a ∈ s.toList ↔ a ∈ s := mem_toList
```

可以使用 `Finset.min` 和 `Finset.max` 从线性序中元素构成的有限集中选取最小或最大元素；类似地，也可以对格中元素构成的有限集使用 `Finset.inf` 和 `Finset.sup`。但这里有一个细节：空有限集的最小元素应当是什么？你可以检查下面这些函数的带撇号版本，它们增加了有限集非空这一前提。无撇号版本 `Finset.min` 和 `Finset.max` 分别向输出类型加入一个顶元素或底元素，用来处理有限集为空的情形。无撇号版本 `Finset.inf` 和 `Finset.sup` 则分别假定该格配备了顶元素或底元素。

```
#check Finset.min
#check Finset.min'
#check Finset.max
#check Finset.max'
#check Finset.inf
#check Finset.inf'
#check Finset.sup
#check Finset.sup'

example : Finset.Nonempty {2, 6, 7} := ⟨6, by trivial⟩
example : Finset.min' {2, 6, 7} ⟨6, by trivial⟩ = 2 := by trivial
```

每个有限集 `s` 都有一个有限基数 `Finset.card s`；当 `Finset` 命名空间打开时，它可写作 `#s`。

```
#check Finset.card

#eval (range 5).card

example (s : Finset ℕ) : s.card = #s := by rfl

example (s : Finset ℕ) : s.card = ∑ i ∈ s, 1 := by rw [card_eq_sum_ones]

example (s : Finset ℕ) : s.card = ∑ i ∈ s, 1 := by simp
```

下一节将专门讨论关于基数的推理。

在形式化数学时，常常必须决定是用集合还是用类型来表述定义和定理。使用类型往往能简化记号和证明，但使用某个类型的子集则可能更灵活。有限集的类型化对应物是 *fintype*，即某个 `α` 上的类型 `Fintype α`。按照定义，有限类型只是一个配备了有限集 `univ` 的数据类型，而该有限集包含其全部元素。

```
variable {α : Type*} [Fintype α]

example : ∀ x : α, x ∈ Finset.univ := by
  intro x; exact mem_univ x
```

`Finset.card α` 等于相应有限集的基数。

```
example : Fintype.card α = (Finset.univ : Finset α).card := rfl
```

我们已经见过有限类型的一个典型例子，即对每个 `n` 给出的类型 `Fin n`。Lean 能够识别有限类型在诸如积这样的操作下封闭。

```
example : Fintype.card (Fin 5) = 5 := by simp
example : Fintype.card ((Fin 5) × (Fin 3)) = 15 := by simp
```

`Finset α` 的任意元素 `s` 都可以强制转换为一个类型 `(↑s : Type α)`，即 `α` 中属于 `s` 的元素所成的子类型。此外，Lean 知道 `↑s` 是有限类型。

```
variable (s : Finset ℕ)

example : (↑s : Type) = {x : ℕ // x ∈ s} := rfl
example : Fintype.card ↑s = s.card := by simp
```

Lean 和 Mathlib 使用 *type class inference* 来跟踪有限类型上的额外结构，也就是包含所有元素的全体有限集。换言之，可以把有限类型看作配备了这项额外数据的代数结构。Chapter 7 将说明这一机制如何工作。

6.2. 计数论证

计数的技巧是组合数学的核心组成部分。Mathlib 含有若干用于计数有限集元素的基本恒等式：

```
open Finset

variable {α β : Type*} [DecidableEq α] [DecidableEq β] (s t : Finset α) (f : α → β)

example : #(s ×s t) = #s * #t := by rw [card_product]
example : #(s ×s t) = #s * #t := by simp

example : #(s ∪ t) = #s + #t - #(s ∩ t) := by rw [card_union]

example (h : Disjoint s t) : #(s ∪ t) = #s + #t := by rw [card_union_of_disjoint h]
example (h : Disjoint s t) : #(s ∪ t) = #s + #t := by simp [h]

example (h : Function.Injective f) : #(s.image f) = #s := by rw [card_image_of_injective _ h]
example (h : Set.InjOn f s) : #(s.image f) = #s := by rw [card_image_of_injOn h]
```

打开 `Finset` 命名空间后，可以使用记号 `#s` 表示 `s.card`，也可以使用 `card_union` 等缩短后的名称。

Mathlib 也可以计数有限类型的元素：

```
open Fintype

variable {α β : Type*} [Fintype α] [Fintype β]

example : card (α × β) = card α * card β := by simp
example : card (α ⊕ β) = card α + card β := by simp
example (n : ℕ) : card (Fin n → α) = (card α)n := by simp
variable {n : ℕ} {γ : Fin n → Type*} [∀ i, Fintype (γ i)]
example : card ((i : Fin n) → γ i) = ∏ i, card (γ i) := by simp
example : card (⊔ i, γ i) = ∑ i, card (γ i) := by simp
```

当 `Fintype` 命名空间未打开时，必须使用 `Fintype.card` 而不是 `card`。

下面是一个计算有限集基数的例子：计算 $\text{range } n$ 与其一个副本的并，其中这个副本被平移了超过 n 的距离。该计算要求证明并中的两个集合不交；证明的第一行产生旁条件 `Disjoint (range n) (image (fun i ↦ m + i) (range n))`，它在证明末尾得到证明。谓词 `Disjoint` 对我们来说过于一般，不能直接使用，但定理 `disjoint_iff_ne` 会把它化为可用的形式。

```
#check Disjoint

example (m n : ℕ) (h : m ≥ n) :
  card (range n ∪ (range n).image (fun i ↦ m + i)) = 2 * n := by
  rw [card_union_of_disjoint, card_range, card_image_of_injective, card_range]; omega
  . apply add_right_injective
  . simp [disjoint_iff_ne]; omega
```

在本节中，`omega` 将是处理算术计算与不等式的主力工具。

下面是一个更有趣的例子。考虑 $\{0, \dots, n\} \times \{0, \dots, n\}$ 中由满足 $i < j$ 的点 (i, j) 构成的子集。如果把它们看成坐标平面中的格点，它们构成以 $(0, 0)$ 和 (n, n) 为顶点的正方形的上三角部分，但不包括对角线。整个正方形的基数为 $(n + 1)^2$ ；减去对角线的大小后再除以二，可知该三角形的基数为 $n(n + 1)/2$ 。

另一种做法是注意该三角形各行的大小为 $0, 1, \dots, n$ ，因此其基数就是前 n 个正整数之和。下面证明中的第一个 `have` 把三角形描述为各行之并，其中第 j 行由数 $0, 1, \dots, j - 1$ 与 j 配对而成。在下面的证明中，记号 `(., j)` 是函数 `fun i ↦ (i, j)` 的缩写。证明的其余部分只是关于有限集基数的计算。

```
def triangle (n : ℕ) : Finset (ℕ × ℕ) := {p ∈ range (n+1) × range (n+1) | p.1 < p.2}

example (n : ℕ) : #(triangle n) = (n + 1) * n / 2 := by
  have : triangle n = (range (n+1)).biUnion (fun j ↦ (range j).image (., j)) := by
    ext p
    simp only [triangle, mem_filter, mem_product, mem_range, mem_biUnion, mem_image]
    constructor
    . rintro ⟨hp1, hp2, hp3⟩
    use p.2, hp2, p.1, hp3
    . rintro ⟨p1, hp1, p2, hp2, rfl⟩
    omega
  rw [this, card_biUnion]; swap
  . -- take care of disjointness first
    intro x _ y _ xney
    simp [disjoint_iff_ne, xney]
  -- continue the calculation
  transitivity (∑ i ∈ range (n + 1), i)
  . congr; ext i
    rw [card_image_of_injective, card_range]
    intros i1 i2; simp
  rw [sum_range_id]; rfl
```

下面的证明变体使用有限类型而非有限集来完成计算。类型 $\alpha \approx \beta$ 是 α 与 β 之间等价的类型，由正向映射、反向映射以及证明二者互为逆的证明组成。证明中的第一个 `have` 表明 `triangle n` 等价于 i 遍历 `Fin (n + 1)` 时各 `Fin i` 的不交并。有趣的是，正向函数和反向函数是用策略构造出来的，而不是显式写出的。由于它们只是搬运数据和信息，`rfl` 即可证明它们互为逆。

之后，`rw [←Fintype.card_coe]` 将 `#(triangle n)` 重写为子类型 `{x // x ∈ triangle n}` 的基数，证明的其余部分就是计算。

```
example (n : ℕ) : #(triangle n) = (n + 1) * n / 2 := by
  have : triangle n ≈ ∑ i : Fin (n + 1), Fin i.val :=
    { toFun := by
        rintro ⟨i, j⟩, hp
        have : (i ≤ n ∧ j ≤ n) ∧ i < j := by simp [triangle] using hp
        exact ⟨j, by linarith⟩, ⟨i, by linarith⟩
      invFun := by
        rintro ⟨i, j⟩
        use ⟨j, i⟩
        suffices j ≤ n ∧ i ≤ n by simp [triangle]
        constructor <.> linarith [i.2, j.2]
      left_inv := by intro i; rfl
      right_inv := by intro i; rfl }
  rw [←Fintype.card_coe]
  trans; apply (Fintype.card_congr this)
  rw [Fintype.card_sigma, sum_fin_eq_sum_range]
  convert Finset.sum_range_id (n + 1)
  simp_all
```

还有另一种方法。下面证明的第一行把问题化归为证明 $2 * \#(\text{triangle } n) = (n + 1) * n$ 。我们可以通过证明两个三角形的副本恰好填满矩形 $\text{range } n \times^s \text{range } (n + 1)$ 来做到这一点。作为练习，请尝试补全该计算的步骤。在解答中，我们在倒数第二步大量依赖 `omega`，但遗憾的是仍须手工完成相当一部分工作。

```
example (n : ℕ) : #(triangle n) = (n + 1) * n / 2 := by
  apply Nat.eq_div_of_mul_eq_right (by norm_num)
  let turn (p : ℕ × ℕ) : ℕ × ℕ := (n - 1 - p.1, n - p.2)
  calc 2 * #(triangle n)
    = #(triangle n) + #(triangle n) := by
      sorry
    _ = #(triangle n) + #(triangle n |>.image turn) := by
      sorry
    _ = #(range n ×^s range (n + 1)) := by
      sorry
    _ = (n + 1) * n := by
      sorry
```

你可以自行确信：若在 `triangle` 的定义中把 `n` 替换为 `n + 1`，并把 `<` 替换为 `≤`，得到的只是向下平移后的同一个三角形。下面的练习要求你用这一事实证明两个三角形大小相同。

```
def triangle' (n : ℕ) : Finset (ℕ × ℕ) := {p ∈ range n ×^s range n | p.1 ≤ p.2}
example (n : ℕ) : #(triangle' n) = #(triangle n) := by sorry
```

本节最后给出一个例子和一个练习，它们来自 Bhavik Mehta 于 2023 年在 *Lean for the Curious Mathematician* 中关于组合数学的[tutorial](#)。设有一个二分图，其顶点集为 `s` 和 `t`，并且对 `s` 中每个 `a`，至少有三条边从 `a` 发出；对 `t` 中每个 `b`，至多有一条边进入 `b`。于是图中的边总数至少是 `s` 的基数的三倍，且至多是 `t` 的基数，由此推出 `s` 的基数的三倍不超过 `t` 的基数。下面的定理实现了这个论证，其中我们用关系 `r` 表示图的边。该证明是一个优雅的计算。

```
open Classical
variable (s t : Finset ℕ) (a b : ℕ)

theorem doubleCounting {α β : Type*} (s : Finset α) (t : Finset β)
  (r : α → β → Prop)
  (h_left : ∀ a ∈ s, 3 ≤ #{b ∈ t | r a b})
  (h_right : ∀ b ∈ t, #{a ∈ s | r a b} ≤ 1) :
  3 * #s ≤ #t := by
  calc 3 * #s
    = ∑ a ∈ s, 3 := by simp [mul_comm]
    _ ≤ ∑ a ∈ s, #{b ∈ t | r a b} := sum_le_sum h_left
    _ = ∑ a ∈ s, ∑ b ∈ t, if r a b then 1 else 0 := by simp
    _ = ∑ b ∈ t, ∑ a ∈ s, if r a b then 1 else 0 := sum_comm
    _ = ∑ b ∈ t, #{a ∈ s | r a b} := by simp
    _ ≤ ∑ b ∈ t, 1 := sum_le_sum h_right
    _ = #t := by simp
```

下面的练习同样取自 Mehta 的教程。设 `A` 是 $\text{range } (2 * n)$ 的一个子集，且有 `n + 1` 个元素。容易看出，`A` 必含两个连续整数，从而含有两个互素的元素。如果观看该教程，你会看到为了建立下面这个事实曾花费了大量精力，而如今它可由 `omega` 自动证明。

```
example (m k : ℕ) (h : m ≠ k) (h' : m / 2 = k / 2) : m = k + 1 ∨ k = m + 1 := by omega
```

Mehta 练习的解法使用抽屉原理，其形式为 `exists_lt_card_fiber_of_mul_lt_card_of_maps_to`，来证明 `A` 中存在两个不同元素 `m` 和 `k`，使得 `m / 2 = k / 2`。请尝试补全这一事实的论证，然后用它完成证明。

```
example {n : ℕ} (A : Finset ℕ)
  (hA : #(A) = n + 1)
  (hA' : A ⊆ range (2 * n)) :
  ∃ m ∈ A, ∃ k ∈ A, Nat.Coprime m k := by
have : ∃ t ∈ range n, 1 < #({u ∈ A | u / 2 = t}) := by
  apply exists_lt_card_fiber_of_mul_lt_card_of_maps_to
  · sorry
  · sorry
rcases this with ⟨t, ht, ht'⟩
simp only [one_lt_card, mem_filter] at ht'
sorry
```

6.3. 归纳定义的类型

Lean 的基础允许我们定义归纳类型，即其实例自底向上生成的数据类型。例如，由 `α` 的元素组成的列表数据类型 `List α`，是从空列表 `nil` 开始，并不断向列表前端添加元素而生成的。下面我们将定义一种二叉树类型 `BinTree`，其元素从空树开始生成，并通过把一个新节点连接到两棵已有树上构造新的树。

在 Lean 中，可以定义对象为无限的归纳类型，例如可数分支的良基树。不过，有限归纳定义在离散数学中很常用，尤其是在与计算机科学相关的离散数学分支中。Lean 不仅提供定义这类类型的方法，也提供归纳原则和递归定义原则。例如，数据类型 `List α` 是归纳定义的：

```
namespace MyListSpace

inductive List (α : Type*) where
| nil : List α
| cons : α → List α → List α

end MyListSpace
```

该归纳定义说明，`List α` 的每个元素要么是空列表 `nil`，要么是 `cons a as`，其中 `a` 是 `α` 的一个元素，而 `as` 是由 `α` 的元素组成的列表。这两个构造子严格说名为 `List.nil` 和 `List.cons`，但当 `List` 命名空间打开时，可以使用较短的记号。当 `List` 命名空间未打开时，只要 Lean 期待一个列表，就可以写 `.nil` 和 `.cons a as`，Lean 会自动插入 `List` 限定。在本节中，我们会把临时定义放在诸如 `MyListSpace` 这样的独立命名空间中，以避免与标准库冲突。在临时命名空间之外，我们则恢复使用标准库定义。

Lean 将记号 `[]` 定义为 `nil`，将 `::` 定义为 `cons`；并且可以把 `[a, b, c]` 写成 `a :: b :: c :: []`。函数 `append` 和 `map` 按如下方式递归定义：

```
def append {α : Type*} : List α → List α → List α
| [], bs => bs
| a :: as, bs => a :: append as bs

def map {α β : Type*} (f : α → β) : List α → List β
| [] => []
| a :: as => f a :: map f as

#eval append [1, 2, 3] [4, 5, 6]
#eval map (fun n => n^2) [1, 2, 3, 4, 5]
```

注意这里有一个基本情形和一个递归情形。在每种情形中，这两个定义子句都按定义成立：

```

theorem nil_append {α : Type*} (as : List α) : append [] as = as := rfl

theorem cons_append {α : Type*} (a : α) (as : List α) (bs : List α) :
  append (a :: as) bs = a :: append as bs := rfl

theorem map_nil {α β : Type*} (f : α → β) : map f [] = [] := rfl

theorem map_cons {α β : Type*} (f : α → β) (a : α) (as : List α) :
  map f (a :: as) = f a :: map f as := rfl

```

函数 `append` 和 `map` 已在标准库中定义，而 `append as bs` 可以写成 `as ++ bs`。

Lean 允许你按照定义的结构写出归纳证明。

```

variable {α β γ : Type*}
variable (as bs cs : List α)
variable (a b c : α)

open List

theorem append_nil : ∀ as : List α, as ++ [] = as
| [] => rfl
| a :: as => by rw [cons_append, append_nil as]

theorem map_map (f : α → β) (g : β → γ) :
  ∀ as : List α, map g (map f as) = map (g ∘ f) as
| [] => rfl
| a :: as => by rw [map_cons, map_cons, map_cons, map_map f g as]; rfl

```

也可以使用策略 `induction'`。

当然，这些定理已经在标准库中。作为练习，请尝试在命名空间 `MyListSpace3` 中定义一个函数 `reverse`（以避免与标准的 `List.reverse` 冲突），用于反转列表。可以用 `#eval reverse [1, 2, 3, 4, 5]` 测试它。最直接的 `reverse` 定义需要二次时间，但不必为此担心。可以跳转到标准库中 `List.reverse` 的定义，查看一个线性时间实现。请尝试证明 `reverse (as ++ bs) = reverse bs ++ reverse as` 和 `reverse (reverse as) = as`。可以使用 `cons_append` 和 `append_assoc`，但你可能需要提出并证明若干辅助引理。

```

def reverse : List α → List α := sorry

theorem reverse_append (as bs : List α) : reverse (as ++ bs) = reverse bs ++ reverse as := by
  sorry

theorem reverse_reverse (as : List α) : reverse (reverse as) = as := by sorry

```

作为另一个例子，考虑下面的二叉树归纳定义，以及用于计算二叉树大小和深度的函数。

```

inductive BinTree where
| empty : BinTree
| node   : BinTree → BinTree → BinTree

namespace BinTree

def size : BinTree → ℕ
| empty   => 0
| node l r => size l + size r + 1

def depth : BinTree → ℕ
| empty   => 0
| node l r => max (depth l) (depth r) + 1

```

把空二叉树计为大小为 0、深度为 0 的二叉树是方便的。在文献中，这种数据类型有时称为 *extended binary trees*。包含空树意味着，例如我们可以定义树 `node empty (node empty empty)`，它由一个根节点、一棵空左子树和一棵只含单个节点的右子树组成。

下面是联系大小与深度的一个重要不等式：

```

theorem size_le : ∀ t : BinTree, size t ≤ 2^depth t - 1
| empty => Nat.zero_le _
| node l r => by
  simp only [depth, size]
  calc l.size + r.size + 1
    ≤ (2^l.depth - 1) + (2^r.depth - 1) + 1 := by
      gcongr <;> apply size_le
    ≤ (2 ^ max l.depth r.depth - 1) + (2 ^ max l.depth r.depth - 1) + 1 := by
      gcongr <;> simp
    ≤ 2 ^ (max l.depth r.depth + 1) - 1 := by
      have : 0 < 2 ^ max l.depth r.depth := by simp
      omega

```

请尝试证明下面这个不等式，它稍微容易一些。记住，如果像前一个定理那样用归纳法证明，必须删除 `:= by`。

```

theorem depth_le_size : ∀ t : BinTree, depth t ≤ size t := by sorry

```

还请定义二叉树上的 `flip` 操作，它递归地交换左子树与右子树。

```

def flip : BinTree → BinTree := sorry

```

如果定义正确，下面命题的证明应当是 *rfl*。

```

example: flip (node (node empty (node empty empty)) (node empty empty)) =
  node (node empty empty) (node (node empty empty) empty) := sorry

```

证明如下命题：

```

theorem size_flip : ∀ t, size (flip t) = size t := by sorry

```

本节最后讨论一些形式逻辑。下面给出命题公式的归纳定义。

```

inductive PropForm : Type where
| var (n : ℕ)          : PropForm
| fls                  : PropForm
| conj (A B : PropForm) : PropForm
| disj (A B : PropForm) : PropForm
| impl (A B : PropForm) : PropForm

```

每个命题公式要么是变量 `var n`，要么是假常量 `fls`，要么是形如 `conj A B`、`disj A B` 或 `impl A B` 的复合公式。用通常的数学记号，它们分别常写为 p_n 、 $\perp$ 、 $A \wedge B$ 、 $A \vee B$ 和 $A \rightarrow B$ 。其他命题联结词可以由这些联结词定义；例如，可以把 $\neg A$ 定义为 $A \rightarrow \perp$ ，把 $A \leftrightarrow B$ 定义为 $(A \rightarrow B) \wedge (B \rightarrow A)$ 。

定义了命题公式这一数据类型之后，我们定义在把布尔真值赋给变量的赋值 `v` 下，如何求一个命题公式的值。

```

def eval : PropForm → (ℕ → Bool) → Bool
| var n, v => v n
| fls, _ => false
| conj A B, v => A.eval v && B.eval v
| disj A B, v => A.eval v || B.eval v
| impl A B, v => ! A.eval v || B.eval v

```

下一个定义给出公式中出现的变量集合，随后定理表明：若两个真值赋值在某公式的变量上相同，则在这两个赋值下计算该公式会得到相同的值。


```

def vars : PropForm → Finset ℕ
| var n    => {n}
| fls      => ∅
| conj A B => A.vars ∪ B.vars
| disj A B => A.vars ∪ B.vars
| impl A B => A.vars ∪ B.vars

theorem eval_eq_eval : ∀ (A : PropForm) (v1 v2 : ℕ → Bool),
  (∀ n ∈ A.vars, v1 n = v2 n) → A.eval v1 = A.eval v2
| var n, v1, v2, h    => by simp_all [vars, eval]
| fls, v1, v2, h      => by simp_all [eval]
| conj A B, v1, v2, h => by
  simp_all [vars, eval, eval_eq_eval A v1 v2, eval_eq_eval B v1 v2]
| disj A B, v1, v2, h => by
  simp_all [vars, eval, eval_eq_eval A v1 v2, eval_eq_eval B v1 v2]
| impl A B, v1, v2, h => by
  simp_all [vars, eval, eval_eq_eval A v1 v2, eval_eq_eval B v1 v2]

```

注意到这些重复，我们可以更聪明地使用自动化。

```

theorem eval_eq_eval' (A : PropForm) (v1 v2 : ℕ → Bool) (h : ∀ n ∈ A.vars, v1 n = v2 n) :
  A.eval v1 = A.eval v2 := by
  cases A <|> simp_all [eval, vars, fun A => eval_eq_eval' A v1 v2]

```

函数 `subst A m C` 描述把公式 `C` 代入公式 `A` 中变量 `var m` 每一次出现处所得的结果。

```

def subst : PropForm → ℕ → PropForm → PropForm
| var n, m, C => if n = m then C else var n
| fls, _, _ => fls
| conj A B, m, C => conj (A.subst m C) (B.subst m C)
| disj A B, m, C => disj (A.subst m C) (B.subst m C)
| impl A B, m, C => impl (A.subst m C) (B.subst m C)

```

作为例子，证明：若一个变量不出现在公式中，则对该变量作代入没有任何影响：

```

theorem subst_eq_of_not_mem_vars :
  ∀ (A : PropForm) (n : ℕ) (C : PropForm), n ∉ A.vars → A.subst n C = A := sorry

```

下面的定理说的是更细致也更有意思的事实：在真值赋值 `v` 下求 `A.subst n C` 的值，等同于在一个把 `var n` 赋为 `C` 的值的真值赋值下求 `A` 的值。请尝试证明它。

```

theorem subst_eval_eq : ∀ (A : PropForm) (n : ℕ) (C : PropForm) (v : ℕ → Bool),
  (A.subst n C).eval v = A.eval (fun m => if m = n then C.eval v else v m) := sorry

```

7. 结构

现代数学本质上依赖代数结构；这些结构封装了可在多种情境中实例化的模式。数学提供了定义这类结构并构造其具体实例的多种方法。

因此，Lean 也提供了形式化定义结构并使用结构的相应方式。你已经在 Lean 中见过一些代数结构的例子，例如Chapter 2中讨论过的环和格。本章将解释那里出现过的神秘方括号标注 `[Ring  $\alpha$ ]` 和 `[Lattice  $\alpha$ ]`，并说明如何自己定义和使用代数结构。

若需要更多技术细节，可以参考 [Theorem Proving in Lean](#)，以及 Anne Baanen 的论文 [Use and abuse of instance parameters in the Lean mathematical library](#)。

7.1. 定义结构

在最广泛的意义上，*structure*是对一组数据的规约，其中可能还包含这些数据必须满足的约束。该结构的一个*instance*，就是满足这些约束的一份具体数据包。例如，可以规定一个点是三个实数构成的元组：

```
@[ext]
structure Point where
  x : ℝ
  y : ℝ
  z : ℝ
```

标注 `@[ext]` 告诉 Lean 自动生成可用于证明结构两个实例相等的定理：只要它们的各个分量相等，就能证明实例相等。这一性质称为*extensionality*。

```
#check Point.ext

example (a b : Point) (hx : a.x = b.x) (hy : a.y = b.y) (hz : a.z = b.z) : a = b := by
  ext
  repeat' assumption
```

随后可以定义 `Point` 结构的具体实例。Lean 提供了多种做法。

```
def myPoint1 : Point where
  x := 2
  y := -1
  z := 4

def myPoint2 : Point :=
  { x := 2, y := -1, z := 4 }

def myPoint3 : Point :=
  (2, -1, 4)

def myPoint4 :=
  Point.mk 2 (-1) 4
```

在第一个例子中，结构的字段被显式命名。`myPoint4` 定义中提到的函数 `Point.mk` 称为 `Point` 结构的*constructor*，因为它用于构造元素。若愿意，也可以指定另一个名称，例如 `build`。

```
structure Point' where build ::
  x : ℝ
  y : ℝ
  z : ℝ

#check Point'.build 2 (-1) 4
```

接下来的两个例子展示如何在结构上定义函数。第二个例子显式写出了构造子 `Point.mk`，而第一个例子为了简洁使用匿名构造子。Lean 可以根据所标明的 `add` 的类型推断出相关构造子。通常约定，将与 `Point` 这类结构相关的定义和定理放入同名命名空间。在下面的例子中，由于已经打开 `Point` 命名空间，`add` 的全名是 `Point.add`。当命名空间未打开时，必须使用全名。不过请记住，匿名投影记号常常很方便，它允许我们写 `a.add b` 而不是 `Point.add a b`。Lean 会把前者解释为后者，因为 `a` 的类型是 `Point`。

```
namespace Point

def add (a b : Point) : Point :=
  (a.x + b.x, a.y + b.y, a.z + b.z)

def add' (a b : Point) : Point where
  x := a.x + b.x
  y := a.y + b.y
  z := a.z + b.z

#check add myPoint1 myPoint2
#check myPoint1.add myPoint2

end Point

#check Point.add myPoint1 myPoint2
#check myPoint1.add myPoint2
```

下面我们仍会把定义放在相关命名空间中，但在引用的代码片段里省略命名空间命令。为了证明加法函数的性质，可以用 `rw` 展开定义，并用 `ext` 把结构两个元素之间的等式化归为各分量之间的等式。下面我们使用关键字 `protected`，使得即使命名空间已经打开，该定理名称仍为 `Point.add_comm`。当我们希望避免与 `add_comm` 这样的通用定理产生歧义时，这很有用。

```
protected theorem add_comm (a b : Point) : add a b = add b a := by
  rw [add, add]
  ext <;> dsimp
  repeat' apply add_comm

example (a b : Point) : add a b = add b a := by simp [add, add_comm]
```

由于 Lean 可以在内部展开定义并化简投影，有时我们想要的等式会按定义成立。

```
theorem add_x (a b : Point) : (a.add b).x = a.x + b.x :=
  rfl
```

也可以用模式匹配在结构上定义函数，方式类似于我们在Section 5.2中定义递归函数的方式。下面的定义 `addAlt` 与 `addAlt'` 本质相同；唯一区别是第二个使用匿名构造子记号。虽然有时这样定义函数很方便，而且结构 η -约化使这个替代定义按定义等价，但在后续证明中它可能不那么方便。特别地，`rw [addAlt]` 会留下一个含有 `match` 语句、更混乱的目标视图。

```
def addAlt : Point → Point → Point
| Point.mk x1 y1 z1, Point.mk x2 y2 z2 => ⟨x1 + x2, y1 + y2, z1 + z2⟩

def addAlt' : Point → Point → Point
| ⟨x1, y1, z1⟩, ⟨x2, y2, z2⟩ => ⟨x1 + x2, y1 + y2, z1 + z2⟩

theorem addAlt_x (a b : Point) : (a.addAlt b).x = a.x + b.x := by
  rfl

theorem addAlt_comm (a b : Point) : addAlt a b = addAlt b a := by
  rw [addAlt, addAlt]
  -- the same proof still works, but the goal view here is harder to read
  ext <;> dsimp
  repeat' apply add_comm
```

数学构造常常需要拆解已打包的信息，再以不同方式重新组合。因此 Lean 和 Mathlib 提供许多高效完成这件事的方法，是很自然的。作为练习，请尝试证明 `Point.add` 满足结合律。然后为点定义标量乘法，并证明它对加法满足分配律。

```
protected theorem add_assoc (a b c : Point) : (a.add b).add c = a.add (b.add c) := by
  sorry

def smul (r : ℝ) (a : Point) : Point :=
  sorry

theorem smul_distrib (r : ℝ) (a b : Point) :
  (smul r a).add (smul r b) = smul r (a.add b) := by
  sorry
```

使用结构只是通向代数抽象的第一步。我们还没有办法把 `Point.add` 与通用符号 `+` 关联起来，也没有办法把 `Point.add_comm` 和 `Point.add_assoc` 与通用定理 `add_comm` 和 `add_assoc` 关联起来。这些任务属于使用结构的 *algebraic* 方面，我们将在下一节解释如何完成。现在，只需把结构看作打包对象和信息的一种方式。

结构特别有用的一点是：它不仅能指定数据类型，也能指定这些数据必须满足的约束。在 Lean 中，后者表示为类型为 `Prop` 的字段。例如，*standard 2-simplex* 定义为满足 $x \geq 0$ 、 $y \geq 0$ 、 $z \geq 0$ 和 $x + y + z = 1$ 的点 (x, y, z) 的集合。如果你不熟悉这个概念，应当画一幅图，并说服自己这个集合就是三维空间中以 $(1, 0, 0)$ 、 $(0, 1, 0)$ 和 $(0, 0, 1)$ 为顶点的等边三角形连同其内部。在 Lean 中可以如下表示它：

```
structure StandardTwoSimplex where
  x : ℝ
  y : ℝ
  z : ℝ
  x_nonneg : 0 ≤ x
  y_nonneg : 0 ≤ y
  z_nonneg : 0 ≤ z
  sum_eq : x + y + z = 1
```

注意，最后四个字段引用了 `x`、`y` 和 `z`，也就是前三个字段。可以定义一个从二单纯形到自身的映射，交换 `x` 与 `y`：

```
def swapXy (a : StandardTwoSimplex) : StandardTwoSimplex
  where
  x := a.y
  y := a.x
  z := a.z
  x_nonneg := a.y_nonneg
  y_nonneg := a.x_nonneg
  z_nonneg := a.z_nonneg
  sum_eq := by rw [add_comm a.y a.x, a.sum_eq]
```

更有趣的是，可以计算单纯形上两个点的中点。为了使用实数上的除法，我们在本文件开头加入了短语 `noncomputable section`。

```
noncomputable section

def midpoint (a b : StandardTwoSimplex) : StandardTwoSimplex
  where
  x := (a.x + b.x) / 2
  y := (a.y + b.y) / 2
  z := (a.z + b.z) / 2
  x_nonneg := div_nonneg (add_nonneg a.x_nonneg b.x_nonneg) (by norm_num)
  y_nonneg := div_nonneg (add_nonneg a.y_nonneg b.y_nonneg) (by norm_num)
  z_nonneg := div_nonneg (add_nonneg a.z_nonneg b.z_nonneg) (by norm_num)
  sum_eq := by field_simp; linarith [a.sum_eq, b.sum_eq]
```

这里我们用简短的证明项建立了 `x_nonneg`、`y_nonneg` 和 `z_nonneg`，但用策略模式通过 `by` 建立 `sum_eq`。

给定满足 $0 \leq \lambda \leq 1$ 的参数 λ ，可以取标准 2-单纯形中两点 a 和 b 的加权平均 $\lambda a + (1 - \lambda)b$ 。请仿照上面的 `midpoint` 函数定义这个函数。

```
def weightedAverage (lambda : Real) (lambda_nonneg : 0 ≤ lambda) (lambda_le : lambda ≤ 1)
  (a b : StandardTwoSimplex) : StandardTwoSimplex :=
  sorry
```

结构可以依赖参数。例如，可以把标准 2-单纯形推广为任意 n 的标准 n -单纯形。在这个阶段，你不需要知道类型 `Fin n` 的任何内容，只需知道它有 n 个元素，并且 Lean 知道如何在其上求和。

```
open BigOperators

structure StandardSimplex (n : ℕ) where
  V : Fin n → ℝ
  NonNeg : ∀ i : Fin n, 0 ≤ V i
  sum_eq_one : (∑ i, V i) = 1

namespace StandardSimplex

def midpoint (n : ℕ) (a b : StandardSimplex n) : StandardSimplex n
  where
  V i := (a.V i + b.V i) / 2
  NonNeg := by
    intro i
    apply div_nonneg
    · linarith [a.NonNeg i, b.NonNeg i]
    norm_num
  sum_eq_one := by
    simp [div_eq_mul_inv, ← Finset.sum_mul, Finset.sum_add_distrib,
      a.sum_eq_one, b.sum_eq_one]
    norm_num

end StandardSimplex
```

作为练习，请看看能否定义标准 n -单纯形中两点的加权平均。可以使用 `Finset.sum_add_distrib` 和 `Finset.mul_sum` 操作相关求和。

我们已经看到，结构可以把数据和性质打包在一起。有趣的是，它们也可以只打包性质而不包含数据。例如，下一个结构 `IsLinear` 把线性的两个组成部分打包在一起。

```
structure IsLinear (f : ℝ → ℝ) where
  is_additive : ∀ x y, f (x + y) = f x + f y
  preserves_mul : ∀ x c, f (c * x) = c * f x

section
variable (f : ℝ → ℝ) (linf : IsLinear f)

#check linf.is_additive
#check linf.preserves_mul

end
```

值得指出的是，结构并不是打包数据的唯一方式。数据结构 `Point` 可以用通用的类型乘积定义，而 `IsLinear` 可以用简单的 `and` 定义。

```
def Point'' :=
  ℝ × ℝ × ℝ

def IsLinear' (f : ℝ → ℝ) :=
  (∀ x y, f (x + y) = f x + f y) ∧ ∀ x c, f (c * x) = c * f x
```

通用类型构造甚至可以替代分量之间存在依赖的结构。例如，*subtype*构造把一份数据与一个性质结合起来。可以把下一个例子中的类型 `PReal` 看作正实数类型。任意 `x : PReal` 都有两个分量：它的值，以及它为正的性质。可以通过 `x.val` 访问前者，其类型为 `ℝ`；通过 `x.property` 访问后者，它表示事实 `0 < x.val`。

```
def PReal :=
  { y : ℝ // 0 < y }

section
variable (x : PReal)

#check x.val
#check x.property
#check x.1
#check x.2

end
```

我们本来也可以用子类型定义标准 2-单纯形，以及任意 n 的标准 n -单纯形。

```
def StandardTwoSimplex' :=
  { p : ℝ × ℝ × ℝ // 0 ≤ p.1 ∧ 0 ≤ p.2.1 ∧ 0 ≤ p.2.2 ∧ p.1 + p.2.1 + p.2.2 = 1 }

def StandardSimplex' (n : ℕ) :=
  { v : Fin n → ℝ // (∀ i : Fin n, 0 ≤ v i) ∧ (∑ i, v i) = 1 }
```

类似地，*Sigma types* 是有序对的推广，其中第二个分量的类型依赖于第一个分量的类型。

```
def StdSimplex := ∑ n : ℕ, StandardSimplex n

section
variable (s : StdSimplex)

#check s.fst
#check s.snd

#check s.1
#check s.2

end
```

给定 `s : StdSimplex`，第一个分量 `s.fst` 是一个自然数，第二个分量是相应单纯形 `StandardSimplex s.fst` 的一个元素。Sigma 类型与子类型的区别在于，Sigma 类型的第二个分量是数据，而不是命题。

不过，尽管可以用乘积、子类型和 Sigma 类型替代结构，使用结构仍有许多优势。定义结构会抽象掉底层表示，并为访问各分量的函数提供自定义名称。这使证明更加稳健：只依赖结构接口的证明，通常在我们改变定义后仍能继续工作，只要把旧访问器用新定义重新定义出来。此外，正如我们马上将看到的，Lean 支持把结构编织成丰富且相互关联的层级体系，并管理它们之间的相互作用。

7.2. 代数结构

为了澄清 “*algebraic structure*” 这一短语的含义，考虑一些例子会很有帮助。

partially ordered set 由一个集合 P 以及 P 上的二元关系 $\leq$ 组成，该关系满足传递性、自反性和反对称性。

group 由一个集合 G 、一个结合的二元运算、一个单位元 1 ，以及一个函数 $g \mapsto g^{-1}$ 组成；该函数为 G 中每个 g 返回一个逆元。若运算可交换，则群称为 *abelian* 或 *commutative*。

lattice 是带有交和并的偏序集。

ring 由一个（以加法方式书写的）阿贝尔群 $(R, +, 0, x \mapsto -x)$ ，连同一个结合的乘法运算 $\cdot$ 和单位元 1 组成，并且乘法对加法满足分配律。若乘法可交换，则环称为 *commutative* 环。

ordered ring $(R, +, 0, -, \cdot, 1, \leq)$ 由一个环连同其元素上的偏序组成，使得对 R 中任意 a, b 和 c ，若 $a \leq b$ ，则 $a + c \leq b + c$ ；并且对 R 中任意 a 和 b ，若 $0 \leq a$ 且 $0 \leq b$ ，则 $0 \leq ab$ 。

metric space 由一个集合 X 和函数 $d : X \times X \rightarrow \mathbb{R}$ 组成，满足如下性质：对 X 中任意 x 和 y ，有 $d(x, y) \geq 0$ 。
 $d(x, y) = 0$ 当且仅当 $x = y$ 。对 X 中任意 x 和 y ，有 $d(x, y) = d(y, x)$ 。对 X 中任意 x, y 和 z ，有 $d(x, z) \leq d(x, y) + d(y, z)$ 。

topological space 由一个集合 X 和 X 的子集族 $\mathcal{T}$ 组成，后者称为 X 的 *open subsets of*，并满足如下性质：空集和 X 是开集。两个开集的交是开集。任意开集族的并是开集。

在这些例子中，结构的元素属于一个集合，即 *carrier set*；它有时代表整个结构。例如，当我们说“令 G 为一个群”，随后又说“令 $g \in G$ ”时，我们同时用 G 表示该结构及其载集。并非每个代数结构都以这种方式关联到单个载集。例如，*bipartite graph* 涉及两个集合之间的关系，*Galois connection* 也是如此。*category* 也涉及两个重要集合，通常称为 *objects* 和 *morphisms*。

这些例子显示，为了支持代数推理，证明助手必须完成一些任务。首先，它需要识别结构的具体实例。数系 $\mathbb{Z}$ 、 $\mathbb{Q}$ 和 $\mathbb{R}$ 都是有序环，因此我们应能在这些任一实例中应用关于有序环的通用定理。有时，一个具体集合可以以不止一种方式成为某结构的实例。例如，除了作为实分析基础的 $\mathbb{R}$ 上通常拓扑之外，我们还可以考虑 $\mathbb{R}$ 上的 *discrete* 拓扑，其中每个集合都是开集。

其次，证明助手需要支持结构上的通用记号。在 Lean 中，记号 `*` 用于所有通常数系中的乘法，也用于通用群和环中的乘法。当我们使用类似 `f x * y` 的表达式时，Lean 必须利用关于 `f`、`x` 和 `y` 的类型信息来判定我们意指哪一种乘法。

第三，它需要处理这样一个事实：结构能够以多种方式从其他结构继承定义、定理和记号。有些结构通过增加更多公理来扩展其他结构。交换环仍然是环，所以任何在环中有意义的定义在交换环中也有意义，任何在环中成立的定理在交换环中也成立。有些结构通过增加更多数据来扩展其他结构。例如，任意环的加法部分是一个加法群。环结构增加了乘法和单位元，以及支配它们并把它们同加法部分联系起来的公理。有时我们可以用一个结构定义另一个结构。任意度量空间都有一个与之关联的典范拓扑，即 *metric space topology*；任意线性序也可以关联多种拓扑。

最后，需要牢记，数学允许我们用函数和运算定义结构，正如我们用函数和运算定义数一样。群的乘积和幂仍是群。对每个 n ，模 n 整数形成一个环；对每个 $k > 0$ ，以该环中元素为系数的多项式的 $k \times k$ 矩阵又形成一个环。因此，我们可以像计算结构的元素一样方便地计算结构本身。这意味着代数结构在数学中具有双重身份：既是对象集合的容器，又是独立的对象。证明助手必须适应这种双重角色。

在处理一个具有相关代数结构的类型的元素时，证明助手需要识别该结构，并找到相关定义、定理和记号。这些听起来像是大量工作，而事实也确实如此。不过 Lean 使用一小组基本机制来完成这些任务。本节的目标就是解释这些机制，并展示如何使用它们。

第一个成分几乎显而易见：从形式上说，代数结构就是 [Section 7.1](#) 意义下的结构。代数结构是一组满足某些公理假设的数据的规约，而我们在 [Section 7.1](#) 中已经看到，`structure` 命令正是为容纳这种内容而设计的。这种配合可谓恰到好处。

给定一个数据类型 `α`，可以如下在 `α` 上定义群结构。

```
structure Group₁ (α : Type*) where
  mul : α → α → α
  one : α
  inv : α → α
  mul_assoc : ∀ x y z : α, mul (mul x y) z = mul x (mul y z)
  mul_one : ∀ x : α, mul x one = x
  one_mul : ∀ x : α, mul one x = x
  inv_mul_cancel : ∀ x : α, mul (inv x) x = one
```

注意，类型 α 是定义 `Group1` 中的一个 *parameter*。因此，应把对象 `struc : Group1  $\alpha$` 理解为 α 上的一个群结构。我们在Section 2.2中看到，`inv_mul_cancel` 的对应形式 `mul_inv_cancel` 可由其他群公理推出，因此无需把它加入定义。

这个群定义类似于 Mathlib 中 `Group` 的定义；我们选择名称 `Group1` 来区分自己的版本。如果写 `#check Group` 并按住 Ctrl 点击定义，会看到 Mathlib 版本的 `Group` 被定义为扩展另一个结构；稍后会解释如何做到这一点。如果输入 `#print Group`，还会看到 Mathlib 版本的 `Group` 有若干额外字段。出于稍后要解释的原因，有时向结构中加入冗余信息是有用的，也就是为可由核心数据定义出的对象和函数增加额外字段。现在不必担心这一点。可以放心，我们的简化版本 `Group1` 在数学本质上与 Mathlib 使用的群定义相同。

有时把类型与结构打包在一起是有用的，Mathlib 也包含一个与下述定义等价的 `Grp` 结构定义：

```
structure Grp1 where
   $\alpha$  : Type*
  str : Group1  $\alpha$ 
```

Mathlib 版本位于 `Mathlib.Algebra.Category.Grp.Basic` 中；若把它加入示例文件开头的导入中，就可以对它执行 `#check`。

出于下面会逐渐清楚的原因，更常用的做法是把类型 α 与结构 `Group  $\alpha$` 分开。我们把这两个对象合称为 *partially bundled structure*，因为这种表示把大多数分量、但不是全部分量合并到一个结构中。在 Mathlib 中，当某个类型用作群的载体类型时，通常用像 `G` 这样的大写罗马字母表示它。

让我们通过定义 `Group1` 类型的一个元素来构造一个群。对于任意一对类型 α 和 β ，Mathlib 定义了类型 `Equiv  $\alpha$   $\beta$` ，表示 α 与 β 之间的 *equivalences*。Mathlib 还为这个类型定义了直观的记号 $\alpha \simeq \beta$ 。元素 `f :  $\alpha \simeq \beta$` 是 α 与 β 之间的一个双射，由四个分量表示：从 α 到 β 的函数 `f.toFun`，从 β 到 α 的逆函数 `f.invFun`，以及说明这两个函数确实互为逆的两个性质。

```
variable ( $\alpha$   $\beta$   $\gamma$  : Type*)
variable (f :  $\alpha \simeq \beta$ ) (g :  $\beta \simeq \gamma$ )

#check Equiv  $\alpha$   $\beta$ 
#check (f.toFun :  $\alpha \rightarrow \beta$ )
#check (f.invFun :  $\beta \rightarrow \alpha$ )
#check (f.right_inv :  $\forall x : \beta, f (f.invFun x) = x$ )
#check (f.left_inv :  $\forall x : \alpha, f.invFun (f x) = x$ )
#check (Equiv.refl  $\alpha$  :  $\alpha \simeq \alpha$ )
#check (f.symm :  $\beta \simeq \alpha$ )
#check (f.trans g :  $\alpha \simeq \gamma$ )
```

注意最后三个构造名称颇具创造性。我们把恒等函数 `Equiv.refl`、取逆操作 `Equiv.symm` 和复合操作 `Equiv.trans` 看作“双射对应”这一性质为等价关系的显式证据。

还要注意，`f.trans g` 要求按相反顺序复合正向函数。Mathlib 已经声明了从 `Equiv  $\alpha$   $\beta$` 到函数类型 $\alpha \rightarrow \beta$ 的 *coercion*，因此可以省略 `.toFun`，让 Lean 为我们插入它。

```
example (x :  $\alpha$ ) : (f.trans g).toFun x = g.toFun (f.toFun x) :=
  rfl

example (x :  $\alpha$ ) : (f.trans g) x = g (f x) :=
  rfl

example : (f.trans g :  $\alpha \rightarrow \gamma$ ) = g  $\circ$  f :=
  rfl
```

Mathlib 还定义了类型 `Perm  $\alpha$` ，表示 α 到自身的等价。


```
example (α : Type*) : Equiv.Perm α = (α ≃ α) :=
  rfl
```

显然，`Equiv.Perm α` 在等价复合下形成一个群。我们约定方向，使 `mul f g` 等于 `g.trans f`，后者的正向函数是 `f ∘ g`。换言之，乘法就是通常意义下双射的复合。下面定义这个群：

```
def permGroup {α : Type*} : Group₁ (Equiv.Perm α)
  where
  mul f g := Equiv.trans g f
  one := Equiv.refl α
  inv := Equiv.symm
  mul_assoc f g h := (Equiv.trans_assoc h g f).symm
  one_mul := Equiv.trans_refl
  mul_one := Equiv.refl_trans
  inv_mul_cancel := Equiv.self_trans_symm
```

事实上，Mathlib 在文件 `Algebra.Group.End` 中为 `Equiv.Perm α` 定义的正是这个 `Group` 结构。照常，可以把鼠标悬停在 `permGroup` 定义中使用的定理上查看其陈述，也可以跳转到原文件中的定义，进一步了解它们如何实现。

在通常数学中，我们一般认为记号独立于结构。例如，可以考虑群 $(G_1, \cdot, 1, \cdot^{-1})$ 、 $(G_2, \circ, e, i(\cdot))$ 和 $(G_3, +, 0, -)$ 。在第一种情况下，把二元运算写作 $\cdot$ ，把单位元写作 1 ，把取逆函数写作 $x \mapsto x^{-1}$ 。在第二和第三种情况下，我们使用所示的替代记号。然而，当在 Lean 中形式化群的概念时，记号与结构联系得更紧密。在 Lean 中，任意 `Group` 的分量命名为 `mul`、`one` 和 `inv`，稍后我们会看到乘法记号如何设置为引用这些分量。如果想使用加法记号，则改用同构的结构 `AddGroup`（即加法群底层的结构）。它的分量命名为 `add`、`zero` 和 `neg`，相应记号也正如预期。

回忆在 Section 7.1 中定义的类型 `Point`，以及在那里定义加法函数。这些定义在本节配套的示例文件中再次给出。作为练习，请定义一个 `AddGroup₁` 结构，它类似于上面定义的 `Group₁` 结构，只是使用刚才描述加法命名方案。请在数据类型 `Point` 上定义取负和零元，并在 `Point` 上定义 `AddGroup₁` 结构。

```
structure AddGroup₁ (α : Type*) where
  (add : α → α → α)
  -- fill in the rest
@[ext]
structure Point where
  x : ℝ
  y : ℝ
  z : ℝ

namespace Point

def add (a b : Point) : Point :=
  (a.x + b.x, a.y + b.y, a.z + b.z)

def neg (a : Point) : Point := sorry

def zero : Point := sorry

def addGroupPoint : AddGroup₁ Point := sorry

end Point
```

我们正在取得进展。现在我们知道如何在 Lean 中定义代数结构，也知道如何定义这些结构的实例。但我们还希望把记号同结构关联起来，使得每个实例都能使用这些记号。此外，我们还希望安排成：可以在一个结构上定义操作，并在任何具体实例中使用它；也可以证明关于一个结构的定理，并在任何实例中使用它。

事实上，Mathlib 已经设置好，可以对 `Equiv.Perm α` 使用通用的群记号、定义和定理。

```

variable {α : Type*} (f g : Equiv.Perm α) (n : ℕ)

#check f * g
#check mul_assoc f g g⁻¹

-- group power, defined for any group
#check g ^ n

example : f * g * g⁻¹ = f := by rw [mul_assoc, mul_inv_cancel, mul_one]

example : f * g * g⁻¹ = f :=
  mul_inv_cancel_right f g

example {α : Type*} (f g : Equiv.Perm α) : g.symm.trans (g.trans f) = f :=
  mul_inv_cancel_right f g

```

你可以检查，上面要求你为 `Point` 定义的加法群结构并非如此。现在我们的任务是理解底层发生了什么“魔法”，使得 `Equiv.Perm α` 的例子能够这样工作。

问题在于，Lean 需要能够利用我们输入的表达式中出现的信息，*find* 相关记号和隐式的群结构。类似地，当我们用类型为 $\mathbb{R}$ 的表达式 `x` 和 `y` 写 `x + y` 时，Lean 需要把符号 `+` 解释为实数上的相关加法函数。它还必须识别类型 $\mathbb{R}$ 是交换环的一个实例，从而可以使用交换环的所有定义和定理。再举一个例子，Lean 中的连续性是相对于任意两个拓扑空间定义的。当有 `f : ℝ → ℂ` 且写 `Continuous f` 时，Lean 必须找到 $\mathbb{R}$ 和 $\mathbb{C}$ 上的相关拓扑。

这种机制由三件事结合实现。

逻辑。应在任意群中解释的定义，把群的类型和群结构作为参数。类似地，关于任意群元素的定理，以对群的类型和群结构的全称量词开头。

隐式参数。类型参数和结构参数通常留作隐式参数，这样我们既不必书写它们，也不必在 Lean 信息窗口中看到它们。

Lean 会在幕后为我们填充这些信息。

类型类推断。又称 *class inference*，这是一种简单但强大的机制，使我们能够注册信息供 Lean 稍后使用。当 Lean 被要求为定义、定理或记号片段填充隐式参数时，它可以利用已经注册的信息。

标注 `(grp : Group G)` 告诉 Lean 应期待显式给出该参数；标注 `{grp : Group G}` 告诉 Lean 应尝试从表达式中的上下文线索推断它；而标注 `[grp : Group G]` 则告诉 Lean 应使用类型类推断来合成相应参数。由于使用这类参数的要点在于我们通常不需要显式引用它们，Lean 允许写 `[Group G]` 并让名称匿名。你大概已经注意到，Lean 会自动选择像 `_inst_1` 这样的名称。当把匿名方括号标注同 `variable` 命令一起使用时，只要变量仍在作用域内，Lean 就会自动向任何提到 `G` 的定义或定理添加参数 `[Group G]`。

如何注册 Lean 执行搜索所需的信息？回到群的例子，只需作两处改动。第一，不使用 `structure` 命令定义群结构，而使用关键字 `class` 表明它是类型类推断的候选。第二，不用 `def` 定义具体实例，而用关键字 `instance` 把该具体实例注册给 Lean。与类变量名一样，实例定义的名称也允许匿名，因为一般而言我们希望 Lean 找到它并使用它，而不必用细节打扰我们。

```

class Group₂ (α : Type*) where
  mul : α → α → α
  one : α
  inv : α → α
  mul_assoc : ∀ x y z : α, mul (mul x y) z = mul x (mul y z)
  mul_one : ∀ x : α, mul x one = x
  one_mul : ∀ x : α, mul one x = x
  inv_mul_cancel : ∀ x : α, mul (inv x) x = one

instance {α : Type*} : Group₂ (Equiv.Perm α) where
  mul f g := Equiv.trans g f
  one := Equiv.refl α
  inv := Equiv.symm
  mul_assoc f g h := (Equiv.trans_assoc h g f).symm
  one_mul := Equiv.trans_refl
  mul_one := Equiv.refl_trans
  inv_mul_cancel := Equiv.self_trans_symm

```

下面说明它们的用法。

```
#check Group₂.mul

def mySquare {α : Type*} [Group₂ α] (x : α) :=
  Group₂.mul x x

#check mySquare

section
variable {β : Type*} (f g : Equiv.Perm β)

example : Group₂.mul f g = g.trans f :=
  rfl

example : mySquare f = f.trans f :=
  rfl

end
```

命令 `#check` 显示, `Group₂.mul` 有一个隐式参数 `[Group₂ α]`, 我们期待它由类型类推断找到, 其中 `α` 是 `Group₂.mul` 的参数类型。换言之, `{α : Type*}` 是群元素类型的隐式参数, `[Group₂ α]` 是 `α` 上群结构的隐式参数。类似地, 当为 `Group₂` 定义通用平方函数 `my_square` 时, 我们使用隐式参数 `{α : Type*}` 表示元素类型, 并使用隐式参数 `[Group₂ α]` 表示 `Group₂` 结构。

在第一个例子中, 当写 `Group₂.mul f g` 时, `f` 和 `g` 的类型告诉 Lean, `Group₂.mul` 的参数 `α` 必须实例化为 `Equiv.Perm β`。这意味着 Lean 必须找到 `Group₂ (Equiv.Perm β)` 的一个元素。前面的 `instance` 声明恰好告诉 Lean 如何做这一点。问题解决了。

这种注册信息、使 Lean 在需要时能够找到它的简单机制极其有用。下面是一种出现方式。在 Lean 的基础中, 数据类型 `α` 可以为空。然而在许多应用中, 知道某个类型至少有一个元素是有用的。例如, 函数 `List.headI` 返回列表的第一个元素; 当列表为空时, 它可以返回默认值。为了使这件事可行, Lean 库定义了类 `Inhabited α`, 它只是存储一个默认值。我们可以证明类型 `Point` 是一个实例:

```
instance : Inhabited Point where default := ⟨0, 0, 0⟩

#check (default : Point)

example : ([] : List Point).headI = default :=
  rfl
```

类型类推断机制也用于通用记号。表达式 `x + y` 是 `Add.add x y` 的缩写, 其中, 如你所料, `Add α` 是一个在 `α` 上存储二元函数的类。写 `x + y` 会告诉 Lean 寻找 `[Add.add α]` 的已注册实例并使用相应函数。下面, 我们为 `Point` 注册加法函数。

```
instance : Add Point where add := Point.add

section
variable (x y : Point)

#check x + y

example : x + y = Point.add x y :=
  rfl

end
```

这样, 也可以把记号 `+` 指派给其他类型上的二元运算。

但还可以做得更好。我们已经看到，`*` 可用于任意群，`+` 可用于任意加法群，而二者都可用于任意环。当在 Lean 中定义一个新的环实例时，不必为该实例定义 `+` 和 `*`，因为 Lean 知道它们已为每个环定义。我们可以用这种方法为类 `Group₂` 指定记号：

```
instance {α : Type*} [Group₂ α] : Mul α :=
  (Group₂.mul)

instance {α : Type*} [Group₂ α] : One α :=
  (Group₂.one)

instance {α : Type*} [Group₂ α] : Inv α :=
  (Group₂.inv)

section
variable {α : Type*} (f g : Equiv.Perm α)

#check f * 1 * g⁻¹

def foo : f * 1 * g⁻¹ = g.symm.trans ((Equiv.refl α).trans f) :=
  rfl

end
```

这种方法之所以有效，是因为 Lean 会执行递归搜索。根据我们声明的实例，Lean 可以通过寻找 `Group₂ (Equiv.Perm α)` 的实例来找到 `Mul (Equiv.Perm α)` 的实例；而它又能找到 `Group₂ (Equiv.Perm α)` 的实例，因为我们已经提供了一个。Lean 能够发现这两个事实并把它们串联起来。

刚才给出的例子是危险的，因为 Lean 库中也有 `Group (Equiv.Perm α)` 的实例，而乘法在任意群上都已定义。因此，究竟找到哪个实例是有歧义的。事实上，除非显式指定不同优先级，Lean 会偏向较新的声明。此外，还有另一种方式告诉 Lean 一个结构是另一个结构的实例，即使用关键字 `extends`。Mathlib 正是以这种方式说明例如每个交换环都是环。更多信息可参见 [Section 8](#) 以及 *Theorem Proving in Lean* 中关于类型类推断的 [section on class inference](#)。

一般来说，对于一个已经定义了记号的代数结构实例，再为 `*` 指定取值并不是好主意。在 Lean 中重新定义 `Group` 的概念只是一个人为例子。不过，在这种情况下，群记号的两种解释都会以同样方式展开为 `Equiv.trans`、`Equiv.refl` 和 `Equiv.symm`。

作为一个类似的人为练习，请仿照 `Group₂` 定义一个类 `AddGroup₂`。使用类 `Add`、`Neg` 和 `Zero`，为任意 `AddGroup₂` 定义通常的加法、取负和零元记号。然后证明 `Point` 是 `AddGroup₂` 的一个实例。请试用一下，并确认加法群记号对 `Point` 的元素有效。

```
class AddGroup₂ (α : Type*) where
  add : α → α → α
  -- fill in the rest
```

上面我们已经为 `Point` 声明了 `Add`、`Neg` 和 `Zero` 的实例，这并不是什么大问题。再次，两种合成记号的方式应当得到相同答案。

类型类推断很微妙，使用时必须谨慎，因为它配置的自动化会在不可见处支配我们输入表达式的解释。不过，若使用得当，类型类推断是一个强有力的工具。正是它使 Lean 中的代数推理成为可能。

7.3. 构造高斯整数

现在我们通过构造一个重要数学对象，即 *Gaussian integers*，并证明它是欧几里得整环，来展示 Lean 中代数层级的用法。换言之，按照一直使用的术语，我们将定义高斯整数，并证明它们是欧几里得整环结构的一个实例。

用通常数学语言说，高斯整数集 $\mathbb{Z}[i]$ 是复数集合 $\{a + bi \mid a, b \in \mathbb{Z}\}$ 。不过，这里我们的目标不是把它们定义为复数的子集，而是把它们定义为一个自身独立的数据类型。为此，我们把高斯整数表示为一对整数，分别看作 *real* 部和 *imaginary* 部。

```
@[ext]
structure GaussInt where
  re : ℤ
  im : ℤ
```

我们首先证明高斯整数具有环结构，其中 0 定义为 $\langle 0, 0 \rangle$ ， 1 定义为 $\langle 1, 0 \rangle$ ，加法逐点定义。为了确定乘法定义，记住我们希望元素 i （表示为 $\langle 0, 1 \rangle$ ）是 -1 的平方根。因此我们希望

$$\begin{aligned}(a + bi)(c + di) &= ac + bci + adi + bdi^2 \\ &= (ac - bd) + (bc + ad)i.\end{aligned}$$

这就解释了下面 `Mul` 的定义。

```
instance : Zero GaussInt :=
  ⟨⟨0, 0⟩⟩

instance : One GaussInt :=
  ⟨⟨1, 0⟩⟩

instance : Add GaussInt :=
  (fun x y ↦ ⟨x.re + y.re, x.im + y.im⟩)

instance : Neg GaussInt :=
  (fun x ↦ ⟨-x.re, -x.im⟩)

instance : Mul GaussInt :=
  (fun x y ↦ ⟨x.re * y.re - x.im * y.im, x.re * y.im + x.im * y.re⟩)
```

如Section 7.1所述，把与某个数据类型相关的所有定义放在同名命名空间中是一个好习惯。因此，在与本章相关的 Lean 文件中，这些定义都放在 `GaussInt` 命名空间中。

注意，这里我们直接定义记号 0 、 1 、 $+$ 、 $-$ 和 $*$ 的解释，而不是把它们命名为 `GaussInt.zero` 之类再把记号指派给这些定义。为定义提供显式名称常常很有用，例如可与 `simp` 和 `rw` 一起使用。

```
theorem zero_def : (0 : GaussInt) = ⟨0, 0⟩ :=
  rfl

theorem one_def : (1 : GaussInt) = ⟨1, 0⟩ :=
  rfl

theorem add_def (x y : GaussInt) : x + y = ⟨x.re + y.re, x.im + y.im⟩ :=
  rfl

theorem neg_def (x : GaussInt) : -x = ⟨-x.re, -x.im⟩ :=
  rfl

theorem mul_def (x y : GaussInt) :
  x * y = ⟨x.re * y.re - x.im * y.im, x.re * y.im + x.im * y.re⟩ :=
  rfl
```

为计算实部和虚部的规则命名，并把它们声明给化简器，也很有用。

```

@[simp]
theorem zero_re : (0 : GaussInt).re = 0 :=
  rfl

@[simp]
theorem zero_im : (0 : GaussInt).im = 0 :=
  rfl

@[simp]
theorem one_re : (1 : GaussInt).re = 1 :=
  rfl

@[simp]
theorem one_im : (1 : GaussInt).im = 0 :=
  rfl

@[simp]
theorem add_re (x y : GaussInt) : (x + y).re = x.re + y.re :=
  rfl

@[simp]
theorem add_im (x y : GaussInt) : (x + y).im = x.im + y.im :=
  rfl

@[simp]
theorem neg_re (x : GaussInt) : (-x).re = -x.re :=
  rfl

@[simp]
theorem neg_im (x : GaussInt) : (-x).im = -x.im :=
  rfl

@[simp]
theorem mul_re (x y : GaussInt) : (x * y).re = x.re * y.re - x.im * y.im :=
  rfl

@[simp]
theorem mul_im (x y : GaussInt) : (x * y).im = x.re * y.im + x.im * y.re :=
  rfl

```

现在，要证明高斯整数是交换环的一个实例出人意料地容易。我们正在有效地使用结构这一概念。每个具体高斯整数都是 `GaussInt` 结构的一个实例，而类型 `GaussInt` 本身连同相关运算则是 `CommRing` 结构的一个实例。反过来，`CommRing` 结构扩展了记号结构 `Zero`、`One`、`Add`、`Neg` 和 `Mul`。

如果输入 `instance : CommRing GaussInt := _`，点击 VS Code 中出现的灯泡，并要求 Lean 填入结构定义的骨架，会看到数量惊人的条目。然而，跳转到该结构的定义会发现，许多字段都有默认定义，Lean 会自动为你填充。下面定义中出现的是本质字段。特殊情况是 `nsmul` 和 `zsmul`，现在应暂时忽略，它们将在下一章解释。在每种情况下，相关恒等式都是通过展开定义、使用 `ext` 策略把等式化归为实部和虚部的等式、进行化简，并在必要时执行整数中的相关环计算来证明的。注意，我们本可以轻易避免重复所有这些代码，但这不是当前讨论的主题。

```

instance instCommRing : CommRing GaussInt where
  zero := 0
  one := 1
  add := (· + ·)
  neg x := -x
  mul := (· * ·)
  nsmul := nsmulRec
  zsmul := zsmulRec
  add_assoc := by
    intros
    ext <|> simp <|> ring
  zero_add := by
    intro
    ext <|> simp
  add_zero := by
    intro
    ext <|> simp
  neg_add_cancel := by
    intro
    ext <|> simp
  add_comm := by
    intros
    ext <|> simp <|> ring
  mul_assoc := by
    intros
    ext <|> simp <|> ring
  one_mul := by
    intro
    ext <|> simp
  mul_one := by
    intro
    ext <|> simp
  left_distrib := by
    intros
    ext <|> simp <|> ring
  right_distrib := by
    intros
    ext <|> simp <|> ring
  mul_comm := by
    intros
    ext <|> simp <|> ring
  zero_mul := by
    intros
    ext <|> simp
  mul_zero := by
    intros
    ext <|> simp

```

Lean 库把 *nontrivial* 类型类定义为至少含有两个不同元素的类型。在环的语境中，这等价于说零不等于一。既然一些常用定理依赖这一事实，我们不妨现在就建立它。

```

instance : Nontrivial GaussInt := by
  use 0, 1
  rw [Ne, GaussInt.ext_iff]
  simp

```

现在我们将证明高斯整数还具有一个重要附加性质。*Euclidean domain* 是一个环 R ，并配备一个 *norm* 函数 $N : R \rightarrow \mathbb{N}$ ，满足下面两个性质：

对 R 中每个 a 和 $b \neq 0$ ，存在 R 中的 q 和 r ，使得 $a = bq + r$ ，并且要么 $r = 0$ ，要么 $N(r) < N(b)$ 。

对每个 a 和 $b \neq 0$ ，有 $N(a) \leq N(ab)$ 。

带有 $N(a) = |a|$ 的整数环 $\mathbb{Z}$ 是欧几里得整环的典型例子。在这种情况下，可以取 q 为 a 除以 b 的整数商，取 r 为余数。注意，当 a 和 b 是整数时， a / b 在 Lean 中表示整数除法， $a \% b$ 表示余数。它们被定义为满足如下性质：

```

example (a b : ℤ) : a = b * (a / b) + a % b :=
  Eq.symm (Int.mul_ediv_add_emod a b)

example (a b : ℤ) : b ≠ 0 → 0 ≤ a % b :=
  Int.emod_nonneg a

example (a b : ℤ) : b ≠ 0 → a % b < |b| :=
  Int.emod_lt_abs a

```

在任意环中，若元素 a 整除 1，则称 a 为 *unit*。若非零元素 a 不能写成 $a = bc$ 的形式，其中 b 和 c 都不是单位，则称它为 *irreducible*。在整数中，每个不可约元素 a 都是 *prime*；也就是说，只要 a 整除乘积 bc ，它就整除 b 或整除 c 。但在其他环中，这个性质可能失败。在环 $\mathbb{Z}[\sqrt{-5}]$ 中，我们有

$$6 = 2 \cdot 3 = (1 + \sqrt{-5})(1 - \sqrt{-5}),$$

并且元素 2、3、 $1 + \sqrt{-5}$ 和 $1 - \sqrt{-5}$ 全都不可约，但它们不是素元。例如，2 整除乘积 $(1 + \sqrt{-5})(1 - \sqrt{-5})$ ，却不整除任一因子。特别地，唯一分解不再成立：数 6 可以以不止一种方式分解为不可约元素。

相比之下，每个欧几里得整环都是唯一分解整环，这蕴含每个不可约元素都是素元。欧几里得整环的公理蕴含，任意非零元素都可以写成不可约元素的有限乘积。它们还蕴含，可以使用欧几里得算法求任意两个非零元素 a 和 b 的最大公因子，也就是可被任意其他公因子整除的元素。这进一步蕴含，不可约元素分解在乘以单位的意义下唯一。

现在证明高斯整数是欧几里得整环，其中范数定义为 $N(a + bi) = (a + bi)(a - bi) = a^2 + b^2$ 。高斯整数 $a - bi$ 称为 $a + bi$ 的 *conjugate*。不难验证，对任意复数 x 和 y ，都有 $N(xy) = N(x)N(y)$ 。

为了看出这个范数定义使高斯整数成为欧几里得整环，只有第一个性质具有挑战。设我们想把 $a + bi = (c + di)q + r$ 写出来，其中 q 与 r 合适。把 $a + bi$ 和 $c + di$ 视为复数，进行除法

$$\frac{a + bi}{c + di} = \frac{(a + bi)(c - di)}{(c + di)(c - di)} = \frac{ac + bd}{c^2 + d^2} + \frac{bc - ad}{c^2 + d^2}i.$$

所得实部和虚部未必是整数，但可以把它舍入到最接近的整数 u 和 v 。于是可将右端写成 $(u + vi) + (u' + v'i)$ ，其中 $u' + v'i$ 是剩余部分。注意有 $|u'| \leq 1/2$ 和 $|v'| \leq 1/2$ ，从而

$$N(u' + v'i) = (u')^2 + (v')^2 \leq 1/4 + 1/4 \leq 1/2.$$

两边乘以 $c + di$ ，得到

$$a + bi = (c + di)(u + vi) + (c + di)(u' + v'i).$$

令 $q = u + vi$ 且 $r = (c + di)(u' + v'i)$ ，则有 $a + bi = (c + di)q + r$ ，只需估计 $N(r)$ ：

$$N(r) = N(c + di)N(u' + v'i) \leq N(c + di) \cdot 1/2 < N(c + di).$$

刚才的论证要求把高斯整数视为复数的子集。因此，在 Lean 中形式化它的一种选择，是把高斯整数嵌入复数，把整数嵌入高斯整数，定义从实数到整数的舍入函数，并非常谨慎地在这些数系之间来回切换。事实上，Mathlib 采用的正是这种方法；其中高斯整数本身被构造为 *quadratic integers* 环的特例。参见文件 [GaussianInt.lean](#)。

这里我们改为给出一个始终停留在整数中的论证。这展示了形式化数学时常遇到的一种选择。给定一个需要库中尚无概念或机制的论证，有两种选择：要么形式化所需概念和机制，要么调整论证以利用已有概念和机制。当前者的结果可在其他语境中使用，通常值得投入时间。从实用角度看，有时寻找更初等的证明会更高效。

整数通常的带余除法定理说，对每个 a 和非零 b ，存在 q 与 r ，使得 $a = bq + r$ 且 $0 \leq r < |b|$ 。这里我们将使用下面的变体：存在 q' 与 r' ，使得 $a = bq' + r'$ 且 $|r'| \leq |b|/2$ 。你可以检查，如果第一个陈述中的 r 满足 $r \leq |b|/2$ ，则可取 $q' = q$ 且 $r' = r$ ；否则可取 $q' = q + 1$ 且 $r' = r - b$ （当 b 为正时；其他情况相应调整）。我们感谢 Heather Macbeth 提出下面这个更优雅的方法，它避免了分类定义。只需在除法前把 $b/2$ 加到 a 上，然后从余数中减去它。

```
def div' (a b : ℤ) :=
  (a + b / 2) / b

def mod' (a b : ℤ) :=
  (a + b / 2) % b - b / 2

theorem div'_add_mod' (a b : ℤ) : b * div' a b + mod' a b = a := by
  rw [div', mod']
  linarith [Int.mul_ediv_add_emod (a + b / 2) b]

theorem abs_mod'_le (a b : ℤ) (h : 0 < b) : |mod' a b| ≤ b / 2 := by
  rw [mod', abs_le]
  constructor
  · linarith [Int.emod_nonneg (a + b / 2) h.ne']
  have := Int.emod_lt_of_pos (a + b / 2) h
  have := Int.mul_ediv_add_emod b 2
  have := Int.emod_lt_of_pos b zero_lt_two
  linarith
```

注意这里使用了我们的老朋友 `linarith`。还需要把 `mod'` 用 `div'` 表示出来。

```
theorem mod'_eq (a b : ℤ) : mod' a b = a - b * div' a b := by linarith [div'_add_mod' a b]
```

我们将使用事实： $x^2 + y^2$ 等于零当且仅当 x 和 y 都为零。作为练习，请证明它在任意有序环中成立。

```
theorem sq_add_sq_eq_zero {α : Type*} [Ring α] [LinearOrder α] [IsStrictOrderedRing α]
  (x y : α) : x ^ 2 + y ^ 2 = 0 ↔ x = 0 ∧ y = 0 := by
  sorry
```

本节剩余所有定义和定理都将放在 `GaussInt` 命名空间中。首先，定义范数函数 `norm`，并请你建立它的一些性质。证明都很短。

```
def norm (x : GaussInt) :=
  x.re ^ 2 + x.im ^ 2

@[simp]
theorem norm_nonneg (x : GaussInt) : 0 ≤ norm x := by
  sorry
theorem norm_eq_zero (x : GaussInt) : norm x = 0 ↔ x = 0 := by
  sorry
theorem norm_pos (x : GaussInt) : 0 < norm x ↔ x ≠ 0 := by
  sorry
theorem norm_mul (x y : GaussInt) : norm (x * y) = norm x * norm y := by
  sorry
```

接着定义共轭函数：

```

def conj (x : GaussInt) : GaussInt :=
  (x.re, -x.im)

@[simp]
theorem conj_re (x : GaussInt) : (conj x).re = x.re :=
  rfl

@[simp]
theorem conj_im (x : GaussInt) : (conj x).im = -x.im :=
  rfl

theorem norm_conj (x : GaussInt) : norm (conj x) = norm x := by simp [norm]

```

最后，用记号 x / y 为高斯整数定义除法，它把复数商舍入到最接近的高斯整数。为此我们使用自定义的 `Int.div'`。如上所算，若 x 为 $a + bi$ 、 y 为 $c + di$ ，则 x / y 的实部和虚部分别是最接近以下数值的整数：

$$\frac{ac + bd}{c^2 + d^2} \quad \text{and} \quad \frac{bc - ad}{c^2 + d^2},$$

这里分子分别是 $(a + bi)(c - di)$ 的实部和虚部，而两个分母都等于 $c + di$ 的范数。

```

instance : Div GaussInt :=
  (fun x y ↦ ⟨Int.div' (x * conj y).re (norm y), Int.div' (x * conj y).im (norm y)⟩)

```

定义了 x / y 之后，我们把 $x \% y$ 定义为余数 $x - (x / y) * y$ 。如上所述，我们把这些定义记录在定理 `div_def` 和 `mod_def` 中，以便与 `simp` 和 `rw` 一起使用。

```

instance : Mod GaussInt :=
  (fun x y ↦ x - y * (x / y))

theorem div_def (x y : GaussInt) :
  x / y = ⟨Int.div' (x * conj y).re (norm y), Int.div' (x * conj y).im (norm y)⟩ :=
  rfl

theorem mod_def (x y : GaussInt) : x % y = x - y * (x / y) :=
  rfl

```

这些定义立即给出对每个 x 和 y 都有 $x = y * (x / y) + x \% y$ ，因此只需证明当 y 非零时， $x \% y$ 的范数小于 y 的范数。

我们刚把 x / y 的实部和虚部分别定义为 `div' (x * conj y).re (norm y)` 与 `div' (x * conj y).im (norm y)`。计算可得

也就是说， $(x \% y) * \text{conj } y = (x - x / y * y) * \text{conj } y = x * \text{conj } y - x / y * (y * \text{conj } y)$

右端的实部和虚部恰好是 `mod' (x * conj y).re (norm y)` 和 `mod' (x * conj y).im (norm y)`。由 `div'` 和 `mod'` 的性质，它们保证小于或等于 $\text{norm } y / 2$ 。因此有

于是得到 $\text{norm} ((x \% y) * \text{conj } y) \leq (\text{norm } y / 2)^2 + (\text{norm } y / 2)^2 \leq (\text{norm } y / 2) * \text{norm } y$ 。

另一方面，有

即 $\text{norm} ((x \% y) * \text{conj } y) = \text{norm} (x \% y) * \text{norm} (\text{conj } y) = \text{norm} (x \% y) * \text{norm } y$ 。

两边同除以 $\text{norm } y$ ，得到所需的 $\text{norm} (x \% y) \leq (\text{norm } y) / 2 < \text{norm } y$ 。

这个繁杂的计算在下一个证明中完成。我们鼓励你逐步查看细节，看看能否找到更漂亮的论证。

```

theorem norm_mod_lt (x : GaussInt) {y : GaussInt} (hy : y ≠ 0) :
  (x % y).norm < y.norm := by
  have norm_y_pos : 0 < norm y := by rw [norm_pos]
  have H1 : x % y * conj y = (Int.mod' (x * conj y).re (norm y), Int.mod' (x * conj y).im (norm y))
  · ext <=> simp [Int.mod'_eq, mod_def, div_def, norm] <=> ring
  have H2 : norm (x % y) * norm y ≤ norm y / 2 * norm y
  · calc
    norm (x % y) * norm y = norm (x % y * conj y) := by simp only [norm_mul, norm_conj]
    _ = |Int.mod' (x.re * y.re + x.im * y.im) (norm y)| ^ 2
      + |Int.mod' (-(x.re * y.im) + x.im * y.re) (norm y)| ^ 2 := by simp [H1, norm, sq_abs]
    _ ≤ (y.norm / 2) ^ 2 + (y.norm / 2) ^ 2 := by gcongr <=> apply Int.abs_mod'_le _ _ norm_y_pos
    _ = norm y / 2 * (norm y / 2 * 2) := by ring
    _ ≤ norm y / 2 * norm y := by gcongr; apply Int.ediv_mul_le; norm_num
  calc norm (x % y) ≤ norm y / 2 := le_of_mul_le_mul_right H2 norm_y_pos
  _ < norm y := by
    apply Int.ediv_lt_of_lt_mul
    · norm_num
    · linarith

```

我们已经接近终点。范数函数 `norm` 把高斯整数映到非负整数。我们需要一个把高斯整数映到自然数的函数，并通过把 `norm` 与函数 `Int.natAbs` 复合来得到它；后者把整数映到自然数。接下来两个引理中的第一个说明，将范数映到自然数再映回整数不会改变值。第二个重新表述范数递减这一事实。

```

theorem coe_natAbs_norm (x : GaussInt) : (x.norm.natAbs : ℤ) = x.norm :=
  Int.natAbs_of_nonneg (norm_nonneg _)

theorem natAbs_norm_mod_lt (x y : GaussInt) (hy : y ≠ 0) :
  (x % y).norm.natAbs < y.norm.natAbs := by
  apply Int.ofNat_lt.1
  simp only [Int.natCast_natAbs, abs_of_nonneg, norm_nonneg]
  exact norm_mod_lt x hy

```

还需要建立欧几里得整环上范数函数的第二个关键性质。

```

theorem not_norm_mul_left_lt_norm (x : GaussInt) {y : GaussInt} (hy : y ≠ 0) :
  ¬(norm (x * y).natAbs < (norm x).natAbs := by
  apply not_lt_of_ge
  rw [norm_mul, Int.natAbs_mul]
  apply le_mul_of_one_le_right (Nat.zero_le _)
  apply Int.ofNat_le.1
  rw [coe_natAbs_norm]
  exact Int.add_one_le_of_lt ((norm_pos _).mpr hy)

```

现在可以把这些合在一起，证明高斯整数是欧几里得整环的一个实例。我们使用已经定义的商函数和余数函数。Mathlib 对欧几里得整环的定义比上面的定义更一般，因为它允许我们证明余数相对于任意良基度量递减。比较一个返回自然数的范数函数的取值，只是这类度量的一个实例；在这种情况下，所需性质就是定理 `natAbs_norm_mod_lt` 和

`not_norm_mul_left_lt_norm`。

```

instance : EuclideanDomain GaussInt :=
{ GaussInt.instCommRing with
  quotient := (· / ·)
  remainder := (· % ·)
  quotient_mul_add_remainder_eq :=
    fun x y => by rw [mod_def, add_comm] ; ring
  quotient_zero := fun x => by
    simp [div_def, norm, Int.div']
    rfl
  r := (measure (Int.natAbs ∘ norm)).1
  r_wellFounded := (measure (Int.natAbs ∘ norm)).2
  remainder_lt := natAbs_norm_mod_lt
  mul_left_not_lt := not_norm_mul_left_lt_norm }

```

一个直接收获是：现在我们知道，在高斯整数中，素元与不可约元这两个概念相互重合。

```

example (x : GaussInt) : Irreducible x ↔ Prime x :=
  irreducible_iff_prime

```

8. 层级结构

我们已在Chapter 7中看到如何定义群类、构造该类的实例，以及如何构造交换环类的实例。但这里当然存在一个层级：交换环特别地也是加法群。本章将研究如何构造这类层级。它们出现在数学的所有分支中，但本章将强调代数方面的例子。

在进一步讨论如何使用已有层级之前就讨论如何构造层级，或许显得有些过早。但使用层级也需要对其底层技术有所理解。因此你大概仍应阅读本章，不过第一次阅读时不必强求记住所有内容；可以先读后续章节，再回到这里重读一遍。

本章中，我们将重新定义 Mathlib 中出现的许多对象的较简化版本，因此会用下标区分我们的版本。例如，`Ring1` 将是我们版本的 `Ring`。由于我们将逐步解释形式化结构的更强方法，这些下标有时会超过 1。

8.1. 基础

在 Lean 所有层级的最底部，我们会遇到携带数据的类。下面的类记录给定类型 `α` 配备了一个称为 `one` 的特殊元素。在这个阶段，它完全没有任何性质。

```
class One1 (α : Type) where
  /-- The element one -/
  one : α
```

由于本章将更频繁地使用类，我们需要更详细地理解 `class` 命令在做什么。首先，上面的 `class` 命令定义了一个结构 `One1`，带参数 `α : Type` 和单一字段 `one`。它还把该结构标记为类，使得对某个类型 `α` 而言类型为 `One1 α` 的参数，只要被标记为实例隐式参数，也就是出现在方括号中，就可以用实例解析过程推断出来。这两个效果也可以通过带 `class` 属性的 `structure` 命令实现，即写 `@[class] structure`。但 `class` 命令还会确保 `One1 α` 在自身字段中作为实例隐式参数出现。比较如下：

```
#check One1.one -- One1.one {α : Type} [self : One1 α] : α

@[class] structure One2 (α : Type) where
  /-- The element one -/
  one : α

#check One2.one
```

在第二个检查中，可以看到 `self : One2 α` 是显式参数。让我们确认第一个版本确实无需任何显式参数即可使用。

```
example (α : Type) [One1 α] : α := One1.one
```

备注：在上面的例子中，参数 `One1 α` 被标记为实例隐式参数，这有点小题大做，因为它只影响声明的 *uses*，而由 `example` 命令创建的声明无法被使用。不过，这使我们可以避免给该参数命名；更重要的是，它帮助我们养成把 `One1 α` 参数标记为实例隐式参数的好习惯。

另一个备注是，只有当 Lean 知道 `α` 是什么时，这一切才会工作。在上面的例子中，如果省略类型标注 `: α`，会生成类似 `typeclass instance problem is stuck, it is often due to metavariables One1 (?m.263 α)` 的错误信息，其中 `?m.263 α` 表示“某个依赖于 `α` 的类型”（263 只是自动生成的下标，用于区分多个未知对象）。避免此问题的另一种方法是使用类型标注，例如：

```
example (α : Type) [One1 α] := (One1.one : α)
```

你在Section 3.6中尝试序列极限时，或许已经遇到过这个问题；例如，若想陈述 $0 < 1$ ，却没有告诉 Lean 这个不等式是在自然数中还是实数中。

下一项任务是给 `One1.one` 指派一个记号。由于不希望与内置记号 `1` 冲突，我们将使用 `11`。下面的命令实现了这一点，其中第一行告诉 Lean 使用 `One1.one` 的文档作为符号 `11` 的文档。

```
@[inherit_doc]
notation "11" => One1.one

example {α : Type} [One1 α] : α := 11

example {α : Type} [One1 α] : (11 : α) = 1 := rfl
```

现在我们想要一个携带数据的类，用来记录一个二元运算。暂时不想在加法和乘法之间作选择，所以将使用菱形符号。

```
class Dia1 (α : Type) where
  dia : α → α → α

infixl:70 " ◊ " => Dia1.dia
```

与 `One1` 的例子一样，此阶段该运算完全没有任何性质。现在定义半群结构类，其中运算记作 `◊`。暂时先手工把它定义为一个带两个字段的结构：一个 `Dia1` 实例，以及一个取值于 `Prop` 的字段 `dia_assoc`，断言 `◊` 满足结合律。

```
class Semigroup0 (α : Type) where
  toDia1 : Dia1 α
  /-- Diamond is associative -/
  dia_assoc : ∀ a b c : α, a ◊ b ◊ c = a ◊ (b ◊ c)
```

注意，在陈述 `dia_assoc` 时，先前定义的字段 `toDia1` 位于局部上下文中，因此 Lean 在寻找 `Dia1 α` 的实例以理解 `a ◊ b` 时可以使用它。然而这个 `toDia1` 字段不会成为类型类实例数据库的一部分。因此，执行

```
example {α : Type} [Semigroup1 α] (a b : α) : α := a ◊ b
```

 会失败，并给出错误信息 `failed to synthesize instance Dia1 α`。

可以稍后添加 `instance` 属性来修正这一点。

```
attribute [instance] Semigroup0.toDia1

example {α : Type} [Semigroup0 α] (a b : α) : α := a ◊ b
```

在继续向上构造之前，需要用一种不同语法加入这个 `toDia1` 字段，告诉 Lean 应把 `Dia1 α` 当作其字段就是 `Semigroup1` 自身的字段来处理。这也会方便地自动添加 `toDia1` 实例。`class` 命令通过 `extends` 语法支持这一点，如下所示：

```
class Semigroup1 (α : Type) extends toDia1 : Dia1 α where
  /-- Diamond is associative -/
  dia_assoc : ∀ a b c : α, a ◊ b ◊ c = a ◊ (b ◊ c)

example {α : Type} [Semigroup1 α] (a b : α) : α := a ◊ b
```

注意，这个语法也可用于 `structure` 命令；不过在那种情况下，由于没有要定义的实例，它只解决书写诸如 `toDia1` 这类字段的麻烦。

在 `extends` 语法中，字段名 `toDia1` 是可选的。默认情况下，它取被扩展类的名称，并在前面加上 “to”。

```
class Semigroup2 (α : Type) extends Dia1 α where
  /-- Diamond is associative -/
  dia_assoc : ∀ a b c : α, a ◊ b ◊ c = a ◊ (b ◊ c)
```

现在尝试把一个菱形运算和一个特殊的一元素结合起来，并加入公理说明该元素在左右两边都是单位元。

```
class DiaOneClass1 (α : Type) extends One1 α, Dia1 α where
  /-- One is a left neutral element for diamond. -/
  one_dia : ∀ a : α, 1 ◊ a = a
  /-- One is a right neutral element for diamond -/
  dia_one : ∀ a : α, a ◊ 1 = a
```

在下一个例子中，我们告诉 Lean：α 具有 DiaOneClass₁ 结构，并陈述一个同时使用 Dia₁ 实例和 One₁ 实例的性质。为了观察 Lean 如何找到这些实例，我们设置一个跟踪选项，其结果可在 Infview 中看到。默认结果相当简略，但可以点击以黑色箭头结尾的行展开。它包含一些失败尝试，即 Lean 在尚未拥有足够类型信息前试图寻找实例。成功的尝试确实会用到由 extends 语法生成的实例。

```
set_option trace.Meta.synthInstance true in
example {α : Type} [DiaOneClass1 α] (a b : α) : Prop := a ◊ b = 1
```

注意，在组合已有类时，不需要加入额外字段。因此可以如下定义幺半群：

```
class Monoid1 (α : Type) extends Semigroup1 α, DiaOneClass1 α
```

虽然上面的定义看起来直接，但它隐藏了一个重要细节。Semigroup₁ α 和 DiaOneClass₁ α 都扩展了 Dia₁ α，因此人们可能担心，拥有一个 Monoid₁ α 实例会给出 α 上两个无关的菱形运算：一个来自字段 Monoid₁.toSemigroup₁，另一个来自字段 Monoid₁.toDiaOneClass₁。

确实，如果尝试手工构造一个幺半群类：

```
class Monoid2 (α : Type) where
  toSemigroup1 : Semigroup1 α
  toDiaOneClass1 : DiaOneClass1 α
```

那么会得到两个完全无关的菱形运算 Monoid₂.toSemigroup₁.toDia₁.dia 和 Monoid₂.toDiaOneClass₁.toDia₁.dia。

使用 extends 语法生成的版本没有这个缺陷。

```
example {α : Type} [Monoid1 α] :
  (Monoid1.toSemigroup1.toDia1.dia : α → α → α) = Monoid1.toDiaOneClass1.toDia1.dia := rfl
```

所以 class 命令为我们做了一些魔法（structure 命令也会如此）。查看这些类的字段的一种简单方法，是检查它们的构造子。比较如下：

```
/- Monoid2.mk {α : Type} (toSemigroup1 : Semigroup1 α) (toDiaOneClass1 : DiaOneClass1 α) : Monoid2 α -/
#check Monoid2.mk

/- Monoid1.mk {α : Type} [toSemigroup1 : Semigroup1 α] [toOne1 : One1 α] (one_dia : ∀ (a : α), 1 ◊ a = a) (dia_one : ∀ (a : α), a ◊ 1 = a) : Monoid1 α -/
#check Monoid1.mk
```

于是可见，Monoid₁ 如预期接受 Semigroup₁ α 参数，但不会接受一个可能重叠的 DiaOneClass₁ α 参数，而是把它拆开，只包含不重叠的部分。它还自动生成了实例 Monoid₁.toDiaOneClass₁；该实例 not 字段，但具有预期的签名。从最终用户角度看，这恢复了两个被扩展类 Semigroup₁ 和 DiaOneClass₁ 之间的对称性。

```
#check Monoid1.toSemigroup1
#check Monoid1.toDiaOneClass1
```

现在我们已经非常接近定义群。可以向幺半群结构添加一个字段，断言每个元素都存在逆元。但这样之后还需要努力访问这些逆元。实践中，把逆元作为数据加入更方便。为了优化可复用性，我们定义一个新的携带数据的类，然后给它一些记号。

```
class Inv1 (α : Type) where
  /-- The inversion function -/
  inv : α → α

@[inherit_doc]
postfix:max "-1" => Inv1.inv

class Group1 (G : Type) extends Monoid1 G, Inv1 G where
  inv_dia : ∀ a : G, a-1 ◊ a = 1
```

上面的定义可能看起来太弱，因为只要求 a^{-1} 是 a 的左逆。但另一边会自动成立。为了证明这一点，需要一个预备引理。

```
lemma left_inv_eq_right_inv1 {M : Type} [Monoid1 M] {a b c : M} (hba : b ◊ a = 1) (hac : a ◊ c = 1) : b = c := by
  rw [← DiaOneClass1.one_dia c, ← hba, Semigroup1.dia_assoc, hac, DiaOneClass1.dia_one b]
```

在这个引理中，给出全名相当烦人，尤其是它要求知道层级的哪一部分提供了这些事实。一种修正方法是使用 `export` 命令，把这些事实复制成根命名空间中的引理。

```
export DiaOneClass1 (one_dia dia_one)
export Semigroup1 (dia_assoc)
export Group1 (inv_dia)
```

于是可以把上面的证明重写为：

```
example {M : Type} [Monoid1 M] {a b c : M} (hba : b ◊ a = 1) (hac : a ◊ c = 1) : b = c := by
  rw [← one_dia c, ← hba, dia_assoc, hac, dia_one b]
```

现在轮到你证明关于我们这些代数结构的事实了。

```
lemma inv_eq_of_dia [Group1 G] {a b : G} (h : a ◊ b = 1) : a-1 = b :=
  sorry

lemma dia_inv [Group1 G] (a : G) : a ◊ a-1 = 1 :=
  sorry
```

在这个阶段，我们想继续定义环，但这里有一个严重问题。一个类型上的环结构同时包含加法群结构、乘法么半群结构，以及关于二者相互作用的一些性质。但到目前为止，我们为所有运算都硬编码了记号 $\diamond$ 。更根本地，类型类系统假定每个类型对每个类型类只有一个实例。有多种方式解决这个问题。令人意外的是，Mathlib 借助某个生成代码的属性，采用朴素思路：为加法理论和乘法理论各复制一份。结构和类都会以加法记号和乘法记号定义，并用属性 `to_additive` 关联起来。对于半群这类多重继承情形，自动生成的“恢复对称性”实例也需要被标记。这有些技术性，不必理解细节。关键点是：引理随后只用乘法记号陈述，并标记属性 `to_additive`，以生成加法版本；例如 `left_inv_eq_right_inv'` 会自动生成加法版本 `left_neg_eq_right_neg'`。为了检查这个加法版本的名称，我们在 `left_inv_eq_right_inv'` 上方使用了命令

```
whatsnew in
```

```

class AddSemigroup₃ (α : Type) extends Add α where
  /-- Addition is associative -/
  add_assoc₃ : ∀ a b c : α, a + b + c = a + (b + c)

@[to_additive AddSemigroup₃]
class Semigroup₃ (α : Type) extends Mul α where
  /-- Multiplication is associative -/
  mul_assoc₃ : ∀ a b c : α, a * b * c = a * (b * c)

class AddMonoid₃ (α : Type) extends AddSemigroup₃ α, AddZeroClass α

@[to_additive AddMonoid₃]
class Monoid₃ (α : Type) extends Semigroup₃ α, MulOneClass α

export Semigroup₃ (mul_assoc₃)
export AddSemigroup₃ (add_assoc₃)

whatsnew in
@[to_additive]
lemma left_inv_eq_right_inv' {M : Type} [Monoid₃ M] {a b c : M} (hba : b * a = 1) (hac : a * c = 1) : b = c := by
  rw [← one_mul c, ← hba, mul_assoc₃, hac, mul_one b]

#check left_neg_eq_right_neg'

```

有了这项技术，我们也可以轻松定义交换半群、交换么半群和交换群，然后定义环。

```

class AddCommSemigroup₃ (α : Type) extends AddSemigroup₃ α where
  add_comm : ∀ a b : α, a + b = b + a

@[to_additive AddCommSemigroup₃]
class CommSemigroup₃ (α : Type) extends Semigroup₃ α where
  mul_comm : ∀ a b : α, a * b = b * a

class AddCommMonoid₃ (α : Type) extends AddMonoid₃ α, AddCommSemigroup₃ α

@[to_additive AddCommMonoid₃]
class CommMonoid₃ (α : Type) extends Monoid₃ α, CommSemigroup₃ α

class AddGroup₃ (G : Type) extends AddMonoid₃ G, Neg G where
  neg_add : ∀ a : G, -a + a = 0

@[to_additive AddGroup₃]
class Group₃ (G : Type) extends Monoid₃ G, Inv G where
  inv_mul : ∀ a : G, a⁻¹ * a = 1

```

应当记得在适当时候给引理标记 `simp`。

```
attribute [simp] Group₃.inv_mul AddGroup₃.neg_add
```

接着，由于切换到标准记号，需要稍微重复一些内容；但至少 `to_additive` 会完成从乘法记号到加法记号的翻译工作。

```

@[to_additive]
lemma inv_eq_of_mul [Group₃ G] {a b : G} (h : a * b = 1) : a⁻¹ = b :=
  sorry

```

注意，可以要求 `to_additive` 给引理标记 `simp`，并如下把该属性传播给加法版本。

```

@[to_additive (attr := simp)]
lemma Group₃.mul_inv {G : Type} [Group₃ G] (a : G) : a * a⁻¹ = 1 := by
  sorry

@[to_additive]
lemma mul_left_cancel₃ {G : Type} [Group₃ G] {a b c : G} (h : a * b = a * c) : b = c := by
  sorry

@[to_additive]
lemma mul_right_cancel₃ {G : Type} [Group₃ G] {a b c : G} (h : b * a = c * a) : b = c := by
  sorry

class AddCommGroup₃ (G : Type) extends AddGroup₃ G, AddCommMonoid₃ G

@[to_additive AddCommGroup₃]
class CommGroup₃ (G : Type) extends Group₃ G, CommMonoid₃ G

```


现在可以定义环了。为了演示，我们不假设加法可交换，然后立即提供一个 `AddCommGroup3` 实例。Mathlib 并不这样做，首先因为实践中这不会让任何环实例更容易；其次是因为 Mathlib 的代数层级经过半环，而半环类似于环但没有相反元，所以下面的证明对它们不适用。这里的收获，除了给从未见过此事的读者一个漂亮练习外，是展示如何使用一种语法构造实例：它允许把父结构作为实例参数提供，然后补充额外字段。这里参数 $\text{Ring}_3 R$ 提供了 $\text{AddCommGroup}_3 R$ 所需的除 `add_comm` 之外的一切。

```
class Ring3 (R : Type) extends AddGroup3 R, Monoid3 R, MulZeroClass R where
  /-- Multiplication is left distributive over addition -/
  left_distrib : ∀ a b c : R, a * (b + c) = a * b + a * c
  /-- Multiplication is right distributive over addition -/
  right_distrib : ∀ a b c : R, (a + b) * c = a * c + b * c

instance {R : Type} [Ring3 R] : AddCommGroup3 R :=
{ add_comm := by
  sorry }
```

当然，也可以构造具体实例，例如整数上的环结构（当然，下面的实例使用了 Mathlib 已完成的全部工作）。

```
instance : Ring3 ℤ where
  add := (· + ·)
  add_assoc3 := add_assoc
  zero := 0
  zero_add := by simp
  add_zero := by simp
  neg := (- ·)
  neg_add := by simp
  mul := (· * ·)
  mul_assoc3 := mul_assoc
  one := 1
  one_mul := by simp
  mul_one := by simp
  zero_mul := by simp
  mul_zero := by simp
  left_distrib := Int.mul_add
  right_distrib := Int.add_mul
```

作为练习，你现在可以为序关系建立一个简单层级，其中包括有序交换么半群的类；这类结构同时具有偏序和交换么半群结构，并满足 $\forall a b : \alpha, a \leq b \rightarrow \forall c : \alpha, c * a \leq c * b$ 。当然，你需要给下面这些类添加字段，也许还要添加 `extends` 子句。

```
class LE1 (α : Type) where
  /-- The Less-or-Equal relation. -/
  le : α → α → Prop

@[inherit_doc] infix:50 " ≤1 " => LE1.le

class Preorder1 (α : Type)

class PartialOrder1 (α : Type)

class OrderedCommMonoid1 (α : Type)

instance : OrderedCommMonoid1 ℕ where
```

现在讨论涉及多个类型的代数结构。首要例子是环上的模。如果你不知道什么是模，可以假装它表示向量空间，并认为所有环都是域。这些结构是交换加法群，并配备来自某个环的元素的标量乘法。

首先定义一个携带数据的类型类，记录某个类型 α 对某个类型 β 的标量乘法，并给它一个右结合记号。

```
class SMul3 (α : Type) (β : Type) where
  /-- Scalar multiplication -/
  smul : α → β → β

infixr:73 " • " => SMul3.smul
```

然后可以定义模（如果不知道什么是模，仍可把它想成向量空间）。

```
class Module₁ (R : Type) [Ring₃ R] (M : Type) [AddCommGroup₃ M] extends SMul₃ R M where
  zero_smul : ∀ m : M, (0 : R) • m = 0
  one_smul : ∀ m : M, (1 : R) • m = m
  mul_smul : ∀ (a b : R) (m : M), (a * b) • m = a • b • m
  add_smul : ∀ (a b : R) (m : M), (a + b) • m = a • m + b • m
  smul_add : ∀ (a : R) (m n : M), a • (m + n) = a • m + a • n
```

这里发生了一件有趣的事。环 R 上的环结构是这个定义参数，这并不太令人惊讶；但你可能预期 `AddCommGroup₃ M` 会像 `SMul₃ R M` 一样成为 `extends` 子句的一部分。尝试这么做会产生一个标记为实例的字段 `Module₃.toAddCommGroup₃`。这个实例会有签名 $(R : \text{Type}) \rightarrow [\text{inst} : \text{Ring}_3 R] \rightarrow \{M : \text{Type}\} \rightarrow [\text{self} : \text{Module}_1 R M] \rightarrow \text{AddCommGroup}_3 M$ 。有了这样的实例在类型类数据库中，每当 Lean 为某个 `M` 寻找 `AddCommGroup₃ M` 实例时，它都必须在开始寻找 `Module₁ R M` 实例这一主任务之前，先追寻完全未指定的类型 `R` 和 `Ring₃ R` 实例。这两个支线任务由错误信息中提到的元变量表示，那里记为 `?R` 和 `?inst1`。这样的 `Module₃.toAddCommGroup₃` 实例会成为实例解析过程的巨大陷阱。

那么 `extends SMul₃ R M` 呢？它会创建字段

```
Module₁.toSMul₃ : {R : Type} → [inst : Ring₃ R] → {M : Type} → [inst_1 : AddCommGroup₃ M] → [self : Module₁ R M] → SMul₃ R M
```

，其最终结果 `SMul₃ R M` 同时提到 `R` 和 `M`，因此这个字段可以安全地作为实例使用。规则很容易记：`extends` 子句中出现的每个类都应提到参数中出现的每个类型。

让我们创建第一个模实例：一个环以其乘法作为标量乘法，是自身上的模。

```
instance selfModule (R : Type) [Ring₃ R] : Module₁ R R where
  smul := fun r s => r*s
  zero_smul := zero_mul
  one_smul := one_mul
  mul_smul := mul_assoc₃
  add_smul := Ring₃.right_distrib
  smul_add := Ring₃.left_distrib
```

作为第二个例子，每个阿贝尔群都是 $\mathbb{Z}$ 上的模（这是允许不可逆标量以推广向量空间理论的原因之一）。首先，对于任意配备零元和加法的类型，可以定义自然数标量乘法：`n • a` 定义为 `a + ... + a`，其中 `a` 出现 `n` 次。然后通过确保 `(-1) • a = -a`，把它扩展为整数标量乘法。

```
def nsmul₁ {M : Type*} [Zero M] [Add M] : ℕ → M → M
| 0, _ => 0
| n + 1, a => a + nsmul₁ n a

def zsmul₁ {M : Type*} [Zero M] [Add M] [Neg M] : ℤ → M → M
| Int.ofNat n, a => nsmul₁ n a
| Int.negSucc n, a => -nsmul₁ n.succ a
```

证明这确实给出模结构有些冗长，而且与当前讨论无关，因此我们会用 `sorry` 填充所有公理。并 *not* 要求你把这些 `sorry` 替换为证明。如果坚持要做，大概需要陈述并证明若干关于 `nsmul₁` 和 `zsmul₁` 的中间引理。

```
instance abGrpModule (A : Type) [AddCommGroup₃ A] : Module₁ ℤ A where
  smul := zsmul₁
  zero_smul := sorry
  one_smul := sorry
  mul_smul := sorry
  add_smul := sorry
  smul_add := sorry
```

一个更重要的问题是，现在 $\mathbb{Z}$ 自身在环 $\mathbb{Z}$ 上有两个模结构：`abGrpModule ℤ`，因为 $\mathbb{Z}$ 是阿贝尔群；以及 `selfModule ℤ`，因为 $\mathbb{Z}$ 是环。这两个模结构对应同一个阿贝尔群结构，但它们是否有相同的标量乘法并不显然。事实上它们确实相同，但这不是按定义成立，而需要证明。对类型类实例解析过程而言，这是非常糟糕的消息，会给该层级的使用者带来令人沮丧的失败。当直接要求寻找一个实例时，Lean 会选取其中一个；我们可以用如下方式查看它选了哪个：

```
#synth Module₁ ℤ ℤ -- abGrpModule ℤ
```

但在更间接的语境中，Lean 可能推断出另一个实例，随后陷入混乱。这种情形称为坏菱形。这与前面使用的菱形运算无关，而是指可以把从 $\mathbb{Z}$ 到其 `Module1  $\mathbb{Z}$` 的路径画成经过 `AddCommGroup3  $\mathbb{Z}$` 或经过 `Ring3  $\mathbb{Z}$` 两条路。

需要理解，并非所有菱形都是坏的。事实上，Mathlib 中到处都有菱形，本章也是如此。早在开头我们就看到，可以从 `Monoid1  $\alpha$` 经过 `Semigroup1  $\alpha$` 或 `DiaOneClass1  $\alpha$` 到达 `Dia1  $\alpha$` ；由于 `class` 命令所做的工作，得到的两个 `Dia1  $\alpha$` 实例按定义相等。特别地，若菱形底部是取值于 `Prop` 的类，它不可能是坏的，因为同一命题的任意两个证明按定义相等。

但我们用模创建出的菱形确实是坏的。出问题的部分是字段 `smul`，它是数据而不是证明，而且两种构造按定义并不相等。稳健的修正方式，是确保从丰富结构到贫乏结构总是通过忘却数据完成，而不是通过定义数据完成。这个著名模式称为“遗忘继承”，并在 <https://inria.hal.science/hal-02463336v2> 中得到广泛讨论。

在我们的具体情形中，可以修改 `AddMonoid3` 的定义，加入一个 `nsmul` 数据字段，以及若干取值于 `Prop` 的字段，以保证此操作可证地等于上面构造的操作。下面定义中，这些字段在类型之后用 `:=` 给出默认值。由于这些默认值，大多数实例仍可与先前定义完全一样地构造。但在 $\mathbb{Z}$ 的特殊情形中，我们将能提供特定取值。

```
class AddMonoid4 (M : Type) extends AddSemigroup3 M, AddZeroClass M where
  /-- Multiplication by a natural number. -/
  nsmul :  $\mathbb{N} \rightarrow M \rightarrow M := nsmul_1$ 
  /-- Multiplication by `(0 :  $\mathbb{N}$ )` gives `0`. -/
  nsmul_zero :  $\forall x, nsmul\ 0\ x = 0 := \text{by intro; rfl}$ 
  /-- Multiplication by `(n + 1 :  $\mathbb{N}$ )` behaves as expected. -/
  nsmul_succ :  $\forall (n : \mathbb{N}) (x), nsmul\ (n + 1)\ x = x + nsmul\ n\ x := \text{by intro; rfl}$ 

instance mySMul {M : Type} [AddMonoid4 M] : SMul  $\mathbb{N}$  M := (AddMonoid4.nsmul)
```

让我们检查，不提供与 `nsmul` 相关的字段时，仍能构造乘积么半群实例。

```
instance (M N : Type) [AddMonoid4 M] [AddMonoid4 N] : AddMonoid4 (M  $\times$  N) where
  add := fun p q  $\mapsto$  (p.1 + q.1, p.2 + q.2)
  add_assoc3 := fun a b c  $\mapsto$  by ext <=> apply add_assoc3
  zero := (0, 0)
  zero_add := fun a  $\mapsto$  by ext <=> apply zero_add
  add_zero := fun a  $\mapsto$  by ext <=> apply add_zero
```

现在处理 $\mathbb{Z}$ 的特殊情况：希望使用从 $\mathbb{N}$ 到 $\mathbb{Z}$ 的强制转换以及 $\mathbb{Z}$ 上的乘法来构造 `nsmul`。特别注意，这里的证明字段比上面默认值中的证明字段包含更多工作。

```
instance : AddMonoid4  $\mathbb{Z}$  where
  add := (· + ·)
  add_assoc3 := Int.add_assoc
  zero := 0
  zero_add := Int.zero_add
  add_zero := Int.add_zero
  nsmul := fun n m  $\mapsto$  (n :  $\mathbb{Z}$ ) * m
  nsmul_zero := Int.zero_mul
  nsmul_succ := fun n m  $\mapsto$  show (n + 1 :  $\mathbb{Z}$ ) * m = m + n * m
    by rw [Int.add_mul, Int.add_comm, Int.one_mul]
```

让我们检查问题是否已经解决。因为 Lean 已经有自然数和整数之间标量乘法的定义，而我们希望确保使用的是自己的实例，所以不使用记号 `·`，而是调用 `SMul.mul` 并显式提供上面定义的实例。

```
example (n :  $\mathbb{N}$ ) (m :  $\mathbb{Z}$ ) : SMul.mul (self := mySMul) n m = n * m := rfl
```

这个故事随后会继续，把 `zsmul` 字段纳入群的定义，并采用类似技巧。现在你已经可以阅读 Mathlib 中么半群、群、环和模的定义了。它们比这里看到的更复杂，因为它们是庞大层级的一部分，但所有原则都已在上文解释。

作为练习，可以回到上面构造的序关系层级，尝试纳入一个携带小于记号 `<1` 的类型类 `LT1`，并确保每个预序都带有一个 `<1`，其默认值由 `≤1` 构造，并带有一个取值于 `Prop` 的字段，断言这两个比较运算之间的自然关系。

8.2. 态射

到目前为止，本章讨论了如何创建数学结构的层级。但在拥有态射之前，结构的定义还不算真正完成。这里有两种主要方法。最明显的一种是在函数上定义谓词。

```
def isMonoidHom1 [Monoid G] [Monoid H] (f : G → H) : Prop :=  
  f 1 = 1 ∧ ∀ g g', f (g * g') = f g * f g'
```

在这个定义中，使用合取略显不便。特别地，用户在想访问两个条件时，需要记住我们选择的顺序。因此可以改用一个结构。

```
structure isMonoidHom2 [Monoid G] [Monoid H] (f : G → H) : Prop where  
  map_one : f 1 = 1  
  map_mul : ∀ g g', f (g * g') = f g * f g'
```

走到这一步，甚至很容易想把它做成一个类，并使用类型类实例解析过程，从较简单函数的实例自动推断复杂函数的 `isMonoidHom2`。例如，幺半群态射的复合仍是幺半群态射，这似乎是一个有用实例。然而，这样的实例会使解析过程非常棘手，因为它需要到处搜寻 `g ∘ f`。若在 `g (f x)` 中找不到，会令人十分沮丧。更一般地说，必须始终记住，识别给定表达式中应用了哪个函数是一个非常困难的问题，称为“高阶合一问题”。因此 Mathlib 不采用这种类方法。

一个更根本的问题是：我们是像上面那样使用谓词（通过 `def` 或 `structure`），还是使用把函数和谓词打包在一起的结构。这部分也是心理层面的问题。考虑一个不是态射的幺半群之间函数极为少见。直觉上，“幺半群态射”并不像是可以赋给裸函数的形容词，而更像是一个名词。另一方面，也可以说拓扑空间之间的连续函数本质上是一个恰好连续的函数。这是 Mathlib 拥有谓词 `Continuous` 的原因之一。例如可以写：

```
example : Continuous (id : ℝ → ℝ) := continuous_id
```

我们仍有连续函数的打包版本，例如它们便于给连续函数空间赋予拓扑；但处理连续性时，它们不是主要工具。

相比之下，幺半群（或其他代数结构）之间的态射则按如下方式打包：

```
@[ext]  
structure MonoidHom1 (G H : Type) [Monoid G] [Monoid H] where  
  toFun : G → H  
  map_one : toFun 1 = 1  
  map_mul : ∀ g g', toFun (g * g') = toFun g * toFun g'
```

当然，我们不希望到处输入 `toFun`，所以使用 `CoeFun` 类型类注册一个强制转换。它的第一个参数是我们想强制转换为函数的类型。第二个参数描述目标函数类型。在我们的情形中，对每个 `f : MonoidHom1 G H`，它总是 `G → H`。我们还用 `coe` 属性标记 `MonoidHom1.toFun`，以确保它在策略状态中几乎不可见地显示，只带一个 `↑` 前缀。

```
instance [Monoid G] [Monoid H] : CoeFun (MonoidHom1 G H) (fun _ ↦ G → H) where  
  coe := MonoidHom1.toFun  
  
attribute [coe] MonoidHom1.toFun
```

让我们检查，确实可以把打包的幺半群态射应用到一个元素上。

```
example [Monoid G] [Monoid H] (f : MonoidHom1 G H) : f 1 = 1 := f.map_one
```

对其他类型的态射也可以这样做，直到得到环同态。

```

@[ext]
structure AddMonoidHom1 (G H : Type) [AddMonoid G] [AddMonoid H] where
  toFun : G → H
  map_zero : toFun 0 = 0
  map_add : ∀ g g', toFun (g + g') = toFun g + toFun g'

instance [AddMonoid G] [AddMonoid H] : CoeFun (AddMonoidHom1 G H) (fun _ ↦ G → H) where
  coe := AddMonoidHom1.toFun

attribute [coe] AddMonoidHom1.toFun

@[ext]
structure RingHom1 (R S : Type) [Ring R] [Ring S] extends MonoidHom1 R S, AddMonoidHom1 R S

```

这种方法有几个问题。一个小问题是，我们不太知道应把 `coe` 属性放在哪里，因为 `RingHom1.toFun` 并不存在，相关函数是 `MonoidHom1.toFun ∘ RingHom1.toMonoidHom1`，而它不是一个可以标记属性的声明（不过仍可定义一个 `CoeFun (RingHom1 R S) (fun _ ↦ R → S)` 实例）。一个重要得多的问题是，关于幺半群态射的引理不会直接应用到环同态上。这样一来，要么每次想应用幺半群态射引理时都要摆弄 `RingHom1.toMonoidHom1`，要么为环同态重述每个这类引理。两种选择都不理想，所以 Mathlib 在这里使用一个新的层级技巧。思路是为“至少是幺半群态射”的对象定义一个类型类，让幺半群态射和环同态都成为该类的实例，并用它来陈述每个引理。在下面的定义中，`F` 可以是 `MonoidHom1 M N`；如果 `M` 和 `N` 有环结构，也可以是 `RingHom1 M N`。

```

class MonoidHomClass1 (F : Type) (M N : Type) [Monoid M] [Monoid N] where
  toFun : F → M → N
  map_one : ∀ f : F, toFun f 1 = 1
  map_mul : ∀ f g g', toFun f (g * g') = toFun f g * toFun f g'

```

不过，上面的实现还有一个问题。我们尚未注册到函数的强制转换实例。现在试着这样做。

```

def badInst [Monoid M] [Monoid N] [MonoidHomClass1 F M N] : CoeFun F (fun _ ↦ M → N) where
  coe := MonoidHomClass1.toFun

```

把它做成实例会很糟糕。当面对类似 `f x` 的表达式且 `f` 的类型不是函数类型时，Lean 会尝试寻找 `CoeFun` 实例，把 `f` 强制转换为函数。上面的函数具有类型

`{M N F : Type} → [Monoid M] → [Monoid N] → [MonoidHomClass1 F M N] → CoeFun F (fun x ↦ M → N)`，因此在尝试应用它时，Lean 事先并不清楚未知类型 `M`、`N` 和 `F` 应按什么顺序推断。这是一种与前面见过的坏实例略有不同的坏实例，但归根到底是同一个问题：在不知道 `M` 的情况下，Lean 将不得不在未知类型上搜索幺半群实例，从而无望地尝试数据库中的 *every* 幺半群实例。如果好奇这种实例会产生什么效果，可以在上述声明上方输入

`set_option synthInstance.checkSynthOrder false in`，把 `def badInst` 替换成 `instance`，然后在本文件中寻找随机失败。

这里的解法很简单：需要告诉 Lean 先搜索 `F` 是什么，然后再推出 `M` 和 `N`。这通过函数 `outParam` 完成。该函数被定义为恒等函数，但类型类机制仍会识别它，并触发所需行为。因此可以注意函数 `outParam`，重新尝试定义我们的类：

```

class MonoidHomClass2 (F : Type) (M N : outParam Type) [Monoid M] [Monoid N] where
  toFun : F → M → N
  map_one : ∀ f : F, toFun f 1 = 1
  map_mul : ∀ f g g', toFun f (g * g') = toFun f g * toFun f g'

instance [Monoid M] [Monoid N] [MonoidHomClass2 F M N] : CoeFun F (fun _ ↦ M → N) where
  coe := MonoidHomClass2.toFun

attribute [coe] MonoidHomClass2.toFun

```

现在可以继续执行实例化这个类的计划。

```

instance (M N : Type) [Monoid M] [Monoid N] : MonoidHomClass2 (MonoidHom1 M N) M N where
  toFun := MonoidHom1.toFun
  map_one := fun f ↦ f.map_one
  map_mul := fun f ↦ f.map_mul

instance (R S : Type) [Ring R] [Ring S] : MonoidHomClass2 (RingHom1 R S) R S where
  toFun := fun f ↦ f.toMonoidHom1.toFun
  map_one := fun f ↦ f.toMonoidHom1.map_one
  map_mul := fun f ↦ f.toMonoidHom1.map_mul

```

如所承诺的，假设有 `MonoidHomClass1 F` 的实例而对 `f : F` 证明的每个引理，都将同时适用于么半群态射和环同态。来看一个示例引理，并检查它适用于两种情形。

```

lemma map_inv_of_inv [Monoid M] [Monoid N] [MonoidHomClass2 F M N] (f : F) {m m' : M} (h : m * m' = 1) :
  f m * f m' = 1 := by
  rw [← MonoidHomClass2.map_mul, h, MonoidHomClass2.map_one]

example [Monoid M] [Monoid N] (f : MonoidHom1 M N) {m m' : M} (h : m * m' = 1) : f m * f m' = 1 :=
  map_inv_of_inv f h

example [Ring R] [Ring S] (f : RingHom1 R S) {r r' : R} (h : r * r' = 1) : f r * f r' = 1 :=
  map_inv_of_inv f h

```

乍看之下，这似乎又回到了把 `MonoidHom1` 做成类这一旧坏主意。但事实并非如此。一切都上移了一个抽象层级。类型类解析过程不会搜索函数，而会搜索 `MonoidHom1` 或 `RingHom1`。

我们的方法还有一个遗留问题：围绕字段 `toFun`、相应的 `CoeFun` 实例和 `coe` 属性存在重复代码。更好的做法还应记录这一模式只用于带有额外性质的函数，这意味着到函数的强制转换应当是单射。因此 Mathlib 通过基类 `DFunLike` 又增加了一层抽象（其中“DFun”表示依赖函数）。让我们在这个基础层之上重新定义 `MonoidHomClass`。

```

class MonoidHomClass3 (F : Type) (M N : outParam Type) [Monoid M] [Monoid N] extends
  DFunLike F M (fun _ ↦ N) where
  map_one : ∀ f : F, f 1 = 1
  map_mul : ∀ (f : F) g g', f (g * g') = f g * f g'

instance (M N : Type) [Monoid M] [Monoid N] : MonoidHomClass3 (MonoidHom1 M N) M N where
  coe := MonoidHom1.toFun
  coe_injective' _ _ := MonoidHom1.ext
  map_one := MonoidHom1.map_one
  map_mul := MonoidHom1.map_mul

```

当然，态射层级并不止于此。我们可以继续定义一个扩展 `MonoidHomClass3` 的类 `RingHomClass3`，并在 `RingHom` 上实例化它，之后还可在 `AlgebraHom` 上实例化它（代数是在环上添加一些额外结构得到的）。不过，我们已经覆盖了 Mathlib 中用于态射的主要形式化思想，你应当已经准备好理解 Mathlib 中态射是如何定义的。

作为练习，你应尝试定义有序类型之间打包的保序函数类，然后定义保序么半群态射。这只是训练用途。与连续函数一样，在 Mathlib 中保序函数主要以未打包形式使用，并由谓词 `Monotone` 定义。当然，你需要补全下面的类定义。

```

@[ext]
structure OrderPresHom (α β : Type) [LE α] [LE β] where
  toFun : α → β
  le_of_le : ∀ a a', a ≤ a' → toFun a ≤ toFun a'

@[ext]
structure OrderPresMonoidHom (M N : Type) [Monoid M] [LE M] [Monoid N] [LE N] extends
  MonoidHom1 M N, OrderPresHom M N

class OrderPresHomClass (F : Type) (α β : outParam Type) [LE α] [LE β]

instance (α β : Type) [LE α] [LE β] : OrderPresHomClass (OrderPresHom α β) α β where

instance (α β : Type) [LE α] [Monoid α] [LE β] [Monoid β] :
  OrderPresHomClass (OrderPresMonoidHom α β) α β where

instance (α β : Type) [LE α] [Monoid α] [LE β] [Monoid β] :
  MonoidHomClass3 (OrderPresMonoidHom α β) α β
:= sorry

```


8.3. 子对象

定义了一些代数结构及其态射之后，下一步是考虑继承这些代数结构的集合，例如子群或子环。这与前一个主题有很大重叠。事实上，`X` 中的集合实现为从 `X` 到 `Prop` 的函数，因此子对象就是满足某个谓词的函数。于是可以复用许多导致 `DFunLike` 类及其后代的思想。我们不会复用 `DFunLike` 本身，因为这会打破从 `Set X` 到 `X → Prop` 的抽象屏障。相反，有一个 `SetLike` 类。这个类不是把一个到函数类型的嵌入打包起来，而是把一个到 `Set` 类型的嵌入打包起来，并定义相应的强制转换和 `Membership` 实例。

```
@[ext]
structure Submonoid₁ (M : Type) [Monoid M] where
  /-- The carrier of a submonoid. -/
  carrier : Set M
  /-- The product of two elements of a submonoid belongs to the submonoid. -/
  mul_mem {a b} : a ∈ carrier → b ∈ carrier → a * b ∈ carrier
  /-- The unit element belongs to the submonoid. -/
  one_mem : 1 ∈ carrier

  /-- Submonoids in `M` can be seen as sets in `M`. -/
instance [Monoid M] : SetLike (Submonoid₁ M) M where
  coe := Submonoid₁.carrier
  coe_injective' _ _ := Submonoid₁.ext
```

有了上面的 `SetLike` 实例，我们已经可以自然地陈述子么半群 `N` 包含 `1`，而不必使用 `N.carrier`。也可以默认地把 `N` 当作 `M` 中的集合，并取它在某个映射下的直接像。

```
example [Monoid M] (N : Submonoid₁ M) : 1 ∈ N := N.one_mem

example [Monoid M] (N : Submonoid₁ M) (α : Type) (f : M → α) := f '' N
```

我们还有一个使用 `Subtype` 的到 `Type` 的强制转换。因此，给定子么半群 `N`，可以写参数 `(x : N)`，它可以被强制转换为属于 `N` 的 `M` 的元素。

```
example [Monoid M] (N : Submonoid₁ M) (x : N) : (x : M) ∈ N := x.property
```

使用这个到 `Type` 的强制转换，还可以处理给子么半群配备么半群结构的任务。我们将使用上面与 `N` 相关的类型给出的强制转换，以及断言该强制转换为单射的引理 `SetCoe.ext`。二者都由 `SetLike` 实例提供。

```
instance SubMonoid₁Monoid [Monoid M] (N : Submonoid₁ M) : Monoid N where
  mul := fun x y ↦ (x*y, N.mul_mem x.property y.property)
  mul_assoc := fun x y z ↦ SetCoe.ext (mul_assoc (x : M) y z)
  one := (1, N.one_mem)
  one_mul := fun x ↦ SetCoe.ext (one_mul (x : M))
  mul_one := fun x ↦ SetCoe.ext (mul_one (x : M))
```

注意，在上面的实例中，除了使用到 `M` 的强制转换并调用 `property` 字段之外，也可以如下使用解构绑定器。

```
example [Monoid M] (N : Submonoid₁ M) : Monoid N where
  mul := fun (x, hx) (y, hy) ↦ (x*y, N.mul_mem hx hy)
  mul_assoc := fun (x, _) (y, _) (z, _) ↦ SetCoe.ext (mul_assoc x y z)
  one := (1, N.one_mem)
  one_mul := fun (x, _) ↦ SetCoe.ext (one_mul x)
  mul_one := fun (x, _) ↦ SetCoe.ext (mul_one x)
```

为了把关于子么半群的引理应用到子群或子环上，需要一个类，就像态射的情形一样。注意，这个类把 `SetLike` 实例作为参数，因此不需要载体字段，并且可以在其字段中使用成员关系记号。

```
class SubmonoidClass₁ (S : Type) (M : Type) [Monoid M] [SetLike S M] : Prop where
  mul_mem : ∀ (s : S) {a b : M}, a ∈ s → b ∈ s → a * b ∈ s
  one_mem : ∀ s : S, 1 ∈ s

instance [Monoid M] : SubmonoidClass₁ (Submonoid₁ M) M where
  mul_mem := Submonoid₁.mul_mem
  one_mem := Submonoid₁.one_mem
```

作为练习，你应定义一个 `Subgroup1` 结构，为它配备 `SetLike` 实例和 `SubmonoidClass1` 实例，在与 `Subgroup1` 相关的子类型上放置 `Group` 实例，并定义一个 `SubgroupClass1` 类。

关于 Mathlib 中给定代数对象的子对象，还有一个非常重要的事实：它们总是形成完备格，而且这个结构经常被使用。例如，你可能想寻找“子么半群的交仍是子么半群”的引理。但它不会是一个引理，而会是一个下确界构造。我们来处理两个子么半群的情形。

```
instance [Monoid M] : Min (Submonoid1 M) :=
  (fun S1 S2 ↦
    { carrier := S1 ∩ S2
      one_mem := ⟨S1.one_mem, S2.one_mem⟩
      mul_mem := fun ⟨hx, hx'⟩ ⟨hy, hy'⟩ ↦ ⟨S1.mul_mem hx hy, S2.mul_mem hx' hy'⟩ })
```

这使我们可以把两个子么半群的交作为子么半群得到。

```
example [Monoid M] (N P : Submonoid1 M) : Submonoid1 M := N ∩ P
```

你可能觉得可惜，上面的例子中必须使用下确界符号 $\cap$ ，而不是交符号 $\cap$ 。但请想一想上确界。两个子么半群的并不是子么半群。然而，子么半群仍然形成格（甚至完备格）。实际上， $N \cup P$ 是由 N 与 P 的并生成的子么半群，当然若把它记为 $N \cup P$ 会非常令人困惑。所以可以看出，使用 $N \cap P$ 要一致得多。它在各种代数结构之间也一致得多。一开始看到两个向量空间 E 和 F 的和记为 $E \cup F$ 而不是 $E + F$ ，可能有些奇怪。但你会习惯的。很快你会觉得记号 $E + F$ 反而是一种干扰，因为它强调 $E \cup F$ 的元素可以写成 E 中一个元素与 F 中一个元素之和这一轶事性事实，而不是强调 $E \cup F$ 是同时包含 E 和 F 的最小向量空间这一根本事实。

本章最后一个主题是商。我们仍想解释 Mathlib 中如何构造方便记号并避免代码重复。这里的主要装置是类

`HasQuotient`，它允许类似 M / N 的记号。注意，商符号 $/$ 是一个特殊 Unicode 字符，不是普通 ASCII 除号。

作为例子，我们将构造交换么半群对一个子么半群的商，并把证明留给你。在最后一个例子中，可以使用 `Setoid.refl`，但它不会自动拾取相关的 `Setoid` 结构。可以用 `@` 语法提供所有参数来修正这个问题，例如 `@Setoid.refl M N.Setoid`。

```
def Submonoid.Setoid [CommMonoid M] (N : Submonoid M) : Setoid M where
  r := fun x y ↦ ∃ w ∈ N, ∃ z ∈ N, x*w = y*z
  iseqv := {
    refl := fun x ↦ ⟨1, N.one_mem, 1, N.one_mem, rfl⟩
    symm := fun ⟨w, hw, z, hz, h⟩ ↦ ⟨z, hz, w, hw, h.symm⟩
    trans := by
      sorry
  }

instance [CommMonoid M] : HasQuotient M (Submonoid M) where
  Quotient := fun N ↦ Quotient N.Setoid

def QuotientMonoid.mk [CommMonoid M] (N : Submonoid M) : M → M / N := Quotient.mk N.Setoid

instance [CommMonoid M] (N : Submonoid M) : Monoid (M / N) where
  mul := Quotient.map₂ (· * ·) (by
    sorry
  )
  mul_assoc := by
    sorry
  one := QuotientMonoid.mk N 1
  one_mul := by
    sorry
  mul_one := by
    sorry
```


9. 群与环

我们在Section 2.2中看到如何对群和环中的运算进行推理。随后在Section 7.2中，我们看到如何定义抽象代数结构，例如群结构，以及高斯整数上的环结构这类具体实例。Chapter 8解释了 Mathlib 如何处理抽象结构的层级。

本章将更详细地处理群与环。我们无法涵盖 Mathlib 中关于这些主题的所有内容，尤其是 Mathlib 还在不断增长。不过，我们会提供进入库的入口，并展示基本概念如何使用。这里与Chapter 8的讨论有一些重叠，但我们将重点放在如何使用 Mathlib，而不是这些主题处理方式背后的设计决策。因此，理解某些例子可能需要回顾Chapter 8的背景。

9.1. 幺半群与群

9.1.1. 幺半群及其态射

抽象代数课程常常从群开始，然后推进到环、域和向量空间。讨论环上的乘法时，这会带来一些扭曲，因为乘法运算并非来自群结构，但许多证明可以从群论逐字搬到这个新情境中。用纸笔做数学时，最常见的补救办法是把这些证明留作练习。一种效率稍低但更安全、也更适合形式化的做法，是使用`monoid`。类型 M 上的幺半群结构是一个内部合成律，满足结合律并有单位元。幺半群主要用于同时容纳群和环的乘法结构。不过也有许多自然例子；例如，自然数集合配备加法后形成幺半群。

从实践角度看，使用 Mathlib 时大多可以忽略幺半群。但在浏览 Mathlib 文件寻找引理时，需要知道它们存在。否则，你可能会在群论文件中寻找某个陈述，而它其实在幺半群相关文件中，因为它并不要求元素可逆。

类型 M 上幺半群结构的类型写作 `Monoid M`。函数 `Monoid` 是类型类，因此几乎总会以实例隐式参数出现（换言之，在方括号中）。默认情况下，`Monoid` 对运算使用乘法记号；若要使用加法记号，则用 `AddMonoid`。这些结构的交换版本会在 `Monoid` 前加前缀 `Comm`。

```
example {M : Type*} [Monoid M] (x : M) : x * 1 = x := mul_one x
example {M : Type*} [AddCommMonoid M] (x y : M) : x + y = y + x := add_comm x y
```

注意，虽然库中有 `AddMonoid`，但对非交换运算使用加法记号通常会造成混淆。

M 与 N 之间幺半群态射的类型称为 `MonoidHom M N`，记作 $M \rightarrow^* N$ 。当把这样的态射应用到 M 的元素上时，Lean 会自动把它看作从 M 到 N 的函数。加法版本称为 `AddMonoidHom`，记作 $M \rightarrow^+ N$ 。

```
example {M N : Type*} [Monoid M] [Monoid N] (x y : M) (f : M →* N) : f (x * y) = f x * f y :=
  f.map_mul x y
example {M N : Type*} [AddMonoid M] [AddMonoid N] (f : M →+ N) : f 0 = 0 :=
  f.map_zero
```

这些态射是打包映射，也就是说，它们把一个映射及其若干性质打包在一起。记住，Section 8.2解释了打包映射；这里我们只指出一个略显遗憾的后果：不能用普通函数复合来复合这些映射。相反，需要使用 `MonoidHom.comp` 和

`AddMonoidHom.comp`。

```
example {M N P : Type*} [AddMonoid M] [AddMonoid N] [AddMonoid P]
  (f : M →+ N) (g : N →+ P) : M →+ P := g.comp f
```

9.1.2. 群及其态射

关于群，我们还有更多内容要说；群就是额外满足每个元素都有逆元这一性质的么半群。

```
example {G : Type*} [Group G] (x : G) : x * x⁻¹ = 1 := mul_inv_cancel x
```

类似于前面见过的 `ring` 策略，也有一个 `group` 策略可以证明任意群中成立的恒等式。（等价地，它证明自由群中成立的恒等式。）

```
example {G : Type*} [Group G] (x y z : G) : x * (y * z) * (x * z)⁻¹ * (x * y * x⁻¹)⁻¹ = 1 := by
  group
```

还有一个用于交换加法群恒等式的策略，称为 `abel`。

```
example {G : Type*} [AddCommGroup G] (x y z : G) : z + x + (y - z - x) = y := by
  abel
```

有趣的是，群态射不过是群之间的么半群态射。因此，可以复制粘贴前面的一个例子，把 `Monoid` 替换为 `Group`。

```
example {G H : Type*} [Group G] [Group H] (x y : G) (f : G →* H) : f (x * y) = f x * f y :=
  f.map_mul x y
```

当然，我们确实会得到一些新性质，例如下面这个：

```
example {G H : Type*} [Group G] [Group H] (x : G) (f : G →* H) : f (x⁻¹) = (f x)⁻¹ :=
  f.map_inv x
```

你可能担心，构造群态射会要求做不必要的工作，因为么半群态射的定义要求单位元映到单位元，而对群态射来说这是自动的。实践中，额外工作并不困难；不过，为避免它，有一个函数可以从群之间与合成律相容的函数构造群态射。

```
example {G H : Type*} [Group G] [Group H] (f : G → H) (h : ∀ x y, f (x * y) = f x * f y) :
  G →* H :=
  MonoidHom.mk' f h
```

还有一个群（或么半群）同构类型 `MulEquiv`，记作 $\simeq^*$ （加法记号中对应 `AddEquiv`，记作 $\simeq^+$ ）。`f : G  $\simeq^*$  H` 的逆是 `MulEquiv.symm f : H  $\simeq^*$  G`，`f` 和 `g` 的复合是 `MulEquiv.trans f g`，`G` 的恒等同构是 `MulEquiv.refl G`。使用匿名投影记号，前两者可分别写作 `f.symm` 和 `f.trans g`。必要时，该类型的元素会自动强制转换为态射和函数。

```
example {G H : Type*} [Group G] [Group H] (f : G  $\simeq^*$  H) :
  f.trans f.symm = MulEquiv.refl G :=
  f.self_trans_symm
```

可以使用 `MulEquiv.ofBijective` 从双射态射构造同构。这样做会使逆函数不可计算。

```
noncomputable example {G H : Type*} [Group G] [Group H]
  (f : G →* H) (h : Function.Bijective f) :
  G  $\simeq^*$  H :=
  MulEquiv.ofBijective f h
```

9.1.3. 子群

正如群态射是打包的，`G` 的子群也是打包结构，由 `G` 中的一个集合及相关闭包性质组成。

```
example {G : Type*} [Group G] (H : Subgroup G) {x y : G} (hx : x ∈ H) (hy : y ∈ H) :
  x * y ∈ H :=
  H.mul_mem hx hy

example {G : Type*} [Group G] (H : Subgroup G) {x : G} (hx : x ∈ H) :
  x⁻¹ ∈ H :=
  H.inv_mem hx
```

在上面的例子中，重要的是要理解 `Subgroup G` 是 `G` 的子群类型，而不是谓词 `IsSubgroup H`，其中 `H` 是 `Set G` 的元素。`Subgroup G` 配备了到 `Set G` 的强制转换，以及 `G` 上的成员谓词。关于如何以及为何这样做，可参见Section 8.3。

当然，两个子群相同当且仅当它们有相同的元素。这个事实已注册给 `ext` 策略使用；它可以像证明两个集合相等一样，用来证明两个子群相等。

例如，要陈述并证明 `ℤ` 是 `ℚ` 的加法子群，我们真正想做的是构造一个类型为 `AddSubgroup ℚ` 的项，其到 `Set ℚ` 的投影为 `ℤ`，或者更准确地说，是 `ℤ` 在 `ℚ` 中的像。

```
example : AddSubgroup ℚ where
  carrier := Set.range ((↑) : ℤ → ℚ)
  add_mem' := by
    rintro _ _ (n, rfl) (m, rfl)
    use n + m
    simp
  zero_mem' := by
    use 0
    simp
  neg_mem' := by
    rintro _ (n, rfl)
    use -n
    simp
```

借助类型类，Mathlib 知道群的子群继承群结构。

```
example {G : Type*} [Group G] (H : Subgroup G) : Group H := inferInstance
```

这个例子有些微妙。对象 `H` 不是类型，但 Lean 会把它解释为 `G` 的子类型，从而自动强制转换为类型。因此，上面的例子可以更显式地重述为：

```
example {G : Type*} [Group G] (H : Subgroup G) : Group {x : G // x ∈ H} := inferInstance
```

拥有类型 `Subgroup G` 而不是谓词 `IsSubgroup : Set G → Prop` 的一个重要好处是，可以很容易地给 `Subgroup G` 赋予额外结构。尤其重要的是，相对于包含关系，它具有完备格结构。例如，我们不是有一个说明 `G` 的两个子群之交仍为子群的引理，而是使用格运算 `⊓` 来构造这个交。随后可以把关于格的任意引理应用到该构造上。

让我们检查，两个子群下确界的底层集合按定义确实就是它们的交。

```
example {G : Type*} [Group G] (H H' : Subgroup G) :
  ((H ⊓ H' : Subgroup G) : Set G) = (H : Set G) ⊓ (H' : Set G) := rfl
```

对底层集合的交使用不同记号可能看起来奇怪，但这种对应关系不能迁移到上确界运算和集合并上，因为子群的并通常不是子群。相反，需要使用由并生成的子群，这通过 `Subgroup.closure` 完成。

```
example {G : Type*} [Group G] (H H' : Subgroup G) :
  ((H ⊔ H' : Subgroup G) : Set G) = Subgroup.closure ((H : Set G) ⊔ (H' : Set G)) := by
  rw [Subgroup.sup_eq_closure]
```

另一个微妙点是，`G` 本身不具有类型 `Subgroup G`，所以需要一种方式谈论把 `G` 看成 `G` 的子群。这同样由格结构提供：全子群就是该格的顶元素。

```
example {G : Type*} [Group G] (x : G) : x ∈ (⊤ : Subgroup G) := trivial
```

类似地，该格的底元素是唯一元素为单位元的子群。

```
example {G : Type*} [Group G] (x : G) : x ∈ (⊥ : Subgroup G) ↔ x = 1 := Subgroup.mem_bot
```

作为操作群和子群的练习，可以定义子群被母群中一个元素共轭后的结果。

```
def conjugate {G : Type*} [Group G] (x : G) (H : Subgroup G) : Subgroup G where
  carrier := {a : G | ∃ h, h ∈ H ∧ a = x * h * x⁻¹}
  one_mem' := by
    dsimp
    sorry
  inv_mem' := by
    dsimp
    sorry
  mul_mem' := by
    dsimp
    sorry
```

把前两个主题联系起来，可以使用群态射对子群进行推出和拉回。Mathlib 中的命名约定是把这些操作称为 `map` 和 `comap`。这些不是常见的数学术语，但优点是比“推出”和“直接像”更短。

```
example {G H : Type*} [Group G] [Group H] (G' : Subgroup G) (f : G →* H) : Subgroup H :=
  Subgroup.map f G'

example {G H : Type*} [Group G] [Group H] (H' : Subgroup H) (f : G →* H) : Subgroup G :=
  Subgroup.comap f H'

#check Subgroup.mem_map
#check Subgroup.mem_comap
```

特别地，底子群在态射 `f` 下的原像是一个子群，称为 `f` 的 *kernel*；而 `f` 的值域也是一个子群。

```
example {G H : Type*} [Group G] [Group H] (f : G →* H) (g : G) :
  g ∈ MonoidHom.ker f ↔ f g = 1 :=
  f.mem_ker

example {G H : Type*} [Group G] [Group H] (f : G →* H) (h : H) :
  h ∈ MonoidHom.range f ↔ ∃ g : G, f g = h :=
  f.mem_range
```

作为操作群态射和子群的练习，让我们证明一些初等性质。它们在 Mathlib 中已经被证明；若想从这些练习中获益，不要太快使用 `exact?`。

```
section exercises
variable {G H : Type*} [Group G] [Group H]

open Subgroup

example (ϕ : G →* H) (S T : Subgroup H) (hST : S ≤ T) : comap ϕ S ≤ comap ϕ T := by
  sorry

example (ϕ : G →* H) (S T : Subgroup G) (hST : S ≤ T) : map ϕ S ≤ map ϕ T := by
  sorry

variable {K : Type*} [Group K]

-- Remember you can use the `ext` tactic to prove an equality of subgroups.
example (ϕ : G →* H) (ψ : H →* K) (U : Subgroup K) :
  comap (ψ.comp ϕ) U = comap ϕ (comap ψ U) := by
  sorry

-- Pushing a subgroup along one homomorphism and then another is equal to
-- pushing it forward along the composite of the homomorphisms.
example (ϕ : G →* H) (ψ : H →* K) (S : Subgroup G) :
  map (ψ.comp ϕ) S = map ψ (S.map ϕ) := by
  sorry

end exercises
```

让我们用两个非常经典的结果结束对 Mathlib 中子群的介绍。拉格朗日定理说明，有限群的子群基数整除该群的基数。Sylow 第一定理是拉格朗日定理的一个著名部分逆命题。

虽然 Mathlib 的这一角落部分设置为允许计算，我们仍可使用下面的 `open scoped` 命令告诉 Lean 使用非构造逻辑。

```
open scoped Classical

example {G : Type*} [Group G] (G' : Subgroup G) : Nat.card G' | Nat.card G :=
  (G'.index, mul_comm G'.index _ ▸ G'.index_mul_card.symm)

open Subgroup

example {G : Type*} [Group G] [Finite G] (p : ℕ) {n : ℕ} [Fact p.Prime]
  (hdvd : p ^ n | Nat.card G) : ∃ K : Subgroup G, Nat.card K = p ^ n :=
  Sylow.exists_subgroup_card_pow_prime p hdvd
```

接下来的两个练习推出拉格朗日引理的一个推论。（这同样已经在 Mathlib 中，所以不要太快使用 `exact?`。）

```
lemma eq_bot_iff_card {G : Type*} [Group G] {H : Subgroup G} :
  H = ⊥ ↔ Nat.card H = 1 := by
  suffices (∀ x ∈ H, x = 1) ↔ ∃ x ∈ H, ∀ a ∈ H, a = x by
    simpa [eq_bot_iff_forall, Nat.card_eq_one_iff_exists]
  sorry

#check card_dvd_of_le

lemma inf_bot_of_coprime {G : Type*} [Group G] (H K : Subgroup G)
  (h : (Nat.card H).Coprime (Nat.card K)) : H ⊓ K = ⊥ := by
  sorry
```

9.1.4. 具体群

也可以在 Mathlib 中操作具体群，尽管这通常比使用抽象理论更复杂。例如，给定任意类型 `X`，`X` 的置换群是 `Equiv.Perm X`。特别地，对称群 $\mathfrak{S}_n$ 是 `Equiv.Perm (Fin n)`。可以陈述关于该群的抽象结果，例如说若 `X` 有限，则 `Equiv.Perm X` 由轮换生成。

```
open Equiv

example {X : Type*} [Finite X] : Subgroup.closure {σ : Perm X | Perm.IsCycle σ} = ⊤ :=
  Perm.closure_isCycle
```

也可以完全具体地计算轮换的实际乘积。下面使用命令 `#simp`，它会在给定表达式上调用 `simp` 策略。记号 `c[]` 用于定义循环置换。在例子中，结果是 $\mathbb{N}$ 的一个置换。可以在出现的第一个数字上使用类型标注，例如 `(1 : Fin 5)`，使其成为 `Perm (Fin 5)` 中的计算。

```
#simp [mul_assoc] c[1, 2, 3] * c[2, 3, 4]
```

处理具体群的另一种方式是使用自由群和群表示。类型 `α` 上的自由群是 `FreeGroup α`，包含映射是 `FreeGroup.of : α → FreeGroup α`。例如，定义一个有三个元素的类型 `S`，这些元素记为 `a`、`b` 和 `c`，并定义相应自由群中的元素 `ab⁻¹`。

```
section FreeGroup

inductive S | a | b | c

open S

def myElement : FreeGroup S := (.of a) * (.of b)⁻¹
```

注意，我们给出了定义的预期类型，使 Lean 知道 `.of` 表示 `FreeGroup.of`。

自由群的泛性质体现为等价 `FreeGroup.lift`。例如，定义一个从 `FreeGroup S` 到 `Perm (Fin 5)` 的群态射，它把 `a` 送到 `c[1, 2, 3]`，把 `b` 送到 `c[2, 3, 1]`，并把 `c` 送到 `c[2, 3]`。

```
def myMorphism : FreeGroup S →* Perm (Fin 5) :=
  FreeGroup.lift fun | .a => c[1, 2, 3]
                  | .b => c[2, 3, 1]
                  | .c => c[2, 3]
```

作为最后一个具体例子，来看如何定义由单个元素生成且其三次方为一的群（因此该群同构于 $\mathbb{Z}/3$ ），并构造从该群到 `Perm (Fin 5)` 的态射。

作为恰有一个元素的类型，我们将使用 `Unit`，其唯一元素记作 `()`。函数 `PresentedGroup` 接受一个关系集合，即某个自由群元素的集合，并返回该自由群对这些关系生成的正规子群取商得到的群。（我们将在Section 9.1.6中看到如何处理更一般的商。）由于我们以某种方式把这件事隐藏在定义背后，所以使用 `deriving Group` 强制在 `myGroup` 上创建一个群实例。

```
def myGroup := PresentedGroup { .of () ^ 3 } deriving Group
```

表示群的泛性质保证，从该群出发的态射可以由把关系送到目标群单位元的函数构造。因此需要这样一个函数，以及该条件成立的证明。然后可以把这个证明传给 `PresentedGroup.toGroup`，得到所需的群态射。

```
def myMap : Unit → Perm (Fin 5)
| () => c[1, 2, 3]

lemma compat_myMap :
  ∀ r ∈ ({ .of () ^ 3 } : Set (FreeGroup Unit)), FreeGroup.lift myMap r = 1 := by
  rintro _ rfl
  simp
  decide

def myNewMorphism : myGroup →* Perm (Fin 5) := PresentedGroup.toGroup compat_myMap

end FreeGroup
```

9.1.5. 群作用

群论与数学其他部分相互作用的一种重要方式是使用群作用。群 `G` 在某个类型 `X` 上的作用，不过是从 `G` 到 `Equiv.Perm X` 的态射。因此，从某种意义上说，群作用已经被前面的讨论覆盖了。但我们不希望随身携带这个态射；相反，希望它尽可能由 Lean 自动推断。因此有一个类型类 `MulAction G X`。这种设置的缺点是，同一个群在同一个类型上有多个作用时，需要一些迂回办法，例如定义类型同义词，并为每个同义词携带不同的类型类实例。

特别地，这允许我们用 `g • x` 表示群元素 `g` 对点 `x` 的作用。

```
noncomputable section GroupActions

example {G X : Type*} [Group G] [MulAction G X] (g g' : G) (x : X) :
  g • (g' • x) = (g * g') • x :=
  (mul_smul g g' x).symm
```

还有用于加法群版本，称为 `AddAction`，其中作用记作 `+v`。例如，它用于仿射空间的定义。

```
example {G X : Type*} [AddGroup G] [AddAction G X] (g g' : G) (x : X) :
  g +v (g' +v x) = (g + g') +v x :=
  (add_vadd g g' x).symm
```

底层群态射称为 `MulAction.toPermHom`。

```
open MulAction

example {G X : Type*} [Group G] [MulAction G X] : G →* Equiv.Perm X :=
  toPermHom G X
```

作为说明，来看如何定义 Cayley 同构嵌入：把任意群 G 嵌入置换群，即 $\text{Perm } G$ 。为了实例化 $[\text{MulAction } G \ G]$ ，Lean 会找到由左乘给出的标准群作用，这正是我们想要的。

```
def CayleyIsoMorphism (G : Type*) [Group G] : G ≃* (toPermHom G G).range :=
  Equiv.Perm.subgroupOfMulAction G G
```

注意，在上面的定义之前，没有任何内容真正要求拥有群而非么半群（事实上，只要类型配备乘法运算即可）。

当我们想把 X 分解为轨道时，群条件才真正进入图景。 X 上相应的等价关系称为 $\text{MulAction.orbitRel}$ 。它没有被声明为全局实例。

```
example {G X : Type*} [Group G] [MulAction G X] : Setoid X := orbitRel G X
```

利用这一点，可以陈述 X 在 G 的作用下被划分为轨道。更准确地说，我们得到 X 与依赖积

$(\omega : \text{orbitRel.Quotient } G \ X) \times (\text{orbit } G \ (\text{Quotient.out}' \ \omega))$ 之间的双射，其中 $\text{Quotient.out}' \ \omega$ 只是选择一个投影到 ω 的元素。回忆一下，这个依赖积的元素是对 $\langle \omega, x \rangle$ ，其中 x 的类型 $\text{orbit } G \ (\text{Quotient.out}' \ \omega)$ 依赖于 ω 。

```
example {G X : Type*} [Group G] [MulAction G X] :
  X ≃ (ω : orbitRel.Quotient G X) × (orbit G (Quotient.out' ω)) :=
  MulAction.selfEquivSigmaOrbits G X
```

特别地，当 X 有限时，可以把它与 $\text{Fintype.card_congr}$ 和 $\text{Fintype.card_sigma}$ 结合起来，推出 X 的基数等于各轨道基数之和。此外，轨道与 G 在稳定子左平移作用下的商之间双射。子群通过左平移的这个作用被用于定义群对子群的商，记号为 $/$ ，所以可以使用下面这个简洁陈述。

```
example {G X : Type*} [Group G] [MulAction G X] (x : X) :
  orbit G x ≃ G / stabilizer G x :=
  MulAction.orbitEquivQuotientStabilizer G x
```

结合上述两个结果的一个重要特例，是 X 为群 G ，并配备子群 H 通过平移给出的作用。在这种情况下，所有稳定子都是平凡的，所以每个轨道都与 H 双射，于是得到：

```
example {G : Type*} [Group G] (H : Subgroup G) : G ≃ (G / H) × H :=
  groupEquivQuotientProdSubgroup
```

这是上面所见拉格朗日定理版本的概念性变体。注意，这个版本不作有限性假设。

作为本节练习，请使用前一个练习中定义的 `conjugate`，构造群在其子群上由共轭给出的作用。

```
variable {G : Type*} [Group G]

lemma conjugate_one (H : Subgroup G) : conjugate 1 H = H := by
  sorry

instance : MulAction G (Subgroup G) where
  smul := conjugate
  one_smul := by
    sorry
  mul_smul := by
    sorry
end GroupActions
```

9.1.6. 商群

在上面关于子群作用于群的讨论中，我们看到了商 G/H 。一般而言，它只是一个类型。当且仅当 H 是正规子群时，它才能被赋予群结构，使商映射成为群态射（而该群结构随后是唯一的）。

正规性假设是一个类型类 `Subgroup.Normal`，因此类型类推断可以使用它来导出商上的群结构。

```
noncomputable section QuotientGroup

example {G : Type*} [Group G] (H : Subgroup G) [H.Normal] : Group (G / H) := inferInstance

example {G : Type*} [Group G] (H : Subgroup G) [H.Normal] : G →* G / H :=
  QuotientGroup.mk' H
```

商群的泛性质通过 `QuotientGroup.lift` 使用：只要群态射 ϕ 的核包含 N ，它就下降到 G/N 。

```
example {G : Type*} [Group G] (N : Subgroup G) [N.Normal] {M : Type*}
  [Group M] (ϕ : G →* M) (h : N ≤ MonoidHom.ker ϕ) : G / N →* M :=
  QuotientGroup.lift N ϕ h
```

上面片段中目标群称为 M ，这暗示在 M 上有么半群结构其实已经足够。

一个重要特例是 $N = \ker \phi$ 。在这种情况下，下降得到的态射是单射，并得到到其像上的群同构。这个结果通常称为第一同构定理。

```
example {G : Type*} [Group G] {M : Type*} [Group M] (ϕ : G →* M) :
  G / MonoidHom.ker ϕ →* MonoidHom.range ϕ :=
  QuotientGroup.quotientKerEquivRange ϕ
```

把泛性质应用到态射 $\phi : G \rightarrow* G'$ 与商群投影 `Quotient.mk' N'` 的复合，也可以得到从 G/N 到 G'/N' 的态射。对 ϕ 所需的条件通常表述为“ ϕ 应把 N 映入 N' ”。但这等价于要求 ϕ 把 N' 拉回到 N 之上；后一条件更便于使用，因为拉回的定义不含有量词。

```
example {G G' : Type*} [Group G] [Group G']
  {N : Subgroup G} [N.Normal] {N' : Subgroup G'} [N'.Normal]
  {ϕ : G →* G'} (h : N ≤ Subgroup.comap ϕ N') : G / N →* G' / N' :=
  QuotientGroup.map N N' ϕ h
```

需要记住一个微妙点：类型 G/N 确实依赖于 N （直至定义相等），所以有一个证明说明两个正规子群 N 和 M 相等，并不足以使相应商类型按定义相等。不过，在这种情况下，泛性质确实给出一个同构。

```
example {G : Type*} [Group G] {M N : Subgroup G} [M.Normal]
  [N.Normal] (h : M = N) : G / M ≃* G / N := QuotientGroup.quotientMulEquivOfEq h
```

作为本节最后一组练习，我们将证明：若有限群 G 的正规子群 H 与 K 互不相交，并且它们基数的乘积等于 G 的基数，则 G 同构于 $H \times K$ 。回忆，在此语境中“互不相交”表示 $H \cap K = \perp$ 。

首先稍微操作一下拉格朗日引理，此时不假设子群正规或互不相交。

```
section
variable {G : Type*} [Group G] {H K : Subgroup G}

open MonoidHom

#check Nat.card_pos -- The nonempty argument will be automatically inferred for subgroups
#check Subgroup.index_eq_card
#check Subgroup.index_mul_card
#check Nat.eq_of_mul_eq_mul_right

lemma aux_card_eq [Finite G] (h' : Nat.card G = Nat.card H * Nat.card K) :
  Nat.card (G / H) = Nat.card K := by
  sorry
```

从现在起，假设我们的子群正规且互不相交，并假设基数条件。现在构造所需同构的第一个构件。


```

variable [H.Normal] [K.Normal] [Fintype G] (h : Disjoint H K)
(h' : Nat.card G = Nat.card H * Nat.card K)

#check Nat.bijective_iff_injective_and_card
#check ker_eq_bot_iff
#check restrict
#check ker_restrict

def iso1 : K ≃* G / H := by
  sorry

```

现在可以定义第二个构件。我们将需要 `MonoidHom.prod`，它由从 G_0 到 G_1 和 G_2 的态射，构造从 G_0 到 $G_1 \times G_2$ 的态射。

```

def iso2 : G ≃* (G / K) × (G / H) := by
  sorry

```

现在可以把所有部分合在一起。

```

#check MulEquiv.prodCongr

def finalIso : G ≃* H × K :=
  sorry

```

9.2. 环

9.2.1. 环、单位、态射与子环

类型 R 上环结构的类型是 `Ring R`。假设乘法可交换的变体是 `CommRing R`。我们已经看到，策略 `ring` 会证明任意由交换环公理推出的等式。

```

example {R : Type*} [CommRing R] (x y : R) : (x + y) ^ 2 = x ^ 2 + y ^ 2 + 2 * x * y := by ring

```

更特殊的变体不要求 R 上的加法形成群，只要求它形成加法么半群。相应的类型类是 `Semiring R` 和 `CommSemiring R`。自然数类型是 `CommSemiring R` 的一个重要实例，任何取值于自然数的函数类型也是如此。另一个重要例子是环中理想的类型，稍后会讨论。策略 `ring` 的名称有双重误导性，因为它假设交换性，却也能在半环中工作。换言之，它适用于任意 `CommSemiring`。

```

example (x y : ℕ) : (x + y) ^ 2 = x ^ 2 + y ^ 2 + 2 * x * y := by ring

```

环和半环类还有一些版本，不假设乘法单位元存在，也不假设乘法满足结合律。这里不讨论这些版本。

一些传统上在环论入门中讲授的概念，实际上只涉及底层乘法么半群。一个突出例子是环的单位的定义。每个（乘法）么半群 M 都有一个谓词 `IsUnit : M → Prop`，断言双侧逆元存在；有一个单位类型 `Units M`，记作 $M^\times$ ；并有到 M 的强制转换。类型 `Units M` 把一个可逆元素、它的逆，以及保证二者确实互为逆的性质打包在一起。这个实现细节主要在定义可计算函数时相关。在大多数情形中，可以用 `IsUnit.unit {x : M} : IsUnit x → M×` 构造单位。在交换情形中，还有

```

Units.mkOfMulEqOne (x y : M) : x * y = 1 → M×

```

，它把 x 视为单位来构造。

```

example (x : ℤ×) : x = 1 ∨ x = -1 := Int.units_eq_one_or x

example {M : Type*} [Monoid M] (x : M×) : (x : M) * x-1 = 1 := Units.mul_inv x

example {M : Type*} [Monoid M] : Group M× := inferInstance

```

两个（半）环 R 与 S 之间环态射的类型是 `RingHom R S`，记作 $R \rightarrow^+ S$ 。

```
example {R S : Type*} [Ring R] [Ring S] (f : R →+* S) (x y : R) :
  f (x + y) = f x + f y := f.map_add x y

example {R S : Type*} [Ring R] [Ring S] (f : R →+* S) : Rx →+* Sx :=
  Units.map f
```

同构版本是 `RingEquiv`，记号为 $\simeq_{+*}$ 。

与子幺半群和子群一样，对环 `R` 的子环也有类型 `Subring R`；但这个类型远不如子群类型有用，因为不能用子环对环取商。

```
example {R : Type*} [Ring R] (S : Subring R) : Ring S := inferInstance
```

还要注意，`RingHom.range` 产生一个子环。

9.2.2. 理想与商

由于历史原因，Mathlib 只对交换环有理想理论。（环库最初是为快速推进现代代数几何基础而开发的。）因此，本节将使用交换（半）环。`R` 的理想被定义为把 `R` 看作 `R`-模时的子模。模将在后面的线性代数章节中介绍，但这个实现细节通常可以安全忽略，因为大多数（但不是全部）相关引理都在理想这一特殊语境中重新陈述。不过，匿名投影记号并不总会按预期工作。例如，下面片段中不能把 `Ideal.Quotient.mk I` 替换为 `I.Quotient.mk`，因为其中有两个 `.`，所以它会被解析为 `(Ideal.Quotient I).mk`；但 `Ideal.Quotient` 本身并不存在。

```
example {R : Type*} [CommRing R] (I : Ideal R) : R →+* R / I :=
  Ideal.Quotient.mk I

example {R : Type*} [CommRing R] {a : R} {I : Ideal R} :
  Ideal.Quotient.mk I a = 0 ↔ a ∈ I :=
  Ideal.Quotient.eq_zero_iff_mem
```

商环的泛性质是 `Ideal.Quotient.lift`。

```
example {R S : Type*} [CommRing R] [CommRing S] (I : Ideal R) (f : R →+* S)
  (H : I ≤ RingHom.ker f) : R / I →+* S :=
  Ideal.Quotient.lift I f H
```

特别地，它导出环的第一同构定理。

```
example {R S : Type*} [CommRing R] [CommRing S] (f : R →+* S) :
  R / RingHom.ker f ≃+* f.range :=
  RingHom.quotientKerEquivRange f
```

理想相对于包含关系形成完备格结构，同时也形成半环结构。这两个结构之间相互配合良好。

```
variable {R : Type*} [CommRing R] {I J : Ideal R}

example : I + J = I ∪ J := rfl

example {x : R} : x ∈ I + J ↔ ∃ a ∈ I, ∃ b ∈ J, a + b = x := by
  simp [Submodule.mem_sup]

example : I * J ≤ J := Ideal.mul_le_left

example : I * J ≤ I := Ideal.mul_le_right

example : I * J ≤ I ∩ J := Ideal.mul_le_inf
```

可以使用环态射分别通过 `Ideal.map` 和 `Ideal.comap` 推出和拉回理想。照常，后者更方便使用，因为它不涉及存在量词。这解释了为什么在陈述允许构造商环之间态射的条件时使用它。

```
example {R S : Type*} [CommRing R] [CommRing S] (I : Ideal R) (J : Ideal S) (f : R →+* S)
(H : I ≤ Ideal.comap f J) : R / I →+* S / J :=
Ideal.quotientMap J f H
```

一个微妙点是，类型 R/I 确实依赖于 I （直至定义相等），所以有一个证明说明两个理想 I 和 J 相等，并不足以使相应商类型按定义相等。不过，泛性质在这种情况下确实提供一个同构。

```
example {R : Type*} [CommRing R] {I J : Ideal R} (h : I = J) : R / I ≃+* R / J :=
Ideal.quotEquivOfEq h
```

现在可以把中国剩余同构作为例子给出。注意带指标下确界符号 $\prod$ 与类型的大积符号 Π 之间的区别。根据字体不同，它们可能很难区分。

```
example {R : Type*} [CommRing R] {ι : Type*} [Fintype ι] (f : ι → Ideal R)
(hf : ∀ i j, i ≠ j → IsCoprime (f i) (f j)) : (R / ∏ i, f i) ≃+* ∏ i, R / f i :=
Ideal.quotientInfRingEquivPiQuotient f hf
```

中国剩余定理的初等版本，即关于 $\mathbb{Z}/m$ 的陈述，可以很容易从前一个定理推出：

```
open BigOperators PiNotation

example {ι : Type*} [Fintype ι] (a : ι → ℕ) (coprime : ∀ i j, i ≠ j → (a i).Coprime (a j)) :
ZMod (∏ i, a i) ≃+* ∏ i, ZMod (a i) :=
ZMod.prodEquivPi a coprime
```

作为一组练习，我们将在一般情形中重新证明中国剩余定理。

首先，需要使用商环的泛性质，把定理中出现的映射定义为一个环态射。

```
variable {ι R : Type*} [CommRing R]
open Ideal Quotient Function

#check Pi.ringHom
#check ker_Pi_Quotient_mk

/-- The homomorphism from `R / ∏ i, I i` to `∏ i, R / I i` featured in the Chinese
Remainder Theorem. -/
def chineseMap (I : ι → Ideal R) : (R / ∏ i, I i) →+* ∏ i, R / I i :=
sorry
```

请确认接下来的两个引理可以用 `refl` 证明。

```
lemma chineseMap_mk (I : ι → Ideal R) (x : R) :
chineseMap I (Quotient.mk _ x) = fun i : ι ↦ Ideal.Quotient.mk (I i) x :=
sorry

lemma chineseMap_mk' (I : ι → Ideal R) (x : R) (i : ι) :
chineseMap I (mk _ x) i = mk (I i) x :=
sorry
```

下一个引理证明中国剩余定理的容易一半，并且不对理想族作任何假设。证明不到一行。

```
#check injective_lift_iff

lemma chineseMap_inj (I : ι → Ideal R) : Injective (chineseMap I) := by
sorry
```

现在准备进入定理的核心，它将证明 `chineseMap` 的满射性。首先，需要了解表达互素性（也称余极大性假设）的不同方式。下面只会用到前两个。

```
#check IsCoprime
#check isCoprime_iff_add
#check isCoprime_iff_exists
#check isCoprime_iff_sup_eq
#check isCoprime_iff_codisjoint
```

我们借此机会对 `Finset` 作归纳。关于 `Finset` 的相关引理列在下面。记住，`ring` 策略适用于半环，而环的理想形成半环。

```
#check Finset.mem_insert_of_mem
#check Finset.mem_insert_self

theorem isCoprime_Inf {I : Ideal R} {J :  $\iota \rightarrow$  Ideal R} {s : Finset  $\iota$ }
(hf :  $\forall j \in s, \text{IsCoprime } I (J j)$ ) : IsCoprime I ( $\bigcap j \in s, J j$ ) := by
  classical
  simp_rw [isCoprime_iff_add] at *
  induction s using Finset.induction with
  | empty =>
    simp
  | @insert i s _ hs =>
    rw [Finset.iInf_insert, inf_comm, one_eq_top, eq_top_iff,  $\leftarrow$  one_eq_top]
    set K :=  $\bigcap j \in s, J j$ 
    calc
      1 = I + K                               := sorry
      _ = I + K * (I + J i)                     := sorry
      _ = (1 + K) * I + K * J i                 := sorry
      _  $\leq$  I + K  $\bigcap$  J i                       := sorry
```

现在可以证明中国剩余定理中出现的映射是满射。

```
lemma chineseMap_surj [Fintype  $\iota$ ] {I :  $\iota \rightarrow$  Ideal R}
(hI :  $\forall i j, i \neq j \rightarrow \text{IsCoprime } (I i) (I j)$ ) : Surjective (chineseMap I) := by
  classical
  intro g
  choose f hf using fun i  $\mapsto$  Ideal.Quotient.mk_surjective (g i)
  have key :  $\forall i, \exists e : R, \text{mk } (I i) e = 1 \wedge \forall j, j \neq i \rightarrow \text{mk } (I j) e = 0$  := by
    intro i
    have hI' :  $\forall j \in (\{i\} : \text{Finset } \iota)^c, \text{IsCoprime } (I i) (I j)$  := by
      sorry
    sorry
  choose e he using key
  use mk _ ( $\sum i, f i * e i$ )
  sorry
```

现在所有部分在下面汇合：

```
noncomputable def chineseIso [Fintype  $\iota$ ] (f :  $\iota \rightarrow$  Ideal R)
(hf :  $\forall i j, i \neq j \rightarrow \text{IsCoprime } (f i) (f j)$ ) : ( $R / \bigcap i, f i$ )  $\simeq$ +*  $\prod i, R / f i$  :=
{ Equiv.ofBijective _ (chineseMap_inj f, chineseMap_surj hf),
  chineseMap f with }
```

9.2.3. 代数与多项式

给定交换（半）环 `R`，一个 *algebra over* `R` 的代数，是一个半环 `A`，并配备从 `R` 到 `A` 的环态射，且该态射的像与 `A` 的每个元素交换。这被编码为类型类 `Algebra R A`。从 `R` 到 `A` 的态射称为结构映射，在 Lean 中记作 `algebraMap R A : R  $\rightarrow$ +* A`。对某个 `r : R`，用 `algebraMap R A r` 乘以 `a : A`，称为 `a` 由 `r` 作标量乘法，记作 `r • a`。注意，这里的代数概念有时称为 *associative unital algebra*，以强调存在更一般的代数概念。

`algebraMap R A` 是环态射这一事实，把许多关于标量乘法的性质打包在一起，例如下面这些：

```
example {R A : Type*} [CommRing R] [Ring A] [Algebra R A] (r r' : R) (a : A) :
  (r + r') • a = r • a + r' • a :=
  add_smul r r' a

example {R A : Type*} [CommRing R] [Ring A] [Algebra R A] (r r' : R) (a : A) :
  (r * r') • a = r • r' • a :=
  mul_smul r r' a
```

两个 `R`-代数 `A` 与 `B` 之间的态射，是与 `R` 中元素的标量乘法交换的环态射。它们是打包态射，类型为 `AlgHom R A B`，记作 `A  $\rightarrow$ _a[R] B`。

非交换代数的重要例子包括自同态代数和方阵代数，二者都会在线性代数章节中涉及。本章将讨论一个最重要的交换代数例子，即多项式代数。

以 R 为系数的一元多项式代数称为 `Polynomial R`；一旦打开 `Polynomial` 命名空间，它就可以写成 `R[X]`。从 R 到 `R[X]` 的代数结构映射记作 `C`，表示“常数”，因为相应的多项式函数总是常函数。不定元记作 `X`。

```
open Polynomial

example {R : Type*} [CommRing R] : R[X] := X

example {R : Type*} [CommRing R] (r : R) := X - C r
```

在上面第一个例子中，给 Lean 预期类型至关重要，因为无法从定义体确定它。在第二个例子中，由于 `r` 的类型已知，目标多项式代数可以从我们使用 `C r` 推断出来。

因为 `C` 是从 R 到 `R[X]` 的环态射，所以在 `R[X]` 中计算之前，可以使用所有环态射引理，例如 `map_zero`、`map_one`、`map_mul` 和 `map_pow`。例如：

```
example {R : Type*} [CommRing R] (r : R) : (X + C r) * (X - C r) = X ^ 2 - C (r ^ 2) := by
  rw [C.map_pow]
  ring
```

可以使用 `Polynomial.coeff` 访问系数。

```
example {R : Type*} [CommRing R] (r : R) : (C r).coeff 0 = r := by simp

example {R : Type*} [CommRing R] : (X ^ 2 + 2 * X + C 3 : R[X]).coeff 1 = 2 := by simp
```

定义多项式次数总是棘手，因为零多项式是特殊情形。Mathlib 有两个变体：`Polynomial.natDegree : R[X] → ℕ` 把零多项式的次数指定为 `0`，而 `Polynomial.degree : R[X] → WithBot ℕ` 指定为 `⊥`。后者中，`WithBot ℕ` 可以看作 $\mathbb{N} \cup \{-\infty\}$ ，只是 `-\infty` 记为 `⊥`，这与完备格中底元素使用同一符号。这个特殊值用作零多项式的次数，并且对加法具有吸收性。（它对乘法几乎也具有吸收性，例外是 `⊥ * 0 = 0`。）

从数学本质上说，`degree` 版本是正确的。例如，它允许我们陈述乘积次数的预期公式（假设基环没有零因子）。

```
example {R : Type*} [Semiring R] [NoZeroDivisors R] {p q : R[X]} :
  degree (p * q) = degree p + degree q :=
  Polynomial.degree_mul
```

而 `natDegree` 版本则需要假设多项式非零。

```
example {R : Type*} [Semiring R] [NoZeroDivisors R] {p q : R[X]} (hp : p ≠ 0) (hq : q ≠ 0) :
  natDegree (p * q) = natDegree p + natDegree q :=
  Polynomial.natDegree_mul hp hq
```

然而，`ℕ` 比 `WithBot ℕ` 好用得多，所以 Mathlib 同时提供两个版本，并提供在二者之间转换的引理。此外，在计算复合的次数时，`natDegree` 是更方便的定义。多项式的复合是 `Polynomial.comp`，并且有：

```
example {R : Type*} [Semiring R] [NoZeroDivisors R] {p q : R[X]} :
  natDegree (comp p q) = natDegree p * natDegree q :=
  Polynomial.natDegree_comp
```

多项式给出多项式函数：任意多项式都可以使用 `Polynomial.eval` 在 R 上求值。

```
example {R : Type*} [CommRing R] (P : R[X]) (x : R) := P.eval x

example {R : Type*} [CommRing R] (r : R) : (X - C r).eval r = 0 := by simp
```

特别地，有一个谓词 `IsRoot`，它对 `R` 中使多项式为零的元素 `r` 成立。

```
example {R : Type*} [CommRing R] (P : R[X]) (r : R) : IsRoot P r ↔ P.eval r = 0 := Iff.rfl
```

我们希望说，若 `R` 没有零因子，则一个多项式的根数（按重数计）至多为其次数。但零多项式的情形再次令人棘手。因此，Mathlib 定义 `Polynomial.roots`，把多项式 `P` 送到一个多重集：若 `P` 为零，则该有限集定义为空；否则为 `P` 的根，带重数。这个定义只在底层环为整环时可用，因为否则该定义没有良好性质。

```
example {R : Type*} [CommRing R] [IsDomain R] (r : R) : (X - C r).roots = {r} :=
  roots_X_sub_C r

example {R : Type*} [CommRing R] [IsDomain R] (r : R) (n : ℕ):
  ((X - C r) ^ n).roots = n • {r} :=
  by simp
```

`Polynomial.eval` 和 `Polynomial.roots` 都只考虑系数环。它们无法表达 $X^2 - 2 : \mathbb{Q}[X]$ 在 $\mathbb{R}$ 中有根，或 $X^2 + 1 : \mathbb{R}[X]$ 在 $\mathbb{C}$ 中有根。为此，需要 `Polynomial.aeval`，它会在任意 `R`-代数中对 `P : R[X]` 求值。更精确地说，给定一个半环 `A` 和一个 `Algebra R A` 实例，`Polynomial.aeval` 把每个元素 `a : A` 送到 `R`-代数态射 $R[X] \rightarrow A$ ，该态射把 `X` 求值为 `a`。由于 `AlgHom` 有到函数的强制转换，可以把它应用到多项式上。但 `aeval` 不以多项式作为参数，所以不能像上面的 `P.eval` 一样使用点记号。

```
example : aeval Complex.I (X ^ 2 + 1 : ℝ[X]) = 0 := by simp
```

在此语境中，与 `roots` 对应的函数是 `aroots`，它接受一个多项式，再接受一个代数，并输出一个多重集（对于零多项式有与 `roots` 相同的注意事项）。

```
open Complex Polynomial

example : aroots (X ^ 2 + 1 : ℝ[X]) ℂ = {Complex.I, -I} := by
  suffices roots (X ^ 2 + 1 : ℂ[X]) = {I, -I} by simpa [aroots_def]
  have factored : (X ^ 2 + 1 : ℂ[X]) = (X - C I) * (X - C (-I)) := by
    have key : (C I * C I : ℂ[X]) = -1 := by simp [← C_mul]
    rw [C_neg]
    linear_combination key
  have p_ne_zero : (X - C I) * (X - C (-I)) ≠ 0 := by
    intro H
    apply_fun eval 0 at H
    simp [eval] at H
  simp only [factored, roots_mul p_ne_zero, roots_X_sub_C]
  rfl

-- Mathlib knows about D'Alembert-Gauss theorem: ``ℂ`` is algebraically closed.
example : IsAlgClosed ℂ := inferInstance
```

更一般地，给定环态射 $f : R \rightarrow S$ ，可以用 `Polynomial.eval₂` 在 `S` 中的一点对 `P : R[X]` 求值。它产生从 `R[X]` 到 `S` 的实际函数，因为它不假设存在 `Algebra R S` 实例，所以点记号会按预期工作。

```
#check (Complex.ofRealHom : ℝ →+* ℂ)

example : (X ^ 2 + 1 : ℝ[X]).eval₂ Complex.ofRealHom Complex.I = 0 := by simp
```

最后简要提一下多元多项式。给定交换半环 `R`，以 `R` 为系数、由类型 `σ` 索引不定元的 `R`-代数是 `MVPolynomial σ R`。给定 `i : σ`，相应多项式是 `MvPolynomial.X i`。（照常，可以打开 `MVPolynomial` 命名空间，把它缩写为 `X i`。）例如，如果想要两个不定元，可以用 `Fin 2` 作为 `σ`，并把定义 $\mathbb{R}^{2^1}$ 中单位圆的多项式写作：

```
open MVPolynomial

def circleEquation : MvPolynomial (Fin 2) ℝ := X 0 ^ 2 + X 1 ^ 2 - 1
```

回忆，函数应用具有很高优先级，因此上面的表达式读作 $(x_0)^2 + (x_1)^2 - 1$ 。可以对它求值，以确认坐标为 $(1, 0)$ 的点在圆上。回忆，记号 $![...]$ 表示 $\text{Fin } n \rightarrow X$ 的元素，其中自然数 n 由参数个数决定，而类型 X 由参数类型决定。

```
example : MvPolynomial.eval ![1, 0] circleEquation = 0 := by simp [circleEquation]
```

10. 线性代数

10.1. 向量空间与线性映射

10.1.1. 向量空间

我们将直接从抽象线性代数开始，即任意域上的向量空间中的线性代数。不过，关于矩阵的内容可见[Section 10.4.1](#)，该节在逻辑上并不依赖这套抽象理论。实际上，Mathlib 处理的是更一般的线性代数，其中会出现“模”这一术语；但暂时我们可以把这看作一种略显特殊的命名习惯。

要表达“设 K 为域，设 V 为 K 上的向量空间”（并使它们成为后续结果的隐式参数），可以写作：

```
variable {K : Type*} [Field K] {V : Type*} [AddCommGroup V] [Module K V]
```

我们在[Chapter 8](#)中解释过，为什么需要两个分开的类型类 `[AddCommGroup V] [Module K V]`。简短地说，数学上我们想表达的是：具有 K -向量空间结构蕴含具有加法交换群结构。我们当然可以把这一点告诉 Lean。但这样一来，每当 Lean 需要在某个类型 V 上寻找这样的群结构时，它就会开始寻找相对于某个完全未指定的域 K 的向量空间结构，而这个域无法从 V 推断出来。这会严重损害类型类合成系统的工作。

向量 v 乘以标量 a 记作 $a \bullet v$ 。下面的例子列出这一运算与加法相互作用的若干代数规则。当然，`simp` 或 `apply?` 都可以找到这些证明。还有一个 `module` 策略，它会解决由向量空间和域的公理推出的目标，正如在交换环中使用 `ring` 策略、在群中使用 `group` 策略一样。不过仍然有必要记住，在引理名称中，标量乘法通常缩写为 `smul`。

```
example (a : K) (u v : V) : a • (u + v) = a • u + a • v :=
  smul_add a u v

example (a b : K) (u : V) : (a + b) • u = a • u + b • u :=
  add_smul a b u

example (a b : K) (u : V) : a • b • u = b • a • u :=
  smul_comm a b u
```

给较熟悉代数的读者补充一点：正如术语所暗示的，Mathlib 的线性代数也涵盖（不一定交换的）环上的模。事实上，它甚至涵盖半环上的半模。若你认为自己并不需要这种一般性，可以思考下面的例子；它很好地概括了理想作用于子模时的大量代数规则：

```
example {R M : Type*} [CommSemiring R] [AddCommMonoid M] [Module R M] :
  Module (Ideal R) (Submodule R M) :=
  inferInstance
```

要表达 `M` 是环 `R` 上的模，可以写作：

```
section
variable {R : Type*} [Ring R] {M : Type*} [AddCommGroup M] [Module R M]
end
```

如果把这与声明向量空间的方式比较，可以看到两个差异。除了变量改名之外，唯一的变化就是把假设 `Field K` 替换为假设 `Ring R`。

要表达 `M` 是半环 `R` 上的半模，我们把假设 `Ring R` 替换为假设 `Semiring R`，同时把 `AddCommGroup M` 替换为 `AddCommMonoid M`；这反映了半模没有取负运算：


```
section
variable {R : Type*} [Semiring R] {M : Type*} [AddCommMonoid M] [Module R M]
end
```

即使 R 有取负运算，Lean 也不能自动推断 M 有取负运算，因为对任意给定的 M ，它先验上无法猜出正确的 R 。因此，如果我们声明 R 是环，且 M 是 R 上的模，那么要么必须声明 M 是一个 `AddCommGroup`，要么手动把相应实例加入上下文：

```
example {R : Type*} [Ring R] {M : Type*} [AddCommMonoid M] [Module R M] : M → M :=
-- Make sure to use `letI` (instead of `haveI` or `let`) to make it easier to
-- identify the existing additive structure from the `AddCommMonoid M` hypothesis
-- with the new one introduced in the line below.
letI : AddCommGroup M := Module.addCommMonoidToAddCommGroup R
fun x => - x
```

10.1.2. 线性映射

接下来需要线性映射。与群态射一样，Mathlib 中的线性映射是打包映射，也就是由一个映射及其线性性质的证明组成的对象。在应用这些打包映射时，它们会被转换为通常的函数。关于这种设计，参见 [Chapter 8](#)。

两个 K -向量空间 V 与 W 之间的线性映射类型记作 $V \rightarrow_{[K]} W$ 。下标 $/$ 表示 linear。初看之下，在这个记号中指定 K 或许有些奇怪；但当多个域同时出现时，这一点非常关键。例如，从 $\mathbb{C}$ 到 $\mathbb{C}$ 的实线性映射是所有形如 $z \mapsto az + b\bar{z}$ 的映射，而复线性映射只有形如 $z \mapsto az$ 的映射，这一区别在复分析中至关重要。

```
variable {W : Type*} [AddCommGroup W] [Module K W]

variable (ϕ : V →[K] W)

example (a : K) (v : V) : ϕ (a • v) = a • ϕ v :=
  map_smul ϕ a v

example (v w : V) : ϕ (v + w) = ϕ v + ϕ w :=
  map_add ϕ v w
```

注意， $V \rightarrow_{[K]} W$ 本身也带有有意义的代数结构（这也是将这些映射打包的动机之一）。它是一个 K -向量空间，因此可以对线性映射相加，也可以用标量乘以线性映射。

```
variable (ψ : V →[K] W)

#check (2 • ϕ + ψ : V →[K] W)
```

使用打包映射的一个代价是，不能直接使用通常的函数复合。我们需要使用 `LinearMap.comp`，或者使用记号 $\circ_1$ 。

```
variable (θ : W →[K] V)

#check (ϕ.comp θ : W →[K] W)
#check (ϕ ∘1 θ : W →[K] W)
```

构造线性映射主要有两种方式。第一种是直接构造该结构，给出函数以及线性性的证明。照常，结构代码操作可以简化这一步：你可以输入 `example : V →[K] V := _`，然后在下划线处使用“Generate a skeleton”代码操作生成骨架。

```
example : V →[K] V where
  toFun v := 3 • v
  map_add' _ _ := smul_add ..
  map_smul' _ _ := smul_comm ..
```

你可能会疑惑，为什么 `LinearMap` 的证明字段名称以撇号结尾。这是因为它们定义时，到函数的强制转换尚未定义，所以这些字段要用 `LinearMap.toFun` 来表述。随后，它们又以关于到函数的强制转换的形式被重新陈述为 `LinearMap.map_add` 和 `LinearMap.map_smul`。事情还没有到此为止。我们还希望有一个适用于任何保持加法的（打包）映射的 `map_add` 版本，例如

加法群态射、线性映射、连续线性映射、 K -代数映射等。这个版本就是根命名空间中的 `map_add`。中间版本 `LinearMap.map_add` 有些冗余，但允许使用点记号，有时很方便。`map_smul` 也有类似情况；一般框架见Chapter 8。

```
#check (φ.map_add' : ∀ x y : V, φ.toFun (x + y) = φ.toFun x + φ.toFun y)
#check (φ.map_add : ∀ x y : V, φ (x + y) = φ x + φ y)
#check (map_add φ : ∀ x y : V, φ (x + y) = φ x + φ y)
```

还可以借助各种组合子，从 Mathlib 中已经定义好的线性映射构造新的线性映射。例如，上面的例子已经作为

`LinearMap.lsmul K V 3` 存在。这里把 `K` 和 `V` 作为显式参数有若干原因。最直接的原因是，如果只有 `LinearMap.lsmul 3`，Lean 无法推断 `V`，甚至也无法推断 `K`。此外，`LinearMap.lsmul K V` 本身也是一个有意义的对象：它的类型是 `K →[K] V →[K] V`，意思是一个从 `K`（视为自身上的向量空间）到从 `V` 到 `V` 的 `K`-线性映射空间的 `K`-线性映射。

```
#check (LinearMap.lsmul K V 3 : V →[K] V)
#check (LinearMap.lsmul K V : K →[K] V →[K] V)
```

还有一个线性同构类型 `LinearEquiv`，记作 `V ≃[K] W`。若 `f : V ≃[K] W`，则其逆为 `f.symm : W ≃[K] V`；`f` 与 `g` 的复合是 `f.trans g`，也记作 `f <<>[K] g`；`V` 的恒等同构是 `LinearEquiv.refl K V`。必要时，这个类型的元素会自动强制转换为态射和函数。

```
example (f : V ≃[K] W) : f <<>[K] f.symm = LinearEquiv.refl K V :=
  f.self_trans_symm
```

可以使用 `LinearEquiv.ofBijective` 从一个双射态射构造同构。这样做会使逆函数成为不可计算对象。

```
noncomputable example (f : V →[K] W) (h : Function.Bijective f) : V ≃[K] W :=
  .ofBijective f h
```

注意，在上面的例子中，Lean 利用已声明的类型来理解 `.ofBijective` 指的是 `LinearEquiv.ofBijective`，无需打开任何命名空间。

10.1.3. 向量空间的和与积

可以用直和与直积从已有向量空间构造新的向量空间。先考虑两个向量空间的情形。此时和与积没有区别，可以直接使用积类型。在下面的代码片段中，我们只是展示如何把所有结构映射（嵌入与投影）作为线性映射得到，以及如何使用泛性质来构造进入积和离开和的线性映射；如果你不熟悉范畴论中和与积的区别，可以忽略泛性质这一术语，只关注下面例子的类型。

```

section binary_product

variable {W : Type*} [AddCommGroup W] [Module K W]
variable {U : Type*} [AddCommGroup U] [Module K U]
variable {T : Type*} [AddCommGroup T] [Module K T]

-- First projection map
example : V × W →[K] V := LinearMap.fst K V W

-- Second projection map
example : V × W →[K] W := LinearMap.snd K V W

-- Universal property of the product
example (ϕ : U →[K] V) (ψ : U →[K] W) : U →[K] V × W := LinearMap.prod ϕ ψ

-- The product map does the expected thing, first component
example (ϕ : U →[K] V) (ψ : U →[K] W) : LinearMap.fst K V W ∘[K] LinearMap.prod ϕ ψ = ϕ := rfl

-- The product map does the expected thing, second component
example (ϕ : U →[K] V) (ψ : U →[K] W) : LinearMap.snd K V W ∘[K] LinearMap.prod ϕ ψ = ψ := rfl

-- We can also combine maps in parallel
example (ϕ : V →[K] U) (ψ : W →[K] T) : (V × W) →[K] (U × T) := ϕ.prodMap ψ

-- This is simply done by combining the projections with the universal property
example (ϕ : V →[K] U) (ψ : W →[K] T) :
  ϕ.prodMap ψ = (ϕ ∘[K] .fst K V W).prod (ψ ∘[K] .snd K V W) := rfl

-- First inclusion map
example : V →[K] V × W := LinearMap.inl K V W

-- Second inclusion map
example : W →[K] V × W := LinearMap.inr K V W

-- Universal property of the sum (aka coproduct)
example (ϕ : V →[K] U) (ψ : W →[K] U) : V × W →[K] U := ϕ.coproduct ψ

-- The coproduct map does the expected thing, first component
example (ϕ : V →[K] U) (ψ : W →[K] U) : ϕ.coproduct ψ ∘[K] LinearMap.inl K V W = ϕ :=
  LinearMap.coproduct_inl ϕ ψ

-- The coproduct map does the expected thing, second component
example (ϕ : V →[K] U) (ψ : W →[K] U) : ϕ.coproduct ψ ∘[K] LinearMap.inr K V W = ψ :=
  LinearMap.coproduct_inr ϕ ψ

-- The coproduct map is defined in the expected way
example (ϕ : V →[K] U) (ψ : W →[K] U) (v : V) (w : W) :
  ϕ.coproduct ψ (v, w) = ϕ v + ψ w :=
  rfl

end binary_product

```

现在转向任意族向量空间的和与积。这里我们只说明如何定义一个向量空间族，以及如何使用和与积的泛性质。注意，直和记号限定在 `DirectSum` 命名空间中，并且直和的泛性质要求指标类型具有可判定相等性（这多少是实现层面的偶然现象）。

```

section families
open DirectSum

variable {ι : Type*} [DecidableEq ι]
      (V : ι → Type*) [V i, AddCommGroup (V i)] [V i, Module K (V i)]

-- The universal property of the direct sum assembles maps from the summands to build
-- a map from the direct sum
example (ϕ : Π i, (V i →ι[K] W)) : (⊕ i, V i) →ι[K] W :=
  DirectSum.toModule K ι W ϕ

-- The universal property of the direct product assembles maps into the factors
-- to build a map into the direct product
example (ϕ : Π i, (W →ι[K] V i)) : W →ι[K] (Π i, V i) :=
  LinearMap.pi ϕ

-- The projection maps from the product
example (i : ι) : (Π j, V j) →ι[K] V i := LinearMap.proj i

-- The inclusion maps into the sum
example (i : ι) : V i →ι[K] (⊕ i, V i) := DirectSum.lof K ι V i

-- The inclusion maps into the product
example (i : ι) : V i →ι[K] (Π i, V i) := LinearMap.single K V i

-- In case `ι` is a finite type, there is an isomorphism between the sum and product.
example [Fintype ι] : (⊕ i, V i) ≈ι[K] (Π i, V i) :=
  linearEquivFunOnFintype K ι V

end families

```

10.2. 子空间与商空间

10.2.1. 子空间

正如线性映射是打包的，`V` 的线性子空间也是一个打包结构，由 `V` 中的一个集合（称为该子空间的载体）以及相关闭包性质组成。这里同样出现“模”而不是“向量空间”一词，是因为 Mathlib 的线性代数实际工作在更一般的语境中。

```

section
variable {K : Type*} [Field K] {V : Type*} [AddCommGroup V] [Module K V]

example (U : Submodule K V) {x y : V} (hx : x ∈ U) (hy : y ∈ U) :
  x + y ∈ U :=
  U.add_mem hx hy

example (U : Submodule K V) {x : V} (hx : x ∈ U) (a : K) :
  a • x ∈ U :=
  U.smul_mem a hx

end

```

在上面的例子中，必须理解 `Submodule K V` 是 `V` 的 `K`-线性子空间的类型，而不是谓词 `IsSubmodule U`，其中 `U` 是 `Set V` 的元素。`Submodule K V` 带有到 `Set V` 的强制转换，并且在 `V` 上有成员谓词。关于这样做的方式与原因，参见 [Section 8.3](#)。

当然，两个子空间相同当且仅当它们具有相同元素。这个事实已注册给 `ext` 策略使用；因此可以像证明两个集合相等一样，用该策略证明两个子空间相等。

例如，要陈述并证明 `ℝ` 是 `ℂ` 的一个 `ℝ`-线性子空间，我们真正想构造的是一个类型为 `Submodule ℝ ℂ` 的项，使其到 `Set ℂ` 的投影为 `ℝ`；更准确地说，是 `ℝ` 在 `ℂ` 中的像。

```

noncomputable example : Submodule ℝ ℂ where
  carrier := Set.range ((↑) : ℝ → ℂ)
  add_mem' := by
    rintro _ _ (n, rfl) (m, rfl)
    use n + m
    simp
  zero_mem' := by
    use 0
    simp
  smul_mem' := by
    rintro c - (a, rfl)
    use c*a
    simp

```

`Submodule` 中证明字段末尾的撇号与 `LinearMap` 中的情形类似。这些字段用 `carrier` 字段来表述，因为它们定义时 `Membership` 实例尚未定义。随后，我们上面见过的 `Submodule.add_mem`、`Submodule.zero_mem` 和 `Submodule.smul_mem` 会取代这些字段。

作为一个操作子空间和线性映射的练习，你将定义线性映射下子空间的原像（当然，下面会看到 Mathlib 已经知道这一构造）。请记住，`Set.mem_preimage` 可以用来改写涉及成员关系和原像的命题。除了上面讨论过的关于 `LinearMap` 与 `Submodule` 的引理之外，这是唯一需要的引理。

```

variable {K : Type*} [Field K] {V : Type*} [AddCommGroup V] [Module K V]

def preimage {W : Type*} [AddCommGroup W] [Module K W] (ϕ : V →[K] W) (H : Submodule K W) :
  Submodule K V where
  carrier := ϕ ⁻¹ H
  zero_mem' := by
    sorry
  add_mem' := by
    sorry
  smul_mem' := by
    sorry

```

借助类型类，Mathlib 知道向量空间的子空间继承向量空间结构。

```

example (U : Submodule K V) : Module K U := inferInstance

```

这个例子有些微妙。对象 `U` 并不是一个类型，但 Lean 会自动把它强制转换为类型，方式是把它解释为 `V` 的子类型。因此，上面的例子可以更显式地重述为：

```

example (U : Submodule K V) : Module K {x : V // x ∈ U} := inferInstance

```

10.2.2. 完备格结构与内直和

使用类型 `Submodule K V` 而不是谓词 `IsSubmodule : Set V → Prop` 的一个重要好处，是可以很容易地给 `Submodule K V` 赋予额外结构。尤其重要的是，按包含关系它具有完备格结构。例如，我们不会使用一个断言 `V` 的两个子空间的交仍为子空间的引理，而是使用格运算 `⊓` 来构造交。随后便可以对这一构造应用任意关于格的引理。

让我们检查两个子空间的下确界的底层集合确实按定义就是它们的交。

```

example (H H' : Submodule K V) :
  ((H ⊓ H' : Submodule K V) : Set V) = (H : Set V) ⊓ (H' : Set V) := rfl

```

对于底层集合的交使用一个不同记号，看起来也许有些奇怪；但这种对应并不能延伸到上确界运算与集合并，因为子空间的并通常不是子空间。相反，需要使用由并集生成的子空间，这由 `Submodule.span` 完成。

```

example (H H' : Submodule K V) :
  ((H ⊔ H' : Submodule K V) : Set V) = Submodule.span K ((H : Set V) ⊔ (H' : Set V)) := by
  simp [Submodule.span_union]

```

另一个微妙点是， V 本身并不具有类型 `Submodule K V`，所以需要一种方式来谈论把 V 视为 V 的子空间。这同样由格结构提供：全子空间就是这个格的顶元。

```
example (x : V) : x ∈ (⊤ : Submodule K V) := trivial
```

类似地，这个格的底元是唯一元素为零元素的子空间。

```
example (x : V) : x ∈ (⊥ : Submodule K V) ↔ x = 0 := Submodule.mem_bot K
```

特别地，我们可以讨论若干子空间构成（内部）直和的情形。对于两个子空间，使用通用谓词 `IsCompl`；它适用于任何有界偏序类型。对于一般的子空间族，则使用 `DirectSum.IsInternal`。

```
-- If two subspaces are in direct sum then they span the whole space.
example (U V : Submodule K V) (h : IsCompl U V) :
  U ⊔ V = ⊤ := h.sup_eq_top

-- If two subspaces are in direct sum then they intersect only at zero.
example (U V : Submodule K V) (h : IsCompl U V) :
  U ⊓ V = ⊥ := h.inf_eq_bot

section
open DirectSum
variable {ι : Type*} [DecidableEq ι]

-- If subspaces are in direct sum then they span the whole space.
example (U : ι → Submodule K V) (h : DirectSum.IsInternal U) :
  ⊔ i, U i = ⊤ := h.submodule_iSup_eq_top

-- If subspaces are in direct sum then they pairwise intersect only at zero.
example {ι : Type*} [DecidableEq ι] (U : ι → Submodule K V) (h : DirectSum.IsInternal U)
  {i j : ι} (hij : i ≠ j) : U i ⊓ U j = ⊥ :=
  (h.submodule_iSupIndep.pairwiseDisjoint hij).eq_bot

-- Those conditions characterize direct sums.
#check DirectSum.isInternal_submodule_iff_iSupIndep_and_iSup_eq_top

-- The relation with external direct sums: if a family of subspaces is
-- in internal direct sum then the map from their external direct sum into `V`
-- is a linear isomorphism.
noncomputable example {ι : Type*} [DecidableEq ι] (U : ι → Submodule K V)
  (h : DirectSum.IsInternal U) : (⊕ i, U i) ≈[K] V :=
  LinearEquiv.ofBijective (coeLinearMap U) h
end
```

10.2.3. 由集合张成的子空间

除了从已有子空间构造子空间之外，还可以从任意集合 S 出发，使用 `Submodule.span K S` 构造包含 S 的最小子空间。纸笔数学中常说这个空间由 S 中元素的所有线性组合组成。但在形式化中，更高效的做法往往是由

`Submodule.span_le` 表达的泛性质，以及伽罗瓦连接的整个理论。

```
example {s : Set V} (E : Submodule K V) : Submodule.span K s ≤ E ↔ s ⊆ E :=
  Submodule.span_le

example : GaloisInsertion (Submodule.span K) ((↑) : Submodule K V → Set V) :=
  Submodule.gi K V
```

当这些还不够时，可以使用相应的归纳原理 `Submodule.span_induction`。它保证：只要某个性质对零、对 S 中元素成立，并且在加法和标量乘法下稳定，那么它就对 S 的张成子空间中的每个元素成立。

作为练习，我们重新证明 `Submodule.mem_sup` 的一个方向。请记住，可以使用 `module` 策略关闭那些由 V 上各代数运算的公理推出的目标。

```

example {S T : Submodule K V} {x : V} (h : x ∈ S ∪ T) :
  ∃ s ∈ S, ∃ t ∈ T, x = s + t := by
  rw [← S.span_eq, ← T.span_eq, ← Submodule.span_union] at h
  induction h using Submodule.span_induction with
  | mem y h =>
    sorry
  | zero =>
    sorry
  | add x y hx hy hx' hy' =>
    sorry
  | smul a x hx hx' =>
    sorry

```

10.2.4. 子空间的推出与拉回

如前所述，现在说明如何用线性映射推出和拉回子空间。照 Mathlib 的惯例，第一个操作称为 `map`，第二个操作称为 `comap`。

```

section

variable {W : Type*} [AddCommGroup W] [Module K W] (ϕ : V →[K] W)

variable (E : Submodule K V) in
#check (Submodule.map ϕ E : Submodule K W)

variable (F : Submodule K W) in
#check (Submodule.comap ϕ F : Submodule K V)

```

注意，这些操作位于 `Submodule` 命名空间中，因此可以使用点记号，把 `E.map ϕ` 写成 `Submodule.map ϕ E` 的形式；不过这样读起来相当别扭（尽管一些 Mathlib 贡献者确实采用这种写法）。

特别地，线性映射的值域和核都是子空间。这些特殊情形足够重要，因此有专门的声明。

```

example : LinearMap.range ϕ = .map ϕ T := LinearMap.range_eq_map ϕ

example : LinearMap.ker ϕ = .comap ϕ ⊥ := Submodule.comap_bot ϕ -- or `rfl`

```

注意，不能把 `LinearMap.ker ϕ` 写成 `ϕ.ker`，因为 `LinearMap.ker` 还适用于保持更多结构的映射类，所以它并不期待一个类型以 `LinearMap` 开始的参数，点记号在这里无法工作。然而，在等式右端我们可以使用另一种点记号。由于在 elaboration 左端之后，Lean 期待一个类型为 `Submodule K V` 的项，它会把 `.comap` 解释为 `Submodule.comap`。

下面的引理给出这些子模与 ϕ 的性质之间的关键关系。

```

open Function LinearMap

example : Injective ϕ ↔ ker ϕ = ⊥ := ker_eq_bot.symm

example : Surjective ϕ ↔ range ϕ = T := range_eq_top.symm

```

作为练习，我们证明 `map` 与 `comap` 的伽罗瓦连接性质。可以使用下面的引理，但这并非必要，因为这些结论按定义即成立。

```

#check Submodule.mem_map_of_mem
#check Submodule.mem_map
#check Submodule.mem_comap

example (E : Submodule K V) (F : Submodule K W) :
  Submodule.map ϕ E ≤ F ↔ E ≤ Submodule.comap ϕ F := by
  sorry

```

10.2.5. 商空间

商向量空间使用一般的商记号（输入 `\quot`，而不是通常的 `/`）。到商空间的投影是 `Submodule.mkQ`，其泛性质是 `Submodule.liftQ`。

```
variable (E : Submodule K V)

example : Module K (V / E) := inferInstance

example : V →[K] V / E := E.mkQ

example : ker E.mkQ = E := E.ker_mkQ

example : range E.mkQ = E := E.range_mkQ

example (hφ : E ≤ ker φ) : V / E →[K] W := E.liftQ φ hφ

example (F : Submodule K W) (hφ : E ≤ .comap φ F) : V / E →[K] W / F := E.mapQ F φ hφ

noncomputable example : (V / LinearMap.ker φ) ≃[K] range φ := φ.quotKerEquivRange
```

作为练习，我们证明商空间子空间的对应定理。Mathlib 中有一个稍微更精确的版本，称为 `Submodule.comapMkQRelIso`。

```
open Submodule

#check Submodule.map_comap_eq
#check Submodule.comap_map_eq

example : Submodule K (V / E) ≃ { F : Submodule K V // E ≤ F } where
  toFun := sorry
  invFun := sorry
  left_inv := sorry
  right_inv := sorry
```

10.3. 自同态

线性映射的一个重要特例是自同态：从一个向量空间到自身的线性映射。它们很有趣，因为它们形成一个 K -代数。特别地，可以把 K 系数多项式代入其中，也可以讨论它们的特征值和特征向量。

Mathlib 使用缩写 `Module.End K V := V →[K] V`；当大量使用这类对象时，这很方便（尤其是在打开 `Module` 命名空间之后）。

```
variable {K : Type*} [Field K] {V : Type*} [AddCommGroup V] [Module K V]

variable {W : Type*} [AddCommGroup W] [Module K W]

open Polynomial Module LinearMap End

example (φ ψ : End K V) : φ * ψ = φ ∘ ψ :=
  End.mul_eq_comp φ ψ -- `rfl` would also work

-- evaluating `P` on `φ`
example (P : K[X]) (φ : End K V) : V →[K] V :=
  aeval φ P

-- evaluating `X` on `φ` gives back `φ`
example (φ : End K V) : aeval φ (X : K[X]) = φ :=
  aeval_X φ
```

作为一个操作自同态、子空间和多项式的练习，我们证明（二元）核引理：对任意自同态 φ 以及任意两个互素多项式 P 和 Q ，有 $\ker P(\varphi) \oplus \ker Q(\varphi) = \ker (PQ(\varphi))$ 。

注意，`IsCoprime x y` 被定义为 $\exists a b, a * x + b * y = 1$ 。


```

#check Submodule.eq_bot_iff
#check Submodule.mem_inf
#check LinearMap.mem_ker

example (P Q : K[X]) (h : IsCoprime P Q) (φ : End K V) : ker (aeval φ P) ⊓ ker (aeval φ Q) = ⊥ := by
  sorry

#check Submodule.add_mem_sup
#check map_mul
#check End.mul_apply
#check LinearMap.ker_le_ker_comp

example (P Q : K[X]) (h : IsCoprime P Q) (φ : End K V) :
  ker (aeval φ P) ⊔ ker (aeval φ Q) = ker (aeval φ (P*Q)) := by
  sorry

```

现在转向特征空间与特征值的讨论。与自同态 φ 和标量 a 对应的特征空间，是 $\varphi - aId$ 的核。特征空间对所有 a 都有定义，尽管只有非零时才有意义。不过，特征向量按定义是特征空间中的非零元素。相应的谓词是 `End.HasEigenvector`。

```

example (φ : End K V) (a : K) : φ.eigenspace a = LinearMap.ker (φ - a • 1) :=
  End.eigenspace_def

```

此外还有谓词 `End.HasEigenvalue`，以及对应的子类型 `End.Eigenvalues`。

```

example (φ : End K V) (a : K) : φ.HasEigenvalue a ↔ φ.eigenspace a ≠ ⊥ :=
  Iff.rfl

example (φ : End K V) (a : K) : φ.HasEigenvalue a ↔ ∃ v, φ.HasEigenvector a v :=
  (End.HasEigenvalue.exists_hasEigenvector, fun ⟨_, hv⟩ ↦ φ.hasEigenvalue_of_hasEigenvector hv)

example (φ : End K V) : φ.Eigenvalues = {a // φ.HasEigenvalue a} :=
  rfl

-- Eigenvalue are roots of the minimal polynomial
example (φ : End K V) (a : K) : φ.HasEigenvalue a → (minpoly K φ).IsRoot a :=
  φ.isRoot_of_hasEigenvalue

-- In finite dimension, the converse is also true (we will discuss dimension below)
example [FiniteDimensional K V] (φ : End K V) (a : K) :
  φ.HasEigenvalue a ↔ (minpoly K φ).IsRoot a :=
  φ.hasEigenvalue_iff_isRoot

-- Cayley-Hamilton
example [FiniteDimensional K V] (φ : End K V) : aeval φ φ.charpoly = 0 :=
  φ.aeval_self_charpoly

```

10.4. 矩阵、基与维数

10.4.1. 矩阵

在介绍抽象向量空间的基之前，我们回到更初等的设置：域 K 上的 K^n 中的线性代数。这里的主要对象是向量和矩阵。对具体向量，可以使用 `![...]` 记号，其中分量用逗号分隔。对具体矩阵，可以使用 `!![...]` 记号，行之间用分号分隔，每行中的元素用逗号分隔。当项具有可计算类型，例如 $\mathbb{N}$ 或 $\mathbb{Q}$ 时，可以使用 `eval` 命令尝试基本运算。

```

section matrices

-- Adding vectors
#eval ![1, 2] + ![3, 4] -- ![4, 6]

-- Adding matrices
#eval !![1, 2; 3, 4] + !![3, 4; 5, 6] -- !![4, 6; 8, 10]

-- Multiplying matrices
#eval !![1, 2; 3, 4] * !![3, 4; 5, 6] -- !![13, 16; 29, 36]

```

必须理解，这里使用 `#eval` 只对探索有意义，并不意在取代 Sage 之类的计算机代数系统。这里用于矩阵的数据表示在计算上完全不高效。它使用函数而不是数组，并针对证明而非计算优化。`#eval` 所使用的虚拟机也并未为这种用途优化。

注意，矩阵记号列出的是行，但向量记号既不是行向量也不是列向量。矩阵从左侧（相应地，从右侧）与向量相乘时，会把该向量解释为行向量（相应地，列向量）。这对应于操作 `Matrix.vecMul`（记号为 $\mathbf{v}^*$ ）和 `Matrix.mulVec`（记号为 $\mathbf{v}$ ）。这些记号限定在 `Matrix` 命名空间中，因此我们需要打开该命名空间。

```
open Matrix

-- matrices acting on vectors on the left
#eval ![1, 2; 3, 4] *v ![1, 1] -- ![3, 7]

-- matrices acting on vectors on the left, resulting in a size one matrix
#eval ![1, 2] *v ![1, 1] -- ![3]

-- matrices acting on vectors on the right
#eval ![1, 1, 1] *v ![1, 2; 3, 4; 5, 6] -- ![9, 12]
```

为了生成由某个向量指定、且各行或各列相同的矩阵，可以使用 `Matrix.replicateRow` 和 `Matrix.replicateCol`；其参数为行或列的指标类型以及该向量。例如，可以得到单行或单列矩阵（更准确地说，是行或列由 `Fin 1` 作指标的矩阵）。

```
#eval replicateRow (Fin 1) ![1, 2] -- ![1, 2]
#eval replicateCol (Fin 1) ![1, 2] -- ![1; 2]
```

其他熟悉的运算包括向量点积、矩阵转置，以及方阵的行列式和迹。

```
-- vector dot product
#eval ![1, 2] ·v ![3, 4] -- `11`

-- matrix transpose
#eval ![1, 2; 3, 4]T -- `![1, 3; 2, 4]`

-- determinant
#eval ![(1 : ℤ), 2; 3, 4].det -- `-2`

-- trace
#eval ![(1 : ℤ), 2; 3, 4].trace -- `5`
```

当项没有可计算类型时，例如它们是实数，我们不能指望 `#eval` 提供帮助。此外，这类求值若要用于证明，会显著扩大可信代码基，即检查证明时必须信任的那部分 Lean。

因此，在证明中也应使用 `simp` 和 `norm_num` 策略；若只是快速探索，则可使用它们对应的命令形式。

```
#simp ![(1 : ℝ), 2; 3, 4].det -- `4 - 2*3`
#norm_num ![(1 : ℝ), 2; 3, 4].det -- `-2`
#norm_num ![(1 : ℝ), 2; 3, 4].trace -- `5`

variable (a b c d : ℝ) in
#simp ![a, b; c, d].det -- `a * d - b * c`
```

方阵的下一个重要运算是求逆。正如数的除法总是有定义，并且在除以零时返回人为指定的值零一样，矩阵求逆也对所有矩阵有定义，并在矩阵不可逆时返回零矩阵。

更精确地说，有一个一般函数 `Ring.inverse`，它在任意环中完成这一点；对任意矩阵 A ， A^{-1} 被定义为 `Ring.inverse A.det • A.adjugate`。根据克拉默法则，当 A 的行列式不为零时，这确实是 A 的逆。

```
#norm_num [Matrix.inv_def] ![(1 : ℝ), 2; 3, 4]-1 -- ![-2, 1; 3 / 2, -(1 / 2)]
```

当然，这个定义真正有用的情形是可逆矩阵。有一个一般类型类 `Invertible` 可用于记录这一事实。例如，下一个例子中的 `simp` 调用会使用引理 `inv_mul_of_invertible`，而该引理有一个 `Invertible` 类型类假设；因此只有当类型类合成系统能找到它时，该引理才会触发。这里我们用一个 `have` 语句使这一事实可用。

```
example : !![(1 : ℝ), 2; 3, 4]⁻¹ * !![(1 : ℝ), 2; 3, 4] = 1 := by
  have : Invertible !![(1 : ℝ), 2; 3, 4] := by
    apply Matrix.invertibleOfIsUnitDet
    norm_num
  simp
```

在这个完全具体的情形中，也可以使用 `norm_num` 机制，并用 `apply?` 找到最后一行：

```
example : !![(1 : ℝ), 2; 3, 4]⁻¹ * !![(1 : ℝ), 2; 3, 4] = 1 := by
  norm_num [Matrix.inv_def]
  exact one_fin_two.symm
```

上面所有具体矩阵的行和列都由某个 `n` 的 `Fin n` 作指标（行与列的大小不必相同）。但有时用任意有限类型给矩阵作指标更方便。例如，有限图的邻接矩阵，其行和列自然由图的顶点作指标。

事实上，如果只是想定义矩阵而不定义其上的任何运算，甚至不需要指标类型有限，系数也可以是任意类型，不必有任何代数结构。因此，Mathlib 简单地把 `Matrix m n α` 定义为 `m → n → α`，其中 `m`、`n` 和 `α` 是任意类型；到目前为止我们使用的矩阵具有诸如 `Matrix (Fin 2) (Fin 2) ℝ` 这样的类型。当然，代数运算需要对 `m`、`n` 和 `α` 作更多假设。

注意，我们不直接使用 `m → n → α` 的主要原因，是为了让类型类系统理解我们的意图。例如，对一个环 `R`，类型 `n → R` 带有逐点乘法；类似地，`m → n → R` 也带有这种运算，而这不是我们希望用于矩阵的乘法。

在下面第一个例子中，我们强制 Lean 展开 `Matrix` 的定义，使其接受该命题有意义，然后通过检查所有项来证明它。

但接下来的两个例子显示，Lean 对 `Fin 2 → Fin 2 → ℤ` 使用逐点乘法，而对 `Matrix (Fin 2) (Fin 2) ℤ` 使用矩阵乘法。

```
section

example : (fun _ ↦ 1 : Fin 2 → Fin 2 → ℤ) = !![1, 1; 1, 1] := by
  ext i j
  fin_cases i <> fin_cases j <> rfl

example : (fun _ ↦ 1 : Fin 2 → Fin 2 → ℤ) * (fun _ ↦ 1 : Fin 2 → Fin 2 → ℤ) = !![1, 1; 1, 1] := by
  ext i j
  fin_cases i <> fin_cases j <> rfl

example : !![1, 1; 1, 1] * !![1, 1; 1, 1] = !![2, 2; 2, 2] := by
  norm_num
```

为了把矩阵定义为函数，同时又不丢失 `Matrix` 对类型类合成带来的好处，可以使用函数与矩阵之间的等价 `Matrix.of`。这个等价在背后是用 `Equiv.refl` 定义的。

例如，可以定义与向量 `v` 对应的范德蒙德矩阵。

```
example {n : ℕ} (v : Fin n → ℝ) :
  Matrix.vandermonde v = Matrix.of (fun i j : Fin n ↦ v i ^ (j : ℕ)) :=
  rfl
end
end matrices
```

10.4.2. 基

现在讨论向量空间的基。非形式化地说，这一概念有多种定义方式。可以使用泛性质；可以说基是一族线性无关且张成全空间的向量；也可以把这些性质合并，直接说基是一族向量，使得每个向量都能唯一地写成基向量的线性组合。还有一种说法是，基给出了与基域 `K` 的幂的线性同构，其中 `K` 被视为自身上的向量空间。

这个同构版本实际上是 Mathlib 底层采用的定义，其他刻画则由此证明出来。对于 “ K 的幂” 这一想法，在无限基的情形中必须稍加谨慎。事实上，在这个代数语境中，只有有限线性组合有意义。因此，我们需要的参照向量空间不是若干份 K 的直积，而是直和。若某个类型 ι 为基作指标，可以使用 $\oplus i : \iota, K$ ；但实际采用的是更专门的写法 $\iota \rightarrow_0 K$ ，意思是 “从 ι 到 K 的有限支撑函数”，即在 ι 的某个有限集之外取零的函数（这个有限集不是固定的，而依赖于具体函数）。若这样的函数来自基 B ，在向量 v 和 $i : \iota$ 处求值，会返回 v 在第 i 个基向量上的分量（或坐标）。

以类型 ι 为指标的 V 作为 K -向量空间的基的类型是 `Basis  $\iota$  K V`。相应的同构称为 `Basis.repr`。

```
variable {K : Type*} [Field K] {V : Type*} [AddCommGroup V] [Module K V]

section

open Module

variable { $\iota$  : Type*} (B : Basis  $\iota$  K V) (v : V) (i :  $\iota$ )

-- The basis vector with index ``i``
#check (B i : V)

-- the linear isomorphism with the model space given by ``B``
#check (B.repr : V  $\simeq$  [ $K$ ]  $\iota \rightarrow_0 K$ )

-- the component function of ``v``
#check (B.repr v :  $\iota \rightarrow_0 K$ )

-- the component of ``v`` with index ``i``
#check (B.repr v i : K)
```

不从这种同构出发，也可以从一族线性无关且张成全空间的向量 b 出发；这就是 `Basis.mk`。

这族向量张成全空间的假设写作 `T  $\leq$  Submodule.span K (Set.range b)`。这里 T 是 V 的子模，也就是把 V 视为自身的子模。这种写法看起来有些曲折，但下面会看到，它几乎按定义等价于更易读的 `$\forall v, v \in \text{Submodule.span } K (\text{Set.range } b)$` （下面代码片段中的下划线对应无用的信息 `v  $\in T$` ）。

```
noncomputable example (b :  $\iota \rightarrow V$ ) (b_indep : LinearIndependent K b)
  (b_spans :  $\forall v, v \in \text{Submodule.span } K (\text{Set.range } b)$ ) : Basis  $\iota$  K V :=
  Basis.mk b_indep (fun v _  $\mapsto$  b_spans v)

-- The family of vectors underlying the above basis is indeed ``b``.
example (b :  $\iota \rightarrow V$ ) (b_indep : LinearIndependent K b)
  (b_spans :  $\forall v, v \in \text{Submodule.span } K (\text{Set.range } b)$ ) (i :  $\iota$ ) :
  Basis.mk b_indep (fun v _  $\mapsto$  b_spans v) i = b i :=
  Basis.mk_apply b_indep (fun v _  $\mapsto$  b_spans v) i
```

特别地，模型向量空间 $\iota \rightarrow_0 K$ 有一个所谓的标准基；其 `repr` 函数作用于任意向量时就是恒等同构。它称为 `Finsupp.basisSingleOne`，其中 `Finsupp` 表示有限支撑函数，而 `basisSingleOne` 指基向量是在除单个输入值以外处都为零的函数。更精确地说，由 $i : \iota$ 作指标的基向量是 `Finsupp.single i 1`，即在 i 处取值 `1`、在其他地方取值 `0` 的有限支撑函数。

```
variable [DecidableEq  $\iota$ ]

example : Finsupp.basisSingleOne.repr = LinearEquiv.refl K ( $\iota \rightarrow_0 K$ ) :=
  rfl

example (i :  $\iota$ ) : Finsupp.basisSingleOne i = Finsupp.single i 1 :=
  rfl
```

当指标类型有限时，不需要有限支撑函数的这些讨论。在这种情况下，可以使用更简单的 `Pi.basisFun`，它给出整个 $\iota \rightarrow K$ 的一组基。

```
example [Finite  $\iota$ ] (x :  $\iota \rightarrow K$ ) (i :  $\iota$ ) : (Pi.basisFun K  $\iota$ ).repr x i = x i := by
  simp
```

回到抽象向量空间基的一般情形，可以把任意向量表示为基向量的线性组合。先看有限基的简单情形。

```
example [Fintype  $\iota$ ] :  $\sum i : \iota, B.repr\ v\ i \bullet (B\ i) = v :=$ 
  B.sum_repr v
```

当 ι 不是有限类型时，上面的陈述先验上没有意义：不能在 ι 上取和。然而，被求和函数的支撑是有限的（它就是 $B.repr\ v$ 的支撑）。因此需要使用能体现这一点的构造。这里 Mathlib 使用一个需要花时间熟悉的专门函数：

`Finsupp.linearCombination`（它建立在更一般的 `Finsupp.sum` 之上）。给定一个从类型 ι 到基域 K 的有限支撑函数 c ，以及任意函数 f 从 ι 到 V ，`Finsupp.linearCombination K f c` 是在 c 的支撑上对标量乘法 $c \bullet f$ 求和。特别地，可以把它替换成在任意包含 c 支撑的有限集上求和。

```
example (c :  $\iota \rightarrow_0 K$ ) (f :  $\iota \rightarrow V$ ) (s : Finset  $\iota$ ) (h : c.support  $\subseteq$  s) :
  Finsupp.linearCombination K f c =  $\sum i \in s, c\ i \bullet f\ i :=$ 
  Finsupp.linearCombination_apply_of_mem_supported K h
```

也可以假设 f 是有限支撑的，并仍然得到一个良定义的和。但 `Finsupp.linearCombination` 所作的选择才与这里关于基的讨论相关，因为它允许陈述 `Basis.sum_repr` 的一般化版本。

```
example : Finsupp.linearCombination K B (B.repr v) = v :=
  B.linearCombination_repr v
```

也许会疑惑，为什么这里 K 是显式参数，尽管它可以从 c 的类型推断出来。关键在于，部分应用后的 `Finsupp.linearCombination K f` 本身是有意义的对象。它不是一个从 $\iota \rightarrow_0 K$ 到 V 的普通函数，而是一个 K -线性映射。

```
variable (f :  $\iota \rightarrow V$ ) in
#check (Finsupp.linearCombination K f : ( $\iota \rightarrow_0 K$ )  $\rightarrow_1[K]$  V)
```

回到数学讨论，需要理解的是，在形式化数学中，基下的向量表示并没有你可能想象的那样有用。事实上，直接使用基的更抽象性质往往高效得多。特别地，基与代数中其他自由对象相联系的泛性质允许通过指定基向量的像来构造线性映射。这就是 `Basis.constr`。对任意 K -向量空间 W ，我们的基 B 给出从 $\iota \rightarrow W$ 到 $V \rightarrow_1[K] W$ 的线性同构 `Basis.constr B K`。这个同构由如下性质刻画：它把任意函数 $u : \iota \rightarrow W$ 送到一个线性映射；该线性映射对每个 $i : \iota$ 都把基向量 $B\ i$ 送到 $u\ i$ 。

```
section

variable {W : Type*} [AddCommGroup W] [Module K W]
  (phi : V  $\rightarrow_1[K]$  W) (u :  $\iota \rightarrow W$ )

#check (B.constr K : ( $\iota \rightarrow W$ )  $\simeq_1[K]$  (V  $\rightarrow_1[K]$  W))

#check (B.constr K u : V  $\rightarrow_1[K]$  W)

example (i :  $\iota$ ) : B.constr K u (B i) = u i :=
  B.constr_basis K u i
```

这个性质确实是刻画性的，因为线性映射由其在基上的取值决定：

```
example (phi psi : V  $\rightarrow_1[K]$  W) (h :  $\forall i, \phi (B i) = \psi (B i)$ ) : phi = psi :=
  B.ext h
```

如果目标空间上也有一组基 B' ，那么可以把线性映射与矩阵等同起来。这个等同是一个 K -线性同构。

```

variable {ι' : Type*} (B' : Basis ι' K W) [Fintype ι'] [DecidableEq ι'] [Fintype ι'] [DecidableEq ι']

open LinearMap

#check (toMatrix B B' : (V →ι[K] W) ≈ι[K] Matrix ι' ι K)

open Matrix -- get access to the ``*_v`` notation for multiplication between matrices and vectors.

example (ϕ : V →ι[K] W) (v : V) : (toMatrix B B' ϕ) *_v (B.repr v) = B'.repr (ϕ v) :=
  toMatrix_mulVec_repr B B' ϕ v

variable {ι'' : Type*} (B'' : Basis ι'' K W) [Fintype ι''] [DecidableEq ι'']

example (ϕ : V →ι[K] W) : (toMatrix B B'' ϕ) = (toMatrix B' B'' .id) * (toMatrix B B' ϕ) := by
  simp
end

```

作为关于这一主题的练习，我们将证明保证自同态具有良定义行列式的定理的一部分。具体而言，我们要证明：当两组基由同一类型作指标时，它们附加到任意自同态上的矩阵具有相同的行列式。为了得到完整结果，还需要补充使用所有基的指标类型彼此同构这一事实。

当然，Mathlib 已经知道这一点，并且 `simp` 可以立即关闭目标；因此不要过早使用它，而应使用给定的引理。

```

open Module LinearMap Matrix

-- Some Lemmas coming from the fact that `LinearMap.toMatrix` is an algebra morphism.
#check toMatrix_comp
#check id_comp
#check comp_id
#check toMatrix_id

-- Some Lemmas coming from the fact that `Matrix.det` is a multiplicative monoid morphism.
#check Matrix.det_mul
#check Matrix.det_one

example [Fintype ι] (B' : Basis ι K V) (ϕ : End K V) :
  (toMatrix B B ϕ).det = (toMatrix B' B' ϕ).det := by
  set M := toMatrix B B ϕ
  set M' := toMatrix B' B' ϕ
  set P := (toMatrix B B') LinearMap.id
  set P' := (toMatrix B' B) LinearMap.id
  sorry
end

```

10.4.3. 维数

回到单个向量空间的情形，基也可用于定义维数概念。这里同样有有限维向量空间的初等情形。对这类空间，我们期望维数是一个自然数。这就是 `Module.finnrank`。它把基域作为显式参数，因为同一个交换群可以成为不同域上的向量空间。

```

section

#check (Module.finnrank K V : ℕ)

-- `Fin n → K` is the archetypical space with dimension `n` over `K`.
example (n : ℕ) : Module.finnrank K (Fin n → K) = n :=
  Module.finnrank_fin_fun K

-- Seen as a vector space over itself, `ℂ` has dimension one.
example : Module.finnrank ℂ ℂ = 1 :=
  Module.finnrank_self ℂ

-- But as a real vector space it has dimension two.
example : Module.finnrank ℝ ℂ = 2 :=
  Complex.finnrank_real_complex

```

注意，`Module.finnrank` 对任意向量空间都有定义。对无限维向量空间，它返回零，正如除以零返回零一样。

当然，许多引理需要有限维假设。这正是类型类 `FiniteDimensional` 的作用。例如，请思考如果没有这个假设，下一个例子会如何失败。

```
example [FiniteDimensional K V] : 0 < Module.finrank K V ↔ Nontrivial V :=
Module.finrank_pos_iff
```

在上面的陈述中，`Nontrivial V` 表示 `V` 至少有两个不同元素。注意，`Module.finrank_pos_iff` 没有显式参数。从左到右使用它时这没有问题；但从右到左使用时就不行，因为 Lean 无法从命题 `Nontrivial V` 猜出 `K`。此时，在检查该引理是关于一个名为 `R` 的环陈述之后，使用命名参数语法很有帮助。因此可以写作：

```
example [FiniteDimensional K V] (h : 0 < Module.finrank K V) : Nontrivial V := by
  apply (Module.finrank_pos_iff (R := K)).1
  exact h
```

上面的写法有些奇怪，因为我们已经有假设 `h`，所以完全可以直接给出完整证明 `Module.finrank_pos_iff.1 h`；不过在更复杂的情形中，了解这种写法很有用。

按定义，`FiniteDimensional K V` 可以从任意一组基读出。

```
variable {ι : Type*} (B : Module.Basis ι K V)

example [Finite ι] : FiniteDimensional K V := Module.Basis.finiteDimensional_of_finite B

example [FiniteDimensional K V] : Finite ι :=
  (FiniteDimensional.fintypeBasisIndex B).finite
end
```

利用线性子空间对应的子类型具有向量空间结构这一事实，可以讨论子空间的维数。

```
section
variable (E F : Submodule K V) [FiniteDimensional K V]

open Module

example : finrank K (E ⊔ F : Submodule K V) + finrank K (E ⊓ F : Submodule K V) =
  finrank K E + finrank K F :=
  Submodule.finrank_sup_add_finrank_inf_eq E F

example : finrank K E ≤ finrank K V := Submodule.finrank_le E
```

在上面第一个陈述中，类型标注的目的，是确保到 `Type*` 的强制转换不会过早触发。

现在可以做一个关于 `finrank` 与子空间的练习。

```
example (h : finrank K V < finrank K E + finrank K F) :
  Nontrivial (E ⊓ F : Submodule K V) := by
  sorry
end
```

现在转向维数理论的一般情形。在这种情况下，`finrank` 没有用；但我们仍然知道，同一向量空间的任意两组基，其指标类型之间存在一个双射。因此仍可希望把秩定义为一个基数，即“所有类型按存在双射这一等价关系取商”所得对象的一个元素。

讨论基数时，就更难像本书其他地方那样忽略与罗素悖论有关的基础问题。不存在所有类型组成的类型，因为那会导致逻辑不一致。这个问题由宇宙层级解决，而我们通常尽量不去显式处理它。

每个类型都有一个宇宙层级，这些层级的行为类似自然数。特别地，有第零层级，相应的宇宙 `Type 0` 简记为 `Type`。这个宇宙足以容纳几乎所有经典数学。例如， $\mathbb{N}$ 和 $\mathbb{R}$ 的类型都是 `Type`。每个层级 `u` 都有一个后继，记作 `u + 1`；并且 `Type u` 的类型是 `Type (u+1)`。

但宇宙层级并不是自然数，它们具有完全不同的性质，并且自身没有类型。特别地，不能在 Lean 中陈述诸如 `u ≠ u + 1` 这样的命题。根本不存在一个使这句话可安放其中的类型。即使陈述 `Type u ≠ Type (u+1)` 也没有意义，因为 `Type u` 与 `Type (u+1)` 具有不同的类型。

每当写 `Type*` 时，Lean 都会插入一个名为 `u_n` 的宇宙层级变量，其中 `n` 是一个数字。这使定义和陈述能够生活在所有宇宙中。

给定宇宙层级 `u`，可以在 `Type u` 上定义一个等价关系：若类型 `α` 与 `β` 之间存在双射，则称它们等价。商类型 `Cardinal.{u}` 生活在 `Type (u+1)` 中。花括号表示宇宙变量。`α : Type u` 在这个商中的像是 `Cardinal.mk α : Cardinal.{u}`。

但我们不能直接比较不同宇宙中的基数。因此，从技术上说，不能把向量空间 `V` 的秩定义为所有可作为 `V` 的基指标类型的基数。于是，秩改定义为 `V` 中所有线性无关集的基数的上确界 `Module.rank K V`。若 `V` 的宇宙层级为 `u`，则其秩的类型为 `Cardinal.{u}`。

```
#check V -- Type u_2
#check Module.rank K V -- Cardinal.{u_2}
```

仍然可以把这个定义与基联系起来。事实上，宇宙层级上还有一个交换的最大值运算 `max`；给定两个宇宙层级 `u` 和 `v`，有一个操作 `Cardinal.lift.{u, v} : Cardinal.{v} → Cardinal.{max v u}`，它允许把基数放入共同的宇宙中，从而陈述维数定理。

```
universe u v -- `u` and `v` will denote universe levels

variable {ι : Type u} (B : Module.Basis ι K V)
         {ι' : Type v} (B' : Module.Basis ι' K V)

example : Cardinal.lift.{v, u} (.mk ι) = Cardinal.lift.{u, v} (.mk ι') :=
  mk_eq_mk_of_basis B B'
```

可以通过从自然数到有限基数的强制转换，把有限维情形与上述讨论联系起来（更精确地说，是到生活在 `Cardinal.{v}` 中的有限基数，其中 `v` 是 `V` 的宇宙层级）。

```
example [FiniteDimensional K V] :
  (Module.finrank K V : Cardinal) = Module.rank K V :=
  Module.finrank_eq_rank K V
```


11. 拓扑

微积分建立在函数概念之上，函数用于刻画相互依赖的量。例如，研究随时间变化的量是很常见的。极限概念同样是基础性的。我们可以说函数 $f(x)$ 在 x 趋近于 a 时的极限为 b ，也可以说当 x 趋近于 a 时， $f(x)$ 收敛到 b 。等价地，可以说当 x 趋近于 a 时， $f(x)$ 趋近于 b ；或说当 x 趋向于 a 时，它趋向 b 。我们已经在Section 3.6中开始考察这类概念。

拓扑学是对极限与连续性的抽象研究。在第 2 章至第 7 章介绍完形式化的基本内容之后，本章将解释 Mathlib 中如何形式化拓扑概念。拓扑抽象不仅适用于大得多的一般性范围，而且有些反直觉的是，它还使我们在具体实例中更容易推理极限和连续性。

拓扑概念建立在若干层数学结构之上。第一层是朴素集合论，如Chapter 4所述。下一层是滤子理论，我们将在Section 11.1中介绍。在此之上，还叠加了拓扑空间、度量空间以及一个稍显特殊的中间概念，称为一致空间。

前几章依赖的数学概念大多可能为你所熟悉；相比之下，滤子概念即使对许多职业数学家也不那么常见。然而，为了有效形式化数学，这一概念是必不可少的。下面说明原因。设 $f : \mathbb{R} \rightarrow \mathbb{R}$ 为任意函数。我们可以考虑当 x 趋近于某个值 x_0 时 $f x$ 的极限，也可以考虑当 x 趋向正无穷或负无穷时 $f x$ 的极限。此外，还可以考虑当 x 从右侧趋近于 x_0 （惯常写作 x_0^+ ）或从左侧趋近于 x_0^- 时 $f x$ 的极限。还有一些变体，其中 x 趋近于 x_0 、 x_0^+ 或 x_0^- ，但不允许取值为 x_0 本身。这样， x 趋近某物的方式至少有八种。还可以把 x 限制为有理数，或对定义域施加其他约束；但这里先只考虑这 8 种情形。

在余定义域上也有类似的多种选择：可以指定 $f x$ 从左侧或右侧趋近某个值，或者趋向正无穷或负无穷，等等。例如，可能想表达：当 x 从右侧趋近 x_0 且不等于 x_0 时， $f x$ 趋向 $+\infty$ 。这会产生 64 种不同的极限陈述，而我们甚至还没有开始处理序列极限，后者曾在Section 3.6中出现。

涉及支撑引理时，问题会进一步复杂化。例如，极限可以复合：若当 x 趋向 x_0 时 $f x$ 趋向 y_0 ，且当 y 趋向 y_0 时 $g y$ 趋向 z_0 ，则当 x 趋向 x_0 时 $g \circ f x$ 趋向 z_0 。这里有三个“趋向”概念，每一个都可以按前一段所述的八种方式实例化。于是得到 512 个引理；若都加入库中，数量相当庞大。非形式化地说，数学家通常证明其中两三个，然后只说明其余可以“同样地”证明。形式化数学要求把相关的“相同方式”完全明确化，而这正是布尔巴基的滤子理论所做到的。

11.1. 滤子

类型 α 上的滤子是 α 的若干集合组成的族，并满足下面将列出的三个条件。这个概念支撑两类相关想法：

极限，包括上文讨论的所有极限类型：序列的有限与无限极限，函数在一点或无穷远处的有限与无限极限，等等。

最终成立的性质，包括对充分大的 $n : \mathbb{N}$ 成立、在某点 x 的足够近处成立、对足够接近的点对成立，或在测度论意义下几乎处处成立。对偶地，滤子也可以表达频繁成立的性质：对任意大的 n 、在给定点任意邻域中的某一点，等等。

与这些描述对应的滤子将在本节稍后定义，但我们已经可以先给出它们的名称：

$(\text{atTop} : \text{Filter } \mathbb{N})$ ，由那些包含某个 N 对应的集合 $\{n \mid n \geq N\}$ 的 $\mathbb{N}$ 的子集组成

$\mathcal{N} x$ ，由拓扑空间中 x 的邻域组成

$\mathcal{U} x$ ，由一致空间的 entourage 组成（一致空间推广了度量空间和拓扑群）

$\mu.\text{ae}$ ，由相对于测度 μ 其补集测度为零的集合组成。

一般定义如下：滤子 $F : \text{Filter } X$ 是由若干集合组成的族 $F.\text{sets} : \text{Set } (\text{Set } X)$ ，并满足下列条件：

条件 $F.\text{univ_sets} : \text{univ} \in F.\text{sets}$

条件 $F.\text{sets_of_superset} : \forall \{U V\}, U \in F.\text{sets} \rightarrow U \subseteq V \rightarrow V \in F.\text{sets}$

条件 $F.\text{inter_sets} : \forall \{U V\}, U \in F.\text{sets} \rightarrow V \in F.\text{sets} \rightarrow U \cap V \in F.\text{sets}$ 。

第一个条件说， X 的全体元素组成的集合属于 $F.\text{sets}$ 。第二个条件说，若 U 属于 $F.\text{sets}$ ，则任何包含 U 的集合也属于 $F.\text{sets}$ 。第三个条件说， $F.\text{sets}$ 在有限交下封闭。在 Mathlib 中，滤子 F 被定义为一个打包了 $F.\text{sets}$ 及其三个性质的结构；但这些性质不携带额外数据，因此混同 F 与 $F.\text{sets}$ 很方便。于是我们把 $U \in F$ 定义为 $U \in F.\text{sets}$ 。这解释了为什么有些提及 $U \in F$ 的引理名称中会出现 `sets` 一词。

把滤子理解为定义了“充分大”集合的概念，可能会有所帮助。第一个条件于是说 `univ` 是充分大的；第二个条件说，包含充分大集合的集合仍然充分大；第三个条件说，两个充分大集合的交仍然充分大。

更有用的想法可能是，把类型 X 上的滤子视为 $\text{Set } X$ 的广义元素。例如，`atTop` 是“非常大的数的集合”，而 $\mathcal{N}_{x_0}$ 是“非常接近 x_0 的点的集合”。这种观点的一个体现是，可以把任意 $s : \text{Set } X$ 关联到所谓的主滤子，它由所有包含 s 的集合组成。这个定义已在 Mathlib 中，并有记号 $\mathcal{P}$ （局部化在 `Filter` 命名空间中）。为了演示，我们请你借此机会在这里完成这个定义。

```
def principal {α : Type*} (s : Set α) : Filter α
  where
    sets := { t | s ⊆ t }
    univ_sets := sorry
    sets_of_superset := sorry
    inter_sets := sorry
```

作为第二个例子，我们请你定义滤子 `atTop : Filter  $\mathbb{N}$` 。（也可以用任何带预序的类型替代 $\mathbb{N}$ 。）

```
example : Filter  $\mathbb{N}$  :=
  { sets := { s | ∃ a, ∀ b, a ≤ b → b ∈ s }
    univ_sets := sorry
    sets_of_superset := sorry
    inter_sets := sorry }
```

还可以直接定义任意 $x : \mathbb{R}$ 的邻域滤子 $\mathcal{N}_x$ 。在实数中， x 的一个邻域是包含开区间 $(x_0 - \varepsilon, x_0 + \varepsilon)$ 的集合；该开区间在 Mathlib 中定义为 `Ioo (x_0 - ε) (x_0 + ε)`。（这种邻域概念只是 Mathlib 中更一般构造的一个特例。）

有了这些例子，已经可以如下定义函数 $f : X \rightarrow Y$ 沿某个 $F : \text{Filter } X$ 收敛到某个 $G : \text{Filter } Y$ 的含义：

```
def Tendsto₁ {X Y : Type*} (f : X → Y) (F : Filter X) (G : Filter Y) :=
  ∀ V ∈ G, f ⁻¹ V ∈ F
```

当 X 为 $\mathbb{N}$ 且 Y 为 $\mathbb{R}$ 时，`Tendsto₁ u atTop ( $\mathcal{N}_x$ )` 等价于说序列 $u : \mathbb{N} \rightarrow \mathbb{R}$ 收敛到实数 x 。当 X 与 Y 都是 $\mathbb{R}$ 时，`Tendsto f ( $\mathcal{N}_{x_0}$ ) ( $\mathcal{N}_{y_0}$ )` 等价于熟悉的概念 $\lim_{x \rightarrow x_0} f(x) = y_0$ 。引言中提到的所有其他极限类型，也都等价于对源和目标上的滤子作适当选择后得到的 `Tendsto₁` 实例。

上面的概念 `Tendsto₁` 与 Mathlib 中定义的 `Tendsto` 按定义等价，但后者定义得更加抽象。`Tendsto₁` 的定义问题在于，它显式暴露了量词以及 G 的元素，并隐藏了把滤子看作广义集合时得到的直观。通过使用更多代数和集合论机制，可以隐藏量词 $\forall V$ ，并让直观更突出。第一个成分是与任意映射 $f : X \rightarrow Y$ 对应的 *pushforward* 操作 f_* ，在 Mathlib 中记作 `Filter.map f`。给定 X 上的滤子 F ，`Filter.map f F : Filter Y` 被定义为使 $V \in \text{Filter.map } f F \Leftrightarrow f^{-1} V \in F$ 按定义成

立。在示例文件中，我们打开了 `Filter` 命名空间，因此 `Filter.map` 可以写作 `map`。这意味着可以用 `Filter Y` 上的序关系重写 `Tendsto` 的定义；这个序关系是成员集合包含关系的反向。换言之，给定 $G H : \text{Filter } Y$ ，有

$$G \leq H \Leftrightarrow \forall V : \text{Set } Y, V \in H \Rightarrow V \in G。$$

```
def Tendsto2 {X Y : Type*} (f : X → Y) (F : Filter X) (G : Filter Y) :=
  map f F ≤ G

example {X Y : Type*} (f : X → Y) (F : Filter X) (G : Filter Y) :
  Tendsto2 f F G ⇔ Tendsto1 f F G :=
  Iff.rfl
```

滤子上的序关系看起来也许是反的。但请回忆，可以通过嵌入 $\mathcal{P} : \text{Set } X \rightarrow \text{Filter } X$ 把 X 上的滤子看成 $\text{Set } X$ 的广义元素；该嵌入把任意集合 s 送到相应的主滤子。这个嵌入保持序，因此 `Filter` 上的序关系确实可以视为广义集合之间的自然包含关系。在这个类比中，推出类似于直接像。事实上， $\text{map } f (\mathcal{P} s) = \mathcal{P} (f '' s)$ 。

现在可以直观地理解，为什么序列 $u : \mathbb{N} \rightarrow \mathbb{R}$ 收敛到点 x_0 当且仅当有 $\text{map } u \text{ atTop} \leq \mathcal{N} x_0$ 。这个不等式表示，“非常大的自然数集合”在 u 下的“直接像”被“包含”在“非常接近 x_0 的点的集合”中。

如所承诺的，`Tendsto2` 的定义不再显式出现量词或集合。它还利用了推出操作的代数性质。首先，每个 `Filter.map f` 都是单调的。其次，`Filter.map` 与复合相容。

```
#check (@Filter.map_mono : ∀ {α β} {m : α → β}, Monotone (map m))

#check
  (@Filter.map_map :
    ∀ {α β γ} {f : Filter α} {m : α → β} {m' : β → γ}, map m' (map m f) = map (m' ∘ m) f)
```

这两个性质合在一起，使我们能够证明极限的复合性；它一次性给出引言中所述复合引理的全部 512 个变体，以及更多结果。你可以练习证明下面的陈述：既可以使用以全称量词表述的 `Tendsto1` 定义，也可以使用代数定义，并结合上面的两个引理。

```
example {X Y Z : Type*} {F : Filter X} {G : Filter Y} {H : Filter Z} {f : X → Y} {g : Y → Z}
  (hf : Tendsto1 f F G) (hg : Tendsto1 g G H) : Tendsto1 (g ∘ f) F H :=
  sorry
```

推出构造用一个映射把滤子从映射的源推到目标。也有一个方向相反的拉回操作 `Filter.comap`。它推广了集合上的原像操作。对任意映射 f ，`Filter.map f` 与 `Filter.comap f` 构成所谓的伽罗瓦连接，也就是说，它们满足

即 $\text{Filter.map } f F \leq G \Leftrightarrow F \leq \text{Filter.comap } f G$

对每个 F 和 G 都成立。这个操作可以用于给出 `Tendsto` 的另一种表述；它与 `Mathlib` 中的定义可证明等价，但不按定义等价。

`comap` 操作可以用来把滤子限制到子类型上。例如，设有 $f : \mathbb{R} \rightarrow \mathbb{R}$ 、 $x_0 : \mathbb{R}$ 和 $y_0 : \mathbb{R}$ ，并且想陈述当 x 在有理数内趋近于 x_0 时， $f x$ 趋近于 y_0 。可以使用强制映射 $(\uparrow) : \mathbb{Q} \rightarrow \mathbb{R}$ ，把滤子 $\mathcal{N} x_0$ 拉回到 $\mathbb{Q}$ ，并陈述

$$\text{Tendsto } (f \circ (\uparrow)) (\mathbb{Q} \rightarrow \mathbb{R}) (\text{comap } (\uparrow) (\mathcal{N} x_0)) (\mathcal{N} y_0)。$$

```
variable (f : ℝ → ℝ) (x₀ y₀ : ℝ)

#check comap ((↑) : ℚ → ℝ) (ℕ x₀)

#check Tendsto (f ∘ (↑)) (comap ((↑) : ℚ → ℝ) (ℕ x₀)) (ℕ y₀)
```

拉回操作也与复合相容，但它是反变的，也就是说，它会反转参数的顺序。

```

section
variable {α β γ : Type*} (F : Filter α) {m : γ → β} {n : β → α}

#check (comap_comap : comap m (comap n F) = comap (n ∘ m) F)

end

```

现在把注意力转向平面 $\mathbb{R} \times \mathbb{R}$ ，并尝试理解点 (x_0, y_0) 的邻域与 $\mathcal{N} x_0$ 、 $\mathcal{N} y_0$ 之间的关系。有一个积操作

`Filter.prod : Filter X → Filter Y → Filter (X × Y)`，记作 x^s ，它回答了这个问题：

```

example :  $\mathcal{N} (x_0, y_0) = \mathcal{N} x_0 \times^s \mathcal{N} y_0 :=$ 
  nhds_prod_eq

```

积操作由拉回操作和 `inf` 运算定义：

即 $F \times^s G = (\text{comap Prod.fst } F) \sqcap (\text{comap Prod.snd } G)$ 。

这里的 `inf` 运算指的是任意类型 X 上 `Filter X` 的格结构；其中 $F \sqcap G$ 是同时小于 F 和 G 的最大滤子。因此，`inf` 运算推广了集合交的概念。

Mathlib 中大量证明会使用上述全部结构（`map`、`comap`、`inf`、`sup` 和 `prod`），给出关于收敛性的代数证明，而从不提及滤子的成员。你可以在证明下面引理时练习这一点；必要时可展开 `Tendsto` 与 `Filter.prod` 的定义。

```

#check le_inf_iff

example (f :  $\mathbb{N} \rightarrow \mathbb{R} \times \mathbb{R}$ ) (x_0 y_0 :  $\mathbb{R}$ ) :
  Tendsto f atTop ( $\mathcal{N} (x_0, y_0)$ ) ↔
  Tendsto (Prod.fst ∘ f) atTop ( $\mathcal{N} x_0$ ) ∧ Tendsto (Prod.snd ∘ f) atTop ( $\mathcal{N} y_0$ ) :=
  sorry

```

有序类型 `Filter X` 实际上是一个完备格，也就是说，它有底元和顶元，并且 X 上任意滤子集合都有一个 `Inf` 和一个 `Sup`。

注意，给定滤子定义中的第二个性质（若 U 属于 F ，则任何比 U 更大的集合也属于 F ），第一个性质（ X 的所有元素组成的集合属于 F ）等价于 F 不是空的集合族。不要把这一点与“空集是否为 F 的元素”这一更微妙的问题混淆。滤子的定义并不禁止 $\emptyset \in F$ ；但如果空集在 F 中，那么每个集合都在 F 中，也就是说 $\forall U : \text{Set } X, U \in F$ 。此时， F 是一个相当平凡的滤子，正是完备格 `Filter X` 的底元。这与布尔巴基的滤子定义不同，后者不允许滤子包含空集。

因为我们的定义包含平凡滤子，有些引理中就需要显式假设非平凡性。不过作为回报，理论具有更好的整体性质。我们已经看到，包含平凡滤子给出了底元。它还允许定义 `principal : Set X → Filter X`，把 $\emptyset$ 映到 $\perp$ ，而无需添加排除空集的前提。它也同样允许无前提地定义拉回操作。事实上，可能出现 `comap f F =  $\perp$` 但 $F \neq \perp$ 的情况。例如，给定 $x_0 : \mathbb{R}$ 和 $s : \text{Set } \mathbb{R}$ ，在与 s 对应的子类型到实数的强制映射下， $\mathcal{N} x_0$ 的拉回非平凡，当且仅当 x_0 属于 s 的闭包。

为了管理那些确实需要假设某个滤子非平凡的引理，Mathlib 有一个类型类 `Filter.NeBot`，库中也有假设

`(F : Filter X) [F.NeBot]` 的引理。例如，实例数据库知道 `(atTop : Filter  $\mathbb{N}$ ).NeBot`，并且知道非平凡滤子的推出仍为非平凡滤子。因此，一个假设 `[F.NeBot]` 的引理会自动适用于任意序列 u 的 `map u atTop`。

我们对滤子的代数性质及其与极限关系的浏览基本完成了，但还没有说明为什么这确实重新获得了通常的极限概念。表面上看，`Tendsto u atTop ( $\mathcal{N} x_0$ )` 似乎强于Section 3.6中定义的收敛概念，因为这里要求 x_0 的每个邻域的原像都属于 `atTop`，而通常定义只要求标准邻域 `Ioo (x_0 - ε) (x_0 + ε)` 满足这一点。关键在于，按定义，每个邻域都包含这样一个标准邻域。这个观察引出滤子基的概念。

给定 $F : \text{Filter } X$ ，若集合族 $s : \iota \rightarrow \text{Set } X$ 满足：对每个集合 U ，都有 $U \in F$ 当且仅当它包含某个 s_i ，则称它是 F 的基。换言之，形式上说， s 是基当且仅当它满足 $\forall U : \text{Set } X, U \in F \leftrightarrow \exists i, s_i \subseteq U$ 。更灵活的做法是在 ι 上考虑一个谓词，只选择指标类型中的某些值 i 。在 $\mathcal{N}_{x_0}$ 的情形中，我们希望 ι 是 $\mathbb{R}$ ，把 i 写作 ε ，并且该谓词应当选出正的 ε 。因此，集合 $I_{00} (x_0 - \varepsilon) (x_0 + \varepsilon)$ 构成 $\mathbb{R}$ 上邻域拓扑的基这一事实，陈述如下：

```
example (x₀ : ℝ) : HasBasis (N x₀) (fun ε : ℝ → 0 < ε) fun ε ↦ Ioo (x₀ - ε) (x₀ + ε) :=
  nhds_basis_Ioo_pos x₀
```

滤子 atTop 也有一个很好的基。引理 `Filter.HasBasis.tendsto_iff` 允许我们在给定 F 和 G 的基之后，重新表述形如 `Tendsto f F G` 的陈述。把这些内容结合起来，基本上就得到我们在Section 3.6中使用的收敛概念。

```
example (u : ℕ → ℝ) (x₀ : ℝ) :
  Tendsto u atTop (N x₀) ↔ ∀ ε > 0, ∃ N, ∀ n ≥ N, u n ∈ Ioo (x₀ - ε) (x₀ + ε) := by
  have : atTop.HasBasis (fun _ : ℕ → True) Ici := atTop_basis
  rw [this.tendsto_iff (nhds_basis_Ioo_pos x₀)]
  simp
```

现在说明滤子如何帮助处理“对充分大的数成立”或“对充分接近某点的点成立”的性质。在Section 3.6中，我们经常遇到这样的情形：已知某性质 P_n 对充分大的 n 成立，且另一个性质 Q_n 也对充分大的 n 成立。使用两次 `cases` 会得到 N_P 与 N_Q ，满足 $\forall n \geq N_P, P_n$ 和 $\forall n \geq N_Q, Q_n$ 。再使用 `set N := max N_P N_Q`，最终可以证明 $\forall n \geq N, P_n \wedge Q_n$ 。反复这样做会变得繁琐。

可以做得更好：注意陈述“ P_n 与 Q_n 对充分大的 n 成立”意味着有 $\{n \mid P_n\} \in \text{atTop}$ 和 $\{n \mid Q_n\} \in \text{atTop}$ 。由于 atTop 是滤子， atTop 的两个元素之交仍在 atTop 中，因此有 $\{n \mid P_n \wedge Q_n\} \in \text{atTop}$ 。写 $\{n \mid P_n\} \in \text{atTop}$ 并不方便，但可以使用更具提示性的记号 $\forall^f n \text{ in } \text{atTop}, P_n$ 。这里上标 f 代表“Filter”。可以把这个记号理解为：对“非常大的数的集合”中的所有 n ， P_n 成立。 $\forall^f$ 记号代表 `Filter.Eventually`，而引理 `Filter.Eventually.and` 使用滤子的交性质完成刚才描述的事情：

```
example (P Q : ℕ → Prop) (hP : ∀f n in atTop, P n) (hQ : ∀f n in atTop, Q n) :
  ∀f n in atTop, P n ∧ Q n :=
  hP.and hQ
```

这个记号非常方便且直观，因此当 P 是等式或不等式陈述时也有专门版本。例如，令 u 和 v 为两个实数序列，证明若 u_n 与 v_n 对充分大的 n 相等，则 u 趋向 x_0 当且仅当 v 趋向 x_0 。我们先使用一般的 `Eventually`，再使用针对等式谓词的专门版本 `EventuallyEq`。这两个陈述按定义等价，因此同一个证明在两种情形中都适用。

```
example (u v : ℕ → ℝ) (h : ∀f n in atTop, u n = v n) (x₀ : ℝ) :
  Tendsto u atTop (N x₀) ↔ Tendsto v atTop (N x₀) :=
  tendsto_congr' h

example (u v : ℕ → ℝ) (h : u =f[atTop] v) (x₀ : ℝ) :
  Tendsto u atTop (N x₀) ↔ Tendsto v atTop (N x₀) :=
  tendsto_congr' h
```

用 `Eventually` 来回顾滤子的定义很有启发。给定 $F : \text{Filter } X$ ，对 X 上任意谓词 P 和 Q ，

条件 $\text{univ} \in F$ 保证 $(\forall x, P x) \rightarrow \forall^f x \text{ in } F, P x$ ，

条件 $U \in F \rightarrow U \subseteq V \rightarrow V \in F$ 保证 $(\forall^f x \text{ in } F, P x) \rightarrow (\forall x, P x \rightarrow Q x) \rightarrow \forall^f x \text{ in } F, Q x$ ，并且

条件 $U \in F \rightarrow V \in F \rightarrow U \cap V \in F$ 保证 $(\forall^f x \text{ in } F, P x) \rightarrow (\forall^f x \text{ in } F, Q x) \rightarrow \forall^f x \text{ in } F, P x \wedge Q x$ 。

```
#check Eventually.of_forall
#check Eventually.mono
#check Eventually.and
```

第二项对应于 `Eventually.mono`，它支持一些很好的滤子用法，尤其是与 `Eventually.and` 结合时。`filter_upwards` 策略允许我们把它们组合起来。比较：

```

example (P Q R :  $\mathbb{N} \rightarrow \text{Prop}$ ) (hP :  $\forall^f n \text{ in atTop, } P n$ ) (hQ :  $\forall^f n \text{ in atTop, } Q n$ )
  (hR :  $\forall^f n \text{ in atTop, } P n \wedge Q n \rightarrow R n$ ) :  $\forall^f n \text{ in atTop, } R n$  := by
  apply (hP.and (hQ.and hR)).mono
  rintro n (h, h', h'')
  exact h'' (h, h')

example (P Q R :  $\mathbb{N} \rightarrow \text{Prop}$ ) (hP :  $\forall^f n \text{ in atTop, } P n$ ) (hQ :  $\forall^f n \text{ in atTop, } Q n$ )
  (hR :  $\forall^f n \text{ in atTop, } P n \wedge Q n \rightarrow R n$ ) :  $\forall^f n \text{ in atTop, } R n$  := by
  filter_upwards [hP, hQ, hR] with n h h' h''
  exact h'' (h, h')

```

了解测度论的读者会注意到，由补集测度为零的集合组成的滤子 `μ.ae`（也就是“几乎每一点组成的集合”）作为 `Tendsto` 的源或目标并不十分有用，但它可以很方便地与 `Eventually` 一起使用，用来表达某个性质几乎处处成立。

$\forall^f x \text{ in } F, P x$ 有一个偶尔很有用的对偶版本： $\exists^f x \text{ in } F, P x$ 意味着 $\{x \mid \neg P x\} \notin F$ 。例如， $\exists^f n \text{ in atTop, } P n$ 表示存在任意大的 n 使得 $P n$ 成立。记号 $\exists^f$ 代表 `Filter.Frequently`。

作为一个更复杂的例子，考虑下面关于序列 `u`、集合 `M` 和值 `x` 的陈述：

若 `u` 收敛到 `x`，并且对充分大的 n ，`u n` 属于 `M`，则 `x` 属于 `M` 的闭包。

这可以形式化如下：

即 `Tendsto u atTop (N x) → (∀f n in atTop, u n ∈ M) → x ∈ closure M`。

这是拓扑库中定理 `mem_closure_of_tendsto` 的一个特例。请尝试使用引用的引理证明它；其中要用到 `ClusterPt x F` 表示 `(N x ⊓ F).NeBot`，并且按定义，假设 $\forall^f n \text{ in atTop, } u n \in M$ 表示 `M ∈ map u atTop`。

```

#check mem_closure_iff_clusterPt
#check le_principal_iff
#check neBot_of_le

example (u :  $\mathbb{N} \rightarrow \mathbb{R}$ ) (M : Set  $\mathbb{R}$ ) (x :  $\mathbb{R}$ ) (hux : Tendsto u atTop (N x))
  (huM :  $\forall^f n \text{ in atTop, } u n \in M$ ) : x ∈ closure M :=
  sorry

```

11.2. 度量空间

上一节的例子集中于实数序列。本节将稍微提高一般性，转向度量空间。度量空间是一个类型 `X`，配备距离函数

`dist : X → X → ℝ`；它推广了 `X = ℝ` 情形中的函数 `fun x y ↦ |x - y|`。

引入这样的空间很容易，并且我们将检查距离函数所需的所有性质。

```

variable {X : Type*} [MetricSpace X] (a b c : X)

#check (dist a b : ℝ)
#check (dist_nonneg : 0 ≤ dist a b)
#check (dist_eq_zero : dist a b = 0 ↔ a = b)
#check (dist_comm a b : dist a b = dist b a)
#check (dist_triangle a b c : dist a c ≤ dist a b + dist b c)

```

注意，还有一些变体允许距离为无穷，或者允许 `dist a b` 为零而不必有 `a = b`，或同时允许这两种情况。它们分别称为 `EMetricSpace`、`PseudoMetricSpace` 和 `PseudoEMetricSpace`（这里的“e”代表“extended”）。

注意，从 `ℝ` 到度量空间的路径跳过了赋范空间这一特殊情形；赋范空间还需要线性代数，并将在微积分章节中解释。

11.2.1. 收敛与连续

使用距离函数，已经可以定义收敛序列以及度量空间之间的连续函数。实际上，它们是在下一节涉及的更一般设置中定义的，但我们有距离重新表述定义的引理。

```
example {u : ℕ → X} {a : X} :
  Tendsto u atTop (N a) ↔ ∀ ε > 0, ∃ N, ∀ n ≥ N, dist (u n) a < ε :=
  Metric.tendsto_atTop

example {X Y : Type*} [MetricSpace X] [MetricSpace Y] {f : X → Y} :
  Continuous f ↔
  ∀ x : X, ∀ ε > 0, ∃ δ > 0, ∀ x', dist x' x < δ → dist (f x') (f x) < ε :=
  Metric.continuous_iff
```

许多引理带有连续性假设，因此我们最终会证明大量连续性结果，并且有一个专门完成这项任务的 `continuity` 策略。先证明下面练习中将要用到的一个连续性陈述。注意，Lean 知道如何把两个度量空间的积视为度量空间；它选择上确界范数，因此 $\text{dist } (x_1, y_1) (x_2, y_2) = \max (\text{dist } x_1 x_2) (\text{dist } y_1 y_2)$ 由自反性成立。因此，考虑从 $X \times X$ 到 $\mathbb{R}$ 的连续函数是有意义的。特别地，距离函数的非柯里化版本就是这样的函数。

```
example {X Y : Type*} [MetricSpace X] [MetricSpace Y] {f : X → Y} (hf : Continuous f) :
  Continuous fun p : X × X ↦ dist (f p.1) (f p.2) := by continuity
```

这个策略有些慢，因此知道如何手动完成也很有用。首先需要使用：函数 `fun p : X × X ↦ f p.1` 是连续的，因为它是 `f` 与投影 `prod.fst` 的复合；其中 `f` 根据假设 `hf` 连续，而 `prod.fst` 的连续性正是引理 `continuous_fst` 的内容。复合性质是 `Continuous.comp`，它位于 `Continuous` 命名空间中，因此可以使用点记号把 `Continuous.comp hf continuous_fst` 压缩为 `hf.comp continuous_fst`；后者实际上更可读，因为它确实读作把我们的假设与引理复合。对第二个分量也可以类似处理，得到 `fun p : X × X ↦ f p.2` 的连续性。然后用 `Continuous.prod_mk` 组装这两个连续性，得到

`(hf.comp continuous_fst).prod_mk (hf.comp continuous_snd) : Continuous (fun p : X × X ↦ (f p.1, f p.2))`，再复合一次即可得到完整证明。

```
example {X Y : Type*} [MetricSpace X] [MetricSpace Y] {f : X → Y} (hf : Continuous f) :
  Continuous fun p : X × X ↦ dist (f p.1) (f p.2) :=
  continuous_dist.comp ((hf.comp continuous_fst).prodMk (hf.comp continuous_snd))
```

即使像上面那样大量使用点记号，通过 `Continuous.comp` 把 `Continuous.prod_mk` 与 `continuous_dist` 组合起来仍显得笨拙。更严重的问题是，这个漂亮的证明需要相当多预先规划。Lean 接受上面的证明项，是因为它是一个完整项，证明了一个与目标按定义等价的陈述；关键是展开函数复合的定义。事实上，我们的目标函数 `fun p : X × X ↦ dist (f p.1) (f p.2)` 并不是以复合形式呈现的。我们给出的证明项证明的是 `dist ∘ (fun p : X × X ↦ (f p.1, f p.2))` 的连续性，而它恰好与目标函数按定义相等。但如果尝试用策略逐步构造这个证明，先写 `apply continuous_dist.comp`，Lean 的 elaborator 将无法识别出复合，从而拒绝应用该引理。当涉及类型的积时，这种情况尤其明显。

这里更适合应用的引理是

`Continuous.dist {f g : X → Y} : Continuous f → Continuous g → Continuous (fun x ↦ dist (f x) (g x))`；它对 Lean 的 elaborator 更友好，并且在直接给出完整证明项时也能产生更短的证明，下面两个对上述陈述的新证明展示了这一点：

```
example {X Y : Type*} [MetricSpace X] [MetricSpace Y] {f : X → Y} (hf : Continuous f) :
  Continuous fun p : X × X ↦ dist (f p.1) (f p.2) := by
  apply Continuous.dist
  exact hf.comp continuous_fst
  exact hf.comp continuous_snd

example {X Y : Type*} [MetricSpace X] [MetricSpace Y] {f : X → Y} (hf : Continuous f) :
  Continuous fun p : X × X ↦ dist (f p.1) (f p.2) :=
  (hf.comp continuous_fst).dist (hf.comp continuous_snd)
```

注意，如果没有来自复合的 elaboration 问题，压缩证明的另一种方式是使用 `Continuous.prod_map`；它有时很有用，并给出

既要在更利于 elaboration 的版本和更短的输入版本之间取舍，多少令人遗憾；因此我们用 `Continuous.fst'` 提供的最后一点压缩来结束这段讨论。它允许把 `hf.comp continuous_fst` 压缩为 `hf.fst'`（对 `snd` 也类似），从而得到最后一个证明；此时已经接近晦涩。

```
example {X Y : Type*} [MetricSpace X] [MetricSpace Y] {f : X → Y} (hf : Continuous f) :
  Continuous fun p : X × X ↦ dist (f p.1) (f p.2) :=
  hf.fst'.dist hf.snd'
```

现在轮到你证明一些连续性引理。尝试 continuity 策略之后，若要手动完成，将需要 `Continuous.add`、`continuous_pow` 和 `continuous_id`。

```
example {f : ℝ → X} (hf : Continuous f) : Continuous fun x : ℝ ↦ f (x ^ 2 + x) :=
  sorry
```

到目前为止，我们把连续性视为全局概念；但也可以定义一点处连续。

```
example {X Y : Type*} [MetricSpace X] [MetricSpace Y] (f : X → Y) (a : X) :
  ContinuousAt f a ↔ ∀ ε > 0, ∃ δ > 0, ∀ {x}, dist x a < δ → dist (f x) (f a) < ε :=
  Metric.continuousAt_iff
```

11.2.2. 球、开集与闭集

一旦有了距离函数，最重要的几何定义就是（开）球和闭球。

```
variable (r : ℝ)

example : Metric.ball a r = { b | dist b a < r } :=
  rfl

example : Metric.closedBall a r = { b | dist b a ≤ r } :=
  rfl
```

注意，这里的 r 是任意实数，没有符号限制。当然，某些陈述确实需要半径条件。

```
example (hr : 0 < r) : a ∈ Metric.ball a r :=
  Metric.mem_ball_self hr

example (hr : 0 ≤ r) : a ∈ Metric.closedBall a r :=
  Metric.mem_closedBall_self hr
```

有了球之后，就可以定义开集。开集实际上是在下一节涉及的更一般设置中定义的，但我们有球重新表述该定义的引理。

```
example (s : Set X) : IsOpen s ↔ ∀ x ∈ s, ∃ ε > 0, Metric.ball x ε ⊆ s :=
  Metric.isOpen_iff
```

于是，闭集就是补集为开集的集合。它们的重要性质是对极限封闭。一个集合的闭包是包含它的最小闭集。

```
example {s : Set X} : IsClosed s ↔ IsOpen (sᶜ) :=
  isOpen_compl_iff.symm

example {s : Set X} (hs : IsClosed s) {u : ℕ → X} (hu : Tendsto u atTop (ℕ) a)
  (hus : ∀ n, u n ∈ s) : a ∈ s :=
  hs.mem_of_tendsto hu (Eventually.of_forall hus)

example {s : Set X} : a ∈ closure s ↔ ∀ ε > 0, ∃ b ∈ s, a ∈ Metric.ball b ε :=
  Metric.mem_closure_iff
```

完成下一个练习时不要使用 `mem_closure_iff_seq_limit`。


```
example {u : ℕ → X} (hu : Tendsto u atTop (ℕ a)) {s : Set X} (hs : ∀ n, u n ∈ s) :
  a ∈ closure s := by
  sorry
```

请回忆滤子部分中邻域滤子在 Mathlib 中的重要作用。在度量空间语境下，关键点是球为这些滤子提供了基。这里的主要引理是 `Metric.nhds_basis_ball` 和 `Metric.nhds_basis_closedBall`，它们分别对正半径开球和闭球断言这一点。中心点是隐式参数，因此可以像下面的例子一样调用 `Filter.HasBasis.mem_iff`。

```
example {x : X} {s : Set X} : s ∈ ℱ x ↔ ∃ ε > 0, Metric.ball x ε ⊆ s :=
  Metric.nhds_basis_ball.mem_iff

example {x : X} {s : Set X} : s ∈ ℱ x ↔ ∃ ε > 0, Metric.closedBall x ε ⊆ s :=
  Metric.nhds_basis_closedBall.mem_iff
```

11.2.3. 紧性

紧性是重要的拓扑概念。它把度量空间中具有与实数线段相同性质的子集同其他区间区分开来：

任意取值于紧集的序列都有一个在该集合中收敛的子序列。

任意定义在非空紧集上且取实数值的连续函数都是有界的，并且在某处达到其界（这称为极值定理）。

紧集是闭集。

先检查实数中的单位区间确实是紧集，然后在一般度量空间中检查上面的紧集性质。第二个陈述只需要在给定集合上的连续性，所以将使用 `ContinuousOn` 而不是 `Continuous`，并且会分别给出最小值和最大值的陈述。当然，这些结果都由更一般的版本推出，其中一些会在后面的章节中讨论。

```
example : IsCompact (Set.Icc 0 1 : Set ℝ) :=
  isCompact_Icc

example {s : Set X} (hs : IsCompact s) {u : ℕ → X} (hu : ∀ n, u n ∈ s) :
  ∃ a ∈ s, ∃ φ : ℕ → ℕ, StrictMono φ ∧ Tendsto (u ∘ φ) atTop (ℕ a) :=
  hs.tendsto_subseq hu

example {s : Set X} (hs : IsCompact s) (hs' : s.Nonempty) {f : X → ℝ}
  (hfs : ContinuousOn f s) :
  ∃ x ∈ s, ∀ y ∈ s, f x ≤ f y :=
  hs.exists_isMinOn hs' hfs

example {s : Set X} (hs : IsCompact s) (hs' : s.Nonempty) {f : X → ℝ}
  (hfs : ContinuousOn f s) :
  ∃ x ∈ s, ∀ y ∈ s, f y ≤ f x :=
  hs.exists_isMaxOn hs' hfs

example {s : Set X} (hs : IsCompact s) : IsClosed s :=
  hs.isClosed
```

也可以用一个额外的取值于 `Prop` 的类型类，指定一个度量空间整体上是紧的：

```
example {X : Type*} [MetricSpace X] [CompactSpace X] : IsCompact (univ : Set X) :=
  isCompact_univ
```

在紧度量空间中，任意闭集都是紧的；这就是 `IsClosed.isCompact`。

11.2.4. 一致连续函数

现在转向度量空间上的一致性概念：一致连续函数、柯西序列和完备性。它们同样是在更一般的语境中定义的，但我们有度量命名空间中的引理可用来访问其初等定义。先从一致连续性开始。

```
example {X : Type*} [MetricSpace X] {Y : Type*} [MetricSpace Y] {f : X → Y} :
  UniformContinuous f ↔
  ∀ ε > 0, ∃ δ > 0, ∀ {a b : X}, dist a b < δ → dist (f a) (f b) < ε :=
  Metric.uniformContinuous_iff
```

为了练习操作这些定义，我们将证明从紧度量空间到度量空间的连续函数是一致连续的（稍后会看到一个更一般的版本）。

先给出一个非形式化证明梗概。设 $f : X \rightarrow Y$ 是从紧度量空间到度量空间的连续函数。固定 $\epsilon > 0$ ，开始寻找某个 δ 。

令 $\phi : X \times X \rightarrow \mathbb{R} := \text{fun } p \mapsto \text{dist } (f \text{ p.1}) (f \text{ p.2})$ ，并令 $K := \{ p : X \times X \mid \epsilon \leq \phi p \}$ 。注意 ϕ 连续，因为 f 和距离函数都连续。并且 K 显然是闭的（使用 `isClosed_le`），因此它是紧的，因为 X 紧且 Lean 知道紧空间的积仍为紧。

然后用 `eq_empty_or_nonempty` 分情况讨论。若 K 为空，则显然完成证明（例如可以取 $\delta = 1$ ）。因此假设 K 非空，并使用极值定理选择 (x_0, x_1) ，使距离函数在 K 上取得下确界。随后可以取 $\delta = \text{dist } x_0 \ x_1$ ，并检查一切成立。

```
example {X : Type*} [MetricSpace X] [CompactSpace X]
  {Y : Type*} [MetricSpace Y] {f : X → Y}
  (hf : Continuous f) : UniformContinuous f := by
  sorry
```

11.2.5. 完备性

度量空间中的柯西序列，是各项彼此越来越接近的序列。表达这一思想有若干等价方式。特别地，收敛序列是柯西序列。反命题只在所谓完备空间中成立。

```
example (u : ℕ → X) :
  CauchySeq u ↔ ∀ ε > 0, ∃ N : ℕ, ∀ m ≥ N, ∀ n ≥ N, dist (u m) (u n) < ε :=
  Metric.cauchySeq_iff

example (u : ℕ → X) :
  CauchySeq u ↔ ∀ ε > 0, ∃ N : ℕ, ∀ n ≥ N, dist (u n) (u N) < ε :=
  Metric.cauchySeq_iff'

example [CompleteSpace X] (u : ℕ → X) (hu : CauchySeq u) :
  ∃ x, Tendsto u atTop (𝓃 x) :=
  cauchySeq_tendsto_of_complete hu
```

我们将通过证明一个方便的判别准则来练习使用这个定义；它是 Mathlib 中某个判别准则的特例。这也是在几何语境中练习使用大和式的好机会。除了滤子部分的说明之外，你可能还需要 `tendsto_pow_atTop_nhds_zero_of_lt_one`、`Tendsto.mul` 和 `dist_le_range_sum_dist`。

```
theorem cauchySeq_of_le_geometric_two' {u : ℕ → X}
  (hu : ∀ n : ℕ, dist (u n) (u (n + 1)) ≤ (1 / 2) ^ n) : CauchySeq u := by
  rw [Metric.cauchySeq_iff']
  intro ε ε_pos
  obtain ⟨N, hN⟩ : ∃ N : ℕ, 1 / 2 ^ N * 2 < ε := by sorry
  use N
  intro n hn
  obtain ⟨k, rfl : n = N + k⟩ := le_iff_exists_add.mp hn
  calc
    dist (u (N + k)) (u N) = dist (u (N + 0)) (u (N + k)) := sorry
    _ ≤ ∑ i ∈ range k, dist (u (N + i)) (u (N + (i + 1))) := sorry
    _ ≤ ∑ i ∈ range k, (1 / 2 : ℝ) ^ (N + i) := sorry
    _ = 1 / 2 ^ N * ∑ i ∈ range k, (1 / 2 : ℝ) ^ i := sorry
    _ ≤ 1 / 2 ^ N * 2 := sorry
    _ < ε := sorry
```

现在准备迎接本节最后的难题：完备度量空间的贝尔定理！下面的证明骨架展示了一些有趣技巧。它使用感叹号版本的 `choose` 策略（你应当尝试去掉这个感叹号看看会发生什么），并展示如何在证明中间使用 `Nat.rec_on` 归纳地定义某个对象。

```

open Metric

example [CompleteSpace X] (f : ℕ → Set X) (ho : ∀ n, IsOpen (f n)) (hd : ∀ n, Dense (f n)) :
  Dense (⋂ n, f n) := by
  let B : ℕ → ℝ := fun n ↦ (1 / 2) ^ n
  have Bpos : ∀ n, 0 < B n
  sorry
  /- Translate the density assumption into two functions `center` and `radius` associating
  to any n, x, δ, δpos a center and a positive radius such that
  `closedBall center radius` is included both in `f n` and in `closedBall x δ`.
  We can also require `radius ≤ (1/2)^(n+1)`, to ensure we get a Cauchy sequence later. -/
  have :
    ∀ (n : ℕ) (x : X),
      ∀ δ > 0, ∃ y : X, ∃ r > 0, r ≤ B (n + 1) ∧ closedBall y r ⊆ closedBall x δ ∧ f n :=
    by sorry
  choose! center radius Hpos HB Hball using this
  intro x
  rw [mem_closure_iff_nhds_basis nhds_basis_closedBall]
  intro ε εpos
  /- `ε` is positive. We have to find a point in the ball of radius `ε` around `x`
  belonging to all `f n`. For this, we construct inductively a sequence
  `F n = (c n, r n)` such that the closed ball `closedBall (c n) (r n)` is included
  in the previous ball and in `f n`, and such that `r n` is small enough to ensure
  that `c n` is a Cauchy sequence. Then `c n` converges to a limit which belongs
  to all the `f n`. -/
  let F : ℕ → X × ℝ := fun n ↦
    Nat.recOn n (Prod.mk x (min ε (B 0)))
      fun n p ↦ Prod.mk (center n p.1 p.2) (radius n p.1 p.2)
  let c : ℕ → X := fun n ↦ (F n).1
  let r : ℕ → ℝ := fun n ↦ (F n).2
  have rpos : ∀ n, 0 < r n := by sorry
  have rB : ∀ n, r n ≤ B n := by sorry
  have incl : ∀ n, closedBall (c (n + 1)) (r (n + 1)) ⊆ closedBall (c n) (r n) ∧ f n := by
    sorry
  have cdist : ∀ n, dist (c n) (c (n + 1)) ≤ B n := by sorry
  have : CauchySeq c := cauchySeq_of_le_geometric_two' cdist
  -- as the sequence `c n` is Cauchy in a complete space, it converges to a limit `y`.
  rcases cauchySeq_tendsto_of_complete this with (y, ylim)
  -- this point `y` will be the desired point. We will check that it belongs to all
  -- `f n` and to `ball x ε`.
  use y
  have I : ∀ n, ∀ m ≥ n, closedBall (c m) (r m) ⊆ closedBall (c n) (r n) := by sorry
  have yball : ∀ n, y ∈ closedBall (c n) (r n) := by sorry
  sorry

```

11.3. 拓扑空间

11.3.1. 基本概念

现在进一步提高一般性，引入拓扑空间。我们将回顾定义拓扑空间的两种主要方式，然后解释为什么拓扑空间范畴比度量空间范畴表现得好多。注意，这里不会使用 Mathlib 的范畴论，只是采取一种略带范畴意味的观点。

从度量空间过渡到拓扑空间的第一种理解方式是：只保留开集概念（或等价地，闭集概念）。从这个观点看，拓扑空间是一个类型，配备一族称为开集的集合。这个集合族必须满足下面给出的若干公理（这份数据略有冗余，但我们将忽略这一点）。

```

section
variable {X : Type*} [TopologicalSpace X]

example : IsOpen (univ : Set X) :=
  isOpen_univ

example : IsOpen (∅ : Set X) :=
  isOpen_empty

example {ι : Type*} {s : ι → Set X} (hs : ∀ i, IsOpen (s i)) : IsOpen (⋃ i, s i) :=
  isOpen_iUnion hs

example {ι : Type*} [Fintype ι] {s : ι → Set X} (hs : ∀ i, IsOpen (s i)) :
  IsOpen (⋂ i, s i) :=
  isOpen_iInter_of_finite hs

```

闭集于是定义为补集为开集的集合。拓扑空间之间的函数称为（全局）连续，当且仅当所有开集的原像都是开集。

```
variable {Y : Type*} [TopologicalSpace Y]

example {f : X → Y} : Continuous f ↔ ∀ s, IsOpen s → IsOpen (f ⁻¹' s) :=
  continuous_def
```

用这个定义已经可以看到，与度量空间相比，拓扑空间只保留了足以讨论连续函数的信息：一个类型上的两个拓扑结构相同，当且仅当它们具有相同的连续函数（事实上，恒等函数在两个方向都连续，当且仅当这两个结构具有相同的开集）。

然而，一旦转向一点处连续，就能看到基于开集的方法的局限。在 Mathlib 中，我们经常把拓扑空间理解为这样的类型：每个点 x 都配有一个邻域滤子 $\mathcal{N} x$ （相应函数 $X \rightarrow \text{Filter } X$ 满足后面解释的若干条件）。请回忆滤子部分中这些对象的两个相关作用。首先， $\mathcal{N} x$ 被看作 X 中接近 x 的点组成的广义集合。其次，它给出一种方式，用于表达对任意谓词 $P : X \rightarrow \text{Prop}$ ，该谓词在足够接近 x 的点处成立。现在陈述 $f : X \rightarrow Y$ 在 x 处连续。完全滤子化的表述是： f 下接近 x 的点组成的广义集合的直接像，包含于接近 $f x$ 的点组成的广义集合。回忆这可以写作 $\text{map } f (\mathcal{N} x) \leq \mathcal{N} (f x)$ 或 $\text{Tendsto } f (\mathcal{N} x) (\mathcal{N} (f x))$ 。

```
example {f : X → Y} {x : X} : ContinuousAt f x ↔ map f (N x) ≤ N (f x) :=
  Iff.rfl
```

也可以同时把邻域看作普通集合，而把邻域滤子看作广义集合来表述：“对 $f x$ 的任意邻域 U ，所有足够接近 x 的点都会被送入 U ”。注意，证明再次是 `Iff.rfl`；这个观点与前一个观点按定义等价。

```
example {f : X → Y} {x : X} : ContinuousAt f x ↔ ∀ U ∈ N (f x), ∀ x' in N x, f x' ∈ U :=
  Iff.rfl
```

现在解释如何在两种观点之间转换。用开集来表述时，可以简单地把 $\mathcal{N} x$ 的成员定义为包含某个含有 x 的开集的集合。

```
example {x : X} {s : Set X} : s ∈ N x ↔ ∃ t, t ⊆ s ∧ IsOpen t ∧ x ∈ t :=
  mem_nhds_iff
```

反向转换时，需要讨论 $\mathcal{N} : X \rightarrow \text{Filter } X$ 必须满足什么条件，才能成为某个拓扑的邻域函数。

第一个约束是：把 $\mathcal{N} x$ 看作广义集合时，它包含把单点集 $\{x\}$ 看作广义集合得到的 $\text{pure } x$ （解释这个奇怪名称会偏离太远，所以暂且接受它）。另一种说法是：如果某个谓词对接近 x 的点成立，那么它在 x 处成立。

```
example (x : X) : pure x ≤ N x :=
  pure_le_nhds x

example (x : X) (P : X → Prop) (h : ∀ y in N x, P y) : P x :=
  h.self_of_nhds
```

然后有一个更微妙的要求：对任意谓词 $P : X \rightarrow \text{Prop}$ 和任意 x ，若对接近 x 的 y 有 $P y$ 成立，则对接近 x 的 y 且接近 y 的 z ，有 $P z$ 成立。更精确地说，有：

```
example {P : X → Prop} {x : X} (h : ∀ y in N x, P y) : ∀ y in N x, ∀ z in N y, P z :=
  eventually_eventually_nhds.mpr h
```

这两个结果刻画了哪些函数 $X \rightarrow \text{Filter } X$ 是 X 上某个拓扑空间结构的邻域函数。仍然有一个函数

`TopologicalSpace.mkOfNhds : (X → Filter X) → TopologicalSpace X`；但只有当输入满足上述两个约束时，它才会把输入作为邻域函数返回。更精确地说，有一个引理 `TopologicalSpace.nhds_mkOfNhds`，它以另一种方式说明这一点；下一个练习将从我们上面的表述推出这种另一种表述。

```
example {α : Type*} (n : α → Filter α) (H₀ : ∀ a, pure a ≤ n a)
  (H : ∀ a : α, ∀ p : α → Prop, (∀f x in n a, p x) → ∀f y in n a, ∀f x in n y, p x) :
  ∀ a, ∀ s ∈ n a, ∃ t ∈ n a, t ≤ s ∧ ∀ a' ∈ t, s ∈ n a' := by
  sorry
end
```

注意，`TopologicalSpace.mkOfNhdns` 并不经常使用；但仍然值得知道，在何种精确意义上，邻域滤子就是拓扑空间结构的全部内容。

为了在 Mathlib 中高效使用拓扑空间，接下来需要知道的是：我们大量使用 `TopologicalSpace : Type u → Type u` 的形式性质。从纯数学观点看，这些形式性质非常清楚地说明了拓扑空间如何解决度量空间存在的问题。从这个观点看，拓扑空间解决的问题是：度量空间的函子性很弱，并且总体上范畴性质很差。除此之外，前面也已经讨论过，度量空间包含大量拓扑上无关的几何信息。

先关注函子性。可以在子集上诱导度量空间结构；等价地，可以沿单射拉回度量结构。但这基本上就是全部。度量结构不能沿一般映射拉回，也不能推出，即使沿满射也不行。

特别地，不存在一种合理的方式给度量空间的商或不可数个度量空间的积赋予距离。举例来说，考虑类型 $\mathbb{R} \rightarrow \mathbb{R}$ ，把它看成由 $\mathbb{R}$ 指标的若干份 $\mathbb{R}$ 的积。我们希望能够说函数序列的逐点收敛是一种正当的收敛概念。但 $\mathbb{R} \rightarrow \mathbb{R}$ 上不存在给出这一收敛概念的距离。相关地，也不存在一种距离，使得映射 $f : X \rightarrow (\mathbb{R} \rightarrow \mathbb{R})$ 连续当且仅当对每个 $t : \mathbb{R}$ ，
`fun x ↦ f x t` 连续。

现在回顾用于解决这些问题的数据。首先，可以用任意映射 $f : X \rightarrow Y$ 从一侧向另一侧推出或拉回拓扑。这两个操作构成伽罗瓦连接。

```
variable {X Y : Type*}

example (f : X → Y) : TopologicalSpace X → TopologicalSpace Y :=
  TopologicalSpace.coinduced f

example (f : X → Y) : TopologicalSpace Y → TopologicalSpace X :=
  TopologicalSpace.induced f

example (f : X → Y) (T_X : TopologicalSpace X) (T_Y : TopologicalSpace Y) :
  TopologicalSpace.coinduced f T_X ≤ T_Y ↔ T_X ≤ TopologicalSpace.induced f T_Y :=
  coinduced_le_iff_le_induced
```

这些操作与函数复合相容。照常，推出是协变的，拉回是反变的；参见 `coinduced_compose` 与 `induced_compose`。纸笔记号中，我们将用 f_*T 表示 `TopologicalSpace.coinduced f T`，用 f^*T 表示 `TopologicalSpace.induced f T`。

下一个重要组成部分是：对任意给定的 X ，`TopologicalSpace X` 上有完备格结构。如果把拓扑主要看作开集数据，那么会期望 `TopologicalSpace X` 上的序关系来自 `Set (Set X)`，也就是说，会期望当集合 u 对 t 开时它也对 t' 开，从而有 $t \leq t'$ 。然而我们已经知道，Mathlib 比开集更重视邻域；所以对任意 $x : X$ ，我们希望从拓扑空间到邻域的映射

`fun T : TopologicalSpace X ↦ @nhds X T x` 保序。并且我们知道 `Filter X` 上的序关系被设计成保证

`principal : Set X → Filter X` 保序，从而可把滤子视为广义集合。因此，我们在 `TopologicalSpace X` 上使用的序关系与来自 `Set (Set X)` 的序关系相反。

```
example {T T' : TopologicalSpace X} : T ≤ T' ↔ ∀ s, T'.IsOpen s → T.IsOpen s :=
  Iff.rfl
```

现在可以把推出（或拉回）操作与序关系结合起来，重新得到连续性。

```
example (T_X : TopologicalSpace X) (T_Y : TopologicalSpace Y) (f : X → Y) :
  Continuous f ↔ TopologicalSpace.coinduced f T_X ≤ T_Y :=
  continuous_iff_coinduced_le
```

借助这个定义以及推出与复合的相容性，可以免费得到如下泛性质：对任意拓扑空间 Z ，函数 $g : Y \rightarrow Z$ 关于拓扑 f_*T_X 连续，当且仅当 $g \circ f$ 连续。

$$\begin{aligned} g \text{ continuous} &\Leftrightarrow g_*(f_*T_X) \leq T_Z \\ &\Leftrightarrow (g \circ f)_*T_X \leq T_Z \\ &\Leftrightarrow g \circ f \text{ continuous} \end{aligned}$$

```
example {Z : Type*} (f : X → Y) (T_X : TopologicalSpace X) (T_Z : TopologicalSpace Z)
  (g : Y → Z) :
  @Continuous Y Z (TopologicalSpace.coinduced f T_X) T_Z g ↔
  @Continuous X Z T_X T_Z (g ∘ f) := by
  rw [continuous_iff_coinduced_le, coinduced_compose, continuous_iff_coinduced_le]
```

于是已经得到了商拓扑（把投影映射作为 f ）。这还没有用到对所有 X ， $\text{TopologicalSpace } X$ 都是完备格。现在看看所有这些结构如何用抽象废话证明积拓扑的存在。上面考虑过 $\mathbb{R} \rightarrow \mathbb{R}$ 的情形；现在考虑一般情形：对某个 $\iota : \text{Type}^*$ 和 $X : \iota \rightarrow \text{Type}^*$ ，取 $\Pi i, X i$ 。我们希望对任意拓扑空间 Z 和任意函数 $f : Z \rightarrow \Pi i, X i$ ， f 连续当且仅当对所有 i ， $(\text{fun } x \mapsto x i) \circ f$ 连续。用 p_i 表示投影 $(\text{fun } (x : \Pi i, X i) \mapsto x i)$ ，在纸面上探索这个约束：

$$\begin{aligned} (\forall i, p_i \circ f \text{ continuous}) &\Leftrightarrow \forall i, (p_i \circ f)_*T_Z \leq T_{X_i} \\ &\Leftrightarrow \forall i, (p_i)_*f_*T_Z \leq T_{X_i} \\ &\Leftrightarrow \forall i, f_*T_Z \leq (p_i)^*T_{X_i} \\ &\Leftrightarrow f_*T_Z \leq \inf [(p_i)^*T_{X_i}] \end{aligned}$$

因此可以看出， $\Pi i, X i$ 上我们想要的拓扑是什么：

```
example (ι : Type*) (X : ι → Type*) (T_X : ∀ i, TopologicalSpace (X i)) :
  (Pi.topologicalSpace : TopologicalSpace (∀ i, X i)) =
  Π i, TopologicalSpace.induced (fun x ↦ x i) (T_X i) :=
  rfl
```

至此，我们结束了这一游览：Mathlib 如何通过更具函子性的理论、以及在任意固定类型上具有完备格结构，来理解拓扑空间如何修正度量空间理论的缺陷。

11.3.2. 分离性与可数性

我们看到，拓扑空间范畴具有非常好的性质。代价是会出现相当病态的拓扑空间。可以对拓扑空间施加若干假设，使其行为更接近度量空间。最重要的是 T2Space ，也称为“Hausdorff”，它保证极限唯一。更强的分离性质是 T3Space ，它还保证 RegularSpace 性质：每个点都有由闭邻域组成的基。

```
example [TopologicalSpace X] [T2Space X] {u : ℕ → X} {a b : X} (ha : Tendsto u atTop (ℕ a))
  (hb : Tendsto u atTop (ℕ b)) : a = b :=
  tendsto_nhds_unique ha hb

example [TopologicalSpace X] [RegularSpace X] (a : X) :
  (ℕ a).HasBasis (fun s : Set X ↦ s ∈ ℕ a ∧ IsClosed s) id :=
  closed_nhds_basis a
```

注意，在每个拓扑空间中，每个点按定义都有一个开邻域基。

```
example [TopologicalSpace X] {x : X} :
  (ℕ x).HasBasis (fun t : Set X ↦ t ∈ ℕ x ∧ IsOpen t) id :=
  nhds_basis_opens' x
```

现在的主要目标，是证明允许由连续性作延拓的基本定理。取布尔巴基《一般拓扑》I.8.5 定理 1 中的非平凡方向：

设 X 为拓扑空间， A 为 X 的稠密子集， $f : A \rightarrow Y$ 为从 A 到 T_3 空间 Y 的连续映射。若对 X 中每个 x ，当 y 在 A 中趋向 x 时， $f(y)$ 在 Y 中趋向某个极限，则存在 f 到 X 的连续延拓 φ 。

实际上，Mathlib 包含上面引理的更一般版本 `IsDenseInducing.continuousAt_extend`；但这里我们坚持使用布尔巴基版本。

请记住，给定 `A : Set X`，`↑A` 是与 `A` 对应的子类型；Lean 会在需要时自动插入那个特殊的上箭头。并且（包含）强制映射是 `(↑) : A → X`。假设“在保持于 A 中的同时趋向 x ”对应于拉回滤子 `comap (↑) (N x)`。

先证明一个辅助引理，把它抽出是为了简化上下文（特别是这里不需要 Y 是拓扑空间）。

```
theorem aux {X Y A : Type*} [TopologicalSpace X] {c : A → X}
  {f : A → Y} {x : X} {F : Filter Y}
  (h : Tendsto f (comap c (N x)) F) {V' : Set Y} (V'_in : V' ∈ F) :
  ∃ V ∈ N x, IsOpen V ∧ c ⁻¹' V ⊆ f ⁻¹' V' := by
  sorry
```

现在转向连续延拓定理的主证明。

当 Lean 需要 `↑A` 上的拓扑时，它会自动使用诱导拓扑。唯一相关的引理是

```
nhds_induced (↑) : ∀ a : ↑A, N a = comap (↑) (N ↑a)
```

（这实际上是关于诱导拓扑的一般引理）。

证明提纲如下：

主假设和选择公理给出一个函数 ϕ ，使得 $\forall x, \text{Tendsto } f \text{ (comap } (\uparrow) \text{ (N } x)) \text{ (N } (\phi x))$ （由于 Y 是 Hausdorff 的， ϕ 完全确定；但直到要证明 ϕ 确实延拓 f 时，才需要这一点）。

先证明 ϕ 连续。固定任意 $x : X$ 。由于 Y 是正则的，只需检查对 ϕx 的每个闭邻域 V' ，都有 $\phi^{-1}' V' \in N x$ 。极限假设通过上面的辅助引理给出某个 $V \in N x$ ，满足 $\text{IsOpen } V \wedge (\uparrow)^{-1}' V \subseteq f^{-1}' V'$ 。由于 $V \in N x$ ，只需证明 $V \subseteq \phi^{-1}' V'$ ，即 $\forall y \in V, \phi y \in V'$ 。固定 V 中的 y 。因为 V 是开的，它是 y 的邻域。特别地， $(\uparrow)^{-1}' V \in \text{comap } (\uparrow) (N y)$ ；进而有 $f^{-1}' V' \in \text{comap } (\uparrow) (N y)$ 。此外，由于 A 稠密， $\text{comap } (\uparrow) (N y) \neq \perp$ 。因为已知 $\text{Tendsto } f \text{ (comap } (\uparrow) \text{ (N } y)) \text{ (N } (\phi y))$ ，这推出 $\phi y \in \text{closure } V'$ ；又因为 V' 是闭的，便证明了 $\phi y \in V'$ 。

剩下的是证明 ϕ 延拓 f 。这一步中会用到 f 的连续性，以及 Y 是 Hausdorff 的事实。

```
example [TopologicalSpace X] [TopologicalSpace Y] [T3Space Y] {A : Set X}
  (hA : ∀ x, x ∈ closure A) {f : A → Y} (f_cont : Continuous f)
  (hf : ∀ x : X, ∃ c : Y, Tendsto f (comap (↑) (N x)) (N c)) :
  ∃ φ : X → Y, Continuous φ ∧ ∀ a : A, φ a = f a := by
  sorry

#check HasBasis.tendsto_right_iff
```

除了分离性之外，使拓扑空间更接近度量空间的主要假设类型是可数性假设。其中最主要的是第一可数性，它要求每个点都有一个可数邻域基。特别地，这保证可以用序列来理解集合闭包。

```
example [TopologicalSpace X] [FirstCountableTopology X]
  {s : Set X} {a : X} :
  a ∈ closure s ↔ ∃ u : ℕ → X, (∀ n, u n ∈ s) ∧ Tendsto u atTop (N a) :=
  mem_closure_iff_seq_limit
```


11.3.3. 紧性

现在讨论拓扑空间中的紧性如何定义。照常，有多种理解方式，而 Mathlib 采用滤子版本。

首先需要定义滤子的聚点。给定拓扑空间 X 上的滤子 F ，点 $x : X$ 称为 F 的聚点，当且仅当把 F 视为广义集合时，它与接近 x 的点组成的广义集合有非空交。

于是可以说，集合 s 是紧的，当且仅当每个包含在 s 中的非空广义集合 F ，即每个满足 $F \leq \mathcal{P} s$ 的滤子，都在 s 中有聚点。

```
variable [TopologicalSpace X]

example {F : Filter X} {x : X} : ClusterPt x F ↔ NeBot (ℕ × ⋂ F) :=
  Iff.rfl

example {s : Set X} :
  IsCompact s ↔ ∀ (F : Filter X) [NeBot F], F ≤ ℙ s → ∃ a ∈ s, ClusterPt a F :=
  Iff.rfl
```

例如，若 F 是 $\text{map } u \text{ atTop}$ ，即 atTop 这个非常大的自然数组成的广义集合在 $u : \mathbb{N} \rightarrow X$ 下的像，则假设 $F \leq \mathcal{P} s$ 意味着对充分大的 n ， $u n$ 属于 s 。说 x 是 $\text{map } u \text{ atTop}$ 的聚点，就是说非常大自然数的像与接近 x 的点的集合相交。当 $\aleph_x$ 有可数基时，可以把它解释为： u 有一个收敛到 x 的子序列；这就恢复了度量空间中紧性的样子。

```
example [FirstCountableTopology X] {s : Set X} {u : ℕ → X} (hs : IsCompact s)
  (hu : ∀ n, u n ∈ s) : ∃ a ∈ s, ∃ φ : ℕ → ℕ, StrictMono φ ∧ Tendsto (u ∘ φ) atTop (ℕ a) :=
  hs.tendsto_subseq hu
```

聚点在连续函数下表现良好。

```
variable [TopologicalSpace Y]

example {x : X} {F : Filter X} {G : Filter Y} (H : ClusterPt x F) {f : X → Y}
  (hfx : ContinuousAt f x) (hf : Tendsto f F G) : ClusterPt (f x) G :=
  ClusterPt.map H hfx hf
```

作为练习，我们将证明紧集在连续映射下的像是紧的。除了已经见过的内容之外，还应使用 `Filter.push_pull` 和 `NeBot.of_map`。

```
example [TopologicalSpace Y] {f : X → Y} (hf : Continuous f) {s : Set X} (hs : IsCompact s) :
  IsCompact (f '' s) := by
  intro F F_ne F_le
  have map_eq : map f (ℙ s ⋂ comap f F) = ℙ (f '' s) ⋂ F := by sorry
  have Hne : (ℙ s ⋂ comap f F).NeBot := by sorry
  have Hle : ℙ s ⋂ comap f F ≤ ℙ s := inf_le_left
  sorry
```

也可以用开覆盖来表达紧性： s 是紧的，当且仅当每个覆盖 s 的开集族都有一个有限子族仍然覆盖它。

```
example {ι : Type*} {s : Set X} (hs : IsCompact s) (U : ι → Set X) (hUo : ∀ i, IsOpen (U i))
  (hsU : s ⊆ ⋃ i, U i) : ∃ t : Finset ι, s ⊆ ⋃ i ∈ t, U i :=
  hs.elim_finite_subcover U hUo hsU
```


12. 微分学

现在考虑分析学中若干概念的形式化：本章从微分开始，下一章转向积分与测度论。在Section 12.1中，我们限于从实数到实数的函数这一设置，这是任何初等微积分课程都熟悉的情形。然后在Section 12.2中，我们将在更广泛的设置下考察导数概念。

12.1. 初等微分学

设 f 是从实数到实数的函数。谈论 f 在单个点处的导数，与谈论导函数，是不同的事情。在 Mathlib 中，第一种概念表示如下。

```
open Real

/-- The sin function has derivative 1 at 0. -/
example : HasDerivAt sin 1 0 := by simp using hasDerivAt_sin 0
```

也可以通过写 `DifferentiableAt  $\mathbb{R}$` 来表达 f 在一点处可微，而不指定它在该点的导数。这里显式指定 $\mathbb{R}$ ，是因为在稍微更一般的语境中，当讨论从 $\mathbb{C}$ 到 $\mathbb{C}$ 的函数时，我们希望能够区分实意义下可微与复导数意义下可微。

```
example (x :  $\mathbb{R}$ ) : DifferentiableAt  $\mathbb{R}$  sin x :=
  (hasDerivAt_sin x).differentiableAt
```

如果每次想引用导数时都必须提供可微性的证明，会很不方便。因此 Mathlib 提供函数 `deriv f :  $\mathbb{R} \rightarrow \mathbb{R}$` ；它对任意函数 $f : \mathbb{R} \rightarrow \mathbb{R}$ 都有定义，并规定在 f 不可微的任何点处取值为 0。

```
example {f :  $\mathbb{R} \rightarrow \mathbb{R}$ } {x a :  $\mathbb{R}$ } (h : HasDerivAt f a x) : deriv f x = a :=
  h.deriv

example {f :  $\mathbb{R} \rightarrow \mathbb{R}$ } {x :  $\mathbb{R}$ } (h : ¬DifferentiableAt  $\mathbb{R}$  f x) : deriv f x = 0 :=
  deriv_zero_of_not_differentiableAt h
```

当然，关于 `deriv` 的许多引理确实需要可微性假设。例如，对于下一个引理，如果没有可微性假设，请思考一个反例。

```
example {f g :  $\mathbb{R} \rightarrow \mathbb{R}$ } {x :  $\mathbb{R}$ } (hf : DifferentiableAt  $\mathbb{R}$  f x) (hg : DifferentiableAt  $\mathbb{R}$  g x) :
  deriv (f + g) x = deriv f x + deriv g x :=
  deriv_add hf hg
```

不过有趣的是，有些陈述可以利用函数不可微时 `deriv` 的值默认等于零这一事实，从而避免可微性假设。因此，要理解下面陈述的意义，需要知道 `deriv` 的精确定义。

```
example {f :  $\mathbb{R} \rightarrow \mathbb{R}$ } {a :  $\mathbb{R}$ } (h : IsLocalMin f a) : deriv f a = 0 := by
  exact?
  --h.deriv_eq_zero
```

甚至可以在没有任何可微性假设的情况下陈述罗尔定理，这看起来更奇怪。

```
open Set

example {f :  $\mathbb{R} \rightarrow \mathbb{R}$ } {a b :  $\mathbb{R}$ } (hab : a < b) (hfc : ContinuousOn f (Icc a b)) (hfI : f a = f b) :
   $\exists c \in \text{Ioo } a \ b, \text{ deriv } f \ c = 0 :=$ 
  exists_deriv_eq_zero hab hfc hfI
```

当然，这个技巧不适用于一般的中值定理。

```
example {f :  $\mathbb{R} \rightarrow \mathbb{R}$ } {a b :  $\mathbb{R}$ } (hab : a < b) (hf : ContinuousOn f (Icc a b))
  (hf' : DifferentiableOn  $\mathbb{R}$  f (Ioo a b)) :  $\exists c \in \text{Ioo } a \ b, \text{ deriv } f \ c = (f \ b - f \ a) / (b - a) :=$ 
  exists_deriv_eq_slope f hab hf hf'
```

Lean 可以使用 `simp` 策略自动计算一些简单导数。

```
example : deriv (fun x : ℝ → x ^ 5) 6 = 5 * 6 ^ 4 := by simp
example : deriv sin π = -1 := by simp
```

12.2. 赋范空间中的微分学

12.2.1. 赋范空间

借助**赋范向量空间**概念，可以把微分推广到 $\mathbb{R}$ 之外；赋范向量空间同时封装了方向和距离。我们从**赋范群**概念开始，它是一个加法交换群，并配备取实数值的范数函数，满足下列条件。

```
variable {E : Type*} [NormedAddCommGroup E]

example (x : E) : 0 ≤ ‖x‖ :=
  norm_nonneg x

example {x : E} : ‖x‖ = 0 ↔ x = 0 :=
  norm_eq_zero

example (x y : E) : ‖x + y‖ ≤ ‖x‖ + ‖y‖ :=
  norm_add_le x y
```

每个赋范空间都是以 $d(x, y) = \|x - y\|$ 为距离函数的度量空间，因此也都是拓扑空间。Lean 与 Mathlib 都知道这一点。

```
example : MetricSpace E := by infer_instance

example {X : Type*} [TopologicalSpace X] {f : X → E} (hf : Continuous f) :
  Continuous fun x ↦ ‖f x‖ :=
  hf.norm
```

为了把范数概念与线性代数中的概念结合使用，我们在 `NormedAddGroup E` 之上添加假设 `NormedSpace ℝ E`。这规定 E 是 $\mathbb{R}$ 上的向量空间，并且标量乘法满足下列条件。

```
variable [NormedSpace ℝ E]

example (a : ℝ) (x : E) : ‖a • x‖ = |a| * ‖x‖ :=
  norm_smul a x
```

完备赋范空间称为**Banach 空间**。每个有限维向量空间都是完备的。

```
example [FiniteDimensional ℝ E] : CompleteSpace E := by infer_instance
```

在前面所有例子中，我们都使用实数作为基域。更一般地，对任何**非平凡赋范域**上的向量空间，也可以讨论微积分。所谓**非平凡赋范域**，是指配备取实数值范数的域，该范数满足乘法性，并且并非每个元素的范数都是零或一（等价地，存在某个元素的范数大于一）。

```
example (ℓk : Type*) [NontriviallyNormedField ℓk] (x y : ℓk) : ‖x * y‖ = ‖x‖ * ‖y‖ :=
  norm_mul x y

example (ℓk : Type*) [NontriviallyNormedField ℓk] : ∃ x : ℓk, 1 < ‖x‖ :=
  NormedField.exists_one_lt_norm ℓk
```

非平凡赋范域上的有限维向量空间是完备的，只要该域本身是完备的。

```
example (ℓk : Type*) [NontriviallyNormedField ℓk] (E : Type*) [NormedAddCommGroup E]
  [NormedSpace ℓk E] [CompleteSpace ℓk] [FiniteDimensional ℓk E] : CompleteSpace E :=
  FiniteDimensional.complete ℓk E
```

12.2.2. 连续线性映射

现在转向赋范空间范畴中的态射，即连续线性映射。在 Mathlib 中，赋范空间 E 与 F 之间的 $\mathbb{k}$ -线性连续映射类型写作 $E \rightarrow L[\mathbb{k}] F$ 。它们实现为打包映射，也就是说，此类型的元素是一个结构，包含函数本身以及线性和连续性的性质。Lean 会插入强制转换，使连续线性映射可以作为函数使用。

```
variable {ℓk : Type*} [NontriviallyNormedField ℓk] {E : Type*} [NormedAddCommGroup E]
[NormedSpace ℓk E] {F : Type*} [NormedAddCommGroup F] [NormedSpace ℓk F]

example : E → L[ℓk] E :=
  ContinuousLinearMap.id ℓk E

example (f : E → L[ℓk] F) : E → F :=
  f

example (f : E → L[ℓk] F) : Continuous f :=
  f.cont

example (f : E → L[ℓk] F) (x y : E) : f (x + y) = f x + f y :=
  f.map_add x y

example (f : E → L[ℓk] F) (a : ℓk) (x : E) : f (a • x) = a • f x :=
  f.map_smul a x
```

连续线性映射具有算子范数，其由下列性质刻画。

```
variable (f : E → L[ℓk] F)

example (x : E) : ‖f x‖ ≤ ‖f‖ * ‖x‖ :=
  f.le_opNorm x

example {M : ℝ} (hMp : 0 ≤ M) (hM : ∀ x, ‖f x‖ ≤ M * ‖x‖) : ‖f‖ ≤ M :=
  f.opNorm_le_bound hMp hM
```

还有打包连续线性同构的概念。这类同构的类型是 $E \simeq L[\mathbb{k}] F$ 。

作为一个有挑战性的练习，你可以证明巴拿赫-施泰因豪斯定理，也称为一致有界性原理。该原理断言：从巴拿赫空间到赋范空间的一族连续线性映射若逐点有界，则这些线性映射的范数一致有界。主要成分是贝尔定理

`nonempty_interior_of_iUnion_of_closed`。（你已经在拓扑章节中证明过它的一个版本。）次要成分包括 `continuous_linear_map.opNorm_le_of_shell`、`interior_subset`、`interior_iInter_subset` 和 `isClosed_le`。

```
variable {ℓk : Type*} [NontriviallyNormedField ℓk] {E : Type*} [NormedAddCommGroup E]
[NormedSpace ℓk E] {F : Type*} [NormedAddCommGroup F] [NormedSpace ℓk F]

open Metric

example {ι : Type*} [CompleteSpace E] {g : ι → E → L[ℓk] F} (h : ∀ x, ∃ C, ∀ i, ‖g i x‖ ≤ C) :
  ∃ C', ∀ i, ‖g i‖ ≤ C' := by
  -- sequence of subsets consisting of those `x : E` with norms `‖g i x‖` bounded by `n`
  let e : ℕ → Set E := fun n ↦ ⋂ i : ι, { x : E | ‖g i x‖ ≤ n }
  -- each of these sets is closed
  have hc : ∀ n : ℕ, IsClosed (e n)
  sorry
  -- the union is the entire space; this is where we use `h`
  have hU : (⋃ n : ℕ, e n) = univ
  sorry
  /- apply the Baire category theorem to conclude that for some `m : ℕ`,
  the interior of `e m` contains some `x` -/
  obtain ⟨m, x, hx⟩ : ∃ m, ∃ x, x ∈ interior (e m) := sorry
  obtain ⟨ε, ε_pos, he⟩ : ∃ ε > 0, ball x ε ⊆ interior (e m) := sorry
  obtain ⟨k, hk⟩ : ∃ k : ℓk, 1 < ‖k‖ := sorry
  -- show all elements in the ball have norm bounded by `m` after applying any `g i`
  have real_norm_le : ∀ z ∈ ball x ε, ∀ (i : ι), ‖g i z‖ ≤ m
  sorry
  have εk_pos : 0 < ε / ‖k‖ := sorry
  refine ⟨(m + m : ℕ) / (ε / ‖k‖), fun i ↦ ContinuousLinearMap.opNorm_le_of_shell ε_pos ?_ hk ?_⟩
  sorry
  sorry
```

12.2.3. 渐近比较

定义可微性还需要渐近比较。Mathlib 有一个覆盖大 O 与小 o 关系的丰富库；下面给出了这些关系的定义。打开 `asymptotics` locale 之后，就可以使用相应记号。这里我们只使用小 o 来定义可微性。

```
open Asymptotics

example {α : Type*} {E : Type*} [NormedGroup E] {F : Type*} [NormedGroup F] (c : ℝ)
  (l : Filter α) (f : α → E) (g : α → F) : IsBigOWith c l f g ↔ ∀ x in l, ‖f x‖ ≤ c * ‖g x‖ :=
  isBigOWith_iff

example {α : Type*} {E : Type*} [NormedGroup E] {F : Type*} [NormedGroup F]
  (l : Filter α) (f : α → E) (g : α → F) : f =O[l] g ↔ ∃ C, IsBigOWith C l f g :=
  isBigO_iff_isBigOWith

example {α : Type*} {E : Type*} [NormedGroup E] {F : Type*} [NormedGroup F]
  (l : Filter α) (f : α → E) (g : α → F) : f =o[l] g ↔ ∀ C > 0, IsBigOWith C l f g :=
  isLittleO_iff_forall_isBigOWith

example {α : Type*} {E : Type*} [NormedAddCommGroup E] (l : Filter α) (f g : α → E) :
  f ~[l] g ↔ (f - g) =o[l] g :=
  Iff.rfl
```

12.2.4. 可微性

现在可以讨论赋范空间之间的可微函数。类似初等一维情形，Mathlib 定义了谓词 `HasFDerivAt` 和函数 `fderiv`。这里字母 “f” 代表 *Fréchet*。

```
open Topology

variable {ℝ : Type*} [NontriviallyNormedField ℝ] {E : Type*} [NormedAddCommGroup E]
  [NormedSpace ℝ E] {F : Type*} [NormedAddCommGroup F] [NormedSpace ℝ F]

example (f : E → F) (f' : E →L[ℝ] F) (x₀ : E) :
  HasFDerivAt f f' x₀ ↔ (fun x ↦ f x - f x₀ - f' (x - x₀)) =o[N x₀] fun x ↦ x - x₀ :=
  hasFDerivAt_iff_isLittleO

example (f : E → F) (f' : E →L[ℝ] F) (x₀ : E) (hff' : HasFDerivAt f f' x₀) : fderiv ℝ f x₀ = f' :=
  hff'.fderiv
```

还有迭代导数，其取值于多线性映射类型 `E [x n] →L[ℝ] F`；也有连续可微函数。类型 `ℕ∞` 是在 `ℕ` 中额外加入一个大于每个自然数的元素 `∞` 得到的。因此， $\mathcal{C}^\infty$ 函数就是满足 `ContDiff ℝ T f` 的函数 `f`。

```
example (n : ℕ) (f : E → F) : E → E[x n] →L[ℝ] F :=
  iteratedFDeriv ℝ n f

example (n : ℕ∞) {f : E → F} :
  ContDiff ℝ n f ↔
  (∀ m : ℕ, (m : WithTop ℕ) ≤ n → Continuous fun x ↦ iteratedFDeriv ℝ m f x) ∧
  (∀ m : ℕ, (m : WithTop ℕ) < n → Differentiable ℝ fun x ↦ iteratedFDeriv ℝ m f x :=
  contDiff_iff_continuous_differentiable
```

`ContDiff` 中的可微性参数也可以取值 `ω : WithTop ℕ∞`，用来表示解析函数。

还有一个更严格的可微性概念，称为 `HasStrictFDerivAt`；它用于反函数定理和隐函数定理的陈述，这两个定理都在 Mathlib 中。对 `ℝ` 或 `ℂ` 而言，连续可微函数都是严格可微的。

```
example {ℝ : Type*} [RCLike ℝ] {E : Type*} [NormedAddCommGroup E] [NormedSpace ℝ E] {F : Type*}
  [NormedAddCommGroup F] [NormedSpace ℝ F] {f : E → F} {x : E} {n : WithTop ℕ∞}
  (hf : ContDiffAt ℝ n f x) (hn : 1 ≤ n) : HasStrictFDerivAt f (fderiv ℝ f x) x :=
  hf.hasStrictFDerivAt (zero_lt_one.trans_le hn).ne'
```

局部反函数定理使用一个操作来陈述：该操作从一个函数以及“该函数在点 `a` 处严格可微且其导数为同构”的假设出发，产生一个反函数。

下面第一个例子取得这个局部反函数。接下来的例子陈述它确实是左、右两侧的局部反函数，并且它严格可微。

```
section LocalInverse
variable [CompleteSpace E] {f : E → F} {f' : E →L[ℓk] F} {a : E}

example (hf : HasStrictFDerivAt f (f' : E →L[ℓk] F) a) : F → E :=
  HasStrictFDerivAt.localInverse f f' a hf

example (hf : HasStrictFDerivAt f (f' : E →L[ℓk] F) a) :
  ∀f x in  $\mathcal{N}$  a, hf.localInverse f f' a (f x) = x :=
  hf.eventually_left_inverse

example (hf : HasStrictFDerivAt f (f' : E →L[ℓk] F) a) :
  ∀f x in  $\mathcal{N}$  (f a), f (hf.localInverse f f' a x) = x :=
  hf.eventually_right_inverse

example (hf : HasStrictFDerivAt f (f' : E →L[ℓk] F) a) :
  HasStrictFDerivAt (HasStrictFDerivAt.localInverse f f' a hf) (f'.symm : F →L[ℓk] E) (f a) :=
  HasStrictFDerivAt.to_localInverse hf

end LocalInverse
```

这只是对 Mathlib 中微分学内容的快速浏览。库中还包含许多这里没有讨论的变体。例如，在一维设置中，你可能想使用单侧导数。Mathlib 在更一般的语境中提供了相应手段；参见 `HasFDerivWithinAt` 或更一般的 `HasFDerivAtFilter`。

13. 积分与测度论

13.1. 初等积分

首先关注 $\mathbb{R}$ 中有限区间上函数的积分。可以对初等函数进行积分。

```
open MeasureTheory intervalIntegral

open Interval
-- this introduces the notation `[[a, b]]` for the segment from `min a b` to `max a b`

example (a b : ℝ) : (∫ x in a..b, x) = (b ^ 2 - a ^ 2) / 2 :=
  integral_id

example {a b : ℝ} (h : (0 : ℝ) ∉ [[a, b]]) : (∫ x in a..b, 1 / x) = Real.log (b / a) :=
  integral_one_div h
```

微积分基本定理把积分与微分联系起来。下面给出该定理两部分的简化陈述。第一部分说，积分提供了微分的逆运算；第二部分说明如何计算导数的积分。（这两部分非常密切相关，但这里没有展示的最佳版本并不等价。）

```
example (f : ℝ → ℝ) (hf : Continuous f) (a b : ℝ) : deriv (fun u ↦ ∫ x in a..u, f x) b = f b :=
  (integral_hasStrictDerivAt_right (hf.intervalIntegrable _ _) (hf.stronglyMeasurableAtFilter _ _)
    hf.continuousAt).hasDerivAt.deriv

example {f : ℝ → ℝ} {a b : ℝ} {f' : ℝ → ℝ} (h : ∀ x ∈ [[a, b]], HasDerivAt f (f' x) x)
(h' : IntervalIntegrable f' volume a b) : (∫ y in a..b, f' y) = f b - f a :=
  integral_eq_sub_of_hasDerivAt h h'
```

卷积也在 Mathlib 中定义，并且其基本性质已经得到证明。

```
open Convolution

example (f : ℝ → ℝ) (g : ℝ → ℝ) : f * g = fun x ↦ ∫ t, f t * g (x - t) :=
  rfl
```

13.2. 测度论

Mathlib 中积分的一般语境是测度论。甚至上一节中的初等积分，实际上也是 Bochner 积分。Bochner 积分是 Lebesgue 积分的一种推广，其中目标空间可以是任意 Banach 空间，而不必有限维。

测度论发展的第一个组成部分，是集合的 σ -代数概念；其中的集合称为可测集合。类型类 `MeasurableSpace` 用来给一个类型配备这种结构。集合 `empty` 与 `univ` 是可测的；可测集的补集是可测的；可测集的可数并可数交也是可测的。注意，这些公理是冗余的；如果执行 `#print MeasurableSpace`，就会看到 Mathlib 实际采用的公理。如下例所示，可数性假设可以用类型类 `Encodable` 表达。

```

variable {α : Type*} [MeasurableSpace α]

example : MeasurableSet (∅ : Set α) :=
  MeasurableSet.empty

example : MeasurableSet (univ : Set α) :=
  MeasurableSet.univ

example {s : Set α} (hs : MeasurableSet s) : MeasurableSet (sᶜ) :=
  hs.compl

example : Encodable ℕ := by infer_instance

example (n : ℕ) : Encodable (Fin n) := by infer_instance

variable {ι : Type*} [Encodable ι]

example {f : ι → Set α} (h : ∀ b, MeasurableSet (f b)) : MeasurableSet (⋃ b, f b) :=
  MeasurableSet.iUnion h

example {f : ι → Set α} (h : ∀ b, MeasurableSet (f b)) : MeasurableSet (⋂ b, f b) :=
  MeasurableSet.iInter h

```

一旦一个类型是可测的，就可以对它作测度。纸笔数学中，在配备 σ -代数的集合（或类型）上，一个测度是从可测集到扩展非负实数的函数，并且对可数不交并满足可加性。在 Mathlib 中，我们不希望每次把测度应用于集合时都携带可测性假设。因此，我们把测度扩展到任意集合 s ，定义为包含 s 的可测集的测度的下确界。当然，许多引理仍需要可测性假设，但并非全部如此。

```

open MeasureTheory Function
variable {μ : Measure α}

example (s : Set α) : μ s = ⋂ (t : Set α) ( _ : s ⊆ t) ( _ : MeasurableSet t), μ t :=
  measure_eq_iInf s

example (s : ι → Set α) : μ (⋃ i, s i) ≤ ∑' i, μ (s i) :=
  measure_iUnion_le s

example {f : ℕ → Set α} (hmeas : ∀ i, MeasurableSet (f i)) (hdis : Pairwise (Disjoint on f)) :
  μ (⋃ i, f i) = ∑' i, μ (f i) :=
  μ.m_iUnion hmeas hdis

```

一旦某个类型关联了测度，如果性质 P 不成立的元素组成的集合测度为 0，就说 P 几乎处处成立。几乎处处成立的性质组成一个滤子，但 Mathlib 引入了专门记号来表达性质几乎处处成立。

```

example {P : α → Prop} : (∀ᵐ x ∂μ, P x) ↔ ∀ᶠ x in ae μ, P x :=
  Iff.rfl

```

13.3. 积分

现在有了可测空间和测度，就可以考虑积分。如上所述，Mathlib 使用非常一般的积分概念，允许任何 Banach 空间作为目标。照常，我们不希望记号携带各种假设，因此把积分定义成：若所讨论的函数不可积，则其积分等于零。多数与积分有关的引理都带有可积性假设。

```

section
variable {E : Type*} [NormedAddCommGroup E] [NormedSpace ℝ E] [CompleteSpace E] {f : α → E}

example {f g : α → E} (hf : Integrable f μ) (hg : Integrable g μ) :
  ∫ a, f a + g a ∂μ = ∫ a, f a ∂μ + ∫ a, g a ∂μ :=
  integral_add hf hg

```

作为各种约定之间复杂相互作用的例子，来看如何积分常值函数。回忆测度 μ 取值于 $\mathbb{R}_{\geq 0\infty}$ ，即扩展非负实数类型。有一个函数 `ENNReal.toReal :  $\mathbb{R}_{\geq 0\infty} \rightarrow \mathbb{R}$` ，它把无穷远点 $\top$ 送到零。对任意 $s : \text{Set } \alpha$ ，若 $\mu s = \top$ ，则非零常值函数在 s 上不可积。在这种情况下，它们的积分按定义等于零，而 `( $\mu s$ ).toReal` 也是如此。因此，在所有情形中都有下面的引理。

```
example {s : Set α} (c : E) : ∫ _ in s, c ∂μ = (μ s).toReal • c :=
  setIntegral_const c
```

现在快速说明如何访问积分理论中最重要的定理，从支配收敛定理开始。Mathlib 中有若干版本，这里只展示最基本的一个。

```
open Filter

example {F : ℕ → α → E} {f : α → E} (bound : α → ℝ) (hmeas : ∀ n, AESTronglyMeasurable (F n) μ)
  (hint : Integrable bound μ) (hbound : ∀ n, ∀m a ∂μ, ‖F n a‖ ≤ bound a)
  (hlim : ∀m a ∂μ, Tendsto (fun n : ℕ ↦ F n a) atTop (ℳ (f a))) :
  Tendsto (fun n ↦ ∫ a, F n a ∂μ) atTop (ℳ (∫ a, f a ∂μ)) :=
  tendsto_integral_of_dominated_convergence bound hmeas hint hbound hlim
```

随后有关于积类型上积分的 Fubini 定理。

```
example {α : Type*} [MeasurableSpace α] {μ : Measure α} [SigmaFinite μ] {β : Type*}
  [MeasurableSpace β] {ν : Measure β} [SigmaFinite ν] (f : α × β → E)
  (hf : Integrable f (μ.prod ν)) : ∫ z, f z ∂ μ.prod ν = ∫ x, ∫ y, f (x, y) ∂ν ∂μ :=
  integral_prod f hf
```

卷积还有一个非常一般的版本，适用于任意连续双线性型。

```
open Convolution

variable {ℓk : Type*} {G : Type*} {E : Type*} {E' : Type*} {F : Type*} [NormedAddCommGroup E]
  [NormedAddCommGroup E'] [NormedAddCommGroup F] [NontriviallyNormedField ℓk] [NormedSpace ℓk E]
  [NormedSpace ℓk E'] [NormedSpace ℓk F] [MeasurableSpace G] [NormedSpace ℝ F] [CompleteSpace F]
  [Sub G]

example (f : G → E) (g : G → E') (L : E →L[ℓk] E' →L[ℓk] F) (μ : Measure G) :
  f *[L, μ] g = fun x ↦ ∫ t, L (f t) (g (x - t)) ∂μ :=
  rfl
```

最后，Mathlib 有一个非常一般的变量代换公式版本。在下面的陈述中，`BorelSpace E` 表示 `E` 上的 σ -代数由 `E` 的开集生成；`IsAddHaarMeasure μ` 表示测度 `μ` 左不变，对紧集赋予有限质量，并对开集赋予正质量。

```
example {E : Type*} [NormedAddCommGroup E] [NormedSpace ℝ E] [FiniteDimensional ℝ E]
  [MeasurableSpace E] [BorelSpace E] (μ : Measure E) [μ.IsAddHaarMeasure] {F : Type*}
  [NormedAddCommGroup F] [NormedSpace ℝ F] [CompleteSpace F] {s : Set E} {f : E → E}
  {f' : E → E →L[ℝ] E} (hs : MeasurableSet s)
  (hf : ∀ x : E, x ∈ s → HasFDerivWithinAt f (f' x) s x) (h_inj : InjOn f s) (g : E → F) :
  ∫ x in f '' s, g x ∂μ = ∫ x in s, |(f' x).det| • g (f x) ∂μ :=
  integral_image_eq_integral_abs_det_fderiv_smul μ hs hf h_inj g
```